GnssProcessor: Mathematical Reference

Revision date: July 31, 2026

Contents

1 Introduction **5**

I Foundations **7**

2 Notation and conventions **7**

3 State vector layout **9**
3.1 Schematic layout . . . 9
3.2 Component-by-component description . . . 9
3.3 Dynamic indexing . . . 13

4 Observation equations **14**
4.1 Code (pseudorange) observation . . . 14
4.2 Carrier-phase observation . . . 14
4.3 Geometric range and linearisation . . . 15
4.4 Stochastic model . . . 15

5 Differencing transformations **16**
5.1 Between-satellite single difference (BSSD) . . . 16
5.2 Between-receiver single difference (BRSD) . . . 17
5.3 Double difference (DD) . . . 18
5.4 S-transform structure . . . 18

II Reference systems and measurement models **19**

6 Coordinate Frames and Time Systems **19**
6.1 The WGS84 ellipsoid: ECEF, geodetic, and local frames . . . 19
6.2 Station reference points: marker, ARP, and phase centre . . . 21
6.3 GNSS time scales and the GPS reference epoch . . . 22
6.4 Leap seconds and per-system dispatch . . . 23
6.5 GLONASS time-of-day to full GPS week . . . 24
6.6 BeiDou time: the 14 s offset and independent week base . . . 25
6.7 Legacy ECI→ECEF: the equinox-based chain . . . 25
6.8 CIO-based ITRS↔GCRS (IAU 2000A/2006) . . . 26
6.9 Earth-orientation parameters: parse and interpolation . . . 27

6.10 The Sun and Moon series these transforms position . . . 27

7 Orbit and Clock Product Evaluation 28
7.1 Precise orbit: derotated Neville interpolation . . . 28
7.2 Precise orbit: sidereal-trigonometric LSQ fit . . . 30
7.3 Precise clock interpolation . . . 31
7.4 Broadcast Keplerian evaluation . . . 32
7.5 Broadcast GLONASS: PZ-90 RK4 integration . . . 33
7.6 Broadcast satellite clock . . . 33
7.7 Ephemeris selection and IOD bookkeeping . . . 34

8 Atmospheric Delay Modelling: Troposphere and Ionosphere 35
8.1 The slant tropospheric delay . . . 35
8.1.1 Saastamoinen zenith hydrostatic delay . . . 36
8.1.2 The Marini mapping form . . . 36
8.1.3 GPT3 empirical pressure, temperature and VMF3 a-coefficients . . . 37
8.1.4 The gradient mapping and a degenerate fallback . . . 38
8.2 The slant ionospheric delay . . . 38
8.2.1 The $1/f^2$ scaling and its sign . . . 38
8.2.2 IONEX / GIM thin-shell slant delay . . . 39
8.2.3 The GIM iono-weighting pseudo-observation . . . 39
8.2.4 The co-estimated spherical-harmonic VTEC field . . . 40
8.2.5 Klobuchar broadcast seed . . . 43
8.3 Higher-order ionospheric corrections . . . 43
8.3.1 The geomagnetic dipole field . . . 44

9 Site Displacement and Geophysical Corrections 44
9.1 Sign convention and projection into the range . . . 45
9.2 Solid Earth tide (IERS 2010, Steps 1 + 2) . . . 45
9.2.1 Step 1 — in-phase degree 2 and 3 . . . 46
9.2.2 Step 1 — out-of-phase anelasticity and the L^1 term . . . 46
9.2.3 Step 2 — frequency dependence (Doodson tables) . . . 47
9.3 Ocean tidal loading (OTL) . . . 47
9.3.1 Astronomical argument and the Doodson–Warburg phase bias . . . 48
9.3.2 Coefficient sources: SIF, BLQ, and the EOT20 grid . . . 48
9.4 Pole tide (rotational deformation, IERS 7.1.4) . . . 49
9.5 Ocean pole tide (Desai 2002, IERS 7.1.5) . . . 49
9.6 Atmospheric pressure loading (S1/S2 thermal tide) . . . 50
9.7 Integration into the measurement model . . . 50

10 Observation Model Physics: Antenna, Attitude, Relativity, and Range Corrections 51
10.1 Satellite attitude: the body frame . . . 51
10.2 Antenna phase-centre corrections . . . 53
10.3 Carrier-phase wind-up . . . 54
10.4 Periodic relativistic clock correction . . . 55
10.5 Shapiro (gravitational-delay) correction . . . 56
10.6 Sagnac (Earth-rotation) correction . . . 56

10.7 Group delays and inter-frequency biases . . . 56
10.8 Provenance and assembly . . . 57

III Estimation engine **58**

11 Estimation engine: the Kalman filter and RTS smoother **58**
11.1 Time update . . . 58
11.2 Measurement update . . . 60
11.3 Iterated relinearisation (IEKF) . . . 61
11.4 Innovation and residual gating . . . 62
11.5 Backward RTS smoother . . . 63

12 Stochastic Model and Quality Control **65**
12.1 The stochastic-model dispatch . . . 65
12.2 A-posteriori variance factor . . . 67
12.3 Redundancy numbers, the Baarda w-test, and the MDB . . . 68
12.4 Data-snooping gate override . . . 70
12.5 Correlation-aware full-$\boldsymbol{R}$ kernel . . . 70
12.6 Producer-side RAIM: within-epoch adaptation . . . 71
12.7 Pooled variance factors and the per-constellation carve-out . . . 72
12.8 Station-relative deweighting of the ionosphere constraint . . . 73
12.9 State-domain emission gates . . . 74

13 Estimator Infrastructure: WLS Seeding, State Initialisation, and Pivot Management **74**
13.1 The WLS epoch estimator: receiver-clock seed and post-gap re-seed . . . 75
13.2 First-epoch state seeding by residual differencing . . . 76
13.3 BSSD pivot management . . . 78
13.3.1 Lexicographic candidate ranking . . . 79
13.3.2 Ephemeris-freshness guardrail . . . 79
13.3.3 Pivot disturbance detection and handover . . . 80

IV Cycle-slip detection and repair **80**

14 Cycle-slip detection and repair **81**
14.1 Detector registry and composition . . . 81
14.2 Test-statistic combinations . . . 82
14.2.1 Melbourne–Wübbena (wide-lane) . . . 82
14.2.2 Extra-wide-lane (EWL) . . . 83
14.2.3 GRAPHIC (single-frequency half-sum) . . . 83
14.2.4 Geometry-free phase (staged) . . . 84
14.2.5 Ionosphere-free phase (staged on the user path; armed on the network path) 84
14.3 Network slip QC: raw evidence and refloat over repair . . . 86
14.4 TurboEdit dual-frequency resolution . . . 87
14.4.1 The 2×2 system . . . 87
14.4.2 Integer rounding and validation . . . 88
14.4.3 MW-only / code-event null line . . . 89

14.4.4 Gap-spanning GF bridge . . . 89
14.4.5 Atomic reconciliation . . . 90

V Integer ambiguity resolution **90**

15 Integer ambiguity resolution **91**
15.1 The integer least-squares problem . . . 91
15.2 Decorrelation: LDL^{T} factorisation and the integer $\boldsymbol{Z}$-transform . . . 92
15.3 Wide-lane / narrow-lane decomposition (PPP path) . . . 93
15.4 Partial / subset ambiguity resolution . . . 94
15.5 Validation: ratio test and fixed-failure-rate gate . . . 96
15.6 Feedback: constrained pseudo-observation update . . . 96
15.7 Network-producer AR (both mechanisms default off) . . . 97
15.8 Configuration summary . . . 99

VI Inertial aiding **99**

16 Inertial aiding **99**
16.1 Strapdown mechanization . . . 100
16.2 INS error-state dynamics . . . 101
16.3 Covariance splice and attitude feedback . . . 102
16.4 Coupling modes and the GNSS update . . . 102

VII Real-time SSR corrections **103**

17 Real-time SSR corrections **103**
17.1 Orbit correction — refines ρ_r^i . . . 103
17.2 Clock correction — refines $-c\,\mathrm{d}t^i$ . . . 104
17.3 IOD gate at correction-set boundaries . . . 105
17.4 Satellite biases — refine $-B_s^i$ and $-b_s^i$ . . . 105
17.5 Satellite-bias STATE gate vs. SSR code biases . . . 106
17.6 GLONASS . . . 107
17.7 Ionosphere — $\mu_s I_{r,1}^i$ off-wire, via the slant channel . . . 107
17.8 Provenance and code pointers . . . 108

VIII Real-time engine **109**

18 Real-time engine **109**
18.1 Streaming control flow . . . 109
18.2 NTRIP transport . . . 110
18.3 RTCM 3 framing and CRC . . . 110
18.4 MSM observation decode . . . 111
18.5 Broadcast nav, SSR and station coordinates . . . 112
18.6 Time-system handling . . . 113
18.7 Observation/SSR synchronisation . . . 113

18.8 Ephemeris management . . . 114
18.9 A-priori seeding from RTCM 1005/1006 . . . 114
18.10 The per-epoch real-time update . . . 115

IX Network mode **116**

19 Network mode **116**
19.1 Producer architecture: the decomposed epoch loop . . . 117
19.2 Joint multi-station observation model . . . 120
19.3 The explicit satellite phase-bias state . . . 120
19.4 Rank deficiency and the minimum-constraint S-basis . . . 121
19.5 Per-station Schur reduction (arrowhead normals) . . . 122
19.6 Constellation scope . . . 123
19.7 The closed loop: producer → held-out user . . . 124

20 Worked example: estimability in a two-receiver network **125**
20.1 Code only: clocks and one gauge freedom . . . 125
20.2 Adding carrier phase: the satellite phase bias needs the network . . . 126

X Reference **128**

21 Pipeline and architecture diagrams **128**

A Generated reference tables **128**
A.1 Processing modes . . . 128
A.2 Execution mode and data source . . . 129
A.3 Differencing regime . . . 129
A.4 Config-to-differencing mapping . . . 129
A.5 Feature toggles . . . 135
A.6 Model-correction switches . . . 141
A.7 Logging and trace channels . . . 144
A.8 Cycle-slip detector registry . . . 146
A.9 State-vector key schema . . . 146

1 Introduction

This document is the mathematical companion to `GnssProcessor`, a C++17 post-processing and real-time GNSS engine implementing PPP, PPP-RTK (single-receiver and network), and double-differenced baseline modes. Its scope is the estimation mathematics and design rationale — the material that ages slowly because it is settled theory or algebra, not churning code.

The reference is organised as a set of *component documents*, each of which also compiles to a standalone PDF. This combined volume gathers them in roughly the order the engine applies them:

- **Foundations** (Sections 2 to 5) — notation, the variable-dimension state vector, the *uncombined* pseudorange / carrier-phase observation model, and the between-satellite single-difference (BSSD) and double-difference (DD) transformations. Every other component builds on this.

- **Reference systems and measurement models** (Sections 6 to 10) — the deterministic substrate and physics behind the observation model the Foundations document writes abstractly: the coordinate frames and time systems every other chapter assumes; the baseline satellite orbit/clock product evaluation that SSR refines; tropospheric and ionospheric delay modelling; the geophysical site displacements (solid Earth, ocean, and pole tides) that correct the receiver coordinate; and the antenna, attitude, relativity, and range corrections that map the raw measurement to the modelled observable.
- **Estimation engine** (Sections 11 to 13) — the variable-dimension Kalman filter (Joseph-form update, prior-anchored iterated relinearisation, robust gating) and the backward RTS smoother; the measurement stochastic model and the a-posteriori quality-control layer (variance factor, Baarda data-snooping, minimal detectable bias); and the supporting infrastructure that bootstraps the filter — the weighted least-squares seed, the first-epoch residual-difference state initialisation, and the pivot selection / disturbance logic.
- **Cycle-slip detection and repair** (Section 14) — the linear-combination test statistics (Melbourne–Wübbena, geometry-free, GRAPHIC, extra-wide-lane) and the TurboEdit 2×2 integer repair.
- **Integer ambiguity resolution** (Section 15) — the MLAMBDA integer least-squares search, wide-lane / narrow-lane decomposition, validation, constrained feedback, and the (default-off) network-producer AR machinery.
- **Inertial aiding** (Section 16) — the strapdown mechanisation and 15-state INS error model of the tightly-coupled IMU path.
- **Real-time SSR corrections** (Section 17) — how State Space Representation orbit, clock, and bias corrections refine the undifferenced observation model.
- **Real-time engine** (Section 18) — the streaming architecture: NTRIP transport, RTCM/MSM decoding, observation/SSR time synchronisation, and the per-epoch real-time update path.
- **Network mode** (Section 19) — the multi-station PPP-RTK network estimator that *generates* the SSR-style corrections consumed by the single-receiver modes: the joint observation model, the S-basis / minimum-constraint datum, the per-station Schur reduction, and the single-concern seam architecture of the per-epoch producer loop.
- **Pipeline and architecture diagrams** (Section 21) — the per-mode data-flow figures that place every chapter above in the epoch loop.
- **Generated reference tables** (Section A, appendix) — processing modes, feature toggles, the cycle-slip detector registry, and the state-vector key schema, produced directly from the source tree by `scripts/gen_report_tables.py` rather than maintained by hand.

Implementation details (function signatures, class responsibilities, config field reference) live in the headers under `include/` and are rendered via Doxygen. When this document references a specific algorithmic component, it cites the responsible *symbol* — a function, class, or JSON config key — together with its source file (e.g. `KalmanFilter::update` in `src/core/filter/KalmanFilter.cpp`, or `network.cold_clock_seed`). Line numbers are deliberately never cited: they rot within weeks, symbol names do not.

The notation follows the IGS conventions adopted by Subirana, Zornoza, and Hernández-Pajares [23] and Teunissen and Montenbruck [27]; see Section 2 for the symbol table this document depends on.

Part I
Foundations

2 Notation and conventions

Indices and sets. We use lower-case Latin letters for indices and calligraphic upper-case for sets:

- $r \in \mathcal{R}$ — receiver. Single-receiver modes (PPP, PPP-RTK) have $|\mathcal{R}| = 1$; baseline modes have a fixed reference and one or more rovers; the network producer (`PPP_RTK_Network`) carries a peer set of $|\mathcal{R}| \geq 2$ stations (50–305 in practice) with no reference/rover asymmetry.
- $g \in \mathcal{G} = \{\text{GPS}, \text{GAL}, \text{BDS}, \text{GLO}, \text{QZS}, \dots\}$ — constellation.
- $i \in \mathcal{S}_g$ — satellite of constellation g. p_g denotes the pivot satellite of constellation g under the BSSD parameterisation (Section 5).
- $s \in \mathcal{B}$ — signal/observation type, identified by its RINEX-3 three-character code (e.g. `C1C`, `L2W`, `L5X`).
- $k \in \mathbb{N}$ — discrete epoch index.

Vectors and matrices. Bold lower-case for vectors ($\boldsymbol{x}$, $\boldsymbol{r}$); bold upper-case for matrices ($\boldsymbol{H}$, $\boldsymbol{P}$). $\boldsymbol{a}^{\mathrm{T}}$ is the transpose. A hat denotes a state estimate; a tilde a one-step prediction: $\hat{\boldsymbol{x}}_{k|k}$ vs. $\tilde{\boldsymbol{x}}_{k|k-1}$.

Observables. For each receiver/satellite/signal triple, the engine consumes two real-valued observables (in metres) per epoch:

- $P^i_{r,s}$ — pseudorange (code), in metres.
- $\Phi^i_{r,s}$ — carrier phase, scaled to metres via λ_s. Beyond the range, clock, and atmosphere terms it shares with the code, its ambiguous (non-range) part is $\lambda_s N^i_{r,s}$ plus the fractional phase biases $b_{r,s} - b^i_s$ (Equation (5)).

Where the distinction matters, s may further denote the band ($L_1, L_2, L_5, \dots$) and the tracking attribute (e.g. `C`, `W`, `X`). The leading observable-type letter (`C`/`L`) selects code vs. phase; the frequency-only quantities f_s, λ_s, and $N^i_{r,s}$ depend only on the (band, attribute) part, so a code and its sibling phase share them. This matches the state keys, which carry the full phase code for ambiguities (`"G01_L1C"`) but only the band digit for receiver code biases (`"GPS_IFB2"`).

Frequencies and wavelengths. f_s is the carrier frequency (Hz) and $\lambda_s = c/f_s$ the wavelength (m). The frequency-squared scaling factor used to project the L1-equivalent slant ionospheric delay onto signal s is

$$\mu_s \;:=\; \frac{f_1^2}{f_s^2}. \tag{1}$$

Here f_1 is the GPS L1 frequency (1575.42 MHz), used as the common reference for *every* constellation — not the band-1 frequency of the satellite's own system. The estimated state $I^i_{r,1}$ is therefore

GPS-L1-equivalent even for BDS/GLO/GAL satellites (`MeasurementModel::buildJacobianRow`: `f_ref = Constants::GPS_L1_FREQ`).

Differencing operators. Three differencing operations recur throughout the document:

$$\Delta q^{i,p_g}_{r,s} := q^i_{r,s} - q^{p_g}_{r,s}, \quad \nabla q^i_{r_1 r_2,s} := q^i_{r_1,s} - q^i_{r_2,s}, \quad \nabla\Delta q^{i,p_g}_{r_1 r_2,s} := \Delta\big(\nabla q^i_{r_1 r_2,s}\big).$$

Read Δq as "between-satellite single difference of q relative to pivot p_g," ∇q as "between-receiver single difference," and $\nabla\Delta q$ as "double difference."

Sign conventions. We adopt the convention that the slant ionospheric delay $I^i_{r,s}$ enters *additively* on the code observable and *subtractively* on the carrier phase observable; this is the source of the well-known "code–phase divergence." All other range-additive terms (geometry, clocks, troposphere, biases) carry the same sign on both observables. See Equations (4) and (5) for the explicit equations.

Symbol table.

Symbol	Units	Meaning
$\boldsymbol{r}_r, \boldsymbol{r}^i$	m	ECEF position of receiver/satellite
ρ^i_r	m	geometric range $\|\boldsymbol{r}^i - \boldsymbol{r}_r\|$
$\boldsymbol{e}^i_r$	—	unit LOS from r to i in ECEF
$c\,\mathrm{d}t_r$	m	receiver clock offset (scaled by c)
$c\,\mathrm{d}t^i$	m	satellite clock offset (CLK product in user modes; estimated state `"<sat>|SC"` in `PPP_RTK_Network`)
$c\,\delta t^{(g)}_r$	m	receiver inter-system time offset for constellation $g \neq g_{\mathrm{ref}}$
T^i_r	m	total tropospheric path delay
$\mathrm{ZHD}^i_r, \mathrm{ZWD}^i_r$	m	zenith hydrostatic / wet components
$I^i_{r,1}$	m	L1-equivalent slant ionospheric delay
$N^i_{r,s}$	cyc	integer carrier-phase ambiguity
$b_{r,s}, b^i_s$	m	receiver / satellite phase biases
$\bar{b}^i_s$	m	estimable satellite phase bias (network mode; Section 3)
$B_{r,s}, B^i_s$	m	receiver / satellite code biases (OSB)
$B^{(g,b)}_{r,k}$	m	per-FCN-slot GLONASS receiver IFB, band b; here $k \in [-7,+6]$ is the signed FDMA channel number (a context-local reuse, not the epoch index)
$\varepsilon^P, \varepsilon^\Phi$	m	residual (multipath + thermal)
$\boldsymbol{v}_r$	m/s	receiver velocity (optional / IMU state)
$\lambda_{\mathrm{WL}},\ \lambda_{\mathrm{NL}},\ \lambda_{\mathrm{EWL}}$	m	wide- / narrow- / extra-wide-lane wavelength $c/(f_a \mp f_b)$ for the lane's band pair
λ_{IF}	m	effective ionosphere-free wavelength $\|c_1\lambda_1 - c_2\lambda_2\|$ with the IF coefficients $c_1 - c_2 = 1$; not an integer wavelength (Section 14)
$\boldsymbol{C}^e_b$	—	body-to-ECEF attitude (direction-cosine) matrix

3 State vector layout

The Kalman filter operates on a state vector $\boldsymbol{x}_k \in \mathbb{R}^{n_k}$ whose dimension n_k *varies across epochs*: per-satellite and per-(satellite, signal) substates are added when a new sat/signal is first observed and pruned (or marked stale) when the satellite drops below the elevation mask or its phase track is interrupted by a cycle slip without recovery. All bookkeeping — key→index maps, free-list re-use, and dimensional reshaping of $\boldsymbol{x}$ and $\boldsymbol{P}$ — is encapsulated in `StateManager`; `KalmanFilter` consumes the state through that interface and never indexes raw memory by hard-coded offsets.

3.1 Schematic layout

In schematic form, suppressing the epoch index k:

$$\begin{aligned}\boldsymbol{x} \;=\; \big[\; & \underbrace{\boldsymbol{r}_r^{\mathrm{T}}}_{3},\; \underbrace{\boldsymbol{v}_r^{\mathrm{T}}}_{0 \text{ or } 3},\; \underbrace{c\,\mathrm{d}t_r}_{1},\; \underbrace{(c\,\delta t_r^{(g)})^{\mathrm{T}}_{g\neq g_{\mathrm{ref}}}}_{|\mathcal{G}|-1},\\ & \underbrace{\mathrm{ZWD}_r,\,(g_N, g_E)}_{1+0 \text{ or } 1+2},\; \underbrace{(c\,\mathrm{d}t_r^{\phi})}_{0 \text{ or } 1},\\ & \underbrace{(I^i_{r,1})^{\mathrm{T}}_{i\in\mathcal{V}_k}}_{|\mathcal{V}_k|},\; \underbrace{(N^i_{r,s})^{\mathrm{T}}_{(i,s)\in\mathcal{A}_k}}_{|\mathcal{A}_k|},\\ & \underbrace{(B_{r,s})^{\mathrm{T}}_{s\in\mathcal{B}}}_{|\mathcal{B}|},\; \underbrace{(B^i_s)^{\mathrm{T}}_{(i,s)\in\mathcal{D}_k}}_{|\mathcal{D}_k|}\;\big]^{\mathrm{T}}. \end{aligned} \tag{2}$$

where

- $\mathcal{V}_k$ is the set of satellites currently visible at epoch k (i.e. above the elevation mask, with valid orbit/clock products and at least one tracked signal),
- $\mathcal{A}_k \subseteq \mathcal{V}_k \times \mathcal{B}$ is the set of phase-tracked (satellite, signal) pairs,
- $\mathcal{D}_k \subseteq \mathcal{V}_k \times \mathcal{B}$ is the subset of (satellite, signal) pairs carrying a satellite code-bias state: the pairs for which the BIA product lacks a satellite OSB or — under the network producer's satellite-OSB mode — every raw code pair (see `estimation.estimate_sat_biases` and `network.estimate_sat_code_bias`).

For brevity the schematic suppresses several conditional refinements detailed below: the rate states that accompany velocity (a receiver clock-rate and one inter-system clock-rate offset per non-reference constellation), the optional per-satellite slant-iono *rate* companion under `enable_iono_rate`, the network-producer satellite-clock and receiver-phase-bias states, the nine-state inertial augmentation of Section 16, and the fact that the entire receiver-side clock and inter-system-offset block is removed under the BSSD parameterisation (Section 5.1).

3.2 Component-by-component description

Position $\boldsymbol{r}_r \in \mathbb{R}^3$. ECEF Cartesian receiver position. Always present. Initialised either from the RINEX header approximate position or, after the first epoch, from a Weighted Least-Squares (WLS) seed solution to within the linearisation radius of the geometric range model (Section 4.3).

Velocity $\boldsymbol{v}_r \in \mathbb{R}^3$. Optional. Enabled by `estimation.estimate_velocity = true`, or forced on by tightly-coupled IMU integration. When present, position and velocity form a coupled continuous-white-noise-acceleration (CWNA, integrated-random-walk) block driven by a *single* acceleration spectral density `process_noise.acceleration_psd_q` (Equation (141)); the scalar `position_q` of the velocity-free case is then unused. Enabling velocity also introduces a receiver clock-rate state $c\,\dot{\mathrm{d}}t_r$, and one inter-system clock-rate offset per non-reference constellation, each CWNA-coupled to its bias counterpart (`clock_rate_q`, `system_time_rate_offset_q`).

Receiver clock $c\,\mathrm{d}t_r$. Single state, in metres. Process noise is large (`clock_bias_q` $\sim 10^4\,\mathrm{m}^2$ per second) so the state is effectively re-estimated each epoch; this captures oscillator drift without imposing a stochastic model that may not match the receiver's actual oscillator. Under the BSSD (between-satellite single-difference) parameterisation (Section 5.1) the receiver clock is unobservable and the state is dropped entirely — as are the inter-system offsets below — so the whole receiver-side clock block vanishes in PPP-RTK SD mode.

Inter-system time offsets $c\,\delta t_r^{(g)}$. One per active constellation other than the chosen reference (default GPS). These absorb the per-constellation clock biases that arise from mixing analysis-centre-defined clock products across systems ("GAL − GPS time offset," etc.).

Tropospheric ZWD and gradients. ZWD_r models the zenith wet delay; the zenith hydrostatic component ZHD_r is treated deterministically via the Saastamoinen model [22]. Total slant tropospheric delay is then

$$T_r^i \;=\; m_h(\theta_r^i)\,\mathrm{ZHD}_r \;+\; m_w(\theta_r^i)\,\mathrm{ZWD}_r \;+\; m_w(\theta_r^i)\cot\theta_r^i\big[g_N\cos\alpha_r^i + g_E\sin\alpha_r^i\big], \tag{3}$$

with elevation θ_r^i, azimuth α_r^i, and the Niell or VMF mapping functions m_h, m_w. Gradients (g_N, g_E) are optional.

Per-satellite slant ionosphere $I_{r,1}^i$. PPP-RTK-only. One state per visible satellite, expressed at the L1 frequency; signal-specific delays follow Equation (1). Initial value seeded from the Klobuchar broadcast model when no external ionospheric product is available. Its process noise is a random walk whose zenith spectral density `ionosphere_q` is projected onto the slant by the squared obliquity factor $1/\sin^2\theta_r^i$ (Equation (140)), letting low-elevation states vary faster. Modelling these per-sat states explicitly is what permits PPP-RTK to retain integer ambiguity recovery without forming the ionosphere-free combination (which amplifies code noise by a factor of ~ 3). Optionally, under `estimation.enable_iono_rate` (default off), each visible satellite carries a companion constant-velocity block $[\,I_{r,1}^i,\ \dot{I}_{r,1}^i\,]$: the slant-iono *rate* $\dot{I}_{r,1}^i$ is unobserved directly ($H = 0$) and driven only by the F-coupling into the level state, with its own spectral density `process_noise.iono_rate_psd` and its own slip response (`reinflateIonosphereRateVariance`), letting the filter track a linear slant ramp rather than a pure random walk.

Carrier-phase ambiguities $N_{r,s}^i$. One state per phase-tracked (i, s) pair, in metres (the state value is λN; the `nl_wl` basis below is the exception, carrying integers in cycles). Treated as float during the prediction/update cycle and resolved to integers post-hoc by the LAMBDA-family algorithms in `filter/AmbiguityResolver` [26] when AR is enabled. Cycle slips trigger a re-seed via `StateManager::resetAmbiguity`, which *decorrelates* the state rather than repairing it: the ambiguity's covariance row and column are zeroed and its diagonal reset to σ_m^2, while the state *value* is left unchanged (it is overwritten by the geometry-based seed in `ensureRequiredStates`).

This follows the project's reset-not-repair convention — a slipped ambiguity carries no trustworthy correlation to the rest of the state, so the cross-state correlations are broken, not preserved.

Receiver code biases $B_{r,s}$. These absorb the receiver-side inter-frequency biases that the OSB convention does not reference to zero. They are *not* one per signal: `StateInitializer::resolveReceiverBias` keys them per (system, frequency band) as inter-frequency biases (`"<system>_IFB<band>"`) plus, in the alternative scheme, a single per-system differential code bias (`"<system>_DCB"`). Like the receiver clock they are receiver-common and so are suppressed under BSSD. On the network path these columns have no Kalman time update at all — the producer runs no `KalmanFilter` (Section 11) — so they are re-estimated afresh each epoch unless `network.carry_rx_code_bias` is armed: an opt-in, default-off diagonal random-walk carry ($\mathrm{var} + q\,\Delta t$) held in the network run context rather than in `StateManager`, guarded at load by the requirement of a time-linked code datum (`network.code_datum_time_linked`, absent which the loader forces the carry off) and at inject time by an innovation gate (`rxCbInjectGate` in `filter/NetworkRunTypes`) that withholds the fold when the carried level and the station's own estimate disagree at receiver-reset scale. It is not yet field-validated.

Per-slot GLONASS inter-frequency bias $B_{r,k}^{(g,b)}$. Optional, off by default. GLONASS FDMA assigns each slot $k \in [-7, +6]$ a distinct carrier frequency $f = f_{\mathrm{base}} + k\,\Delta f$, so the receiver group delay differs *per slot* and a single per-band IFB state cannot represent it. When `estimation.glonass_ifb.enable = true` (and the prerequisite `estimation.estimate_ifb`; the loader warns and the flag is inert otherwise), `resolveReceiverBias` instead emits one state per (FDMA band $b \in \{\mathrm{G1}, \mathrm{G2}\}$, slot k) with unit measurement coefficient, keyed `"GLONASS_IFB<band>_k<±slot>"`. The central channel $k = 0$ is the held datum and produces no state; its constant is absorbed by the GLONASS inter-system time offset. CDMA bands (G3/G4/G6) have a slot-independent frequency and retain the ordinary per-band treatment above. Crucially, unlike the receiver-common biases these columns *survive* between-satellite single differencing — two GLONASS satellites on different slots do not cancel — so `BssdTransform` and `StateManager` special-case them (via `isGlonassPerSlotIfbKey`) to exempt them from the receiver-block zeroing; only the inter-slot *differences* remain estimable in SD.

Satellite code biases B_s^i. Per (satellite, signal) pair, but *only when the BIA file does not supply an OSB* for that pair — typically a third-frequency signal in a triple-frequency setup. That fallback rule is the user-path and network-V1 behaviour. Under the network producer's satellite-OSB mode (`network.estimate_sat_code_bias`, also implied and forced on by `network.estimate_iono_sh`) the loader instead *promotes* the state to every raw code (i, s) rather than only the uncovered pairs, and suppresses the external `.BIA` entirely so the network's own estimate is the single code-bias source; the datum that makes it estimable is derived in Section 19. When present, these states are weakly stochastic (`sat_bias_q` $\sim 10^{-10}$) since the underlying physical biases are nearly stationary on the daily timescale.

Satellite phase biases $\bar{b}_s^i$ (network mode). Present only in the network producer (`ProcessingMode::PPP_RTK_Network`) and gated by `estimation.estimate_sat_phase_biases`. The JSON default is off and the flag is inert in every other mode, but this is not an experimental opt-in: in the one mode where the states exist the network-producer loader force-enables it unconditionally (the *forcing estimate_sat_clocks + estimate_sat_phase_biases* branch in `GnssConfigLoadNetwork`), and the mirror `GnssConfig::applyNetworkConsumerCouplings` turns

it back off for the closed-loop consumer, which *consumes* these states rather than estimating them. The user side imports the satellite phase bias as a known SSR correction (Section 17) or absorbs it into the float ambiguity; the network instead carries an *explicit* per-(satellite, signal) phase-bias state on the undifferenced phase rows (the estimable bias of Odijk [17], lumping the network pivot-receiver ambiguity). On a single station's row it is co-linear with the per-receiver float ambiguity and is separable only *across* stations in the joint network estimator (Section 19). It lives in a *distinct* state map from the satellite *code* biases above (`StateManager::sat_phase_bias_indices_`), keyed `"<sat>|<signal>|PB"`, and is near-stationary between epochs at the rate `sat_phase_bias_q` — which on the network path is applied as the inflation of a shared-border *carry prior*, not as a Kalman $\boldsymbol{Q}(\Delta t)$ entry (see the satellite-clock paragraph below).

Receiver phase biases $\delta_{r,s}$ (network mode). The receiver-side counterpart of the satellite phase bias, present only in the network producer and gated by `network.phase_datum_eq34` (the Eq-34 / single-receiver-chain phase datum; see Section 19). Giving the receiver fractional phase bias its *own* estimable state $\delta_{r,s}$ stops it lumping into the emitted satellite phase bias, so the emitted between-satellite reference stays integer-consistent across *all* satellites (each chain hop then contributes only an integer). It is per (receiver, system, signal), keyed `"<system>_RPB_<signal>"` in its own map (`StateManager::receiver_phase_bias_indices_`). Structurally it is unlike every other bias block described here: these are *local* states, each observed by a single station. Whether they carry across epochs is a knob. At `network.phase_datum_eq34_rpb_q_m2s` $= 0$ each $\delta_{r,s}$ is re-estimated fresh every epoch (the original Card-3 behaviour); at the non-zero default the producer folds a local time-linking prior toward the previous estimate with variance $\mathrm{var} + q\,\Delta t$, which suppresses the per-epoch bounce that an only weakly observable local state otherwise injects, anti-correlated, into every individual ambiguity float. That carry lives *outside* `StateManager` — in the network run context (the `rpb_carry` map of `app/NetworkRunContext`), read during the parallel block assembly, harvested post-solve, and erased for any (i, s) whose chain frame stepped at that epoch's election (`filter/NetworkChainDatum`). It is forced off under `network.carry_invariant`, where the loader warns that the q is inert. Either way the receiver-side wander sinks into $\delta_{r,s}$ while the time-linked integer information stays in the ambiguity carry.

Satellite clocks $c\,\Delta\delta t^i$ (network mode). Also network-producer-only, gated by `estimation.estimate_sat_clocks` — like the phase bias above, off by JSON default and inert elsewhere, but force-enabled unconditionally by the network-producer loader at the same site. The network estimates an *explicit* per-satellite clock correction $\Delta\delta t^i$ (in metres, relative to the a-priori precise/real-time product clock already removed from `obs.modeled`) so the emitted clocks stay consistent with the chosen S-system datum. It is *per satellite*, not per signal — one state per satellite, with no signal field — and the satellite *orbit* stays fixed from the product; only the clock is estimated. It lives in a distinct map (`StateManager::sat_clock_indices_`), keyed `"<sat>|SC"`. Its cross-epoch rate `sat_clock_q` is near-white, but on the network path it is *not* a Kalman $\boldsymbol{Q}(\Delta t)$ entry: the producer runs no `KalmanFilter` (Section 11), and the rate instead inflates the shared-border carry prior as $\mathrm{var} + q\,\Delta t$. The same holds for `sat_phase_bias_q` and `sat_code_bias_q`; all three are authorable either through their legacy `filter.process_noise.*` keys or through the network-scoped `network.process_noise` class table, whose per-class fallbacks equal those legacy defaults. Between full solves the emitted clock may additionally be refreshed by the split-cadence fast tier (`network.fast_clock`, default off and currently gate-failed); that tier estimates a clock *delta* in its own least-squares tick and creates no state-vector row, so it does not appear here — see Section 19.

Inertial augmentation. When a tightly-coupled IMU is active the core state grows by nine further entries — a three-axis attitude error and the accelerometer and gyroscope biases — propagated by the strapdown mechanisation rather than the random-walk process model. These are treated in full in Section 16.

Baseline differential troposphere. In baseline mode the per-station tropospheric, clock, ionosphere, and bias states are absent; an optional single differential zenith-wet-delay state $\Delta\mathrm{ZWD}_{r_{\mathrm{rov}}r_{\mathrm{ref}}}$ may be carried (`baseline.estimate_differential_tropo`, Section 5.3).

3.3 Dynamic indexing

The variable-dimension blocks are addressed via key→index maps maintained by `StateManager`. The canonical key formats are defined once in `include/filter/StateKeySchema.hpp`, each with a `make`/`parse` pair:

- iono key: `"<sys><prn>"`, e.g. `"G05"` (delimiter-free);
- ambiguity key: `"<sys><prn>_<signal>"`, e.g. `"G05_L1C"` — split on the *first* underscore, so an iono-free combination `"G05_L1C_L5Q"` retains its compound signal;
- satellite signal-bias key (code): `"<sys><prn>|<signal>"`, e.g. `"G05|C1W"` — a *pipe* delimiter and a distinct map, *not* the underscore ambiguity namespace;
- satellite phase-bias key (network): `"<sys><prn>|<signal>|PB"`, e.g. `"G05|L1C|PB"` — the trailing `|PB` keeps it disjoint from the code-bias namespace, in its own map (`sat_phase_bias_indices_`);
- receiver phase-bias key (network): `"<system>_RPB_<signal>"`, e.g. `"Galileo_RPB_L5Q"` — the `_RPB_` infix keeps it disjoint from the receiver code-bias namespace (which matches on `IFB`/`DCB`), in its own map (`receiver_phase_bias_indices_`);
- satellite clock key (network): `"<sat>|SC"`, e.g. `"G05|SC"` — per satellite, *not* per signal; the trailing `|SC` discriminator gives the clock its own map (`sat_clock_indices_`);
- receiver code-bias key: `"<system>_IFB<band>"` or `"<system>_DCB"`; the per-slot GLONASS variant appends an FCN marker, `"GLONASS_IFB<band>_k<±slot>"` (e.g. `"GLONASS_IFB1_k-4"`);
- inter-system time-offset key: the bare system name (`"Galileo"`, …; GPS is the reference and never appears);
- double-difference ambiguity key (baseline): `"DD|<cons>|<sat>|<code>|<pivot>"`.

This single source of truth eliminates the off-by-one errors that plague hand-managed dynamic state vectors and makes the BSSD/DD S-transforms (Section 5) implementable as block operations on the relevant key sets.

Two further categories in the schema header are deliberately absent from the list above, because they are not rows of $\boldsymbol{x}$: the BSSD pivot absolute-seed lookup key (`"<cons>|<band>"`), and the shared spherical-harmonic VTEC coefficient keys (`"SH|<n>|<m>|<C|S>"`, with the second shell carried on the distinct `SH2` prefix), which name columns of the network estimator's *shared border* rather than `StateManager` states. Both nonetheless follow the same `make`/`parse` discipline, and both appear in Table 9 — machine-derived from the schema header, and therefore the completeness reference against which the list above should be checked.

Ambiguity basis. By default the ambiguity block carries one per-band float ambiguity per tracked (satellite, signal) in metres. Setting `estimation.state_ambiguity_basis = "nl_wl"` switches the whole run to a narrow-lane / wide-lane parameterisation: per satellite a narrow-lane state (key `"<sat>_NL_<band>"`) and one wide-lane state per band pair (`"<sat>_WL_<a>_<b>"`, semantically $N_a - N_b$), both in *cycles* rather than metres so the wide-lane and LAMBDA arithmetic stays native to integers, with the wavelength scaling moved into the Jacobian. The two bases never coexist in a single state vector.

4 Observation equations

This section gives the full *undifferenced* observation model that the filter linearises around at each epoch. The PPP-RTK parameterisation is uncombined: per-signal pseudorange and carrier phase are consumed directly, without forming ionosphere-free or wide-lane combinations as a preprocessing step. Combinations (IF, MW, GF) are still computed for diagnostic and slip-detection purposes (Section 14; see `slip/MwSlipDetector`, `slip/GraphicSlipDetector`) but they do not enter the measurement update path. This holds whenever the uncombined parameterisation is active; the loader forces `uncombined_enabled=true` for the whole PPP-RTK family (user, network producer, network consumer) in `GnssConfig::loadProcessingSection_`; the same site forces it for the baseline double-difference modes, which process each signal independently (Section 5.3). Plain PPP mode leaves the flag at its default `false` and instead builds ionosphere-free combined observables (keys such as `C1W_C5X`) that *do* feed `MeasurementModel::buildJacobianRow` and the update; the sequel is written for the uncombined case.

4.1 Code (pseudorange) observation

For receiver r tracking satellite i on signal s, the observed pseudorange is

$$\boxed{P^i_{r,s} \;=\; \rho^i_r \;+\; c(\mathrm{d}t_r - \mathrm{d}t^i) \;+\; c\,\delta t_r^{(g(i))} \;+\; T^i_r \;+\; \mu_s\, I^i_{r,1} \;+\; B_{r,s} - B^i_s \;+\; \varepsilon^P_{r,i,s}.} \tag{4}$$

4.2 Carrier-phase observation

The corresponding carrier-phase observation, scaled to metres by λ_s, is

$$\boxed{\Phi^i_{r,s} \;=\; \rho^i_r \;+\; c(\mathrm{d}t_r - \mathrm{d}t^i) \;+\; c\,\delta t_r^{(g(i))} \;+\; T^i_r \;-\; \mu_s\, I^i_{r,1} \;+\; \lambda_s\, N^i_{r,s} \;+\; b_{r,s} - b^i_s \;+\; \varepsilon^\Phi_{r,i,s}.} \tag{5}$$

The defining structural difference between Equations (4) and (5) is the sign of the ionospheric term: $+\mu_s I$ on code, $-\mu_s I$ on phase. Three additional remarks:

1. The phase ambiguity $N^i_{r,s}$ is integer in principle but is contaminated by the receiver-plus-satellite phase biases $b_{r,s} - b^i_s$. In a *float* solution these biases are absorbed into $N^i_{r,s}$, leaving a non-integer state with no structural advantage over the code observation. Recovering integer-valued ambiguities therefore requires either external Fractional Cycle Bias (FCB) products or differencing (Section 5), which cancels common-mode bias terms.
2. The clock product $c\,\mathrm{d}t^i$ is read from the analysis-centre CLK file (post-processing) or refined from the broadcast clock by a streamed SSR correction (real-time, Section 17) and treated as deterministic in the filter; any residual product error appears on the observation and is absorbed by the receiver clock state.

3. The system time offset $c\,\delta t_r^{(g(i))}$ is zero by construction when $g(i)$ is the reference constellation (default GPS). Together with the receiver clock $c\,\mathrm{d}t_r$ this means a multi-constellation run carries $|\mathcal{G}|$ clock-like states in total.

4.3 Geometric range and linearisation

The geometric range $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|_2$ enters Equations (4) and (5) non-linearly through the receiver position $\boldsymbol{r}_r$. The filter linearises about the current state estimate $\hat{\boldsymbol{r}}_r$:

$$\rho_r^i \;=\; \hat{\rho}_r^i \;-\; (\boldsymbol{e}_{\hat{r}}^i)^{\mathrm{T}}(\boldsymbol{r}_r - \hat{\boldsymbol{r}}_r) \;+\; \mathcal{O}\big(\|\boldsymbol{r}_r - \hat{\boldsymbol{r}}_r\|^2\big)\,, \tag{6}$$

with the unit line-of-sight

$$\boldsymbol{e}_{\hat{r}}^i \;:=\; \frac{\boldsymbol{r}^i - \hat{\boldsymbol{r}}_r}{\|\boldsymbol{r}^i - \hat{\boldsymbol{r}}_r\|_2}. \tag{7}$$

The Jacobian row $\boldsymbol{H}_{(i,s)}$ contributing to the design matrix at the position columns is therefore $-(\boldsymbol{e}_{\hat{r}}^i)^{\mathrm{T}}$. For sub-metre residual targets this single linearisation per epoch is sufficient; the engine optionally relinearises within the epoch (`processing.relinearize_geometry`, which defaults to off in code and is enabled in the PPP, PPP-RTK, baseline, and real-time configs but left off in the six network-producer configs) to handle long initial position offsets.

The satellite position $\boldsymbol{r}^i$ is itself a function of the transmission time. Rather than iterate the emission time against the geometric range — which depends on $\boldsymbol{r}_r$ — `Geometry::computeEmissionState` solves it in a single step from the *pseudorange*: the time of flight is $\tau = P/c$, subtracted (with the satellite clock) from the receiver epoch. Because the receiver clock bias $\mathrm{d}t_r$ enters the receiver epoch as $+\mathrm{d}t_r$ and the pseudorange as $+c\,\mathrm{d}t_r$, it cancels exactly, leaving a residual $(I+T+\varepsilon)/c \approx 30$–$60$ ns of satellite displacement — sub-millimetre, below any PPP threshold. This deliberately avoids a range-based fixed point: iterating against ρ_r^i would re-introduce an uncancelled $\mathrm{d}t_r$ and drive a positive-feedback loop through the filter's receiver-clock state. The only second pass (the `model_corrections.refine_emission_time` block in `Geometry::computeSatelliteGeometry`) re-runs `computeEmissionState` on a cleaned pseudorange, having first subtracted the slant hydrostatic delay and the IF-combined ATX projection from the seed — a pseudorange-cleanliness refinement, not a range-versus-position iteration.

4.4 Stochastic model

Observation noise covariance is structured per epoch as a block diagonal matrix with entries

$$\sigma_{r,i,s}^2 \;=\; \sigma_{0,\mathrm{type}(s)}^2 \;\cdot\; w(\theta_r^i)^{-1} \;\cdot\; \xi(\mathrm{C/N_0 \text{ at } } (r,i,s)), \tag{8}$$

combining a per-type baseline ($\sigma_{0,\mathrm{code}}$, $\sigma_{0,\mathrm{phase}}$ from `filter.measurement_noise`), an elevation-dependent down-weighting factor $w(\theta_r^i)^{-1} = (\sin\theta_r^i)^{-2p_{\mathrm{type}}}$ whose exponent is *continuous* and *type-dependent* — code and Doppler rows take `filter.code_elevation_power`, carrier-phase rows `filter.phase_elevation_power` — with the weighting strategy itself selected by `filter.stochastic_model` (derived, together with the low-elevation guard that leaves σ_0^2 unamplified, in Equation (157)), and an optional carrier-to-noise ratio scaling ξ parameterised by `snr_model_params`. Cross-correlations between observations are assumed zero in the undifferenced parameterisation; differencing introduces structured correlations handled in Section 5.

5 Differencing transformations

PPP-RTK and baseline modes do not consume the undifferenced observations of Equations (4) and (5) directly. Each forms a linear combination across satellites (BSSD), receivers (BRSD), or both (DD), absorbing common-mode terms into the cancellation and recovering integer-friendly residual structure. This section gives both the algebraic cancellations and the matrix-form S-transforms that map undifferenced parameters to differenced ones.

5.1 Between-satellite single difference (BSSD)

For each constellation g, choose a pivot satellite p_g (elected by `PivotManager::selectPivotCandidates` under the "sticky pivot" retention policy; the selection ordering is detailed in the *Pivot selection* note below). For any non-pivot satellite $i \in \mathcal{S}_g \setminus \{p_g\}$ and signal s, define

$$\begin{aligned} \Delta P_{r,s}^{i,p_g} &:= P_{r,s}^{i} - P_{r,s}^{p_g}, \\ \Delta \Phi_{r,s}^{i,p_g} &:= \Phi_{r,s}^{i} - \Phi_{r,s}^{p_g}. \end{aligned} \tag{9}$$

Substituting Equations (4) and (5) and cancelling the receiver-side common terms ($c\,\mathrm{d}t_r$, $c\,\delta t_r^{(g)}$, $B_{r,s}$, $b_{r,s}$) yields

$$\boxed{\begin{aligned} \Delta P_{r,s}^{i,p_g} &= \Delta\rho_r^{i,p_g} - c\,\Delta\mathrm{d}t^{i,p_g} + \Delta T_r^{i,p_g} + \mu_s\,\Delta I_{r,1}^{i,p_g} - \Delta B_s^{i,p_g} + \Delta\varepsilon^{P}, \qquad (10) \\ \Delta \Phi_{r,s}^{i,p_g} &= \Delta\rho_r^{i,p_g} - c\,\Delta\mathrm{d}t^{i,p_g} + \Delta T_r^{i,p_g} - \mu_s\,\Delta I_{r,1}^{i,p_g} + \lambda_s\,\Delta N_{r,s}^{i,p_g} - \Delta b_s^{i,p_g} + \Delta\varepsilon^{\Phi}. \qquad (11) \end{aligned}}$$

The cancellations are exact only when both observations enter the filter with the same modelling: same tropospheric mapping (which is why ΔT is small but not zero), same antenna PCV correction, and so on.

The receiver-bias cancellation of $B_{r,s}$ in Equation (10) holds for CDMA constellations, whose group delay is a single per-signal receiver term. GLONASS is the exception: its FDMA inter-frequency biases are per-FCN-slot states, so two satellites on different slots carry different receiver delays and the term does *not* cancel between them — a residual $B_{r,s}^{(k(i))} - B_{r,s}^{(k(p_g))}$ survives the single difference. Accordingly `BssdTransform` zeroes the receiver-DCB columns after differencing *except* the per-slot GLONASS IFB states (guarded by `StateKeys::isGlonassPerSlotIfbKey`), and `PivotManager::buildEpochMaps` drops GLONASS from the BSSD candidate maps altogether — not merely from pivot election — so with the flag off the constellation contributes no single-differenced row at all. GLONASS BSSD is default-OFF (`estimation.between_satellite_sd.enable_glonass`); when enabled, Equation (10) acquires the surviving inter-slot IFB difference as an additional estimated column.

Pivot selection. Election is not by elevation alone. The primary key is a *hard* lexicographic tier on phase-signal coverage: a full-coverage candidate outranks any partial-coverage candidate regardless of its score. Only within a coverage tier does the additive score $-\min\bigl(1, \sqrt{\sigma^2_{\hat{N},\max}}\bigr) + \mathrm{el}/90 - \frac{1}{2}\mathbb{1}[\mathrm{el} < 20^\circ]$ apply, in which float-ambiguity convergence $\sqrt{\sigma^2_{\hat{N},\max}}$ and elevation enter as co-equal terms (with a hard low-elevation guardrail below 20°). Layered on top is an ephemeris-freshness bar: a candidate whose broadcast $|\mathrm{obs} - \mathrm{toe}|$ exceeds its constellation's freshest qualified peer by more than `kPivotToeSpreadMaxS` (1200 s) is barred from *pivot* election (though still usable as a non-pivot), which prevents a stale-eph pivot from stepping every SD row it anchors. Candidates below

`estimation.between_satellite_sd.pivot_min_elevation` are excluded outright. The baseline DD path scores differently: `DdPivotManager` ranks on avg_snr + 0.01 · num_sigs and does not use elevation.

Pivot transitions. A handover fires on two kinds of trigger. The visibility-driven one is simple — when the incumbent pivot leaves the visible set, p_g is re-elected from the remaining satellites — but the dominant one in practice is *event-driven*: the post-fit residual analyser calls `PivotManager::requestForcedHandover` when the pivot's *observation* slipped while its filter *state* is still intact (so the resting `isPivotDisturbed` check, which tests only that the pivot ambiguity sits at exactly $(0, 0)$, would not catch it). The forced request carries an exclusion set so the handover cannot bounce onto another offset satellite, a `retired_pivots_this_epoch_` backstop that terminates cascades, a same-epoch re-entry gate (`hasPendingForcedHandover`), and a re-inflation set for states that absorbed a coherent pivot-side code bias (the information-conserving transform below cannot evict that offset on its own). The baseline DD path exposes the symmetric `DdPivotManager::requestForcedRerank`. This event response is load-bearing precisely because the pivot's states are frozen at zero variance (below): a stiff, time-linked zero-q quantity requires a wired jump response.

On election the float ambiguities $\{\Delta N_{r,s}^{i,p_g^{\mathrm{old}}}\}_{i,s}$ must be re-expressed relative to p_g^{new}. Where the new pivot already carries an established ambiguity in the group, the engine applies the linear S-transformation $\boldsymbol{x}^{\mathrm{new}} = \boldsymbol{S}\,\boldsymbol{x}^{\mathrm{old}}$ with $\boldsymbol{S}$ chosen so that $\Delta N_{r,s}^{p_g^{\mathrm{new}},p_g^{\mathrm{old}}}$ becomes the new reference (zero), preserving the integer information accumulated since the last fix; the covariance transforms as $\boldsymbol{P}^{\mathrm{new}} = \boldsymbol{S}\,\boldsymbol{P}^{\mathrm{old}}\,\boldsymbol{S}^{\mathrm{T}}$. When it cannot — the new pivot has no existing ambiguity in the (cons, code) group (`DdPivotManager`, logged as "reset-and-reseed"), or a disturbed pivot has no healthy replacement in a single-satellite constellation (`PivotManager::handleDisturbedPivot` re-zeroes $\boldsymbol{x}$ and the corresponding rows/columns of $\boldsymbol{P}$ in place) — the transform is skipped and the stale rows are discarded and re-seeded. In the uncombined PPP-RTK parameterisation the same $\boldsymbol{S}\,\boldsymbol{P}\,\boldsymbol{S}^{\mathrm{T}}$ re-reference is applied not only to the ambiguity block but to *three* state groups — `reReferenceAmbiguities`, `reReferenceIonosphere`, and `reReferenceSatBiases` — which is why `BssdTransform` zeroes the pivot's ionosphere and sat-bias $\boldsymbol{H}$-columns after differencing, and why $\Delta I_{r,1}^{i,p_g}$ in Equations (10) and (11) is itself a pivot-referenced SD quantity, not an absolute slant delay. The S-basis constraint is realised operationally by freezing the pivot's ambiguity, ionosphere, and sat-bias covariances to exactly zero (and suppressing their process noise) — the very condition `isPivotDisturbed` tests against.

`PivotManager::getCleanHandovers` classifies each transition as lossless-invertible (*clean*) or discontinuous (disturbed/forced/re-seeded), so the RTS smoother can inverse-reference across a clean handover rather than force a segment boundary; the losslessness claim above holds only for the clean subset. These are two distinct code paths sharing the algebra above: in PPP-RTK the pivot re-election and the "sticky-pivot" retention policy live in `filter/PivotManager`, whereas the baseline DD path performs the analogous re-reference through the ambiguity-rename S-transform in `filter/DdPivotManager`, governed by the `baseline.enable_pivot_handover` flag.

5.2 Between-receiver single difference (BRSD)

In baseline modes there is at least one rover r_{rov} and exactly one fixed-coordinate reference r_{ref}. The BRSD operator forms

$$\nabla P^i_{r_{\mathrm{rov}} r_{\mathrm{ref}},s} \;:=\; P^i_{r_{\mathrm{rov}},s} - P^i_{r_{\mathrm{ref}},s}, \tag{12}$$

and analogously for $\nabla\Phi$. The cancellations produced by $\nabla\cdot$ are dual to those produced by $\Delta\cdot$: BRSD cancels *satellite*-side common terms ($c\,\mathrm{d}t^i$, B^i_s, b^i_s) but preserves receiver clock and biases of

both stations. No state-space estimation of satellite biases is required when ∇ is applied — they disappear before reaching the filter.

5.3 Double difference (DD)

The composition $\nabla\Delta\cdot = \Delta(\nabla\,\cdot\,)$ is order-independent up to noise correlation: $\Delta(\nabla\,\cdot\,) = \nabla(\Delta\,\cdot\,)$. Applying it to the carrier-phase model and collecting common-mode cancellations:

$$\nabla\Delta\Phi^{i,p_g}_{r_{\mathrm{rov}}r_{\mathrm{ref}},s} \;=\; \nabla\Delta\rho^{i,p_g}_{r_{\mathrm{rov}}r_{\mathrm{ref}}} + \nabla\Delta T^{i,p_g}_{r_{\mathrm{rov}}r_{\mathrm{ref}}} - \mu_s\,\nabla\Delta I^{i,p_g}_{r_{\mathrm{rov}}r_{\mathrm{ref}},1} + \lambda_s\,\nabla\Delta N^{i,p_g}_{r_{\mathrm{rov}}r_{\mathrm{ref}},s} + \nabla\Delta\varepsilon^{\Phi}. \tag{13}$$

What survives in Equation (13) is precisely the geometry term (linear in the rover position perturbation), the differential atmospheric delays, and the integer DD ambiguity — the structural form that makes carrier-phase RTK possible.

Short vs. long baselines. On short baselines ($\lesssim 10\,\mathrm{km}$) the differential atmospheric terms in Equation (13) are negligible at the millimetre level and need not be modelled. As baseline length grows, the differential ionosphere $\nabla\Delta I$ scales linearly with separation and saturates at typically 1 m for separations of a few hundred kilometres, swamping the carrier wavelength. Two mitigations are available in `GnssProcessor`:

- `baseline.use_iono_free` forms the dual-frequency ionosphere-free combination at the DD level, eliminating the first-order ionospheric term at the cost of a non-integer ambiguity (AR is automatically excluded for IF rows) and $\sim 3\times$ amplified code noise;
- `baseline.estimate_differential_tropo` adds a single $\Delta\mathrm{ZWD}_{r_{\mathrm{rov}}r_{\mathrm{ref}}}$ state with slow random-walk dynamics, capturing the residual differential wet delay that survives the BRSD on baselines with significant altitude difference.

5.4 S-transform structure

Both BSSD and DD admit a matrix-form representation as $\boldsymbol{x}_{\mathrm{diff}} = \boldsymbol{S}\,\boldsymbol{x}_{\mathrm{undiff}}$ with $\boldsymbol{S}$ acting as a block design transform. For BSSD restricted to the ambiguity block of one constellation g on one signal s, with $|\mathcal{S}_g|$ tracked sats, $\boldsymbol{S}$ is a $(|\mathcal{S}_g|-1)\times|\mathcal{S}_g|$ matrix of the form

$$\boldsymbol{S}_{\mathrm{BSSD}} \;=\; \begin{bmatrix} 1 & 0 & \cdots & 0 & -1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & -1 & 0 & \cdots & 0 \\ \vdots & & \ddots & & \vdots & & & \\ 0 & 0 & \cdots & 0 & -1 & 0 & \cdots & 1 \end{bmatrix}, \tag{14}$$

where the column of -1's sits at the index of the pivot p_g. The DD operator is the *composition* of the two single-difference operators applied in sequence, $\boldsymbol{S}_{\mathrm{DD}} = \boldsymbol{S}_{\mathrm{BSSD}}\,\boldsymbol{S}_{\mathrm{BRSD}}$, realised in code as `BrsdTransform::apply` followed by `BssdTransform::differenceBrsd` (`include/measurement/BssdTransform.hpp`). Only on a fully populated satellite $\times$ receiver observation grid does this composition coincide with the Kronecker product $\mathbf{D}_{\mathrm{sat}} \otimes \mathbf{D}_{\mathrm{rx}}$ of the between-satellite and between-receiver differencing matrices acting on the vectorised observations — a useful mental model, but not what the non-rectangular real-data case computes. The transformed covariance is $\boldsymbol{P}_{\mathrm{diff}} = \boldsymbol{S}\,\boldsymbol{P}_{\mathrm{undiff}}\,\boldsymbol{S}^{\mathrm{T}}$, a generally dense matrix even when $\boldsymbol{P}_{\mathrm{undiff}}$ is diagonal (because each differenced row borrows from the same pivot column). This dense correlation structure is the algebraic reason DD-based filters cannot be naively implemented as independent per-pair scalar updates — the correctly correlated joint update is what unlocks integer ambiguity recovery via the LAMBDA decorrelation. The general theory and a thorough catalogue of admissible S-transforms is given in Teunissen [25] and in the rank-defect treatment of Jonge [10].

Implementation notes. Two facts about how `GnssProcessor` realises the transforms above are worth stating explicitly. First, the chosen S-system is never materialised as an explicit minimum-constraint $\boldsymbol{S}$-basis matrix; it is realised *implicitly* through the pivot-differencing operator (`measurement/BssdTransform`), a faithful, S-basis-invariant realisation of the uncombined parameterisation of Odijk [17]. The three regimes — undifferenced, BSSD, and DD — are dispatched polymorphically by a `DifferencingMode` strategy in `measurement/MeasurementBuilder`, not by inline config branches. Second, integer recovery on a DD row requires that all four contributing observations share the same RINEX signal code. The row is marked `integer_capable` only when all three conjuncts hold: `!brsd_aliased` on *both* the i_i and pivot BRSD entries (rover and reference agree on each, i.e. no cross-attribute alias) *and* the rover uses the same code on i_i as on the pivot ($\texttt{rov_code}_i = \texttt{rov_code}_{\text{piv}}$). The third conjunct is independent of the `brsd_aliased` flags: a row can be un-aliased on both entries yet still carry a non-integer DCB/IFB residual if the rover mixes codes across i_i and the pivot. Rows failing any conjunct are carried as float and excluded from ambiguity resolution.

Part II
Reference systems and measurement models

6 Coordinate Frames and Time Systems

Every model in this reference — the tropospheric and ionospheric mappings of Section 8, the solid-Earth, ocean, and pole tides of Section 9, the antenna, attitude, and phase-wind-up terms of Section 10, and the orbit evaluation of Section 7 — presumes a fixed geodetic-frame and time-scale substrate. This document specifies that substrate: the WGS84 ellipsoid and the three coordinate representations (Cartesian ECEF, geodetic, and local ENU) the engine moves between; the family of GNSS time scales together with the leap- second machinery that ties them to UTC; and the Earth-orientation chain that rotates the celestial frame — in which the Sun, Moon, and the satellite ephemerides are most naturally expressed — into the terrestrial frame in which the receiver sits. None of the conversions here is an estimator state. The one frame quantity that *is* estimated — the inter-system time offset (ISB) that absorbs the unmodelled bias between constellation time scales — belongs to Section 3; this document covers only the *deterministic* scale conversions that the ISB state is defined relative to. Code references are to `src/core/math/Coordinates.cpp`, `src/core/math/TimeUtils.cpp`, `src/core/model/Planetary.cpp`, `src/core/products/LiveEphemerisStore.cpp`, `src/core/products/ProductParser.cpp`, and `src/core/rinex/RinexParser.cpp`.

6.1 The WGS84 ellipsoid: ECEF, geodetic, and local frames

The terrestrial reference frame is the WGS84 Earth-Centred, Earth-Fixed (ECEF) Cartesian system: origin at the geocentre, z along the conventional pole, x toward the IERS reference meridian, y completing a right-handed triad. The defining ellipsoid has semi-major axis a and flattening f, from which the first eccentricity squared follows (`Coordinates::ecefToLlh` and `Coordinates::llhToEcef` each derive it from f):

$$e^2 \;=\; f\,(2 - f). \tag{15}$$

The radius of curvature in the prime vertical at geodetic latitude φ is

$$N(\varphi) \;=\; \frac{a}{\sqrt{1 - e^2\,\sin^2\varphi}}, \tag{16}$$

which scales the entire forward map from geodetic coordinates (φ, λ, h) to ECEF (`Coordinates::llhToEcef`):

$$\boxed{\begin{aligned} x &= \big(N + h\big)\cos\varphi\,\cos\lambda, \\ y &= \big(N + h\big)\cos\varphi\,\sin\lambda, \\ z &= \big((1 - e^2)\,N + h\big)\sin\varphi. \end{aligned}} \tag{17}$$

The factor $(1 - e^2)N$ on the z component is the source of all ellipsoidal subtlety: a sphere would carry N unmodified on every axis, and the $(1 - e^2)$ correction is precisely the contraction that distinguishes the geodetic normal (perpendicular to the ellipsoid surface) from the geocentric radius vector.

Inverse map (Bowring fixed point). The inverse problem — recovering (φ, λ, h) from (x, y, z) — has no closed form for the latitude because N in (17) itself depends on φ. The longitude is immediate, $\lambda = \operatorname{atan2}(y, x)$. For the latitude the engine runs a Bowring fixed-point iteration on the equatorial projection $p = \sqrt{x^2 + y^2}$ (`Coordinates::ecefToLlh`). The seed neglects height,

$$\varphi^{(0)} = \operatorname{atan2}\big(z,\; p\,(1 - e^2)\big), \tag{18}$$

and each pass recomputes N and the height and re-evaluates the latitude with the height feedback folded back into the eccentricity term:

$$h^{(k)} = \frac{p}{\cos\varphi^{(k)}} - N^{(k)}, \qquad \varphi^{(k+1)} = \operatorname{atan2}\left(z,\; p\left[1 - e^2\,\frac{N^{(k)}}{N^{(k)} + h^{(k)}}\right]\right). \tag{19}$$

The iteration is capped at ten passes and exits early once $|\varphi^{(k+1)} - \varphi^{(k)}| < 10^{-13}$; for terrestrial heights it converges to machine precision in two or three steps.

Pole branch. When $p < 10^{-12}\,\mathrm{m}$ the point lies on (or within a picometre of) the z axis and both the longitude and the $p/\cos\varphi$ height formula degenerate. The code takes an explicit branch (`Coordinates::ecefToLlh`, the $p < 10^{-12}$ guard): the longitude retains the $\operatorname{atan2}(y, x)$ value computed before the guard — formally undefined on the axis, and immaterial there — while the branch fixes $\varphi = \pm\pi/2$ by the sign of z and reads the height directly off the polar semi-minor axis,

$$h = |z| - N\,(1 - e^2) = |z| - a\sqrt{1 - e^2} \quad (\text{at the pole},\; \sin^2\varphi = 1), \tag{20}$$

since $N(1 - e^2)$ evaluated at the pole is exactly the polar radius $a\sqrt{1 - e^2}$. This guard keeps the inverse map total over all of $\mathbb{R}^3$, which matters because the planetary and attitude code occasionally evaluates geometry for satellite positions that transit near the polar axis.

Local East–North–Up frame. Many of the downstream models — the tropospheric mapping function, the elevation mask, the antenna phase-centre projection — are most natural in the

topocentric ENU frame anchored at the receiver's geodetic (φ, λ). The columns of the ENU→ECEF rotation are the three local unit vectors expressed in ECEF (`Coordinates::enuToEcefMatrix`):

$$\boxed{\boldsymbol{R}_{\mathrm{ENU}}(\varphi,\lambda) = \begin{bmatrix} \hat{\boldsymbol{e}}_{\mathrm{E}} & \hat{\boldsymbol{e}}_{\mathrm{N}} & \hat{\boldsymbol{e}}_{\mathrm{U}} \end{bmatrix} = \begin{bmatrix} -\sin\lambda & -\sin\varphi\cos\lambda & \cos\varphi\cos\lambda \\ \cos\lambda & -\sin\varphi\sin\lambda & \cos\varphi\sin\lambda \\ 0 & \cos\varphi & \sin\varphi \end{bmatrix}.} \tag{21}$$

The Up column is the outward geodetic normal — identical to the direction along which h is measured in (17) — East is its horizontal cross-product with the pole, and North completes the right-handed triad. Because the columns are orthonormal, $\boldsymbol{R}_{\mathrm{ENU}}$ is a proper rotation and its inverse is its transpose: an ECEF displacement is resolved into local components by $\boldsymbol{v}_{\mathrm{ENU}} = \boldsymbol{R}_{\mathrm{ENU}}^{\mathrm{T}}\,\boldsymbol{v}_{\mathrm{ECEF}}$ (`Coordinates::ecefToEnu`, with `Coordinates::enuToEcef` the forward direction), avoiding any explicit matrix inversion.

Elevation and azimuth. With the receiver-to-satellite line-of-sight resolved into the local frame, $\boldsymbol{e}_{\mathrm{ENU}} = (e_{\mathrm{E}}, e_{\mathrm{N}}, e_{\mathrm{U}})^{\mathrm{T}} = \boldsymbol{R}_{\mathrm{ENU}}^{\mathrm{T}}\boldsymbol{e}$, the elevation and azimuth that gate the measurement (the elevation mask of Section 4.3 and the mapping-function argument of Section 8) follow directly:

$$\mathrm{el} = \arcsin\!\big(e_{\mathrm{U}}\big), \qquad \mathrm{az} = \operatorname{atan2}\!\big(e_{\mathrm{E}},\, e_{\mathrm{N}}\big), \tag{22}$$

azimuth measured clockwise from North. Equations (21)–(22) are the entire geometric bridge between the ECEF state vector and every elevation-dependent weighting and antenna model in the reference.

6.2 Station reference points: marker, ARP, and phase centre

Nothing above fixes which *point* of a station a coordinate refers to, and three distinct points are in play. The *marker* (monument) is the surveyed benchmark, and is what an IGS SINEX/SSC solution, an NGS Station Information File, and the RINEX `APPROX POSITION XYZ` header record all refer to. The *antenna reference point* (ARP) sits where the antenna meets its mount and is the origin against which the ANTEX phase-centre offsets of Section 10 are tabulated. The *antenna phase centre* is where the measurement is physically realised. The estimator's position state is the ARP — for the simple reason that the receiver PCO/PCV it applies are ARP-referenced — so the ARP→phase-centre leg is closed by the antenna model and only the marker→ARP leg belongs to this document. Confusing the two ends of that leg puts a constant bias, equal to the eccentricity, straight into the vertical.

The leg itself is the RINEX `ANTENNA:DELTA H/E/N` eccentricity $\boldsymbol{e}_{\mathrm{HEN}} = (\delta H,\, \delta E,\, \delta N)^{\mathrm{T}}$, defined as the offset from the marker to the ARP in the topocentric frame *at the marker*. Note the ordering trap: RINEX writes $(H, E, N) = (\mathrm{Up}, \mathrm{East}, \mathrm{North})$, which is not the (E, N, U) order the rotation of (21) consumes, so the triple must be permuted before it is rotated (`Coordinates::markerToArp`, the single owner of this convention):

$$\boxed{\boldsymbol{r}_{\mathrm{ARP}} \;=\; \boldsymbol{r}_{\mathrm{mkr}} \;+\; \boldsymbol{R}_{\mathrm{ENU}}\big(\varphi_{\mathrm{mkr}},\, \lambda_{\mathrm{mkr}}\big) \begin{bmatrix} \delta E \\ \delta N \\ \delta H \end{bmatrix},} \tag{23}$$

with $(\varphi_{\mathrm{mkr}}, \lambda_{\mathrm{mkr}})$ recovered from $\boldsymbol{r}_{\mathrm{mkr}}$ by the inverse map of (19). Write $\boldsymbol{e}'_{\mathrm{HEN}}$ for the permuted $(\delta E, \delta N, \delta H)^{\mathrm{T}}$ that the rotation acts on.

The inverse is not a sign flip. Reducing an ARP-referenced estimate back to the monument — to compare it against, or publish it in, a surveyed coordinate — cannot simply negate (23), because the local frame belongs at the marker and the marker is precisely the unknown. Evaluating $\boldsymbol{R}_{\mathrm{ENU}}$ at the ARP instead leaves a closure error of order $\|\boldsymbol{e}_{\mathrm{HEN}}\|^2/R_\oplus$. The engine therefore takes one fixed-point step (`Coordinates::arpToMarker`):

$$\boldsymbol{r}^{(0)} = \boldsymbol{r}_{\mathrm{ARP}} - \boldsymbol{R}_{\mathrm{ENU}}\big(\mathrm{llh}(\boldsymbol{r}_{\mathrm{ARP}})\big)\,\boldsymbol{e}'_{\mathrm{HEN}}, \qquad \boldsymbol{r}_{\mathrm{mkr}} = \boldsymbol{r}_{\mathrm{ARP}} - \boldsymbol{R}_{\mathrm{ENU}}\big(\mathrm{llh}(\boldsymbol{r}^{(0)})\big)\,\boldsymbol{e}'_{\mathrm{HEN}}, \tag{24}$$

which closes the round trip with (23) to well below the millimetre level this reference grades at.

Where the reduction is applied — and where it is not. Every position derived from a RINEX *header* is reduced: the post-processing filter seed (`Application::initializePosition` and the `PppModel` state initialiser), the real-time seed (`RealTimeEngine::initialize`), the baseline rover a priori, and the closed-loop consumer truth all take the marker through (23) — some calling `Coordinates::markerToArp` directly, others through the accessor `RinexParser::getArpPositionXyz` that pairs the two header fields. A coordinate supplied *directly in the configuration* is not reduced: it is taken at face value, on the stated policy that the user owns its reference point. So the baseline *reference* coordinate is used as given while the rover a priori read from its header is reduced — an asymmetry inside one mode — and, more consequentially, the network producer's station coordinates (`StationConfig::position_ecef`, optionally SIF-resolved under `network.use_sif_coordinates`) are used as given while being held *fixed*, their three position columns dropped from the normal equations. A monument-referenced coordinate there is therefore not absorbed by a position state; it survives into the residuals and hence into the estimated clocks, biases, and ionosphere. A zero or absent eccentricity makes (23) an exact, bitwise no-op, which is what keeps header-less RTCM/SBF streams and `0/0/0` stations untouched; the ordering, sign, no-op, and round-trip properties are pinned by the `AntennaEccentricityTest` suite.

6.3 GNSS time scales and the GPS reference epoch

Time inside the engine is carried as a `GnssTime` $\{\mathrm{week}, \mathrm{tow}\}$ pair — an integer GPS week and a floating-point time-of-week in seconds — and is expanded to a single continuous count $T = \mathrm{week} \times 604800 + \mathrm{tow}$ wherever a monotone scalar is needed (e.g. ERP interpolation, Section 6.9). A week boundary is 604 800 s and a day 86 400 s (`src/core/math/TimeUtils.cpp`, header constants `SECONDS_IN_WEEK`, `SECONDS_IN_DAY`). The common origin is the GPS epoch, 1980-01-06 00:00:00 UTC, whose Julian Date is fixed in the code:

$$\mathrm{JD}_{\mathrm{GPS}} = 2444244.5 \qquad (\texttt{src/core/math/TimeUtils.cpp}, \texttt{GPS_EPOCH_JD}). \tag{25}$$

GPS time is a continuous atomic scale — it never inserts a leap second — so it sits a fixed integer offset below International Atomic Time and a fixed fractional offset below Terrestrial Time (`src/core/math/TimeUtils.cpp`, header constants `TAI_GPS_OFFSET`, `TT_GPS_OFFSET`):

$$\boxed{\mathrm{TAI} - \mathrm{GPS} = 19\ \mathrm{s}, \qquad \mathrm{TT} - \mathrm{GPS} = 51.184\ \mathrm{s} = (32.184 + 19)\ \mathrm{s}.} \tag{26}$$

The 32.184 s in the TT relation is the constant TT−TAI offset; together these let any GPS time be promoted to the TT scale on which precession and nutation are evaluated. Terrestrial Time relative to the J2000.0 epoch is measured in Julian centuries,

$$t = \frac{\mathrm{JD}_{\mathrm{TT}} - 2451545.0}{36525.0}, \tag{27}$$

the dimensionless argument of the astronomical series of Sections 6.7 and 6.10 (`TimeUtils::jdToJ2000Centuries`, constants `J2000_JD`, `DAYS_PER_JULIAN_CENTURY`).

Gregorian↔Julian Date. The calendar conversions follow the Meeus algorithm [14]. The forward map, with the month shifted so January and February count as months 13 and 14 of the prior year and a Gregorian-reform correction $b = 2 - A + \lfloor A/4 \rfloor$ for $A = \lfloor y/100 \rfloor$, is (`TimeUtils::gregorianToJulianDay`):

$$\mathrm{JD} = \lfloor 365.25\,(y + 4716) \rfloor + \lfloor 30.6001\,(m + 1) \rfloor + d + b - 1524.5 + \frac{(\text{time of day})}{86400}, \tag{28}$$

and the inverse (`TimeUtils::julianDayToGregorian`) reverses it through the integer auxiliaries $z = \lfloor \mathrm{JD} + 0.5 \rfloor$ and the Gregorian-reform branch at $z \geq 2299161$. The crucial distinction the code maintains throughout is between a calendar date that is already on the GPS scale (no leap correction: `gregorianGpsToGpsTime`) and one that is on UTC and must have the epoch's leap seconds added back to land on GPS time (`TimeUtils::gregorianToGpsTime`); conflating the two is the classic source of whole-second timing errors.

6.4 Leap seconds and per-system dispatch

UTC is tied to GPS by the historical table of leap-second insertions. The engine stores the table as $\big(\mathrm{JD}_k,\ \Delta_{\mathrm{GU}}(k)\big)$ pairs — the GPS−UTC offset that takes effect at each insertion boundary, the first row being the GPS epoch itself with offset zero (`src/core/math/TimeUtils.cpp`, `LEAP_TABLE`). A query resolves the offset by a reverse scan that returns the offset of the latest boundary not after the epoch:

$$\Delta_{\mathrm{GU}}(\mathrm{JD}) = \Delta_{\mathrm{GU}}(k^{\star}), \qquad k^{\star} = \max\{\, k : \mathrm{JD}_k \leq \mathrm{JD} \,\}, \tag{29}$$

falling back to zero before the epoch (`TimeUtils::leapSecondsGpsUtc`). TAI−UTC is then simply $\Delta_{\mathrm{GU}} + 19$ s from (26) (`TimeUtils::leapSecondsTaiUtc`). The epoch-dependence is essential: the same UTC calendar instant maps to different GPS times in 2005 and 2010 because a leap second was inserted between them, and only a boundary-aware lookup recovers the correct offset.

The wall-clock leap constant and its runtime override. The boundary-aware table of (29) is the authority for every *epoch-referenced* conversion — a dated calendar timestamp, GLONASS day arithmetic, UTC formatting — because each of those carries the epoch at which the correct offset is known. The one conversion that does not is the promotion of the *system wall clock* to GPS time (`TimeUtils::nowContinuousGpsSeconds`, `nowGpsTime`), used only on the real-time decode and latency-stamping paths: POSIX `system_clock` is UTC and carries no epoch of its own, so the offset cannot be looked up and must instead be supplied as a fixed count. That count is the compiled constant `TimeUtils::LEAP_SECONDS` = 18 s (valid from 2017-01-01), read through an atomic accessor (`currentLeapSeconds`) so that a future insertion can be applied to a live binary without a rebuild: `setLeapSecondsOverride`, driven by the config key `realtime.leap_seconds` (`src/core/config/GnssConfigLoad.cpp`), replaces it at real-time startup. The override deliberately touches *only* the wall-clock path; the epoch-dependent (29) lookups are left alone, since they are correct at every historical epoch outside the insertion instants themselves[1] and an override there would corrupt rather than update them.

Per-constellation Gregorian dispatch. The four supported time systems differ only in how a RINEX/broadcast calendar timestamp is referred to the GPS scale, dispatched by a system tag (`TimeUtils::GnssTimeFromDate`):

[1]The comparison in `TimeUtils::leapSecondsGpsUtc` is made on the GPS scale against boundary Julian Dates tabulated at the *UTC* insertion instant, so the offset advances Δ_{GU} seconds early — a known, deferred discrepancy confined to a Δ_{GU}-second window at each boundary (18 s at the most recent insertion, 2017-01-01).

- **GPS / Galileo** — already on the GPS scale; the timestamp is converted with no leap correction (GST is steered to GPS time and shares the same week/second framing to within the few nanoseconds the ISB state of Section 3 absorbs).
- **BeiDou** — the timestamp is on BDT, which runs 14 s behind GPS (Section 6.6); the code converts on the GPS-scale assumption and then *adds* 14 s, carrying any week roll-over (`TimeUtils::GnssTimeFromDate`, BeiDou branch).
- **GLONASS** — the RINEX timestamp is UTC; the full leap correction of (29) is applied to bring it onto GPS time (`TimeUtils::GnssTimeFromDate`, GLONASS branch).

GPS and Galileo therefore carry no leap correction, BeiDou a fixed 14 s, and GLONASS the full epoch-dependent leap count — a dispatch that exactly mirrors the constellations' defining time references. There is a fourth branch: an unrecognised time-system tag falls through to the GPS-scale conversion with only a trace-level note, and a calendar date that fails validation returns the sentinel $\{0, 0\}$ rather than a fabricated epoch (`TimeUtils::GnssTimeFromDate`). The fallback is silent at default verbosity, so an unexpected tag mis-times by the full leap count rather than failing loudly.

6.5 GLONASS time-of-day to full GPS week

GLONASS broadcast and SSR records frequently carry only a time-of-day — seconds since the start of the GLONASS day — with no week and no absolute date. GLONASS time runs on UTC(SU) advanced by three hours (Moscow), so its day boundary is three hours ahead of the UTC midnight. Reconstructing a full GPS week from a bare time-of-day therefore needs an *anchor*: an approximate GPS-seconds estimate of the true epoch (typically the current obs epoch), against which the day and the ± 12 h wrap are resolved (`TimeUtils::glonassTodToGpsTime`, after RTKLIB's `adjday_glot`). The procedure, with day length $D = 86\,400$ s, half-day $D/2$, and Moscow offset $M = 3 \times 3600$ s:

1. Convert the anchor GPS seconds to UTC by subtracting the leap count at that epoch, $t_{\mathrm{UTC}} = t_{\mathrm{anchor}} - \Delta_{\mathrm{GU}}$, then shift into the Moscow-day frame $t_{\mathrm{M}} = t_{\mathrm{UTC}} + M$ and locate that day's midnight $t_0 = \lfloor t_{\mathrm{M}}/D \rfloor\, D$, giving the anchor's time-of-day $\tau_{\mathrm{a}} = t_{\mathrm{M}} - t_0$.
2. Place the reported time-of-day τ in the anchor's day, resolving the midnight wrap toward the nearest match:

$$\boxed{\tau \leftarrow \begin{cases} \tau + D, & \tau < \tau_{\mathrm{a}} - D/2, \\ \tau - D, & \tau > \tau_{\mathrm{a}} + D/2, \\ \tau, & \text{otherwise}, \end{cases} \qquad t_{\mathrm{GPS}} = \underbrace{(t_0 + \tau)}_{\text{Moscow}} - M + \Delta_{\mathrm{GU}}.} \tag{30}$$

The final line undoes the Moscow offset to recover UTC and then *re-adds the same leap count* Δ_{GU} to climb back onto the GPS scale. The leap term enters once with each sign and so cancels exactly between the two legs — it is present only because the intermediate arithmetic must pass through UTC to frame the Moscow day correctly. If no usable anchor is supplied (a non-positive value) the routine returns the sentinel $\{0, 0\}$ rather than fabricate a week, so the caller can detect the failure and fall back; the continuous-seconds-to-week split that closes the routine carries the standard negative-TOW correction (`gpsSecondsToGnssTime`).

6.6 BeiDou time: the 14 s offset and independent week base

BeiDou time (BDT) is offset from GPS by a sub-second-level integer and by an independent week numbering, and the two pieces are owned by different modules. The *sub-second* relation,

$$\mathrm{BDT} = \mathrm{GPS} - 14\ \mathrm{s}, \tag{31}$$

reflects that BDT was synchronised to UTC at its 2006 zero epoch, by which time GPS already led UTC by 14 s; it is applied in `TimeUtils` when a BeiDou calendar timestamp is converted to GPS time (`TimeUtils::GnssTimeFromDate`, BeiDou branch; Section 6.4). The *week* relation is separate. BDT counts weeks from its own zero epoch (2006-01-01), which is GPS week 1356, and this base is applied at query time by the ephemeris store, not by `TimeUtils` (the file-local constant `kBdtToGpsOffsetSec`, consumed by `LiveEphemerisStore::computeSatellitePV`). When the store lifts a BeiDou ephemeris reference time onto the common GPS scale for an honest cross-system age check it adds the combined offset

$$\boxed{\Delta_{\mathrm{BDT}\to\mathrm{GPS}} = 1356 \times 604800\ \mathrm{s} + 14\ \mathrm{s},} \tag{32}$$

the week base plus the 14 s of (31) (`kBdtToGpsOffsetSec`, applied by the file-local `toeGpsContinuous`). Omitting the week base would make every BeiDou satellite read roughly 1356 weeks (∼26 years) stale and be silently rejected by the staleness guard that follows; the offset does not cancel automatically because the query time is GPS-scale while the stored t_{oe} is on the native BDT week (the staleness guard in `LiveEphemerisStore::computeSatellitePV`).

6.7 Legacy ECI→ECEF: the equinox-based chain

The Sun, Moon, and (for some product streams) the satellite positions are most naturally given in a quasi-inertial celestial frame; the tides, attitude, and phase wind-up that consume them live in the terrestrial ECEF frame. The bridge is the Earth-orientation matrix $\boldsymbol{U}$ such that $\boldsymbol{r}_{\mathrm{ECEF}} = \boldsymbol{U}\,\boldsymbol{r}_{\mathrm{ECI}}$. The default (always-available) path is the classical equinox-based chain following the IAU-1976 precession and IAU-1980 nutation conventions [18], composed as (`Planetary::eci2ecef`):

$$\boxed{\boldsymbol{U} = \boldsymbol{W}_{\mathrm{pm}}\ \boldsymbol{R}_3(\mathrm{GAST})\ \boldsymbol{N}_{\mathrm{nut}}\ \boldsymbol{P}_{\mathrm{prec}}.} \tag{33}$$

Here $\boldsymbol{R}_1, \boldsymbol{R}_2, \boldsymbol{R}_3$ are the elementary right-handed axis rotations of `Planetary::Rxyz` and each factor reverses one piece of the celestial-to-terrestrial orientation: precession and nutation orient the true equator and equinox of date, the sidereal rotation $\boldsymbol{R}_3(\mathrm{GAST})$ spins the frame about the pole by the apparent sidereal time, and polar motion $\boldsymbol{W}_{\mathrm{pm}}$ ties the celestial pole to the conventional terrestrial pole.

Precession (IAU 1976). With t the TT centuries of (27) and arcsecond coefficients converted to radians, the three equatorial precession angles ζ_A, θ_A, z_A and the mean obliquity ε_0 are (the precession block of `Planetary::eci2ecef`):

$$\begin{aligned}
\zeta_A &= 2306.2181\,t + 0.30188\,t^2 + 0.017998\,t^3,\\
\theta_A &= 2004.3109\,t - 0.42665\,t^2 - 0.041833\,t^3,\\
z_A &= 2306.2181\,t + 1.09468\,t^2 + 0.018203\,t^3,\\
\varepsilon_0 &= 84381.448 - 46.8150\,t - 0.00059\,t^2 + 0.001813\,t^3 \quad (\mathrm{arcsec}),
\end{aligned} \tag{34}$$

assembled as $\boldsymbol{P}_{\mathrm{prec}} = \boldsymbol{R}_3(-z_A)\,\boldsymbol{R}_2(\theta_A)\,\boldsymbol{R}_3(-\zeta_A)$.

Nutation (IAU 1980, 106 terms). The nutation in longitude $\Delta\psi$ and in obliquity $\Delta\varepsilon$ are summed over the 106-term IAU-1980 series [18] in the five Delaunay fundamental arguments $f = (l, l', F, D, \Omega)$ (`Planetary::nutIau1980`; arguments from `Planetary::astArgs`):

$$\Delta\psi = 10^{-4}\sum_{i=1}^{106}\left(A_i^{\psi} + \dot{A}_i^{\psi}\,t\right)\sin\left(\boldsymbol{n}_i\cdot f\right), \quad \Delta\varepsilon = 10^{-4}\sum_{i=1}^{106}\left(A_i^{\varepsilon} + \dot{A}_i^{\varepsilon}\,t\right)\cos\left(\boldsymbol{n}_i\cdot f\right), \tag{35}$$

in arcseconds (the 10^{-4} scales the tabulated 0.1 mas units), then converted to radians. The nutation matrix rotates by the true obliquity $\varepsilon_0 + \Delta\varepsilon$: $\boldsymbol{N}_{\text{nut}} = \boldsymbol{R}_1(-\varepsilon_0 - \Delta\varepsilon)\,\boldsymbol{R}_3(-\Delta\psi)\,\boldsymbol{R}_1(\varepsilon_0)$ (the nutation block of `Planetary::eci2ecef`).

Apparent sidereal time and the 0h-UT1 subtlety. Greenwich Apparent Sidereal Time adds the equation of the equinoxes — the nutation in longitude projected on the equator, plus the two leading complementary terms in the lunar node Ω — to the mean sidereal time (the GAST block of `Planetary::eci2ecef`):

$$\text{GAST} = \text{GMST} + \Delta\psi\cos\varepsilon_0 + \left(0.00264\sin\Omega + 0.000063\sin 2\Omega\right)\text{arcsec}. \tag{36}$$

GMST itself is built from the IAU-1982 model evaluated at the *start* of the current UT1 day plus the Earth's rotation through the elapsed day fraction (`Planetary::utc2gmst`). The reference instant must be 0h UT1 of the calendar day, and because a Julian Date is referenced to *noon*, that midnight is recovered as

$$\text{JD}_{0\text{h}} = \left\lfloor \text{JD}_{\text{UT1}} - 0.5 \right\rfloor + 0.5, \tag{37}$$

not $\lfloor \text{JD} \rfloor$ (same routine). Using the bare floor lands on the *previous* noon for any post-midnight epoch — a half-day error that propagates to roughly 0.5° in GMST and, through (33), on the order of 10^6 km in the rotated Sun position; the comment in `Planetary::utc2gmst` records exactly this failure mode.

Polar motion. The terrestrial-pole tie is $\boldsymbol{W}_{\text{pm}} = \boldsymbol{R}_2(-x_p)\,\boldsymbol{R}_1(-y_p)$ for the interpolated pole coordinates (x_p, y_p) in radians (the polar-motion block of `Planetary::eci2ecef`), the small-angle two-axis rotation that aligns the celestial intermediate pole of date with the IERS conventional pole.

6.8 CIO-based ITRS↔GCRS (IAU 2000A/2006)

An opt-in path (config key `estimation.enable_cio_frame`, getter `GnssConfig::enableCioFrame`; **default off**, so the equinox chain of (33) is the shipped path) replaces the equinox chain with the modern Celestial Intermediate Origin (CIO) based transform of the IAU 2000A/2006 conventions [18], evaluated through SOFA [8] and mirroring the M-PAGES `NgsFormC2T` construction (`Planetary::celestialToTerrestrial`). The celestial-to-terrestrial matrix is composed as

$$\boxed{\boldsymbol{U} = \texttt{POM}(x_p, y_p, s')\;\boldsymbol{R}_3(\theta_{\text{ERA}})\;\texttt{C2I}(X, Y, s),} \tag{38}$$

i.e. the SOFA composition `iauC2tcio`. The Celestial Intermediate Pole coordinates (X, Y) and the CIO locator s come from the full IAU-2000A precession-nutation model via `iauXys00a`, after which the *observed* celestial-pole offsets are injected additively,

$$X \leftarrow X + \mathrm{d}X, \qquad Y \leftarrow Y + \mathrm{d}Y, \tag{39}$$

and the celestial-to-intermediate matrix is formed by `iauC2ixys`. The diurnal rotation is the Earth Rotation Angle θ_{ERA} (a strictly linear function of UT1, `iauEra00`) rather than the sidereal time of (36), and the terrestrial-pole tie `POM` uses `iauPom00` with the TIO locator s' from `iauSp00`. Two-part Julian Dates are passed with the (2400000.5, MJD) split for numerical conditioning (same routine). The two paths are designed to agree: with $\mathrm{d}X = \mathrm{d}Y = 0$ the CIO result reproduces the bare IAU-2000A model and differs from the legacy equinox chain only by the (sub-milliarcsecond) frame-bias and model-version difference, which the engine confirms at startup by logging the angular separation of the two Sun directions (the startup diagnostic in `Planetary::computeSunMoonPosition`).

6.9 Earth-orientation parameters: parse and interpolation

Both transforms above need the time-varying Earth-orientation parameters ($x_p, y_p, \mathrm{UT1} - \mathrm{UTC}, \mathrm{d}X, \mathrm{d}Y$). These arrive in a standard IGS `.ERP` file, parsed by `ProductParser::parseErp` (`src/core/products/ProductParser.cpp`). The parser reads the MJD time tag and the pole/UT1 columns and applies the IGS scale factors — the pole coordinates are tabulated in 10^{-6} arcseconds and UT1−UTC in 10^{-7} seconds (`ProductParser::parseErp`):

$$x_p = (\mathrm{col}) \times 10^{-6}\,\mathrm{arcsec}, \quad y_p = (\mathrm{col}) \times 10^{-6}\,\mathrm{arcsec}, \quad (\mathrm{UT1} - \mathrm{UTC}) = (\mathrm{col}) \times 10^{-7}\,\mathrm{s}. \tag{40}$$

The MJD tag is converted to a GPS week/TOW relative to the GPS-epoch MJD (same routine). The CIP offsets $\mathrm{d}X, \mathrm{d}Y$ are *not* carried by the standard `.ERP` format and default to zero; because the IAU-2000A model of (38) already includes the modelled precession-nutation, $\mathrm{d}X = \mathrm{d}Y = 0$ is the correct nominal value, not a missing input (same routine). The parser *only* parses; all temporal interpolation lives in `Planetary::interpolateErp`, which binary-searches the chronologically sorted table on continuous GPS seconds and linearly interpolates every component between the bracketing entries:

$$p(t) = p_0 + \frac{t - t_0}{t_1 - t_0}\,(p_1 - p_0), \qquad p \in \{x_p, y_p, \mathrm{UT1}{-}\mathrm{UTC}, \mathrm{d}X, \mathrm{d}Y\}, \tag{41}$$

clamping to the nearest endpoint outside the table span (same routine). The interpolated (x_p, y_p) feed the polar-motion factor of (33) / (38), while UT1−UTC corrects UTC to UT1 before the sidereal/ERA evaluation. The product-evaluation document (Section 7) cross-references this section rather than re-deriving the polar-motion application, since the same $\boldsymbol{W}_{\mathrm{pm}}$ rotates both the luni-solar vectors and any celestial-frame satellite product.

6.10 The Sun and Moon series these transforms position

The end purpose of the orientation chain is to place the Sun and Moon in ECEF for the geophysical and observation-model corrections. The celestial-frame positions are computed first, from an analytic series in the J2000 centuries t and the Delaunay arguments f (`Planetary::sunMoonPosEci`), an enhanced RTKLIB/Meeus model [14, 15]. The Sun's ecliptic longitude is the mean longitude plus the equation of centre (three harmonics of the solar mean anomaly $M_\odot$) and small Venus, Jupiter, and lunar perturbation terms (same routine); the obliquity rotation

$$\varepsilon = 23.439291 - 0.0130042\,t \quad (\mathrm{deg}) \tag{42}$$

carries the resulting ecliptic $(\lambda_\odot, \beta_\odot)$ into equatorial ECI components:

$$\begin{aligned} r_x &= r\,\cos\beta_\odot \cos\lambda_\odot, \\ r_y &= r\,(\cos\varepsilon\,\cos\beta_\odot \sin\lambda_\odot - \sin\varepsilon\,\sin\beta_\odot), \\ r_z &= r\,(\sin\varepsilon\,\cos\beta_\odot \sin\lambda_\odot + \cos\varepsilon\,\sin\beta_\odot), \end{aligned} \tag{43}$$

with the solar distance r from the three-harmonic radius series. The Moon's longitude, latitude, and horizontal-parallax-derived distance follow the analogous Brown-series truncation in the same five fundamental arguments (same routine). The resulting ECI vectors are then rotated to ECEF by exactly the $\boldsymbol{U}$ of (33) or (38) according to the `estimation.enable_cio_frame` toggle (default off, Section 6.8), in `Planetary::computeSunMoonPosition`; the ECEF Sun and Moon are precisely the inputs consumed by the solid-Earth and ocean tides of Section 9 and by the satellite-attitude and phase-wind-up models of Section 10. The two luni-solar models are only $\sim$100 km accurate in position — ample for the tidal and attitude geometry, which depend on the Sun/Moon *direction* to well under the milliarcsecond level that matters — and a non-finite guard substitutes a nominal 1 AU / mean-lunar-distance fallback rather than propagate a NaN into the correction stack (the output-validation tail of `Planetary::computeSunMoonPosition`).

7 Orbit and Clock Product Evaluation

Every processing mode in the engine — post-processed PPP, PPP-RTK, the baseline modes, and the real-time streaming engine — shares one deterministic satellite *backbone*: a position $\boldsymbol{r}^i$, velocity $\dot{\boldsymbol{r}}^i$, and clock bias δt^i for each satellite at each query epoch. This backbone is evaluated *before* any state-space refinement. In post-processing it is the final word; in the live engine it is the broadcast baseline that the SSR overlay (Section 17) refines by a RAC orbit delta and a scalar clock delta. This document derives the backbone itself — the two precise interpolation paths and the two broadcast evaluation paths — and the ephemeris-selection logic that feeds them. It deliberately stops at the point where the SSR delta is added: the orbit/clock *overlay* math (RAC frame, IOD gate, freshness) lives in Section 17 and is not re-derived here. The Sagnac (Earth-rotation) range correction that turns $\boldsymbol{r}^i$ at transmit time into a consistent line-of-sight belongs to Section 10; the ERP / polar-motion transforms that carry an SP3 sample between the terrestrial and celestial frames belong to Section 6.

The backbone splits cleanly into a *precise* path and a *broadcast* path. The precise path consumes tabulated SP3 orbit samples and CLK clock samples (IGS Final / Rapid products) and *interpolates*; it is the post-processing default and is centimetre-to-millimetre grade. The broadcast path evaluates a Keplerian (or, for GLONASS, a numerically integrated PZ-90) ephemeris analytically from the navigation message; it is metre-grade on its own and is the substrate the live SSR stream corrects. On the default path the two paths do not mix within a satellite at a given epoch: a precise product, when loaded and spanning the query, wins; otherwise the engine falls through to the broadcast evaluation (`src/core/model/ModelGeometry.cpp`, `src/core/corrections/GnssCorrections.cpp`). The resolution is per quantity, however, not structurally coupled: the real-time RTB-4 gate `realtime.use_precise_orbits` splits the orbit and clock sources independently (`GnssConfig::realtimeUsesLiveOrbitSource` vs `realtimeUsesLiveOrbitClockSource`, `ProductParser::loadProducts(bool,bool)`), so an armed "ultra-rapid orbits + network clocks" config evaluates one satellite at one epoch with a precise interpolated orbit *and* a broadcast+SSR clock. The flag defaults false — the orbit gate then returns exactly the clock gate, byte-identical — so the no-mixing statement holds for every non-real-time mode and for the default real-time path.

7.1 Precise orbit: derotated Neville interpolation

SP3 files tabulate the satellite position in the ECEF frame on a coarse grid (typically 900 s for IGS Final, 300 s for Rapid). The ECEF trajectory is *not* a smooth low-degree polynomial: it carries the sidereal-frequency oscillation of the rotating frame relative to the satellite's near-inertial orbit, which a degree-10 fit cannot capture to better than a few millimetres near the SP3 epoch

boundaries. The engine therefore *derotates* each node into a common near-inertial frame anchored at the query epoch before interpolating (`src/core/model/ModelGeometry.cpp`, following the RTKLIB / cssrlib convention [24]). For a node sampled at t_i and a query at t_q, with the Earth-rotation rate $\omega_\oplus = 7.2921151467 \times 10^{-5}$ rad/s (`Constants::EARTH_ANGULAR_VELOCITY`), the in-plane coordinates are rotated by the angle the frame swept between the two epochs:

$$\boldsymbol{r}_{\mathrm{I}}^{i}(t_i) = \boldsymbol{R}_z(\omega_\oplus\,(t_i - t_q))\;\boldsymbol{r}^{i}(t_i), \qquad \boldsymbol{R}_z(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{44}$$

Concretely, with $\alpha_i = \omega_\oplus\,(t_i - t_q)$ the derotated in-plane components written to the interpolation arrays are $\tilde{x}_i = \cos\alpha_i\, x_i - \sin\alpha_i\, y_i$ and $\tilde{y}_i = \sin\alpha_i\, x_i + \cos\alpha_i\, y_i$, with z untouched (`ModelGeometry.cpp`). Because the rotation angle is exactly zero at $t_i = t_q$, the interpolated value at the query is *already* the ECEF position at t_q — no post-rotation is needed.

Bracket-centred window. The node window is the decisive design choice for position *continuity*. The engine selects an IGS_POINTS = 11-node window centred on the *bracket* $[t_a, t_{a+1}]$ that contains t_q, not the 11 nodes nearest to t_q (`ModelGeometry.cpp`). The distinction is subtle but operationally important: a nearest-N rule swaps a node at the *midpoint* between SP3 epochs, where the two candidate polynomials disagree by a few millimetres, producing the well-known synchronous-across-constellation 15-minute sawtooth in PPP position. A bracket-centred rule instead swaps the window exactly when t_q crosses a node t_{a+1}; at that instant t_{a+1} is shared by both windows, so the Neville value is identical on either side and the interpolated orbit is continuous across the swap. A gap guard re-picks a narrower MIN_POINTS = 9-node window and rejects the satellite if the residual span still exceeds the fit interval (thresholds in `ModelGeometry.cpp`; tuning values cited, not reproduced). The "fit interval" is not read from the product header but computed as the *median* of the leading consecutive-node spacings (`ModelGeometry.cpp`); this is deliberate, because a multi-file SP3 merge can interleave a finer margin file's off-grid nodes into a coarser arc (e.g. a 5-minute ultra-rapid file concatenated with a 15-minute one — the 1 s dedup window correctly keeps them as non-duplicates, so the merged series is non-uniformly sampled and `ProductParser` warns on the mixed-rate concat). The median grid gives a stable MAX_SPAN guard on such a series where the nominal grid would be ill-defined.

Neville value and derivative. Within the window the engine runs Neville's algorithm simultaneously for the value and its analytic derivative (`src/core/math/Interpolate.cpp`, `Interpolate::nevilleInterpDeriv`). To avoid catastrophic cancellation, the node abscissae are shifted to $s_i = t_i - t_q$ (raw continuous GPS seconds are $\sim 1.36 \times 10^8$, so an un-shifted difference loses ~ 8 digits); the polynomial value and derivative are both invariant under this unit-Jacobian shift. With the tableau column P_i for the value and D_i for the derivative, the coupled recurrence over degree j (evaluating at the shifted query $s_q = 0$) is

$$P_{i,j} = \frac{(s_q - s_{i+j})\,P_{i,j-1} - (s_q - s_i)\,P_{i+1,j-1}}{s_i - s_{i+j}}, \tag{45}$$

$$D_{i,j} = \frac{(s_q - s_{i+j})\,D_{i,j-1} + P_{i,j-1} - (s_q - s_i)\,D_{i+1,j-1} - P_{i+1,j-1}}{s_i - s_{i+j}}, \tag{46}$$

seeded by $P_{i,0} = y_i$ and $D_{i,0} = 0$; the converged scalars are $P_{0,n-1}$ and $D_{0,n-1}$ (`Interpolate.cpp`). The derivative recurrence (46) is exactly $\partial/\partial s_q$ of the value recurrence (45), so no second pass or finite difference is required.

Inertial-to-ECEF velocity. The Neville derivative is taken on the *derotated* channels, so it is the velocity in the frame fixed at t_q — a snapshot inertial frame. Downstream consumers (Doppler line-of-sight rate, orbit-normal attitude, the relativistic $\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i$ term) require the ECEF velocity, recovered by removing the frame rotation $\dot{\boldsymbol{r}}^i_{\mathrm{ECEF}} = \dot{\boldsymbol{r}}^i_{\mathrm{I}} - \boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i$. For $\boldsymbol{\omega}_\oplus = (0, 0, \omega_\oplus)$ this is the boxed transform applied in `ModelGeometry.cpp`:

$$\boxed{\dot{\boldsymbol{r}}^i_{\mathrm{ECEF}} = \begin{bmatrix} v_{\mathrm{I},x} + \omega_\oplus \, \boldsymbol{r}^i_y \\ v_{\mathrm{I},y} - \omega_\oplus \, \boldsymbol{r}^i_x \\ v_{\mathrm{I},z} \end{bmatrix}.} \tag{47}$$

The relativistic clock term is invariant to this conversion, since $\boldsymbol{r}^i \cdot (\boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i) = 0$; only the Doppler and cross-product attitude paths see the difference (`ModelGeometry.cpp`).

7.2 Precise orbit: sidereal-trigonometric LSQ fit

A config-gated alternative (`config.enableTrigSp3Interp()`) replaces the Neville polynomial with an over-determined least-squares fit on a *sidereal-trigonometric basis*, run on the identical derotated channels and time base (`ModelGeometry.cpp`; `Interpolate::trigSvdInterp`). The basis is a constant plus four sine/cosine harmonic pairs of the fundamental sidereal angular rate — nine terms in all:

$$\boldsymbol{r}^i_{\mathrm{c}}(\tau) = c_0 + \sum_{f=1}^{4} \big[c_{2f-1} \sin(f\,\Omega_0\,\tau) + c_{2f} \cos(f\,\Omega_0\,\tau) \big], \qquad \tau = t - t_0, \tag{48}$$

where t_0 is the first node (the constant term absorbs the offset, the sin/cos pairs the phase, so the fitted curve is invariant to this choice). The fundamental rate is the angular frequency of the *mean sidereal day*: with the sidereal-day ratio $\kappa = 0.99726956634$ and a nominal solar day of 86400 s,

$$\boxed{\Omega_0 = \frac{2\pi}{\kappa \cdot 86400 \text{ s}} \quad [\text{rad/s}], \qquad \kappa = 0.99726956634,} \tag{49}$$

expressed *per second* so the basis argument τ stays in the caller's seconds and the analytic derivative falls out per second (`Interpolate.cpp`; M-PAGES uses rad/day and divides the velocity by 86400, numerically identical). The constant $\kappa = 0.99726956634$ is the IERS mean sidereal-to-solar day ratio and *is* the physics of the model, so it is reproduced here with its source line.

The fit is solved by Eigen's `JacobiSVD` with a relative singular-value threshold of 10^{-15}, which yields the minimum-norm least-squares solution — the Numerical-Recipes `svbksb` convention M-PAGES' `svdfit` uses [19]. Per-node weights derive from the SP3 `++` accuracy codes; because those are a single value per satellite, the weight set is uniform and the fit is effectively unweighted, matching M-PAGES. The analytic value and derivative at the query $\tau_q = t_q - t_0$ are

$$\boldsymbol{r}^i_{\mathrm{c}}(\tau_q) = c_0 + \sum_{f=1}^{4} \big[c_{2f-1} \sin(f\Omega_0\tau_q) + c_{2f} \cos(f\Omega_0\tau_q) \big], \tag{50}$$

$$\dot{\boldsymbol{r}}^i{}_{\mathrm{c}}(\tau_q) = \sum_{f=1}^{4} (f\Omega_0) \big[c_{2f-1} \cos(f\Omega_0\tau_q) - c_{2f} \sin(f\Omega_0\tau_q) \big], \tag{51}$$

with the per-harmonic angular weight $w_f = f\,\Omega_0$ (`Interpolate.cpp`). Crucially this is an *over-determined smoother*: the window is up to 11 nodes against 9 basis terms, so for $N > 9$ the fitted

curve does *not* pass through the nodes — a deliberate divergence from the M-PAGES 9-point exact fit, kept so the trig branch reuses the existing derotated window verbatim. The same inertial-to-ECEF velocity conversion (47) is then applied to $\dot{\boldsymbol{r}}^i_{\mathrm{c}}$.

7.3 Precise clock interpolation

The CLK product tabulates the scalar satellite clock bias δt^i on its own grid (commonly 30 s). Because clock noise is process-driven (a near-Brownian random walk in bias, with occasional rate discontinuities), the engine offers three interpolation modes selected by `config.getSatelliteClockInterpolationMode()` (`src/core/corrections/GnssCorrections.cpp`). All three are bracket-centred for the same continuity reason as the orbit window: the sliding window must swap on a shared node, never at a between-epoch midpoint.

(a) Linear (MMSE for Brownian motion). The default is a two-node linear interpolation across the bracket $[t_0, t_1]$ containing t_q:

$$\delta t^i(t_q) = \delta t^i_0 + \left(\delta t^i_1 - \delta t^i_0\right) \frac{t_q - t_0}{t_1 - t_0}, \tag{52}$$

which is the minimum-mean-square-error estimator for a Wiener process sampled at the two bracketing epochs. Interpolation across a gap larger than a guard threshold (in seconds) is rejected outright rather than fabricated (`GnssCorrections.cpp`; gap value cited, not reproduced).

(b) Rate-anchored weighted LSQ. The `LsqSatClock` mode fits a local line $\delta t^i(t) \approx a + b\,(t - t_q)$ over an $\sim$11-node window by weighted least squares, with the centring shift $s_k = t_k - t_q$ making the fitted value at the query equal to the intercept a directly. The key refinement is that the CLK product may also carry a clock *rate* m_j at the coarse 5-minute epochs; each rate is added as an *explicit observation of the slope*, $m_j = b + \mu_j$. Since a value equation $y_k = a + b\,s_k + \nu_k$ has $\partial/\partial a = 1$, $\partial/\partial b = s_k$ while a rate equation $m_j = b + \mu_j$ has $\partial/\partial a = 0$, $\partial/\partial b = 1$, the rate observations contribute to the slope row only. With value weights $w^{(y)}_k = 1/\sigma^2_{y,k}$ and rate weights $w^{(m)}_j = 1/\sigma^2_{m,j}$, the 2×2 weighted normal equations are

$$\boxed{\begin{bmatrix} \sum_k w^{(y)}_k & \sum_k w^{(y)}_k s_k \\ \sum_k w^{(y)}_k s_k & \sum_k w^{(y)}_k s_k^2 \;+\; \sum_j w^{(m)}_j \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum_k w^{(y)}_k y_k \\ \sum_k w^{(y)}_k s_k y_k \;+\; \sum_j w^{(m)}_j m_j \end{bmatrix}}, \tag{53}$$

solved in closed form for the intercept

$$a = \frac{\left(S_{wxx} + S_m\right) S_{wy} - S_{wx}\left(S_{wxy} + S_{my}\right)}{S_w\left(S_{wxx} + S_m\right) - S^2_{wx}}, \tag{54}$$

with the running sums named exactly as in `GnssCorrections.cpp` ($S_w = \sum w^{(y)}_k$, $S_{wx} = \sum w^{(y)}_k s_k$, $S_{wxx} = \sum w^{(y)}_k s_k^2$, $S_{wy} = \sum w^{(y)}_k y_k$, $S_{wxy} = \sum w^{(y)}_k s_k y_k$, $S_m = \sum w^{(m)}_j$, $S_{my} = \sum w^{(m)}_j m_j$), and the determinant in the denominator guarding a degenerate (coincident-time) window with a linear fallback. The estimate $\delta t^i(t_q) = a$ is returned. Because the fit smooths $\sim N$ value samples instead of touching two, the observation-noise component of the estimate at the bracket midpoint drops by $\sim\sqrt{N/2}$ relative to plain linear (52) — material in the seconds-to-minutes regime where the CLK observation noise dominates the per-sample process noise. The cost is a loss of exact-node passage: the smoother introduces millimetre-level discontinuities at the clock epochs when the window slides.

(c) Barycentric Lagrange. For high-rate (1 Hz) products a barycentric-form Lagrange interpolation over a LAGRANGE_POINTS = 9-node window is available (`GnssCorrections.cpp`; `Interpolate::barycentricWeights` / `barycentricEval`). The barycentric form

$$\delta t^i(t_q) = \sum_{i=1}^{n} \frac{w_i}{t_q - t_i}\,\delta t_i^i \;\Bigg/\; \sum_{i=1}^{n} \frac{w_i}{t_q - t_i}, \qquad w_i = \prod_{k\neq i} \frac{1}{t_i - t_k}, \tag{55}$$

is the numerically stable evaluation of the degree-$(n-1)$ Lagrange polynomial [2]; an extrapolation guard falls back to two-node linear when t_q lies outside the node span. This mode is *not* recommended for standard 30 s IGS Final clocks — the polynomial oscillates between the sparse samples — and is retained only for genuinely high-rate input.

7.4 Broadcast Keplerian evaluation

When no precise product spans the query, GPS, Galileo, BeiDou and QZSS are evaluated from their Keplerian broadcast ephemeris (`src/core/products/LiveEphemerisStore.cpp`, `keplerPosition`). This is the substrate the live SSR stream corrects. Each system carries its own gravitational parameter μ and Earth-rotation rate $\omega_\oplus$ (`LiveEphemerisStore.cpp`): GPS / QZSS use $\mu = 3.986005 \times 10^{14}\,\mathrm{m^3/s^2}$, $\omega_\oplus = 7.2921151467 \times 10^{-5}\,\mathrm{rad/s}$; Galileo / BeiDou use $\mu = 3.986004418 \times 10^{14}\,\mathrm{m^3/s^2}$, with BeiDou taking the slightly different $\omega_\oplus = 7.292115 \times 10^{-5}\,\mathrm{rad/s}$. BeiDou is evaluated entirely on its native BeiDou Time (BDT) scale — BDT week 0 aligns with GPS week 1356 and BDT = GPS − 14 s — with both the query epoch and the stored t_{oe} kept on BDT so the node-rotation term below uses the BDT seconds-of-week directly (`LiveEphemerisStore.cpp`).

With the semi-major axis $a = (\sqrt{a})^2$ and the time-from-ephemeris $t_k = t - t_{oe}$ reduced into $[-302400, +302400]$ s (week rollover), the corrected mean motion adds the broadcast δn to the Keplerian value, and the mean anomaly propagates linearly:

$$n = \sqrt{\frac{\mu}{a^3}} + \delta n, \qquad M = M_0 + n\,t_k. \tag{56}$$

Kepler's equation $M = E - e\sin E$ is solved by Newton iteration (up to 15 steps, convergence at $|\Delta E| < 10^{-13}$):

$$E_{k+1} = E_k + \frac{M - \big(E_k - e\sin E_k\big)}{1 - e\cos E_k}. \tag{57}$$

The true anomaly and the argument of latitude follow, then the broadcast second-harmonic corrections refine the argument of latitude u, the orbital radius r, and the inclination i:

$$\nu = \operatorname{atan2}\big(\sqrt{1-e^2}\,\sin E,\ \cos E - e\big), \qquad \phi = \nu + \omega, \tag{58}$$

$$\begin{aligned} u &= \phi + c_{us}\sin 2\phi + c_{uc}\cos 2\phi, \\ r &= a\big(1 - e\cos E\big) + c_{rs}\sin 2\phi + c_{rc}\cos 2\phi, \\ i &= i_0 + i\,t_k + c_{is}\sin 2\phi + c_{ic}\cos 2\phi. \end{aligned} \tag{59}$$

The corrected longitude of the ascending node folds in the node rate *relative to* the rotating Earth and the fixed node offset at the ephemeris epoch:

$$\boxed{\Omega = \Omega_0 + \big(\dot\Omega - \omega_\oplus\big)\,t_k - \omega_\oplus\,t_{oe},} \tag{60}$$

where the final $-\omega_{\oplus}\, t_{oe}$ uses the seconds-of-week of t_{oe} on the native scale (`LiveEphemerisStore.cpp`). With the in-plane position $(x_o, y_o) = (r\cos u,\, r\sin u)$ the ECEF position is the standard rotation through Ω and i:

$$\boldsymbol{r}^i = \begin{bmatrix} x_o\cos\Omega - y_o\cos i\,\sin\Omega \\ x_o\sin\Omega + y_o\cos i\,\cos\Omega \\ y_o\sin i \end{bmatrix}. \tag{61}$$

The BeiDou GEO additional fixed rotation is not applied (GEO satellites are rarely used for PPP-RTK). Velocity is obtained by central finite difference of (61) over $\pm 0.5\,\mathrm{s}$, $\dot{\boldsymbol{r}}^i \approx [\boldsymbol{r}^i(t+\frac{1}{2}) - \boldsymbol{r}^i(t-\frac{1}{2})]/(2\,\Delta t)$ with $\Delta t = 0.5\,\mathrm{s}$, reusing the same analytic position evaluator (`LiveEphemerisStore.cpp`).

7.5 Broadcast GLONASS: PZ-90 RK4 integration

GLONASS does not broadcast Keplerian elements; it broadcasts an ECEF state vector (position, velocity, and a luni-solar acceleration) valid at t_b, and the ICD prescribes numerical integration of the PZ-90 equations of motion to the query epoch [6]. The engine marches a fixed-step fourth-order Runge–Kutta integrator (`src/core/products/LiveEphemerisStore.cpp`, `glonassState`). The per-axis acceleration combines the central gravitational term, the J_2 oblateness perturbation, the centrifugal and Coriolis terms of the rotating frame, and the broadcast luni-solar acceleration held constant over the integration. Writing $r = \|\boldsymbol{r}^i\|$ and the oblateness coefficient

$$\kappa_{J_2} = \tfrac{3}{2}\, J_2\, \frac{\mu\, a_e^2}{r^5}, \qquad J_2 = 1.0826257 \times 10^{-3}, \tag{62}$$

(with $\mu = 3.9860044 \times 10^{14}\,\mathrm{m^3/s^2}$, $a_e = 6378136\,\mathrm{m}$, $\omega_{\oplus} = 7.292115 \times 10^{-5}\,\mathrm{rad/s}$, all PZ-90 constants at `LiveEphemerisStore.cpp`), the accelerations are

$$\boxed{\begin{aligned} \ddot{x} &= -\frac{\mu}{r^3}x + \kappa_{J_2}\, x\big(1 - 5\,z^2/r^2\big) + \omega_{\oplus}^2 x + 2\omega_{\oplus}\dot{y} + a_x, \\ \ddot{y} &= -\frac{\mu}{r^3}y + \kappa_{J_2}\, y\big(1 - 5\,z^2/r^2\big) + \omega_{\oplus}^2 y - 2\omega_{\oplus}\dot{x} + a_y, \\ \ddot{z} &= -\frac{\mu}{r^3}z + \kappa_{J_2}\, z\big(3 - 5\,z^2/r^2\big) + a_z, \end{aligned}} \tag{63}$$

where (a_x, a_y, a_z) is the broadcast luni-solar acceleration. Note the distinct numerator on the z-axis oblateness term ($3 - 5z^2/r^2$ versus $1 - 5z^2/r^2$ in-plane), and that the centrifugal $\omega_{\oplus}^2$ and Coriolis $2\omega_{\oplus}$ terms appear only on the in-plane axes, as required in the PZ-90 rotating frame. The integrator steps with a fixed maximum $h = 30\,\mathrm{s}$, signed toward the query, clamping the final partial step so the march lands exactly on t_q; each step is the classical RK4 update $y \leftarrow y + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ (`LiveEphemerisStore.cpp`).

7.6 Broadcast satellite clock

The broadcast satellite clock for the Keplerian systems is the standard second-order polynomial in the time-from-clock-epoch $t - t_{oc}$ (on the native scale, with BeiDou shifted to BDT exactly as the orbit node term is):

$$\boxed{\delta t^i = a_{f0} + a_{f1}\,(t - t_{oc}) + a_{f2}\,(t - t_{oc})^2.} \tag{64}$$

GLONASS instead uses its two-term broadcast model with the 1020-message τ_n (DF123) and γ_n (DF120) about $t_b = t_{oe}$:

$$\delta t^i = -\tau_n + \gamma_n\,(t - t_b) \tag{65}$$

(`LiveEphemerisStore.cpp`). This path is exercised only where GLONASS is excluded from `active_systems` — the shipped real-time replay configs, which is the context this store serves; the default `active_systems` and the post-processing configs both enable GLONASS, so the path is not universally dead code.

The precise-clock-baseline convention. A design decision worth stating explicitly: the broadcast clock (64) *deliberately omits* both the periodic relativistic eccentricity term and the group delay (TGD/BGD). This is not an oversight — it is what makes the broadcast clock a clean drop-in for the SSR scalar clock delta. The IGS-RTS clock correction is generated against precisely this bare polynomial, so the engine must *not* pre-apply the periodic relativity or TGD before adding the SSR delta, or the correction would double-count. The periodic relativistic term $-2\,\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i / c^2$ is added later, in the observation-physics path; see Section 10 for its derivation. This convention keeps the orbit, the bare clock, and the SSR overlay mutually consistent on one ephemeris baseline.

7.7 Ephemeris selection and IOD bookkeeping

The broadcast paths above presuppose that the *right* ephemeris record has been selected. The live store holds, per satellite, a `current` and an optional `previous` record, and several interlocking rules govern which one feeds the orbit/clock evaluation (`src/core/products/LiveEphemerisStore.cpp`).

Freshest-toe within a source. Within a single source, the record with the newer reference epoch wins: an incoming record whose t_{oe} is older than the held one (a stale re-broadcast) is rejected, while a sub-second jitter or equal-epoch refresh overwrites in place (`LiveEphemerisStore.cpp`). Selection between `current` and `previous` at query time is by *closest* t_{oe} on a common GPS-continuous scale — BeiDou's BDT t_{oe} is lifted to GPS seconds first (`toeGpsContinuous`), or every BeiDou record would read $\sim$1356 weeks stale and be wrongly skipped.

IGS-source authority over receiver nav. When the engine ingests both an IGS broadcast stream and the receiver's own decoded navigation, the IGS record is *authoritative*. The IGS-RTS SSR delta is computed against IGS broadcast ephemeris, so the rover must pair the delta with the *same* IGS broadcast; even at an identical IOD, the receiver's locally assembled orbit carries slightly different fit parameters (a live Galileo I/NAV assembly was observed to diverge by 100 m+), and applying IGS-SSR on top of a receiver ephemeris detonates the single-difference frame. A receiver record therefore never overwrites a current IGS record at an equal-or-older t_{oe}; it is accepted only when it is strictly newer than the IGS record by more than one constellation refresh (1800 s, the GLONASS nominal step), the signature of a *stalled* IGS feed — so a dead IGS stream degrades gracefully to receiver nav rather than stranding the satellite (`LiveEphemerisStore.cpp`).

IODE-change demote and SSR re-referencing. When the IOD changes, the `current` record is demoted to `previous` (rather than discarded), keeping the just-superseded orbit valid across the cutover instant (`LiveEphemerisStore.cpp`). This matters because, across a cutover, the receiver can hold a version one step ahead of the lagging SSR stream. When an SSR delta names a specific IOD, `pickForSsr` re-references: it returns whichever held record (`current` or `previous`) matches the SSR's IOD, so the orbit and clock are evaluated on the exact baseline the correction was computed for, instead of dropping the satellite to broadcast grade on a `current`-vs-SSR mismatch. The same current/previous retention underwrites a second consumer built on the broadcast clock

polynomial (64): `LiveEphemerisStore::ephCutoverShiftMetres` evaluates $c\,(\delta t^i_{\text{new}} - \delta t^i_{\text{old}})$ across an ephemeris version roll — `computeSatelliteClock` run once on the freshest pick and once pinned to the old IODE — and returns nothing when the old version is no longer retained. It is the product-side half of the network producer's `network.apriori_source = brdc_rts` eph-cutover re-level; that re-level is derived in Section 19 and is not repeated here.

The GNSS-IOD matching convention. The SSR "GNSS IOD" field (IGS-SSR / RTCM `4076`, IDF012) is an 8-bit quantity whose per-constellation *definition* differs from the raw broadcast IODE stored on the ephemeris. `ssrMatchIode` normalises the held record's IODE into the SSR convention in exactly one place (`LiveEphemerisStore.cpp`):

$$\mathrm{IOD}_{\mathrm{SSR}}(i) = \begin{cases} \text{IODE} & \text{GPS, QZSS (8-bit LNAV, passthrough),} \\ \text{IODnav \& } \texttt{0xFF} & \text{Galileo (low 8 bits of the 10-bit I/NAV IODnav),} \\ \left\lfloor t^{\mathrm{sow}}_{oe}/720 \right\rfloor \bmod 240 & \text{BeiDou (toe-derived, per IGS-SSR v1.00),} \end{cases} \tag{66}$$

with t^{sow}_{oe} the BDT seconds-of-week. The BeiDou broadcast AODE that the decoder stores in the IODE field is a *different* quantity and would never match the SSR field, so it is deliberately ignored in favour of the toe-derived value. The raw IODE is left intact for the demote bookkeeping above; only the value handed to the SSR gate is normalised here. How that normalised IOD then *gates* the application of the SSR orbit/clock delta — accept the delta only on an exact IOD match, else fall back to broadcast grade — is the subject of Section 17.3; this document supplies only the matched quantity it consumes.

8 Atmospheric Delay Modelling: Troposphere and Ionosphere

The undifferenced code and phase equations, Equation (4) and Equation (5) of the Foundations document, each carry two propagation terms whose interiors are not states of the filter but *models*: the slant tropospheric delay T^i_r and the first-order slant ionospheric delay $\mu_s\, I^i_r$. The troposphere is split into a deterministic, a-priori *zenith hydrostatic* part that is computed and removed before the filter sees the measurement, plus a small *zenith wet* residual and a pair of horizontal gradients that ARE estimated (Section 3, Equation (3)). The ionosphere, in the uncombined parameterisation, is carried as one slant state per satellite (Section 3.3); the models in this chapter supply its a-priori seed, an optional pseudo-observation that ties the relative slants to a Global Ionosphere Map, and the small second- and third-order corrections that the linear $1/f^2$ scaling cannot represent. The network producer additionally co-estimates its own spherical-harmonic VTEC field on a single thin shell (optionally two, Section 8.2.4), which lets it dispense with an external GIM entirely; that model is derived in Section 8.2.4, and the shared column layout it occupies in the network normal equations in Section 19. This document is the derivation of those interiors: the mapping functions, the pressure reductions, the thin-shell geometry, the geomagnetic field, and the sign conventions that decide whether each term is added to or subtracted from the modelled range.

8.1 The slant tropospheric delay

The neutral atmosphere is non-dispersive at GNSS frequencies, so its delay is identical on code and phase and is absent from any ionosphere-free or geometry-free combination. The standard decomposition splits the slant delay into a hydrostatic ("dry") and a wet zenith delay, each carried

to the line of sight by its own elevation mapping function, plus an azimuthal gradient [23]:

$$\boxed{T_r^i = m_{\mathrm{h}}(\varepsilon)\,\mathrm{ZHD} + m_{\mathrm{w}}(\varepsilon)\,\mathrm{ZWD} + m_{\mathrm{g}}(\varepsilon)\big(G_{\mathrm{N}}\cos\alpha + G_{\mathrm{E}}\sin\alpha\big).} \tag{67}$$

Of these, ZHD is supplied a priori and removed from the prefit residual; ZWD, G_{N} and G_{E} are estimated states whose Jacobian columns are exactly the mapping coefficients $m_{\mathrm{w}}(\varepsilon)$ and $m_{\mathrm{g}}(\varepsilon)\{\cos\alpha, \sin\alpha\}$ (`src/core/measurement/MeasurementModel.cpp`; Equation (3)). The remainder of this subsection derives each factor.

8.1.1 Saastamoinen zenith hydrostatic delay

The zenith hydrostatic delay is fixed to better than a millimetre by the total surface pressure alone, through the Saastamoinen reduction [22] as coded in `src/core/atmosphere/Troposphere.cpp`:

$$\boxed{\mathrm{ZHD} = \frac{0.0022768\ P}{1 - 0.00266\cos 2\phi - 0.00028\,h_{\mathrm{km}}},} \tag{68}$$

with P the total pressure in millibar at the antenna, ϕ the geodetic latitude, and $h_{\mathrm{km}} = h/1000$ the ellipsoidal height in kilometres (`Troposphere::computeSaastamoinenZHD`, denominator `f_lat`). The numerical coefficient $0.0022768\ \mathrm{m\,mbar^{-1}}$ folds the refractivity constant $k_1 = 77.604\ \mathrm{K\,mbar^{-1}}$ with the dry-gas constant and standard gravity; the denominator is the gravity-variation factor $f(\phi, h)$ that converts the column-integrated delay to its value at the local effective gravity.

When neither a surface met sensor nor the GPT3 grid (below) is available, P is taken from a standard-atmosphere barometric reduction of a configured sea-level pressure P_0 and temperature T_0 (`Troposphere.cpp`, `pressureFromHeightStdAtm`):

$$P(h) = P_0\left(1 - \frac{L\,h}{T_0}\right)^{g/(RL)}, \qquad L = 0.0065\ \mathrm{K\,m^{-1}}, \quad \frac{g}{RL} \approx 5.25588, \tag{69}$$

with $g = 9.80665\ \mathrm{m\,s^{-2}}$ and $R = 287.05\ \mathrm{J\,kg^{-1}\,K^{-1}}$ the specific gas constant of dry air. The base $(1 - Lh/T_0)$ is clamped to $[0.1, 2.0]$ and the result to $[100, 1100]$ mbar as a numerical guard.

8.1.2 The Marini mapping form

Both the hydrostatic and wet mappings use the normalised three-parameter continued fraction of Marini, normalised to unity at zenith (`Troposphere::mariniMapping`):

$$\boxed{m(\varepsilon;\,a,b,c) = \frac{1 + \dfrac{a}{1 + \dfrac{b}{1+c}}}{\sin\varepsilon + \dfrac{a}{\sin\varepsilon + \dfrac{b}{\sin\varepsilon + c}}}.} \tag{70}$$

The numerator is the denominator evaluated at $\varepsilon = 90^\circ$ ($\sin\varepsilon = 1$), so $m(90^\circ) = 1$ exactly; the only thing that distinguishes the hydrostatic from the wet mapping, and one model from another, is the triple (a, b, c). Two families are implemented.

Niell / GMF coefficients. The default mapping interpolates the Niell tables [16] of (a, b, c) given at five reference latitudes $15°, 30°, 45°, 60°, 75°$, each as a mean plus an annually varying amplitude (the coefficient tables inside `Troposphere::getTroposphereMapping`). The seasonal value is

$$v(\phi, t) = \bar{v}(\phi) - \delta v(\phi)\,\cos\frac{2\pi\,(\mathrm{doy} - \mathrm{doy}_0)}{365.25}, \qquad \mathrm{doy}_0 = \begin{cases} 28 & \phi \geq 0 \\ 28 + 182.625 & \phi < 0 \end{cases} \tag{71}$$

(`Troposphere::getTroposphereMapping`, over the day of year returned by `Troposphere::getDayOfYear`). The *minus* sign and the southern-hemisphere half-year phase shift are both load-bearing: the amplitude term is subtracted, and the Southern Hemisphere is put in anti-phase by adding half a year to the phase reference rather than by flipping the sign. The latitude argument is the absolute latitude, linearly interpolated and clamped to $[15°, 75°]$; the wet coefficients carry mean values only (no annual term in this model).

A height correction lifts the hydrostatic mapping from sea level to the station, identical in the Niell and VMF3 paths (`Troposphere::getTroposphereMapping`):

$$m_\mathrm{h}(\varepsilon, h) = m_\mathrm{h}(\varepsilon, 0) + \left[\frac{1}{\sin\varepsilon} - m(\varepsilon;\, a_\mathrm{ht}, b_\mathrm{ht}, c_\mathrm{ht})\right] h_\mathrm{km}, \qquad \begin{aligned} a_\mathrm{ht} &= 2.53 \times 10^{-5}, \\ b_\mathrm{ht} &= 5.49 \times 10^{-3}, \\ c_\mathrm{ht} &= 1.14 \times 10^{-3}, \end{aligned} \tag{72}$$

with the same Marini form (70) evaluated at the height triple. Only the hydrostatic mapping receives this correction.

VMF3 coefficients. When configured for VMF3 [12], the wet/hydrostatic a-coefficients come from the GPT3 grid (next paragraph) and the b, c coefficients are synthesised on the fly by a degree- and order-12 spherical-harmonic expansion of their seasonal columns (`src/core/atmosphere/Vmf3.cpp`). A Cartesian-form associated Legendre recursion builds the real and imaginary solid harmonics V_{nm}, W_{nm} at the station co-latitude and longitude, the $91 = \sum_{n=0}^{12}(n+1)$ harmonic terms are accumulated against the stored `Vmf3Coeffs` tables, and each of $b_\mathrm{h}, c_\mathrm{h}, b_\mathrm{w}, c_\mathrm{w}$ is then evaluated as a five-coefficient mean-plus-annual-plus-semiannual harmonic — all three steps inside `Vmf3::computeMapping`:

$$v(\mathrm{doy}) = A_0 + A_1\cos\omega + B_1\sin\omega + A_2\cos 2\omega + B_2\sin 2\omega, \qquad \omega = \frac{2\pi\,\mathrm{doy}}{365.25}. \tag{73}$$

The synthesised (a, b, c) feed the very same Marini fraction (70), followed by the height correction (72). A failed grid query silently falls back to the Niell path so a single bad sample never aborts the run.

8.1.3 GPT3 empirical pressure, temperature and VMF3 a-coefficients

The GPT3 5° grid [12] supplies, per cell, the five harmonic coefficients of pressure, temperature, temperature lapse rate, geoid undulation, and the VMF3 hydrostatic and wet a-coefficients (`Gpt3::load`, in `atmosphere/Gpt3`). Each quantity is the same annual-plus-semiannual harmonic as (73) evaluated per cell (`Gpt3::harmonic`):

$$V(\mathrm{doy}) = a_0 + A_1\cos\omega + B_1\sin\omega + A_2\cos 2\omega + B_2\sin 2\omega. \tag{74}$$

The grid value sits at the cell's orthometric reference height H_s; it is carried to the station orthometric height $h_{\mathrm{ortho}} = h - N$ (undulation N) by a barometric pressure reduction and a linear temperature lapse (`Gpt3::query`):

$$P_{\mathrm{stn}} = P_{\mathrm{grid}}\big(1 - K_{\mathrm{P}}\,\Delta h\big)^{n_{\mathrm{P}}}, \quad T_{\mathrm{stn}} = T_{\mathrm{grid}} + \frac{\mathrm{d}T}{\mathrm{d}z}\,\Delta h, \qquad \begin{aligned} K_{\mathrm{P}} &= 2.26 \times 10^{-5}\,\mathrm{m}^{-1}, \\ n_{\mathrm{P}} &= 5.225, \end{aligned} \tag{75}$$

with $\Delta h = h_{\mathrm{ortho}} - H_s$ and $\mathrm{d}T/\mathrm{d}z$ the grid lapse in $\mathrm{K\,km^{-1}}$ applied to Δh in kilometres. The exponent 5.225 is the same $g/(RL)$ ratio as (69), rounded for the grid. Four corners are read and combined by bilinear interpolation in latitude and (wrapped) longitude (also `Gpt3::query`); the resulting P_{stn} drives the Saastamoinen ZHD of (68), and the interpolated $a_{\mathrm{h}}, a_{\mathrm{w}}$ drive the VMF3 mapping.

8.1.4 The gradient mapping and a degenerate fallback

The azimuthal gradient in (67) uses the Chen–Herring mapping [3], coded identically in the Jacobian and the prefit residual (`MeasurementModel::buildJacobianRow`, `StateInitializer::computeRawResidual`):

$$\boxed{m_{\mathrm{g}}(\varepsilon) = \frac{1}{\sin\varepsilon\tan\varepsilon + 0.0032}, \qquad T^{\mathrm{grad}} = m_{\mathrm{g}}(\varepsilon)\big(G_{\mathrm{N}}\cos\alpha + G_{\mathrm{E}}\sin\alpha\big).} \tag{76}$$

The two partial derivatives $\partial \mathrm{T}/\partial G_{\mathrm{N}} = m_{\mathrm{g}}\cos\alpha$ and $\partial \mathrm{T}/\partial G_{\mathrm{E}} = m_{\mathrm{g}}\sin\alpha$ are exactly the Jacobian columns of the north/east gradient states (Section 3). Finally, a related cold-start guard applies to the *elevation* (hydrostatic/wet) mapping, not the gradient mapping above: when the receiver position is not yet usable (norm below 10^3 m, or non-finite) the geodetic latitude/height needed by the Niell/VMF3 coefficients is unavailable, so the hydrostatic and wet mappings degrade to the geometry-only Black–Eisner form $1/(\sin\varepsilon + a/(\tan\varepsilon + b))$ (the fallback branch of `Troposphere::getTroposphereMapping`) purely so the engine never divides by an undefined elevation during cold start.

8.2 The slant ionospheric delay

The ionosphere is dispersive: to first order its excess phase advance and group delay are $\mp 40.3\,\mathrm{STEC}/f^2$, equal in magnitude and opposite in sign on phase versus code, where STEC is the slant total electron content in $\mathrm{el\,m^{-2}}$. The uncombined model carries this as a per-satellite slant state I_r^i referenced to GPS L1, and projects it onto every signal by the dispersion ratio.

8.2.1 The $1/f^2$ scaling and its sign

For a signal of frequency f on a state referenced to $f_{\mathrm{ref}} = f_{\mathrm{L1}}^{\mathrm{GPS}}$, the dispersion coefficient is

$$\boxed{\mu_s = \frac{f_{\mathrm{ref}}^2}{f_s^2}, \qquad \frac{\partial P_s}{\partial I} = +\mu_s, \qquad \frac{\partial \Phi_s}{\partial I} = -\mu_s,} \tag{77}$$

i.e. the slant-iono Jacobian column is $+\mu_s$ on code and $-\mu_s$ on phase (`MeasurementModel::buildJacobianRow`). This is the model interior of μ_s in Equation (1); referencing every constellation's slant state to GPS L1 is what lets the between-satellite/between-receiver differencing operators (Section 5) cancel the receiver iono datum cleanly. Because $f_{\mathrm{ref}} = f_{\mathrm{L1}}$, the L1 signal has $\mu = 1$ and the state *is* the L1 slant delay in metres.

8.2.2 IONEX / GIM thin-shell slant delay

A Global Ionosphere Map gives vertical TEC on a latitude/longitude grid at a single thin shell of height H above a base radius R_E. The slant delay along a ray at elevation ε follows from single-layer geometry (`computeIppGeometry` in `src/core/atmosphere/SphericalHarmonicIonosphere.cpp`, the shared helper that `GlobalIonosphereMap::slantIonoDelay` and the co-estimated spherical-harmonic VTEC model both call so the two never drift). The zenith angle z' at the ionospheric pierce point (IPP) obeys

$$\sin z' = \frac{R_E}{R_E + H}\cos\varepsilon, \qquad \text{mapping } M(\varepsilon) = \frac{1}{\cos z'}, \tag{78}$$

so that $\text{STEC} = M(\varepsilon)\,\text{VTEC}$ (`GlobalIonosphereMap::slantIonoDelay`). The earth-centred angle to the IPP is $\psi = \frac{\pi}{2} - \varepsilon - z'$, and the IPP latitude and longitude come from the spherical-triangle solution along the satellite azimuth α (`computeIppGeometry`). It propagates the receiver's geodetic latitude ϕ_r across the thin shell as though it were geocentric — an approximation worth well under a millimetre of L1 delay, and the frame in which the reconstructed IPP is later consumed by the higher-order model (96):

$$\phi_{\text{IPP}} = \arcsin\!\big(\sin\phi_r\cos\psi + \cos\phi_r\sin\psi\cos\alpha\big), \tag{79}$$

$$\lambda_{\text{IPP}} = \lambda_r + \arcsin\frac{\sin\psi\,\sin\alpha}{\cos\phi_{\text{IPP}}}. \tag{80}$$

The VTEC at the IPP is interpolated bilinearly in space and, in time, *sun-fixed*: each bracketing map is sampled at the IPP longitude rotated by the Earth's rotation between the map epoch and the query epoch, $\Delta\lambda = (t - t_{\text{map}})\cdot 360^\circ/86400$ s, before the two are linearly blended (`GlobalIonosphereMap::interpolateVtec`, over `GlobalIonosphereMap::sampleMap`). The slant delay in metres at frequency f then uses the dispersion constant

$$\boxed{I^{\text{GIM}} = \text{STEC}\cdot\frac{40.3\times10^{16}}{f^2}\ \ [\text{m}], \qquad \text{STEC} = M(\varepsilon)\,\text{VTEC [TECU]},} \tag{81}$$

the factor 10^{16} converting VTEC from TECU to $\text{el}\,\text{m}^{-2}$ and $40.3 = 40.3\ \text{m}\,\text{Hz}^2\,(\text{el}\,\text{m}^{-2})^{-1}$ the first-order refractivity constant (both the constant and the metre conversion live in `GlobalIonosphereMap::slantIonoDelay`).

8.2.3 The GIM iono-weighting pseudo-observation

The GIM is too coarse to constrain the *absolute* slant of any one satellite, but it constrains the *relative* slants across a constellation well. This is imposed as a Kalman pseudo-observation (`Application::applyGimIonoUpdate`), one row per satellite that owns a slant state above the elevation cutoff. The innovation is the GIM slant minus the current state,

$$z_i = I_i^{\text{GIM}} - I_r^i, \qquad \boldsymbol{H} = \boldsymbol{I}\ \text{(selection of the iono columns)}, \tag{82}$$

with the GIM evaluated at GPS L1 to match the state reference (`Application::applyGimIonoUpdate`). The structure is in the measurement-noise matrix. Per constellation g, with $\mathbf{1}$ the all-ones vector over that constellation's rows,

$$\boxed{\boldsymbol{R} = \bigoplus_{g\in\mathcal{G}}\Big(\text{diag}(\sigma_i^2) + \sigma_c^2\,\mathbf{1}\mathbf{1}^{\text{T}}\Big),} \tag{83}$$

(also `Application::applyGimIonoUpdate`). The rank-one $\sigma_c^2\,\mathbf{1}\mathbf{1}^{\mathrm{T}}$ block assigns an essentially unbounded variance to the per-constellation *common mode* of the innovations: that common mode is the un-modellable receiver iono datum (an arbitrary offset shared by all satellites of a constellation), so the pseudo-observation leaves it free and constrains only the between-satellite differences that the GIM actually knows. The update is applied through the same Joseph-form pseudo-update as every other measurement (Section 11); the iono process noise is documented there (Equation (140)) and not repeated here.

Why frozen pivots are excluded. In the BSSD parameterisation each constellation's pivot satellite holds its slant-iono state frozen at value 0 with variance 0 as the rank-fixing datum. Such a row is skipped entirely (the frozen-pivot guard in `Application::applyGimIonoUpdate`). A zero-variance state receives zero Kalman gain and so could never benefit from the GIM; but it shares the common-mode $\mathbf{1}\mathbf{1}^{\mathrm{T}}$ block of (83) with its constellation peers. Were it included, its spurious innovation $I_{p_g}^{\mathrm{GIM}} - 0 \sim 5$–$30$ m would couple, through the off-diagonal of $\boldsymbol{S}^{-1}$, into the gain of every other satellite of that constellation, dragging the relative slants by the pivot's full absolute STEC. Excluding the pivot row is therefore not an optimisation but a correctness requirement: the common-mode block is meant to absorb the datum offset of the *remaining* rows, so the datum satellite must not itself appear as a measurement.

8.2.4 The co-estimated spherical-harmonic VTEC field

Rather than import an external map, the network producer can estimate the vertical TEC field itself — a truncated real spherical-harmonic expansion on the same kind of thin shell, solved jointly with the satellite code biases. The coefficients are *shared* columns of the network normal equations, registered through `NetworkEstimator::registerVtecCoeff` and reduced onto the shared border like any other network-wide family (Section 19); what follows is the model itself, its pseudo-observation, and its datum.

The switch is `network.estimate_iono_sh`. It defaults false, but it is part of the standing armed network posture and is set by most shipped network configurations, so it is the ionosphere model the producer actually runs on rather than an experiment. Arming it forces `network.estimate_sat_code_bias` on — the field is what makes the satellite code-bias slice observable — and suppresses external GIM loading, so the two ionosphere sources can never be applied to the same run (`config/GnssConfigLoadNetwork.cpp`; the SH-versus-GIM branch guards the `networkAppendGimIonoRows_` call site in `app/ApplicationNetworkAssembly`).

Basis and normalisation. With β the geographic latitude of the pierce point and s its sun-fixed longitude, the field is

$$\boxed{\mathrm{VTEC}(\beta, s) = \sum_{n=0}^{N} \sum_{m=0}^{n} \bar{P}_{nm}(\sin\beta)\,\big[a_{nm}\cos(ms) + b_{nm}\sin(ms)\big] \quad [\mathrm{TECU}],} \tag{84}$$

truncated triangularly at degree N (`estimation.global_vtec_sh.max_degree`), which gives $(N{+}1)^2$ real coefficients: one cosine term per $(n, m \geq 0)$ and one sine term per $(n, m \geq 1)$, enumerated once in a fixed order that the shared-column layout mirrors (`SphericalHarmonicIonosphere::coeffs`). The normalisation is

$$\bar{P}_{nm} = N_{nm}\,P_{nm}, \qquad N_{nm} = \sqrt{(2 - \delta_{0m})\,(2n+1)\,\frac{(n-m)!}{(n+m)!}}, \tag{85}$$

chosen so the real surface harmonics $\{\bar{P}_{nm}\cos(ms),\ \bar{P}_{nm}\sin(ms)\}$ are orthonormal under $(1/4\pi)\int(\cdot)\,\mathrm{d}\Omega$ [23]. Because (84) is linear in the coefficients, the basis values *are* the partial derivatives — $\partial\mathrm{VTEC}/\partial a_{nm}$ and $\partial\mathrm{VTEC}/\partial b_{nm}$ are exactly what `SphericalHarmonicIonosphere::basis` returns, so no numerical differentiation enters the Jacobian. One consequence is used repeatedly below: $\bar{P}_{00}\equiv 1$, so a_{00} *is* the global mean VTEC.

Frame and geometry. The expansion is carried in a sun-fixed frame, $s=\lambda_{\mathrm{IPP}}-\lambda_{\odot}$, with $\lambda_{\odot}=\alpha_{\odot}-\mathrm{GMST}$ the sub-solar geographic longitude (`meanSunLongitude`), so the field is quasi-static over a session instead of sweeping westward at the Earth's rotation rate; `estimation.global_vtec_sh.sun_fixed` selects geographic longitude instead. The pierce point and the thin-shell mapping $M(\varepsilon)$ that carries VTEC to STEC come from *the same* `computeIppGeometry` helper that `GlobalIonosphereMap::slantIonoDelay` calls, evaluated at `estimation.global_vtec_sh.layer_height_km` over base radius `earth_radius_km`. Equations (78), (79) and (80) are therefore the co-estimated model's geometry verbatim: the external map and the co-estimated field share one geometry implementation precisely so they cannot drift apart.

The pin row. The field observes nothing directly; it is tied to the estimated slant states by one pseudo-observation per line of sight (`Application::networkAppendShIonoRows_`). For every GPS or Galileo LOS above `ionosphere_weighting.elevation_cutoff_deg` whose slant state lives in that station's local block, the row asserts that the slant state equals the field's prediction:

$$\boxed{\begin{aligned} &g_r^i \equiv I_r^i \;-\; \kappa\, M(\varepsilon)\sum_k \bar{Y}_k(\mathrm{IPP})\,c_k \;=\; 0, \qquad \kappa=\frac{40.3\times 10^{16}}{f_{\mathrm{L1}}^2},\\ &\frac{\partial g_r^i}{\partial I_r^i}=+1, \qquad \frac{\partial g_r^i}{\partial c_k}=-\,\kappa\, M(\varepsilon)\,\bar{Y}_k(\mathrm{IPP}). \end{aligned}} \tag{86}$$

Here c_k runs over the field coefficients, $\bar{Y}_k$ over the corresponding basis functions of (84), and κ is the TECU-to-L1-metre conversion already introduced in (81). The $+1$ lands in the station's private block and the $-\kappa M\bar{Y}_k$ terms in the *shared* border; that second write is the whole difference between an ionosphere prior and a joint estimate. The right-hand side is $-I_r^i$ at the current state, the shared coefficients entering the absolute solve at zero offset.

The measurement noise on these rows is *the same* common-mode-free per-constellation block as the GIM pseudo-observation, (83), built from the same `ionosphere_weighting.common_mode_sigma_m` over a per-LOS representativeness sigma

$$\sigma_i=\max\bigl(\texttt{floor_m},\ \sigma_{\mathrm{rep}}\,M(\varepsilon)\,\kappa\bigr), \qquad \sigma_{\mathrm{rep}}=\texttt{global_vtec_sh.representation_sigma_tecu}, \tag{87}$$

after which the block is whitened by the Cholesky factor of $\boldsymbol{R}$ into unit-information rows before it joins the normal equations. Reusing (83) is a modelling decision, not an implementation convenience. The rank-one $\sigma_c^2\,\mathbf{1}\mathbf{1}^{\mathrm{T}}$ block leaves the per-(receiver, constellation) iono datum free, so the receiver DCB rides in the common mode and the field constrains only the *between-satellite* slant structure — which is the only thing a global VTEC surface can be asked to know. Its corollary is that a_{00} is observable only weakly, through the spread of $M(\varepsilon)$ across the network, and that is what the datum below has to handle. GLONASS is excluded from the field entirely (`networkAppendShIonoRows_` skips it): its FDMA per-slot inter-frequency bias does not cancel in the common-mode block and would leak into the global coefficients.

The Kaula–Tikhonov prior as the estimability datum. Because the coefficients are shared columns, a coefficient whose pierce-point support has rotated over an ocean or polar gap is near-null in the reduced normal equations and would trip the all-or-nothing rank gate (`NetworkSbasis::assertNullSpaceRank`). A degree-dependent prior is therefore folded on *every* coefficient before the gate runs (`networkInjectShVtecPrior`):

$$\boxed{\begin{aligned} &\sigma_n = \frac{\texttt{kaula_scale_tecu}}{n} \quad (n \geq 1), \qquad \sigma_0 = \texttt{mean_vtec_prior_sigma_tecu}, \\ &\bar{c}_k = \begin{cases} \texttt{mean_vtec_prior_tecu}, & c_k = a_{00}, \\ 0, & \text{otherwise}, \end{cases} \end{aligned}} \tag{88}$$

combined in information space with an optional random-walk anchor toward the previous epoch's estimate, inflated by a per-degree spectral density $q_n = \texttt{q_per_degree}/n^2$ for $n \geq 1$. The two roles the prior plays must be kept apart. The Kaula term is *estimability*: it bounds every coefficient positive-definitely, so the declared null-space dimension stays equal to the number of constellations regardless of coverage, and it pulls unobserved coefficients back toward climatology each epoch instead of letting them random-walk away. The a_{00} term is a deliberately *loose* climatology anchor, and its looseness is the point: a_{00} trades against the absolute satellite-DCB level, whose only rigorous datum is the per-(constellation, signal) sum-to-zero constraint of Section 19. Clamping a_{00} to an external mean VTEC would re-import the very GIM dependency the co-estimation exists to remove, and would bias every emitted satellite code bias by the difference.

Open extensions — default OFF, not yet field-validated. Two extensions of this model are built but off by default and still under validation; neither is the shipped behaviour.

(i) A second thin shell (`estimation.global_vtec_sh.layer2_max_degree`, default 0 = single layer) registers a second coefficient set at its own height, and the pin row (86) then sums the shells, each evaluated at *its own* pierce geometry, $\sum_l \kappa\, M_l(\varepsilon)\, \bar{Y}_k(\mathrm{IPP}_l)\, c_{k,l}$ (`ShLayer`, assembled by `shLayers`, which every consumer of the field loops so a shell can never be evaluated at another shell's geometry). The second shell's a_{00} anchor is deliberately *tight* — the mirror image of the argument above. The inter-layer split of the mean VTEC is quasi-null from ground-only geometry, since only the difference of the two shells' mapping profiles observes it, so the split must be held by prior information while the *total* mean stays data-determined through the first shell's loose anchor.

(ii) A per-(station, satellite) representativeness deviation (`estimation.global_vtec_sh.rep_error_tau_s`, default 0 = off) replaces the pin-to-zero of (86) with a pin to a tracked first-order Gauss–Markov deviation d, carried in vertical TECU so its level and steady state are elevation-stationary. Writing d^-, P^- for the aged prior, $\hat{u}, \hat{V}$ for the solved posterior mean and variance of the pin functional, and f^2 for the pin-row floor variance, the exact scalar posterior is

$$\hat{d} = d^- + \frac{P^-}{a}\big(\hat{u} - d^-\big), \qquad P^+ = P^- \frac{U + f^2}{U + a}, \qquad a = P^- + f^2, \quad U = \frac{\hat{V}\, a}{a - \hat{V}}, \tag{89}$$

with U the data-only information recovered from $\hat{V}$ (`shDevExactUpdate`). Two anti-ratchet treatments sit on top of it, also default OFF. `rep_error_demean` (`shDevDemean`) removes the information-weighted global and per-station means of the carried priors once per epoch, because a deviation component that is common in vertical TECU has row signature $\kappa\, M(\varepsilon)$ — *identical* to the a_{00} column's — so the pair is exactly quasi-null and the harvest–age–re-assert cycle ratchets it; the de-mean enforces the state's own contract, that it carries per-LOS structure only. `rep_error_d5_keep` (`shDevD5Retap`) splits the slip response by provenance: a raw-detector-confirmed slip still

erases the deviation, keeping faith with the rule that a stiff time-linked carry must have a wired slip response (Section 14), while an innovation-gate-only re-tap keeps the level and merely re-taps the variance to the Gauss–Markov steady state. The warm-LOS counts that arm the de-mean are constants in the seam, not configuration; the loader warns that both treatments are inert unless `rep_error_tau_s` is positive, and that arming the erase-policy split without the de-mean re-opens the ratchet.

8.2.5 Klobuchar broadcast seed

For a single-frequency a-priori only — the cold-start seed of the slant state, never a dual-frequency correction — the broadcast Klobuchar half-cosine model is available [11] (`Ionosphere::computeKlobucharDelay`). With elevation and azimuth in semicircles, the earth-centred angle, geomagnetic IPP latitude, local time and obliquity are

$$\psi = \frac{0.0137}{\varepsilon/180 + 0.11} - 0.022, \qquad \phi_i = \mathrm{clip}_{\pm 0.416}\big(\phi/180 + \psi\cos\alpha\big), \tag{90}$$

$$\phi_m = \phi_i + 0.064\cos\big((\lambda_i - 1.617)\pi\big), \qquad F = 1 + 16\,(0.53 - \varepsilon/180)^3, \tag{91}$$

with $\lambda_i = \lambda/180 + \psi\sin\alpha/\cos(\phi_i\pi)$ and local time $t_i = 43200\,\lambda_i + t \pmod{86400}$. Cubic polynomials in the geomagnetic latitude ϕ_m give the amplitude and period,

$$\mathrm{AMP} = \sum_{k=0}^{3} a_k\,\phi_m^k\ (\geq 0), \qquad \mathrm{PER} = \sum_{k=0}^{3} b_k\,\phi_m^k\ (\geq 72000\ \mathrm{s}), \tag{92}$$

and the zenith delay is the truncated half-cosine, mapped to slant by F and scaled to the requested frequency by the f_{L1}^2/f^2 dispersion ratio of (77) (the tail of `Ionosphere::computeKlobucharDelay`):

$$I^{\mathrm{Klob}} = c\,F\Big(5\times10^{-9} + \mathrm{AMP}\,\big[1 - \tfrac{X^2}{2} + \tfrac{X^4}{24}\big]\Big)\frac{f_{\mathrm{L1}}^2}{f^2}, \quad X = \frac{2\pi(t_i - 50400)}{\mathrm{PER}},\ |X| < 1.57, \tag{93}$$

and the constant 5×10^{-9} s night term alone when $|X| \geq 1.57$. The leading c converts the modelled signal delay from seconds to metres.

8.3 Higher-order ionospheric corrections

The $1/f^2$ scaling captures the first-order term; the residual second- and third-order delays scale as $1/f^3$ and $1/f^4$ and are applied as deterministic corrections to the modelled range (`HigherOrderIonosphere::slantHigherOrderComponents`). The second order is the magnetic-field–weighted delay

$$\boxed{I_2^P = 7527\,c\,\mathrm{STEC}\,\frac{\boldsymbol{B}\cdot\hat{\boldsymbol{k}}}{f^3}, \qquad I_2^\Phi = -\tfrac{1}{2}\,I_2^P,} \tag{94}$$

where STEC is in $\mathrm{el\,m^{-2}}$, $\boldsymbol{B}$ is the geomagnetic flux density in tesla at the IPP, $\hat{\boldsymbol{k}}$ the receiver→satellite unit vector, and c the speed of light (the constant $7527\,c = e\,A/(2\pi m_e) \approx 2.2564\times10^{12}$ folds the electron charge, the 40.3-family refractivity constant A, and the electron mass; the constants and the $-\frac{1}{2}$ phase factor are both in `HigherOrderIonosphere::slantHigherOrderComponents`). The third order depends on the path integral of N_e^2, which

a shape factor η relates to the F2-peak density and STEC (also `HigherOrderIonosphere::slantHigherOrderComponents`):

$$\boxed{I_3^P = 2437\,\eta\,N_{\max}\,\frac{\mathrm{STEC}}{f^4}, \qquad N_{\max} = \kappa\,\mathrm{STEC}, \qquad I_3^{\Phi} = -\tfrac{1}{3}\,I_3^P,} \tag{95}$$

with $\eta = 0.66$ the Hoque–Jakowski shape factor [7], $2437 = 3A^2/8$, and $N_{\max} = \kappa\,\mathrm{STEC}$ the linear Hoque–Jakowski fit ($\kappa = (20-6)\times10^{12}/((4.55-1.38)\times10^{18})$ $\mathrm{el\,m^{-3}/(el\,m^{-2})}$) of the F2-peak electron density to slant STEC. The third-order term is thus *linear* in $N_{\max}\,\mathrm{STEC}$, not in STEC^2. Both corrections are sub-millimetre at L1 but enter with the correct sign on each measurement type and survive between-satellite differencing because they are satellite-specific.

8.3.1 The geomagnetic dipole field

The field at the IPP is a centred, tilted magnetic dipole evaluated in ECEF (`MagneticField::fieldAtEcef`):

$$\boxed{\boldsymbol{B}(\boldsymbol{r}) = B_0\left(\frac{R_E}{r}\right)^3\left(\hat{\boldsymbol{m}} - 3(\hat{\boldsymbol{m}}\cdot\hat{\boldsymbol{r}})\,\hat{\boldsymbol{r}}\right),} \tag{96}$$

with $r = \|\boldsymbol{r}\|$, $\hat{\boldsymbol{r}} = \boldsymbol{r}/r$, $B_0 = 30\times10^{-6}$ T the equatorial surface field, $R_E = 6371.2$ km the IGRF reference radius, and $\hat{\boldsymbol{m}}$ the unit axis pointing to the geographic location of the geomagnetic north pole, IGRF-13 epoch 2020.0 at $(80.65°, -72.68°)$ (`MagneticField::dipoleAxisEcef`). With $\hat{\boldsymbol{m}}$ directed at the Arctic geomagnetic pole (which is the dipole's *south* pole), the sign convention in (96) makes $\boldsymbol{B}$ point horizontally toward magnetic north at the magnetic equator, matching compass behaviour; this sign is what fixes the sign of $\boldsymbol{B}\cdot\hat{\boldsymbol{k}}$ and hence of I_2 in (94). The IPP ECEF position itself is reconstructed as a spherical-to-Cartesian map of the geocentric IPP latitude/longitude at the IONEX base-plus-shell radius, since IONEX is a thin-shell model (`HigherOrderIonosphere::slantHigherOrderComponents`).

Together with the deterministic ZHD removal, the estimated ZWD/gradient states, and the uncombined slant-iono state, these models populate every atmospheric term of the undifferenced observation equation; the differencing operators that subsequently eliminate the receiver-common pieces of them are the subject of Section 5.

9 Site Displacement and Geophysical Corrections

The receiver coordinate that enters Equation (7) and Equation (6) of the Foundations document is not the instantaneous crust position: it is the conventional, tide-free station position to which the IGS/ITRF products are referenced, displaced by a sum of *a priori* geophysical corrections. Each correction is a deterministic, modelled 3D displacement $\boldsymbol{d}(t)$ of the antenna reference point (the position state, which the engine reduces from the RINEX monument through the header eccentricity — Section 6.2). None is estimated; all are computed from the (cm-grade) a priori position, the ephemerides of the Sun and Moon, and Earth-orientation parameters, then projected onto the line of sight and folded into the modelled range. This document derives those models as the *intended* IERS 2010 convention [18], so that it serves as the oracle that resolves the sign and phase questions the implementation has surfaced.

Write the corrected receiver position as

$$\boldsymbol{r} = \boldsymbol{r}_0 + \boldsymbol{d}_{\mathrm{SET}} + \boldsymbol{d}_{\mathrm{OTL}} + \boldsymbol{d}_{\mathrm{PT}} + \boldsymbol{d}_{\mathrm{OPT}} + \boldsymbol{d}_{\mathrm{APL}}, \tag{97}$$

with $\boldsymbol{r}_0$ the conventional a priori position and each $\boldsymbol{d}$ a millimetre-to-decimetre ECEF vector. The five terms are the solid Earth tide ($\boldsymbol{d}_{\mathrm{SET}}$, the largest, up to ~ 30 cm radial), ocean tidal loading ($\boldsymbol{d}_{\mathrm{OTL}}$, coastal cm), the rotational-deformation (solid) pole tide ($\boldsymbol{d}_{\mathrm{PT}}$, sub-cm to cm secular-plus-Chandler), the ocean pole tide ($\boldsymbol{d}_{\mathrm{OPT}}$, sub-mm to ~ 2 mm), and atmospheric pressure loading ($\boldsymbol{d}_{\mathrm{APL}}$, 2–20 mm). The last is modelled only in its sun-synchronous S_1/S_2 thermal-tide part; the non-tidal (synoptic) component remains a known gap (Section 9.6). All five are computed in `src/core/corrections/GnssCorrections.cpp` and `src/core/corrections/OceanTideModel.cpp`; the LOS projection and summation into the modelled range live in `src/core/model/ModelGeometry.cpp` and `src/core/model/PppModel.cpp`. Each term carries a config flag (`enable_solid_earth_tide`, `enable_ocean_tide_loading`, `enable_pole_tide`, `enable_ocean_pole_tide`, `enable_atmospheric_loading`); with a flag off, that term contributes a zero vector and the result reduces to the uncorrected position.

9.1 Sign convention and projection into the range

Every displacement is projected onto the unit line of sight before it enters the modelled range. Let $\boldsymbol{e} = (\boldsymbol{r}^i - \boldsymbol{r})/\|\boldsymbol{r}^i - \boldsymbol{r}\|$ be the receiver-to-satellite unit vector (the convention of Equation (7); see `ModelGeometry.cpp` where `los = (sat_pos -rx_pos)/range`). A site displacement $\boldsymbol{d}$ changes the geometric range by the component of $\boldsymbol{d}$ *along* the receiver-to-satellite direction with a sign that shortens the range when the antenna moves *toward* the satellite. Concretely the scalar correction summed into the modelled range is

$$\boxed{\delta\rho \;=\; \boldsymbol{d}\cdot(-\boldsymbol{e}) \;=\; \boldsymbol{d}\cdot\frac{\boldsymbol{r}-\boldsymbol{r}^i}{\|\boldsymbol{r}-\boldsymbol{r}^i\|},} \tag{98}$$

so a displacement that pushes the receiver *toward* the satellite (a positive component along $\boldsymbol{e}$) yields $\delta\rho < 0$ and shortens the modelled range, as required. The ocean-loading, atmospheric-loading, and pole-tide projections use exactly this form (`Geometry::computeSatelliteGeometry`: `otl_corr = otl_ecef.dot(-los)`, `apl_corr = apl_ecef.dot(-los)`; `PppModel::computePoleTideProjection`: `pole_tide_ecef.dot(-los)`), and the modelled range adds them in (`PppModel::computeBaseModeledValue` return sums `+ otl_corr + apl_corr + solid_tide` and the pair processors add `+ pole_tide_corr`). The solid-Earth-tide helper performs its own projection internally with the algebraically identical vector $(\boldsymbol{r}-\boldsymbol{r}^i)/\|\boldsymbol{r}-\boldsymbol{r}^i\|$ (`GnssCorrections.cpp`, `computeSolidEarthTide`: `los = (receiver_pos -sat_pos)/rho_norm`, then `disp.dot(los)`), so all four tide terms share one consistent sign. Throughout this document we therefore give each model as a 3D ECEF (or local) displacement $\boldsymbol{d}$ and leave Equation (98) implicit.

The Sun and Moon positions $\boldsymbol{r}^{\odot}, \boldsymbol{r}^{\mathrm{m}}$ on which the body tides depend, and the ECI↔ECEF rotation and Earth-orientation parameters they require, are produced by `src/core/model/Planetary.cpp` and are documented in Section 6; here we take $\boldsymbol{r}^{\odot}, \boldsymbol{r}^{\mathrm{m}}$ as given geocentric ECEF vectors.

9.2 Solid Earth tide (IERS 2010, Steps 1 + 2)

The solid Earth tide is the elastic deformation of the whole Earth under the lunisolar tide-generating potential. The implementation is a faithful port of the IERS Conventions `dehanttideinel` routine [18], evaluated in two steps: a time-domain Step 1 from the instantaneous Sun/Moon geometry, plus a frequency-domain Step 2 that corrects the small handful of constituents whose Love/Shida numbers depart from the nominal degree-2 values. **This entire Step 1+2 block is oracle-validated**: it reproduces the official IERS `dehanttideinel` test vector and the in-tree POINT reference

to sub-millimetre, cross-checked by `scripts/solid_tide_reference.py` (`GnssCorrections.cpp`, `computeSolidEarthTideDisplacement` and the `SolidEarthTideTest` suite). The permanent (zero-frequency) tide is *retained*, so the corrected coordinate is conventional tide-free, matching IGS/ITRF (see the routine's header comment on the retained zero-frequency tide).

9.2.1 Step 1 — in-phase degree 2 and 3

For each tide-raising body $b \in \{\odot, \mathrm{m}\}$ at geocentric position $\boldsymbol{r}^b$ ($r_b = \|\boldsymbol{r}^b\|$) and the station at $\boldsymbol{r}_0$ ($r = \|\boldsymbol{r}_0\|$, geocentric latitude φ via direction cosines), the degree-2 in-phase displacement is built from the cosine of the body zenith angle $\cos Z_b = \hat{\boldsymbol{r}}_0 \cdot \hat{\boldsymbol{r}}^b$ and a body scale factor

$$\mathrm{fac}_b \;=\; \frac{M_b}{M_\oplus}\, R_e \left(\frac{R_e}{r_b}\right)^3, \tag{99}$$

with $R_e = 6378136.55$ m and mass ratios $M_\odot/M_\oplus = 332945.943062$, $M_\mathrm{m}/M_\oplus = 0.012300034$ (`GnssCorrections.cpp` tide-helper constants `kTideRe`/`kTideMassRatioSun`/`kTideMassRatioMoon`). The radial and tangential weights are

$$p_2^b = 3\Big(\tfrac{h_2}{2} - l_2\Big)\cos^2 Z_b - \tfrac{h_2}{2}, \tag{100}$$

$$x_2^b = 3\, l_2 \cos Z_b, \tag{101}$$

(`GnssCorrections.cpp`, `computeSolidEarthTideDisplacement`), where h_2, l_2 are the *latitude-dependent* degree-2 Love and Shida numbers of IERS Eq. 7.2,

$$h_2 = 0.6078 - 0.0006\,(1 - \tfrac{3}{2}\cos^2\varphi), \qquad l_2 = 0.0847 + 0.0002\,(1 - \tfrac{3}{2}\cos^2\varphi) \tag{102}$$

(`GnssCorrections.cpp`, `computeSolidEarthTideDisplacement`). An analogous degree-3 term uses fixed $h_3 = 0.292$, $l_3 = 0.015$ (`kTideH3`/`kTideL3`) and $\mathrm{fac}_b^{(3)} = \mathrm{fac}_b\,(R_e/r_b)$ (same routine). The Step-1 in-phase displacement sums the radial weight along the station unit vector and the tangential weight along each body's unit vector:

$$\boxed{\boldsymbol{d}^{(1)}_{\mathrm{SET}} = \sum_{b\in\{\odot,\mathrm{m}\}} \mathrm{fac}_b\Big(x_2^b\,\hat{\boldsymbol{r}}^b + p_2^b\,\hat{\boldsymbol{r}}_0\Big) \;+\; (\text{degree-3 analogue}),} \tag{103}$$

matching the per-component accumulation in `computeSolidEarthTideDisplacement`.

9.2.2 Step 1 — out-of-phase anelasticity and the L^1 term

Mantle anelasticity makes the diurnal and semidiurnal degree-2 Love numbers complex; the small imaginary parts produce out-of-phase displacements `ST1IDIU` (diurnal, $\delta h = -0.0025$, $\delta l = -0.0007$) and `ST1ISEM` (semidiurnal, $\delta h = -0.0022$, $\delta l = -0.0007$) (`GnssCorrections.cpp`, helpers `st1Diurnal`, `st1SemiD`). The latitude (L^1) dependence of the Love numbers contributes a horizontal-only correction `ST1L1` with $l^{(1)}_{\mathrm{diu}} = 0.0012$, $l^{(1)}_{\mathrm{sem}} = 0.0024$ (helper `st1L1`). These three helpers return local radial/north/east displacements rotated into ECEF and are added to Equation (103) inside `computeSolidEarthTideDisplacement`; their detailed trigonometric forms follow the IERS reference verbatim and are not reproduced here.

9.2.3 Step 2 — frequency dependence (Doodson tables)

Step 2 corrects the constituents whose actual Love numbers differ from the nominal degree-2 value, summing harmonic terms keyed by the Doodson fundamental arguments. The astronomical arguments τ, s, h, p, N', p_s (lunar time, Moon/Sun mean longitudes, lunar perigee, negative node, solar perigee) are polynomials in Julian centuries T from J2000, with the 15°/hr Earth-rotation term entering τ through the fractional UT hour H:

$$s = 218.31664563^\circ + 481267.88194\,T - 0.0014663889\,T^2 + \ldots, \tag{104}$$

$$\tau = 15^\circ H + 280.4606184^\circ + 36000.7700536\,T + \ldots - s, \tag{105}$$

with the remaining h, p, N', p_s polynomials given to fourth order in `GnssCorrections.cpp` (`computeAngles`). For each table line the phase is $\theta = \tau + n_s s + n_h h + n_p p + n_{N'} N' + n_{p_s} p_s (+\,\lambda$ for diurnal) and the in-/out-of-phase radial and tangential amplitudes (millimetres) are applied with a latitude weight.

Diurnal band (Table 7.3a). Thirty-one lines, dominated by K_1 and P_1 (`GnssCorrections.cpp`, `st2Diurnal`). The displacement for one line is

$$\begin{aligned} \delta R &= (A_R^{\text{in}} \sin\theta + A_R^{\text{out}} \cos\theta)\, 2 \sin\varphi \cos\varphi, \\ \delta N &= (A_T^{\text{in}} \sin\theta + A_T^{\text{out}} \cos\theta)\, \cos 2\varphi, \\ \delta E &= (A_T^{\text{in}} \cos\theta + A_T^{\text{out}} \sin\theta)\, \sin\varphi, \end{aligned} \tag{106}$$

(`st2Diurnal`), summed over the table and rotated radial/north/east → ECEF. The reference amplitudes (mm) include K_1 : (12.00, −0.80, −0.67, −0.03) and P_1 : (−1.23, −0.07, 0.06, 0.01) in the order $(A_R^{\text{in}}, A_R^{\text{out}}, A_T^{\text{in}}, A_T^{\text{out}})$ (the `st2Diurnal` table `T`).

Long-period band (Table 7.3b). Five zonal lines, with no longitude or τ dependence and no east component (`GnssCorrections.cpp`, `st2Lon`). The radial weight is the zonal Legendre form $(3\sin^2\varphi - 1)/2$ and the north weight is $\sin 2\varphi$:

$$\delta R = (A_R^{\text{in}} \cos\theta + A_R^{\text{out}} \sin\theta)\, \frac{3\sin^2\varphi - 1}{2}, \qquad \delta N = (A_T^{\text{in}} \cos\theta + A_T^{\text{out}} \sin\theta)\, \sin 2\varphi, \tag{107}$$

(`st2Lon`). The full solid-tide displacement is $\boldsymbol{d}_{\text{SET}} = \boldsymbol{d}^{(1)}_{\text{SET}}$ + ST1IDIU + ST1ISEM + ST1L1 + STEP2DIU + STEP2LON (`computeSolidEarthTideDisplacement`).

9.3 Ocean tidal loading (OTL)

Ocean tidal loading is the elastic response of the crust to the redistributed mass of the ocean tides; it is significant (∼cm) near coasts and is synthesised as a harmonic sum over the eleven principal constituents M2, S2, N2, K2, K1, O1, P1, Q1, Mf, Mm, Ssa (`OceanTideModel.hpp`, `OTL_CONSTITUENT_NAMES`). Each constituent k contributes, on each local component $c \in \{\text{up}, \text{west}, \text{south}\}$,

$$\boxed{dr_c(t) \;=\; \sum_{k=1}^{11} A_{c,k}\, \cos\big(\chi_k(t) - \phi_{c,k}\big),} \tag{108}$$

where $A_{c,k}$ is the station amplitude, $\phi_{c,k}$ the Greenwich phase lag, and $\chi_k(t)$ the astronomical argument (the eleven-constituent accumulation of `OceanTideModel::computeDisplacement`). The internal accumulation is in the (up, west, south) frame; the conversion to ENU is $(E, N, U) = (-\text{west}, -\text{south}, \text{up})$, then ENU→ECEF (same routine).

9.3.1 Astronomical argument and the Doodson–Warburg phase bias

The argument χ_k is assembled from Doodson multipliers $[\,\tau, s, h, p, N', p_s\,]$ acting on the variables $[\,\mathrm{GMST}+\pi,\ s,\ h,\ p,\ N',\ p_s\,]$, themselves derived from the Delaunay fundamental arguments $s = F+\Omega,\ h = s-D,\ p = s-l,\ N' = -\Omega$ (`OceanTideModel::computeTidalArguments`). Critically, the canonical tide argument to which both the BLQ Greenwich phase lags and the EOT20/FES complex coefficients are referenced carries a *species-dependent Doodson–Warburg phase bias*: a quarter cycle ($\pm 90°$) for the diurnal band, because the Cartwright–Tayler–Edden tide-generating potential refers to a *sine* rather than a cosine when $(\mathrm{degree}+n_\tau)$ is odd. The intended bias column (cycles) is

M2	S2	N2	K2	K1	O1	P1	Q1	Mf	Mm	Ssa
0	0	0	0	+0.25	−0.25	−0.25	−0.25	0	0	0

(109)

applied as $\chi_k \mathrel{+}= 2\pi\,(\mathrm{bias}_k)$ (the `CONSTITUENT_PHASE_BIAS_CYCLES` step of `OceanTideModel::computeTidalArguments`; the table itself in `OceanTideModel.hpp`). **Omitting this bias phases every diurnal constituent (K1/O1/P1/Q1) by a full** $90°$, mismodelling the entire diurnal OTL contribution — this is the intended oracle resolution of the OTL-diurnal-phase question. The bias *must* be present and carry the species signs of Equation (109).

9.3.2 Coefficient sources: SIF, BLQ, and the EOT20 grid

Per-station amplitudes/phases are resolved by a four-way precedence (`Geometry::loadData`, OTL coefficient block): a Station Information File (SIF) → a station BLQ block → the global EOT20 grid → a safe zero. Highest fidelity, when `enableStationInfo()` is set and the SIF carries an entry for the RINEX marker, is a set of per-station three-component FES2014b coefficients. The SIF stores Up/East/North amplitude+phase, whereas the model's internal accumulation frame (Equation (108)) is Up/West/South; since East = −West and South = −North, the horizontals keep their amplitude and shift phase by $+180°$ (`d.phase_west = c.east_phase + 180`, `d.phase_south = c.north_phase + 180`), Up passing through unchanged — the same class of phase-convention resolution as the Doodson–Warburg bias above. When no SIF entry exists the coefficients come from a station BLQ block or, failing that, are synthesised by bilinear interpolation of the global EOT20 complex load grid at the RINEX header *marker* position (`RinexParser::getApproxPositionXyz`), not at the ARP-referenced state — immaterial at this grid resolution, but the two points are distinct in the code (radial only, unlike the three-component SIF); with no source at all the displacement is zero. The EOT20 netCDF packs the real and imaginary load coefficients as `int16` with a `scale_factor`/`add_offset`; the netCDF C library does *not* auto-apply these, so they are unpacked manually as $v = v_{\mathrm{int16}} \cdot \mathrm{scale} + \mathrm{offset}$ (the `scale`$\approx 5.34\times10^{-5}$ m/LSB; omitting it inflates amplitudes by $\sim$20000×) (`OceanTideModel::parseEot20Load`: its `getScaleOffset` lambda and the manual unpack that follows). The complex load $Z = \mathrm{Re} + i\,\mathrm{Im}$ yields the real signal $h(t) = \mathrm{Re}\{Z\,e^{i\chi}\} = A\cos(\chi-\phi)$ with

$$A = \sqrt{\mathrm{Re}^2+\mathrm{Im}^2}, \qquad \phi = \mathrm{atan2}(-\mathrm{Im},\ \mathrm{Re}) \tag{110}$$

(ϕ in BLQ format being the Greenwich phase lag in degrees; same routine). Because EOT20 orders its 17 constituents differently from the BLQ convention, the synthesis remaps the BLQ slot k to the EOT20 index via $\{4, 13, 8, 3, 2, 9, 10, 11, 6, 7, 15\}$ (the `EOT20_INDICES_FOR_BLQ` table in `OceanTideModel::parseEot20Load`).

9.4 Pole tide (rotational deformation, IERS 7.1.4)

The (solid) pole tide is the elastic deformation driven by the centrifugal potential of polar motion: as the instantaneous rotation pole wanders relative to its secular mean, the bulge it raises moves with it. The wobble is the polar-motion offset from the IERS 2018 secular mean pole. In milliarcseconds, with t_y the decimal year,

$$\bar{x}_p = 55.0 + 1.677\,(t_y - 2000), \qquad \bar{y}_p = 320.5 + 3.460\,(t_y - 2000) \tag{111}$$

(`GnssCorrections.cpp`, `computePoleTideCorrection`); this linear model is the convention to which the post-2018 IGS products consumed by the engine are referenced. The wobble components (arcseconds) are

$$m_1 = x_p - \bar{x}_p, \qquad m_2 = -\big(y_p - \bar{y}_p\big), \tag{112}$$

with x_p, y_p the polar-motion values linearly interpolated from the ERP table at the query epoch (`computePoleTideCorrection`; the $\bar{x}_p, \bar{y}_p$ millarcsecond means are scaled to arcseconds in the difference). **The wobble must be carried in arcseconds, not radians**: the displacement coefficients below are metre-valued per arcsecond, so forming m_1, m_2 in radians underscales the whole pole tide by $\sim 4.85 \times 10^{-6}$ (a previously-latent bug noted in the `computePoleTideCorrection` wobble comment).

IERS 2010 Eq. 7.24 gives the rotational-deformation displacement in colatitude $\theta = 90^\circ - \varphi$; substituting geodetic latitude ($\sin 2\theta = \sin 2\varphi$, $\cos 2\theta = -\cos 2\varphi$, $\cos\theta = \sin\varphi$, north $= -S_\theta$) yields the radial, north, and east components (metres):

$$\boxed{\begin{aligned} \boldsymbol{d}^R_{\mathrm{PT}} &= -0.033\,\sin 2\varphi\,\big(m_1\cos\lambda + m_2\sin\lambda\big), \\ \boldsymbol{d}^N_{\mathrm{PT}} &= -0.009\,\cos 2\varphi\,\big(m_1\cos\lambda + m_2\sin\lambda\big), \\ \boldsymbol{d}^E_{\mathrm{PT}} &= +0.009\,\sin\varphi\,\big(m_1\sin\lambda - m_2\cos\lambda\big), \end{aligned}} \tag{113}$$

(`computePoleTideCorrection`). **The east factor is $\sin\varphi$ ($= \cos\theta$), not $\cos\varphi$**: this is the intended oracle resolution of the pole-tide east-term question. An east factor of $\cos\varphi$ is internally inconsistent with the geodetic-φ radial/north terms ($\sim 1.48\times$ scale error at mid-latitude, and the wrong latitude dependence — vanishing at the equator instead of the pole). The local (R, N, E) vector is rotated into ECEF at the end of `computePoleTideCorrection`.

9.5 Ocean pole tide (Desai 2002, IERS 7.1.5)

Polar motion also redistributes ocean mass (the ocean pole tide), whose load deforms the crust by a further sub-millimetre to ~2 mm. It adds onto Equation (113) when the station's six Desai grid load coefficients are available (the `has_opt_coeffs_` branch of `computePoleTideCorrection`). Following Desai (2002) [4], the same wobble m_1, m_2 drives a complex pole-tide coefficient

$$c^R = m_1\,\gamma_2^R + m_2\,\gamma_2^I, \tag{114}$$

$$c^I = m_2\,\gamma_2^R - m_1\,\gamma_2^I, \tag{115}$$

with the IERS/Desai load Love combination $\gamma_2^R = 0.6870$, $\gamma_2^I = 0.0036$ (`kOptGammaReal`/`kOptGammaImag`). Each of the six per-station coefficients u_c^R, u_c^I (north, east, radial real and imaginary parts), bilinearly interpolated from the Desai grid, gives the local displacement

$$\boxed{\boldsymbol{d}^c_{\mathrm{OPT}} = \kappa\,\big(c^R\,u_c^R + c^I\,u_c^I\big), \qquad c \in \{E, N, U\},} \tag{116}$$

(`GnssCorrections.cpp`, `computeOceanPoleTideDisplacement`). The scale κ folds in the IERS load-deformation constants and the arcsec→rad conversion (so that the wobble may stay in arcseconds, consistent with Equation (112)):

$$\kappa = \frac{4\pi\, G\, a\, \rho_w\, H_p}{3\, g_E}\, \frac{\pi}{648000}, \qquad H_p = \sqrt{\tfrac{8\pi}{15}}\, \frac{\Omega^2 a^4}{GM_\oplus}, \tag{117}$$

with $\rho_w = 1025\ \mathrm{kg\,m^{-3}}$, $a = 6378136.6$ m, $g_E = 9.7803278\ \mathrm{m\,s^{-2}}$, $\Omega = 7.292115 \times 10^{-5}\ \mathrm{rad\,s^{-1}}$, $G = 6.67428 \times 10^{-11}$, $GM_\oplus = 3.986004418 \times 10^{14}$ (the `kOpt*` constants; κ assembled in `oceanPoleTideKappa`). The whole ocean-pole-tide path reproduces the NGS M-PAGES implementation and the independent Python oracle `scripts/ocean_pole_tide_reference.py` to $\sim 10^{-12}$ (see the constants' header comment).

9.6 Atmospheric pressure loading (S1/S2 thermal tide)

Atmospheric pressure loading (APL) — the crustal response to surface-pressure redistribution — is $\sim$2–20 mm, predominantly vertical. It splits into a sun-synchronous, thermally driven S_1/S_2 tidal part (diurnal and semidiurnal, fully predictable from local solar time) and a non-tidal (synoptic) part driven by weather systems. The engine models the *tidal* part; the non-tidal part remains a genuine gap (see the honesty note below).

The thermal-tide displacement uses per-station diurnal (S_1) and semidiurnal (S_2) harmonic coefficients evaluated against the local-solar-time argument $\Theta_n = n\,(2\pi\,\mathrm{UT_{frac}} + \lambda)$, $n \in \{1, 2\}$, with $\mathrm{UT_{frac}}$ the UT fraction of day and λ the station longitude (`GnssCorrections.cpp`, `computeAtmosphericTideDisplacement`):

$$\boxed{\boldsymbol{d}^{c}_{\mathrm{APL}} = a^{\cos}_{c,S_1} \cos\Theta_1 + a^{\sin}_{c,S_1} \sin\Theta_1 + a^{\cos}_{c,S_2} \cos\Theta_2 + a^{\sin}_{c,S_2} \sin\Theta_2, \qquad c \in \{E, N, U\},} \tag{118}$$

formed in local ENU and rotated to ECEF, with a sub-cm (1 cm) sanity guard. The twelve coefficients per station are laid out $[\,a^{\cos}_E, a^{\sin}_E, a^{\cos}_N, a^{\sin}_N, a^{\cos}_U, a^{\sin}_U\,]$ for S_1 (indices 0–5) then the same six for S_2 (indices 6–11), bilinearly interpolated from a Ray–Ponte-style S_1/S_2 load grid (`loadAtmosphericTideFromGrid`) at the same RINEX header marker position as the OTL grid (Section 9.3.2); the per-epoch displacement itself is evaluated at the ARP-referenced position state. The term is gated by `model_corrections.enable_atmospheric_loading` (default `false`) and reads its grid from `files.atm_tide_grid_path`, defaulting to `<data>/static/apl/s1_s2_load_cm.dat` (a PROVISIONAL in-code filename, to be pinned to the supplied product); the coefficients are loaded in `Geometry::loadData` and projected to `apl_corr` alongside `otl_corr` in `computeSatelliteGeometry`. With the flag off, or with no usable grid, $\boldsymbol{d}_{\mathrm{APL}} = \mathbf{0}$ in Equation (97) and runs are byte-identical.

Honesty note. In the spirit of Section 17's flag of the iono-not-SSR gap: only the sun-synchronous S_1/S_2 tidal APL is modelled. The non-tidal (synoptic) pressure loading — the larger, weather-driven part, requiring a surface-pressure field convolved with a Green's function or a gridded non-tidal loading product — is *not* modelled and is the leading candidate for the next site-displacement term.

9.7 Integration into the measurement model

Operationally, the per-epoch geometry builder evaluates each enabled displacement, projects it via Equation (98), and the modelled-range assembly adds the scalar corrections. In `Geometry::computeSatelliteGeometry` the OTL ECEF displacement is formed and projected to `otl_corr`, the

S_1/S_2 atmospheric tide to `apl_corr`, and the solid tide is evaluated against the cached Sun/Moon positions and already returns its LOS-projected scalar. In `PppModel` the pole tide (and, when enabled, the ocean pole tide additively inside it) is projected by `computePoleTideProjection` and the base modelled value (`computeBaseModeledValue`) sums `+ otl_corr + apl_corr + solid_tide` while the pair processors add `+ pole_tide_corr`. The resulting corrected geometric range is precisely the $\rho(\boldsymbol{r})$ that enters Equation (6) once $\boldsymbol{r}$ is replaced by Equation (97). Because each term is gated by its own config flag and contributes a zero vector when off, the differenced observables of Section 5.1 and Section 5.3 formulations inherit whatever residual of these corrections survives differencing: solid Earth tide and pole tide are highly spatially correlated and largely cancel over short baselines, whereas ocean loading and the (largely unmodelled non-tidal) atmospheric loading do not cancel and remain the dominant differential site-displacement error on long baselines.

10 Observation Model Physics: Antenna, Attitude, Relativity, and Range Corrections

The corrections in this document are the *deterministic* part of the observation model that lies between pure geometry and the atmosphere on one side, and the streamed SSR overlay on the other. Section 4 Equation (4) and Equation (5) introduce the satellite and receiver hardware biases B_s^i, b_s^i, $B_{r,s}$, $b_{r,s}$ *abstractly*, and Section 4.3 carries the geometric range ρ_r^i and its linearisation; this document supplies every *other* deterministic term that maps the raw measurement to the modelled range: the antenna phase-centre geometry, the satellite attitude frame on which it depends, the carrier-phase wind-up, the relativistic clock and propagation corrections, the Sagnac (Earth-rotation) frame correction, and the deterministic realisation of the bias terms through broadcast group delays and the precise `.BIA` product. The streamed SSR overlay that refines the *same* terms in real time is the subject of Section 17; the two mechanisms are related but distinct, and the differences (frames, sign conventions, gating) are flagged where they matter.

Every term below is assembled into a single modelled observable in `model/PppModel.cpp` (`PppModel::computeBaseModeledValue`, the single shared helper called by both the iono-free-pair and the uncombined observation paths), summed onto the geometric range that `model/ModelGeometry.cpp` produces. The dominant structure of that sum, for the code observable, is

$$\hat{P}_{r,s}^i = \rho_r^i \;-\; c\,\mathrm{d}t^i \;-\; c\,\delta t_{\mathrm{rel}} \;+\; T_r^i \;+\; A_{r,s}^i \;+\; B_{\mathrm{grp}} \;+\; S_r^i \;+\; d_{\mathrm{atm}} \;+\; d_{\mathrm{tide}}, \tag{119}$$

where $A_{r,s}^i$ is the total antenna correction (Section 10.2), δt_{rel} the periodic relativistic clock term (Section 10.4), S_r^i the Shapiro delay (Section 10.5), B_{grp} the deterministic code/phase bias (Section 10.7), and d_{atm}, d_{tide} the loading/site-displacement terms documented in Section 9. The Sagnac correction (Section 10.6) is folded into ρ_r^i itself, not added separately, for reasons made precise below. The signs in (119) are taken verbatim from the return statement of `PppModel::computeBaseModeledValue` (`model/PppModel.cpp`): the satellite clock and relativistic terms *subtract*, the antenna, Shapiro, loading and tide terms *add*.

10.1 Satellite attitude: the body frame

The satellite antenna phase-centre offset, the antenna phase-centre variation, and the carrier wind-up are all defined in the *spacecraft body frame*, so that frame must be constructed first. The engine uses the IGS/ANTEX convention: a right-handed orthonormal triad $(\hat{\boldsymbol{x}}_i, \hat{\boldsymbol{y}}_i, \hat{\boldsymbol{k}}_i)$ with $+Z$ toward the Earth's centre (nadir / antenna boresight), $+X$ in the Sun–satellite–Earth plane pointing toward the Sun, and $+Y$ completing the triad along the solar-panel (cross-track) axis (`model/SatAttitude.cpp`, `model/Antenna.cpp`).

Nominal yaw-steering. With $\boldsymbol{r}^i$ the satellite ECEF position and $\hat{\boldsymbol{e}}_\odot = (\boldsymbol{r}_\odot - \boldsymbol{r}^i)/\|\boldsymbol{r}_\odot - \boldsymbol{r}^i\|$ the unit direction to the Sun, the nadir axis and the cross-track axis are

$$\hat{\boldsymbol{k}}_i = -\frac{\boldsymbol{r}^i}{\|\boldsymbol{r}^i\|}, \qquad \hat{\boldsymbol{y}}_i = \frac{\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot}{\|\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot\|}, \qquad \hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i, \tag{120}$$

the exact operand ordering of `SatAttitude::compute` (`model/SatAttitude.cpp`), to which the antenna path delegates (`Antenna::getAntennaCorrection`) rather than rebuilding the frame inline; the one genuine inline duplicate of this construction lives in the wind-up path and is flagged in Section 10.3 below. Forming $\hat{\boldsymbol{y}}_i$ first and then $\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i$ is algebraically the Gram–Schmidt orthogonalisation of the Sun direction against nadir: by the $\boldsymbol{a} \times (\boldsymbol{b} \times \boldsymbol{c})$ identity,

$$\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i = (\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot) \times \hat{\boldsymbol{k}}_i = \hat{\boldsymbol{e}}_\odot - \hat{\boldsymbol{k}}_i\,(\hat{\boldsymbol{k}}_i \cdot \hat{\boldsymbol{e}}_\odot), \tag{121}$$

i.e. $\hat{\boldsymbol{x}}_i$ is the component of $\hat{\boldsymbol{e}}_\odot$ orthogonal to nadir, normalised — the $+X$ axis points half-toward the Sun, as required by the ANTEX convention. The body-frame matrix collecting the three columns,

$$\boldsymbol{F} = \big[\, \hat{\boldsymbol{x}}_i \;\; \hat{\boldsymbol{y}}_i \;\; \hat{\boldsymbol{k}}_i \,\big], \tag{122}$$

is orthonormal, so its inverse is its transpose — the property exploited below to rotate a body-frame offset into ECEF.

Sun elevation and the mode switch. For non-nominal attitude (eclipse seasons, the noon/midnight yaw turn) the relevant geometric quantity is the Sun's elevation above the orbital plane, β. With the orbit angular-momentum (normal) unit vector $\hat{\boldsymbol{n}} = (\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i)/\|\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i\|$,

$$\boxed{\beta = \arcsin\big(\hat{\boldsymbol{n}} \cdot \hat{\boldsymbol{e}}_\odot\big), \qquad \hat{\boldsymbol{n}} = \frac{\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i}{\|\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i\|},} \tag{123}$$

signed by the projection of the Sun direction onto the $+(\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i)$ normal (`SatAttitude::computeBeta`, `model/SatAttitude.cpp`; the sign is noted in-code as unvalidated against an external attitude product). The mode selector (`SatAttitude::determineMode`) keeps GPS, GLONASS and Galileo in yaw-steering unconditionally, and switches BeiDou and QZSS to orbit-normal attitude below a constellation-specific β threshold (4.0° BeiDou, 20.0° QZSS, `model/SatAttitude.cpp`). In orbit-normal mode the $+Y$ axis is pinned to the orbit normal, $\hat{\boldsymbol{y}}_i = \hat{\boldsymbol{n}}$, and the triad is re-orthogonalised $\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i$, $\hat{\boldsymbol{y}}_i = \hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{x}}_i$. The legacy default path additionally forces the hard-coded BeiDou-2 GEO PRN list (C01–C05, C59, C60) into orbit-normal mode irrespective of β; the β-driven table is active only when `enableOrbitNormalAttitude()` is set, and is constructed so the flag-off result is bit-identical to the historical inline frame (`model/Antenna.cpp`, `model/SatAttitude.cpp`).

Eclipse gating. The body frame above presumes a known yaw angle, which is exactly what fails during Earth-shadow passes: the actual spacecraft yaw during eclipse and the post-shadow recovery is block-specific and not modelled here, so the antenna PCO/PCV and the wind-up are unreliable for an eclipsing satellite. When the user opts in (`processing.exclude_eclipsing_satellites`), the satellite is dropped at the geometry stage by a *cylindrical umbra* test (`Planetary::isSatelliteEclipsing`, `model/Planetary.cpp`, called from `model/ModelGeometry.cpp`). With $\hat{\boldsymbol{u}}_\odot = \boldsymbol{r}_\odot/\|\boldsymbol{r}_\odot\|$ the Earth-to-Sun direction, the satellite's along-axis and perpendicular distances are $s_\parallel = \boldsymbol{r}^i \cdot \hat{\boldsymbol{u}}_\odot$ and $\boldsymbol{s}_\perp = \boldsymbol{r}^i - s_\parallel\,\hat{\boldsymbol{u}}_\odot$, and the predicate is

$$\text{eclipsed} \iff \big(s_\parallel < 0\big) \;\wedge\; \big(\|\boldsymbol{s}_\perp\| < \kappa\, R_\oplus\big), \tag{124}$$

i.e. the satellite is on the anti-Sun side *and* inside the shadow cylinder of radius $\kappa R_\oplus$, with $R_\oplus = 6\,371\,230\,\mathrm{m}$ (`common/Constants.hpp`) and κ the user shadow scale (`processing.eclipse_shadow_scale`, default unity; cited rather than reproduced as a tuning knob). This is a deliberately conservative geometric umbra, not a photometric penumbra model.

10.2 Antenna phase-centre corrections

The range measurement is realised between the satellite and receiver *antenna phase centres*, but the estimated states refer to the satellite centre of mass and the receiver antenna reference point (ARP): the ANTEX receiver PCO/PCV below are ARP-referenced, so the position state must be too. The antenna correction $A^i_{r,s}$ reconciles the two, summing four geometric terms plus the wind-up:

$$\boxed{A^i_{r,s} \;=\; -\,\delta^i_{\mathrm{PCO}} \;+\; \delta^i_{\mathrm{PCV}} \;+\; \delta^r_{\mathrm{PCO}} \;+\; \delta^r_{\mathrm{PCV}} \;+\; \varphi_{\mathrm{wu}},} \tag{125}$$

the exact accumulation of `Antenna::getAntennaCorrection` (`model/Antenna.cpp`). The line-of-sight used throughout is the *satellite-to-receiver* unit vector, $\boldsymbol{e} = (\boldsymbol{r}_r - \boldsymbol{r}^i)/\|\boldsymbol{r}_r - \boldsymbol{r}^i\|$ (`Antenna.cpp`; note this is the negative of the receiver-to-satellite $\boldsymbol{e}^i_r$ of Section 4.3 in Equation (7)). The sign conventions are derived once and reused for each term below.

Which coordinate is the ARP. The ARP frame asserted above holds for every position the engine derives from a RINEX *header*: the marker coordinate is reduced to the ARP through the header eccentricity before it seeds the state (`Coordinates::markerToArp`, packaged as `RinexParser::getArpPositionXyz`; Section 6.2). A coordinate supplied directly in the configuration is *not* reduced — it is used at face value, on the policy that the user owns its reference point. That exception is load-bearing in the network producer, whose station coordinates come from the configuration (`StationConfig::position_ecef`) and are held fixed, their position columns dropped from the normal equations: a monument-referenced coordinate there is not absorbed by a position state but survives into the residuals, and so into the estimated clocks, biases, and ionosphere.

Satellite PCO (subtracted). The satellite phase-centre offset $\boldsymbol{p}^i_{\mathrm{body}}$ is published in the body frame (millimetres in ANTEX). It is rotated to ECEF by the body-frame matrix — because $\boldsymbol{F}$ collects the body axes as *columns*, $\boldsymbol{F}$ itself maps body $\to$ ECEF (and the orthonormal $\boldsymbol{F}^{\mathrm{T}} = \boldsymbol{F}^{-1}$ maps ECEF $\to$ body) — and projected onto the LOS:

$$\delta^i_{\mathrm{PCO}} = \big(\boldsymbol{F}\,\boldsymbol{p}^i_{\mathrm{body}}\big)\cdot\boldsymbol{e}. \tag{126}$$

Because the antenna phase centre sits *nearer the receiver* than the centre of mass, the true range is shorter than the CoM range; the correction is therefore *subtracted* in (125) (`correction -= sat_pco_corr`, `model/Antenna.cpp`). One real-time exception omits this term entirely: when an applied SSR orbit correction is itself APC-referenced (the corrected orbit already points at the phase centre), the `skip_satellite_pco` path of `getAntennaCorrection` drops the satellite-PCO range term to avoid double-counting it, gated by `realtime.honor_ssr_reference_point` and default-off; the SSR reference-point dialect that drives it is documented with Section 17.

Satellite PCV (added). The satellite phase-centre variation is a function of the *nadir angle* — the angle between the boresight $\hat{\boldsymbol{k}}_i$ and the line of sight — and of the azimuth in the body frame. With $\cos\theta_{\mathrm{nad}} = \hat{\boldsymbol{k}}_i \cdot \boldsymbol{e}$,

$$\theta_{\mathrm{nad}} = \arccos\big(\hat{\boldsymbol{k}}_i\cdot\boldsymbol{e}\big), \qquad \alpha_{\mathrm{body}} = \mathrm{atan2}\big((\boldsymbol{F}^{\mathrm{T}}\boldsymbol{e})_y,\,(\boldsymbol{F}^{\mathrm{T}}\boldsymbol{e})_x\big), \tag{127}$$

and the PCV value is read by bilinear interpolation of the ANTEX grid over (zenith/nadir ring, azimuth sector), the look-up argument being $\theta_{\text{nad}} - \texttt{zen1}$ in grid units (`Antenna::interpolatePcv`, `model/Antenna.cpp`). The satellite PCV is *added* to the range correction.

Receiver PCO (added). The receiver PCO is published as a North/East/Up (NEU) triplet relative to the ARP. It is reordered to ENU, rotated to ECEF by the local ENU→ECEF rotation $\boldsymbol{R}_{\text{enu}}(\phi,\lambda)$ at the receiver geodetic latitude/longitude (`Coordinates::enuToEcefMatrix`), and projected on the LOS:

$$\delta^{r}_{\text{PCO}} = \Big(\boldsymbol{R}_{\text{enu}}(\phi,\lambda)\,\big[\,p_E \;\; p_N \;\; p_U\,\big]^{\mathrm{T}}\Big)\cdot\boldsymbol{e}, \qquad \big[\,p_E\,p_N\,p_U\,\big] = \big[\,p_{\text{NEU},2} \;\; p_{\text{NEU},1} \;\; p_{\text{NEU},3}\,\big], \tag{128}$$

where the index swap $(N,E,U)\to(E,N,U)$ is explicit in `model/Antenna.cpp`. It is *added* to the correction.

Receiver PCV (added). The receiver PCV is indexed by *zenith angle* $z = 90^\circ - \text{el}$ (and, optionally, azimuth corrected for the antenna mounting orientation when `enableRxAntennaOrientation()` is set), with the same bilinear interpolation over $z - \texttt{zen1}$ (`model/Antenna.cpp`). It is *added* to the correction.

ATX frequency fallback. ANTEX entries frequently omit the newest carriers (L5/E5a/B2a on older spacecraft, and many receiver antennas published before triple-frequency tracking). Rather than dropping the antenna correction to zero — which leaves a several-centimetre vertical bias — the engine walks a *nearest-carrier* substitution chain when the requested ANTEX frequency key is absent (`pickAtxFrequencyKey`, `model/Antenna.cpp`). The chains are ordered by carrier-frequency proximity, e.g. for GPS `G05` → {`G02`, `G01`} and for Galileo `E05` → {`E08`, `E07`, `E06`, `E01`}, since cross-frequency PCO drift on one physical antenna is typically below a centimetre. The PCO and PCV chains are walked *independently* (an antenna may carry the PCO but not the PCV at a given key). For the *satellite* antenna only the intra-system chain is permitted — borrowing another constellation's spacecraft entry would be meaningless. For the *receiver* antenna, after the intra-system chain is exhausted and only when `enable_receiver_cross_system_atx_fallback` is set, a *cross-system* nearest-GPS-band chain is allowed (`pickReceiverAtxFrequencyKey`), because the receiver phase pattern is a function of frequency on shared hardware: the load-bearing mappings are exact-carrier matches (`E01` → `G01`, `E05` → `G05` at 1575.42 and 1176.45 MHz), the remainder ∼10–70 MHz approximations (`model/Antenna.cpp`). Each substitution raises a coverage flag and emits a fallback log line; this is the same graceful-fallback policy used for biases rather than excluding the measurement.

10.3 Carrier-phase wind-up

A right-hand-circularly-polarised GNSS signal accrues a phase shift equal to the relative geometric rotation between the transmitting and receiving antenna dipoles as either rotates about the line of sight [28]. This wind-up is identical on every signal in cycles (hence a range term proportional to wavelength) and is computed in `GnssCorrections::computePhaseWindUp` (`corrections/GnssCorrections.cpp`). With $\boldsymbol{k}$ the unit line-of-sight, $(\hat{\boldsymbol{x}}_i,\hat{\boldsymbol{y}}_i)$ the satellite body $\pm X, +Y$ axes and $(\boldsymbol{x}_r,\boldsymbol{y}_r)$ the receiver local North/East axes, the *effective dipoles* are

$$\boldsymbol{D}_i = \hat{\boldsymbol{x}}_i - \boldsymbol{k}\,(\boldsymbol{k}\cdot\hat{\boldsymbol{x}}_i) + \boldsymbol{k}\times\hat{\boldsymbol{y}}_i, \qquad \boldsymbol{D}_r = \boldsymbol{x}_r - \boldsymbol{k}\,(\boldsymbol{k}\cdot\boldsymbol{x}_r) + \boldsymbol{k}\times\boldsymbol{y}_r, \tag{129}$$

the standard Wu construction — each dipole is the antenna X axis projected into the plane orthogonal to $\boldsymbol{k}$, plus the $\boldsymbol{k} \times Y$ quadrature term (`corrections/GnssCorrections.cpp`). The receiver North and East axes are built directly from the geodetic latitude ϕ and longitude λ, $\boldsymbol{x}_r = (-\sin\phi\cos\lambda,\ -\sin\phi\sin\lambda,\ \cos\phi)$ and $\boldsymbol{y}_r = (-\sin\lambda,\ \cos\lambda,\ 0)$ (`GnssCorrections.cpp`). The signed wind-up angle is the atan2 of the cross- and dot-products of the unit dipoles, which recovers the full $(-\pi, \pi]$ range and avoids the information loss of an arccos near 0 and π:

$$\boxed{\Delta\varphi = \operatorname{atan2}\Big(\boldsymbol{k} \cdot \big(\hat{\boldsymbol{D}}_i \times \hat{\boldsymbol{D}}_r\big),\ \hat{\boldsymbol{D}}_i \cdot \hat{\boldsymbol{D}}_r\Big), \qquad \varphi_{\mathrm{wu}} = \Delta\varphi\,\frac{\lambda_s}{2\pi},} \tag{130}$$

with $\hat{\boldsymbol{D}}_i, \hat{\boldsymbol{D}}_r$ the normalised dipoles and λ_s the signal wavelength (`corrections/GnssCorrections.cpp`); the $\boldsymbol{k}\cdot(\hat{\boldsymbol{D}}_i \times \hat{\boldsymbol{D}}_r)$ argument fixes the sign of the rotation about the line of sight. The result enters (125) as φ_{wu} on phase observables only (the code branch passes zero wavelength).

Fractional vs. accumulated modes. The raw atan2 returns the wind-up wrapped to a single cycle. In the default *fractional* mode this wrapped angle is used directly: the float ambiguity $N^i_{r,s}$ absorbs any integer-cycle component, so wrapping is harmless. For integer ambiguity resolution the ambiguity must stay a constant integer across the arc, so the engine offers an *accumulated* (unwrapped) mode (`config.accumulatePhaseWindUp()`) that unwraps each epoch relative to the previous stored angle, $\Delta\varphi_t = \Delta\varphi_{t-1} + \mathrm{wrap}_{2\pi}(\Delta\varphi_t - \Delta\varphi_{t-1})$, provided the inter-epoch gap is below the 300 s continuity threshold; a larger gap (or a backward time step) restarts the accumulator from the raw angle (`corrections/GnssCorrections.cpp`). This 300 s reset is the wind-up analogue of an ambiguity-arc boundary: a gap long enough to invalidate the unwrap reference is treated as a fresh arc.

Known frame duplication. The satellite body frame feeding the wind-up dipole $\boldsymbol{D}_i$ (129) is built *inline* inside `computePhaseWindUp` (`corrections/GnssCorrections.cpp`) rather than calling the shared `model/SatAttitude.cpp` helper used by the antenna path (120). The two constructions are deliberately bit-for-bit identical (same operand ordering, same BeiDou-2 GEO orbit-normal special case, same velocity fallback), and both source files note the duplication explicitly. It is flagged here because the invariant is a *maintenance* obligation: a change to one frame must be mirrored in the other or the antenna PCV and the wind-up would silently diverge on the same satellite.

10.4 Periodic relativistic clock correction

A clock on an eccentric orbit runs at a time-varying rate relative to a clock on the geoid; the constant and secular parts are absorbed into the broadcast/precise clock polynomial, but the *periodic* part driven by orbital eccentricity must be applied separately. The standard expression [23], in `GnssCorrections::computeRelativisticCorrection` (`corrections/GnssCorrections.cpp`), is

$$\boxed{\delta t_{\mathrm{rel}} \ = \ -\,\frac{2\,(\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i)}{c^2} \quad [\mathrm{s}], \qquad \text{range term} \ = -\,c\,\delta t_{\mathrm{rel}},} \tag{131}$$

i.e. $-2\,(\boldsymbol{r}\cdot\boldsymbol{v})/c^2$ seconds, applied to the range by the $-c\,\delta t_{\mathrm{rel}}$ term of the assembly (119) (`model/PppModel.cpp`). Two points are worth making precise. First, the dot product $\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i$ is *invariant* to whether the position/velocity pair is expressed in the inertial or the co-rotating ECEF frame: the ECEF velocity differs from the inertial velocity by the frame-rotation term $\dot{\boldsymbol{r}}^i_{\mathrm{ecef}} = \dot{\boldsymbol{r}}^i_{\mathrm{inertial}} - \boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i$, and $\boldsymbol{r}^i \cdot (\boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i) = 0$ because the cross-product is orthogonal to $\boldsymbol{r}^i$; so the same scalar results

from either velocity, and the broadcast-ephemeris ECEF velocity may be used directly. Second, the periodic term is computed *here*, separately, because the IGS precise CLK product by convention carries only the constant-plus-drift clock and excludes the periodic eccentricity term — applying (131) restores the missing piece consistently for both precise and broadcast clock sources.

10.5 Shapiro (gravitational-delay) correction

The signal path bends and slows in the Earth's gravitational potential; the resulting Shapiro delay is a few centimetres [23]. With $r_i = \|\boldsymbol{r}^i\|$, $r_r = \|\boldsymbol{r}_r\|$ the geocentric radii and $r_{ir} = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ the satellite–receiver distance, the delay (in metres) is

$$\boxed{S_r^i \;=\; \frac{2\,GM}{c^2}\,\ln\frac{r_i + r_r + r_{ir}}{r_i + r_r - r_{ir}},} \tag{132}$$

with $GM = 3.986005 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$ (`Constants::GRAVITATIONAL_CONSTANT`, `common/Constants.hpp`) and the formula verbatim at `GnssCorrections.cpp`. Because $r_{ir} < r_i + r_r$ the logarithm is positive, so $S_r^i > 0$ and the term is *added* to the modelled range in (119) (`model/PppModel.cpp`). Its magnitude is ∼1–2 cm for a typical MEO geometry.

10.6 Sagnac (Earth-rotation) correction

The pseudorange is formed between the satellite position at *signal emission* and the receiver position at *signal reception*, but both are needed in a single, consistent ECEF frame. During the ∼70 ms transit the ECEF frame itself rotates with the Earth, so the emission-time satellite coordinates must be rotated forward into the reception-time frame. The engine applies the *rigorous* rotation, not a scalar approximation: with the geometric time-of-flight $\tau = \|\boldsymbol{r}^i_{\mathrm{em}} - \boldsymbol{r}_r\|/c$ and the Earth rotation rate $\omega_\oplus = 7.2921151467 \times 10^{-5}\,\mathrm{rad/s}$ (`common/Constants.hpp`), the emission-time position $\boldsymbol{r}^i_{\mathrm{em}}$ is rotated by $\theta = \omega_\oplus \tau$ about the Z axis,

$$\boxed{\boldsymbol{r}^i = \boldsymbol{R}_z(\omega_\oplus \tau)\,\boldsymbol{r}^i_{\mathrm{em}}, \qquad \boldsymbol{R}_z(\theta) = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix},} \tag{133}$$

after which $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line of sight are computed from the rotated $\boldsymbol{r}^i$ (`model/ModelGeometry.cpp`, the `rotateAndComputeTopo` lambda). The Sagnac correction is thus *folded into the geometric range itself* rather than added as a separate term — which is why (119) carries no explicit Sagnac entry. An equivalent scalar form, $(\omega_\oplus/c)\,(x_i y_r - y_i x_r)$, exists in the codebase as a *diagnostic* only (`GnssCorrections::computeSagnacCorrection`, `corrections/GnssCorrections.cpp`); it is the first-order expansion of the rigorous rotation and is not on the modelling path. The Section 7 cross-references *here* for the Sagnac frame rotation; the derivation is kept in this document. The same $\boldsymbol{R}_z(\omega(t_{\mathrm{node}} - t_{\mathrm{query}}))$ rotation is reused at SP3 interpolation time to pre-rotate ephemeris nodes into the query-epoch frame (`model/ModelGeometry.cpp`), a numerical-conditioning use of the identical rotation documented with the products.

10.7 Group delays and inter-frequency biases

The abstract bias terms of the Foundations observation equations — the satellite code/phase biases B^i_s, b^i_s and the receiver biases $B_{r,s}$, $b_{r,s}$ — are realised deterministically by two products, which this section ties to the equation symbols.

Broadcast group delay. The broadcast navigation message carries a per-system inter-frequency group delay (GPS TGD, Galileo BGD, BeiDou TGD), referencing each signal's code hardware delay to the ionosphere-free combination used to generate the broadcast clock. In the uncombined parameterisation these are absorbed into the deterministic code bias applied through `model/PppModel.cpp`'s bias term; they are the broadcast-grade realisation of B_s^i, used when no precise bias product is available.

Precise OSB / DCB biases. When a precise bias product is loaded, the deterministic correction is the observable-specific bias (OSB) / differential code bias (DCB) entry for the exact (i,s) pair, time-windowed against its validity interval and returned by `GnssCorrections::getBiasCorrection` (`corrections/GnssCorrections.cpp`). This single scalar realises B_s^i (code) or b_s^i (phase) for that signal:

$$B_{\text{grp}} = \text{bias}(i, s, t), \qquad t_{\text{start}} \le t \le t_{\text{end}}, \tag{134}$$

with the application gated by `enable_code_phase_bias` (`model/PppModel.cpp`). Two exclusions are baked into the bias index (`GnssCorrections::buildBiasIndex`): *zero-valued* code entries (e.g. MGEX placeholder `C2I`/`C6I` for BeiDou) are treated as *absent* so the model does not both apply a null correction and suppress the per-(i,s) estimated bias state, leaving the real tens-of-metres delay unmodelled; and *GLONASS code* entries are likewise treated as missing because the FDMA per-channel receiver delay cannot be captured by a single value-per-(i,s) bias, forcing those signals onto the estimated-state path (`corrections/GnssCorrections.cpp`).

Estimated vs. deterministic biases. Whatever the deterministic bias leaves uncorrected — principally the receiver hardware delay $B_{r,s}$, $b_{r,s}$, and any satellite delay not covered by a product — is handled by the estimator. The receiver biases are common to all satellites on a signal and so collapse into the receiver clock / inter-system bias states or cancel under between-satellite differencing (Section 5); the per-(i,s) satellite code bias may be *estimated* as a state when no product covers it (`estimation.estimate_sat_biases`, `measurement/MeasurementModel.cpp`). The state-allocation gate that prevents estimating a bias the products already supply — hasBIA(i, s) from the static product, OR'd with hasSsrCB(i, s) from the live stream — is documented with the SSR overlay (Section 17, Equation (263) there); the present document supplies only the *deterministic* half of that OR.

Relation to the SSR overlay. The streamed SSR code/phase OSB (Section 17) refines the *same* B_s^i, b_s^i terms in real time, summed onto the predicted observable as `ssr_bias_term` in `measurement/MeasurementModel.cpp` alongside the deterministic B_{grp} here. The two are distinct mechanisms and must not be conflated: the SSR bias is a per-(i,s) scalar like (134), but the SSR *orbit* delta lives in a radial/along/cross (RAC) delta frame (Section 17's Equation (255)) that is *not* the satellite attitude body frame (122) of this document — the RAC frame is built from the broadcast position and velocity to express an orbit correction, whereas the body frame is built from the Sun direction to orient the antenna. They coincide neither in construction nor in purpose.

10.8 Provenance and assembly

All deterministic corrections converge in `model/PppModel.cpp`: `computeBaseModeledValue`, the single shared helper for both the iono-free-pair and uncombined paths, returns the sum (119), calling `model/ModelGeometry.cpp` for the Sagnac-rotated geometric range (133), `model/Antenna.cpp` for the total antenna correction (125) (which in turn calls `model/SatAttitude.cpp` for the body frame

and `corrections/GnssCorrections.cpp` for the wind-up), and `corrections/GnssCorrections.cpp` for the relativistic (131), Shapiro (132) and bias (134) terms. The Sun and Moon ephemeris feeding the attitude frame (120), the Sun-elevation (123), the eclipse predicate (124) and the wind-up (129) is produced by `model/Planetary.cpp` (the rigorous IAU precession/nutation Sun position, cached per epoch) and is documented in Section 6. Every term is invariant physics; the only user-facing knobs are enable toggles and the eclipse-shadow scale, all cited rather than reproduced because they tune *whether* a correction is applied, never its functional form.

Part III
Estimation engine

11 Estimation engine: the Kalman filter and RTS smoother

This section documents the recursive estimator at the core of the engine: a variable-dimension extended Kalman filter (EKF) with an optional prior-anchored iterated (IEKF) relinearisation pass, robust post-fit residual gating, and a backward Rauch–Tung–Striebel (RTS) smoother for post-processing modes. The state vector $\boldsymbol{x}$ is the uncombined PPP / PPP-RTK parameterisation of Section 3: receiver position (and optionally velocity), receiver clock and per-system time offsets, zenith wet troposphere and gradients, per-satellite slant ionosphere, receiver/satellite code and phase biases, and one float ambiguity per tracked carrier. Its dimension changes epoch to epoch as satellites rise and set and as ambiguities are reset; the layout and the dynamic key→index maps are owned exclusively by `filter/StateManager`, and the filter operates on that state without any fixed-index assumptions (`src/core/filter/KalmanFilter.cpp`).

Throughout we write $\boldsymbol{F}$ for the state-transition matrix, $\boldsymbol{Q}$ for the discrete process-noise covariance, $\boldsymbol{R}$ for the measurement-noise covariance, $\boldsymbol{S}$ for the innovation covariance, $\boldsymbol{K}$ for the Kalman gain, and $\boldsymbol{\nu}$ for the innovation (measurement minus predicted measurement). Predicted (a-priori) and updated (a-posteriori) quantities use the tilde/hat and $k|k{-}1$ / $k|k$ conditioning convention fixed in Section 2.

11.1 Time update

State transition. The transition matrix $\boldsymbol{F}(\Delta t)$ for epoch interval $\Delta t = t_k - t_{k-1}$ is identity plus a small number of coupling blocks, assembled by `StateManager::getTransitionMatrix` (`src/core/filter/StateManager.cpp`). The kinematic-integrator off-diagonal couplings are

$$\boldsymbol{F}[\,\boldsymbol{r},\boldsymbol{v}\,] = \Delta t\,\boldsymbol{I}_3, \qquad \boldsymbol{F}[\,c\mathrm{d}t,\ c\dot{\mathrm{d}}t\,] = \Delta t, \qquad \boldsymbol{F}[\,c\delta t^{(g)},\ c\dot{\delta} t^{(g)}\,] = \Delta t, \tag{135}$$

present only when the corresponding rate state exists (`hasVelocity()`, clock-rate, per-system rate). A fourth coupling appears when the constant-velocity slant-iono model is armed (`estimation.enable_iono_rate`, default off): a per-satellite level/rate integrator

$$\boldsymbol{F}[\,I^i,\ \dot{I}^i\,] = \Delta t \qquad \text{for every non-frozen-pivot satellite,} \tag{136}$$

realising $I^i(k{+}1) = I^i(k) + \Delta t\,\dot{I}^i(k)$. Every remaining state — ambiguities, zenith wet delay, gradients, code and satellite biases, any rate-free inter-system offset, and (absent `enable_iono_rate`) the per-satellite ionosphere — has $\boldsymbol{F} = \boldsymbol{I}$ and is propagated as a pure random walk with its mean held constant. Under the BSSD (single-difference) parameterisation the receiver-clock,

phase-clock (`getPhaseClockIndex`), inter-system-offset, and frozen-pivot per-satellite ionosphere (`getFrozenPivotSatellites`) rows are unobservable (their $\boldsymbol{H}$ columns are zero); their position rows in Equation (135) are suppressed and, for each, the corresponding cross-covariances are zeroed in place after propagation while the diagonal variance is preserved (the `freeze_state` block in `src/core/filter/KalmanFilter.cpp`). The iono level/rate coupling of Equation (136), by contrast, is deliberately *not* gated off under BSSD — the slant-iono level remains observable under single-differencing (only the pivot column is zeroed), so every non-frozen-pivot satellite keeps its integrator; the frozen-pivot satellite alone is skipped, holding its rate at $(0, 0)$ with its process noise suppressed so a live coupling cannot re-drive the pivot level off the SD datum.

The propagation itself is the standard EKF time update

$$\boxed{\begin{aligned}\tilde{\boldsymbol{x}}_{k|k-1} &= \boldsymbol{F}(\Delta t)\,\hat{\boldsymbol{x}}_{k-1|k-1}, &(137)\\ \tilde{\boldsymbol{P}}_{k|k-1} &= \boldsymbol{F}(\Delta t)\,\boldsymbol{P}_{k-1|k-1}\,\boldsymbol{F}(\Delta t)^{\mathrm{T}} + \boldsymbol{Q}(\Delta t), &(138)\end{aligned}}$$

implemented in `KalmanFilter::predict`. A non-positive Δt is treated as a no-op (duplicate or out-of-order epoch), short-circuiting before Equation (138).

Process-noise model. The discrete process noise $\boldsymbol{Q}(\Delta t)$ is built from continuous spectral densities by `StateManager::getProcessNoiseMatrix`. Two block structures appear. Pure random-walk states have a scalar discrete variance increment

$$\boldsymbol{Q}_{ii} = q_i\,\Delta t, \tag{139}$$

where q_i is the continuous variance rate (m^2/s) read from `filter.process_noise.*`. Representative entries are zenith wet troposphere (`troposphere_q`), the receiver code bias (`code_bias_q`), the satellite *code* bias (`sat_bias_q`, applied to `StateManager::sat_bias_indices_`), the satellite *phase* bias (`sat_phase_bias_q`, a separate density on the separate `sat_phase_bias_indices_` map), phase clock (`phase_clock_q`), and the float ambiguity (`ambiguity_q`). Both satellite-bias blocks — and the satellite-clock block beside them — suppress $\boldsymbol{Q}$ entirely for a frozen-pivot satellite, for the same reason as the freeze discussion above: the frozen state is held at (value, variance) $= (0, 0)$ as the datum, and any process noise would walk it off that datum. The ambiguity density defaults to a numerical safety floor (10^{-12}) rather than a physical value, and per-key overrides are supported for arc-specific reseeding. The per-satellite slant ionosphere uses a random walk whose zenith density is mapped to the slant by the squared obliquity factor,

$$\boldsymbol{Q}_{ii} = \frac{q_{\mathrm{iono}}}{\sin^2\theta^i}\,\Delta t, \qquad \sin^2\theta^i \text{ floored at } 10^{-2}\ (\theta \approx 5.7^\circ), \tag{140}$$

with q_{iono} = `ionosphere_q` and optional per-satellite overrides driven by the adaptive-iono scheme. Under the constant-velocity model of Equation (136) this random walk survives only as the fallback for satellites without a companion rate state: otherwise the level/rate pair takes the CWNA block of Equation (141) below with obliquity-scaled acceleration PSD $q = q_{\dot{\mathrm{iono}}}/\sin^2\theta^i$, where $q_{\dot{\mathrm{iono}}}$ = `process_noise.iono_rate_psd` (an acceleration PSD, $\mathrm{m}^2/\mathrm{s}^3$, default 10^{-10}). The adaptive-monitor override enters this branch as the *dimensionless* ratio $q_{\mathrm{ov}}/q_{\mathrm{iono}}$ multiplying the rate PSD, so the Sage–Husa storm response drives $\dot{I}^i$ rather than the level.

Coupled position/velocity, clock/clock-rate, inter-system offset/rate, and — under `enable_iono_rate` — slant-iono level/rate pairs use the continuous white-noise-acceleration (CWNA, integrated-random-walk) discretisation. For a coupled pair $(\xi, \dot{\xi})$ driven by acceleration PSD q,

$$\boldsymbol{Q}_{[\xi,\dot{\xi}]} = \begin{bmatrix} \frac{1}{3}\,q\,\Delta t^3 & \frac{1}{2}\,q\,\Delta t^2 \\ \frac{1}{2}\,q\,\Delta t^2 & q\,\Delta t \end{bmatrix}, \tag{141}$$

with q given by `acceleration_psd_q` (position/velocity, the 3×3 block-diagonal extension of Equation (141) over the ECEF axes), `clock_rate_q`, `system_time_rate_offset_q`, and the obliquity-scaled `iono_rate_psd`$/\sin^2\theta^i$ respectively. When the velocity state is present the rate-free position density `position_q` is unused. Baseline (double-difference) mode reduces $\boldsymbol{Q}$ to the position block, a per-ambiguity random-walk term, and an optional differential-ZWD entry $\boldsymbol{Q}_{ii} = \sigma^2 \Delta t$ governed by `baseline.differential_tropo_process_noise_m_per_sqrt_s`.

Scope: the recursive estimator only. These densities govern the *recursive* estimator. The network producer runs no `KalmanFilter` at all — `getTransitionMatrix` and `getProcessNoiseMatrix` are reached only from `KalmanFilter::predict` / `predictImu` / `predictAided` and from the smoother's `Application::capturePriorSnapshot`. The producer instead solves per-epoch normal equations for corrections to each station's linearisation point, and imposes its cross-epoch stiffness with explicit information-form carry priors of effective variance $\mathrm{var}_{\mathrm{eff}} = \mathrm{var} + q\,\Delta t$ rather than through $\boldsymbol{Q}(\Delta t)$ (Section 19). The satellite-side densities `sat_clock_q`, `sat_phase_bias_q` and `sat_code_bias_q` are read there as exactly those carry rates, and are additionally authorable through the always-live network-scoped `network.process_noise` class table, whose per-class fallbacks equal the legacy `filter.process_noise.*` defaults so an unmodified config resolves identically either way.

Aided time update. When inertial aiding is active, `KalmanFilter::predictImu` performs a strapdown mechanisation of the nominal position, velocity, and attitude and splices a 15×15 INS error-state transition $\boldsymbol{\Phi}$ and noise $\boldsymbol{Q}_{\mathrm{ins}}$ — with error order $[\delta\boldsymbol{p},\ \delta\boldsymbol{v},\ \delta\boldsymbol{\psi},\ \delta\boldsymbol{b}_a,\ \delta\boldsymbol{b}_g]$ — into the GNSS-dimensioned $\boldsymbol{F}$ and $\boldsymbol{Q}$ over the integrated interval, after which Equation (138) runs on the full state with explicit symmetrisation $\boldsymbol{P} \leftarrow \frac{1}{2}(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}})$. The generalised `predictAided` path applies the same splice for arbitrary aiding sources and delegates byte-identically to the GNSS-only Equation (138) when no aiding payload is present.

11.2 Measurement update

The measurement update (`KalmanFilter::update`) is a *batch* update: all observation rows of the epoch are processed simultaneously, not sequentially/scalar. A sparse-$\boldsymbol{H}$ optimisation accelerates the four design-matrix products ($\boldsymbol{HP}$, $\boldsymbol{HPH}^{\mathrm{T}}$, $\boldsymbol{KH}$, and the $\boldsymbol{H}$-touching parts of the gain), and is numerically identical to the dense computation to within a 10^{-12} structural threshold (`src/core/filter/KalmanFilter.cpp`).

Given the design matrix $\boldsymbol{H}$ from the linearisation of the Equation (4) and Equation (5) of the Foundations document (position rows $-(\boldsymbol{e}^i_{\hat{r}})^{\mathrm{T}}$, the Equation (6)), the innovation $\boldsymbol{\nu}$ (`measurement.innovations`), and the measurement-noise covariance $\boldsymbol{R}$ — a full matrix when supplied, otherwise the diagonal $\boldsymbol{R} = \mathrm{diag}(\sigma^2_{r,i,s})$ of the Equation (8) — the innovation covariance and gain are

$$\boldsymbol{S} = \boldsymbol{H}\,\tilde{\boldsymbol{P}}_{k|k-1}\,\boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R}, \tag{142}$$

$$\boldsymbol{K} = \tilde{\boldsymbol{P}}_{k|k-1}\,\boldsymbol{H}^{\mathrm{T}}\,\boldsymbol{S}^{-1}. \tag{143}$$

The gain is never formed by explicitly inverting $\boldsymbol{S}$. Exploiting the symmetry of $\boldsymbol{S}$, the engine factorises it once with an $\mathrm{LDL}^{\mathrm{T}}$ decomposition and recovers $\boldsymbol{K} = \left(\boldsymbol{S}^{-1}\boldsymbol{H}\tilde{\boldsymbol{P}}\right)^{\mathrm{T}}$ from a single solve against $\boldsymbol{H}\tilde{\boldsymbol{P}}$ (`ldlt.solve(HP).transpose()`); on factorisation failure it falls back to a complete-orthogonal (pseudo-inverse) solve. The state update is

$$\delta\boldsymbol{x} = \boldsymbol{K}\,\boldsymbol{\nu}, \qquad \hat{\boldsymbol{x}}_{k|k} = \tilde{\boldsymbol{x}}_{k|k-1} + \delta\boldsymbol{x}. \tag{144}$$

Joseph-form covariance update. The covariance is updated in the numerically stabilised *Joseph* form, never the short form $(\boldsymbol{I}-\boldsymbol{K}\boldsymbol{H})\tilde{\boldsymbol{P}}$:

$$\boxed{\boldsymbol{P}_{k|k} = (\boldsymbol{I}-\boldsymbol{K}\boldsymbol{H})\,\tilde{\boldsymbol{P}}_{k|k-1}\,(\boldsymbol{I}-\boldsymbol{K}\boldsymbol{H})^{\mathrm{T}} + \boldsymbol{K}\,\boldsymbol{R}\,\boldsymbol{K}^{\mathrm{T}}.} \tag{145}$$

The Joseph form preserves symmetry and positive semi-definiteness for any gain, which matters because the engine deliberately departs from the optimal gain in its robust paths (Section 11.4); the result is symmetrised once more, $\boldsymbol{P} \leftarrow \frac{1}{2}(\boldsymbol{P}+\boldsymbol{P}^{\mathrm{T}})$, to clear round-off. The factor $(\boldsymbol{I}-\boldsymbol{K}\boldsymbol{H})$ is assembled from the sparse $\boldsymbol{K}\boldsymbol{H}$ product with a unit shift on the diagonal. The same Joseph kernel, with the explicit gain $\boldsymbol{K} = \boldsymbol{P}\boldsymbol{H}^{\mathrm{T}}\boldsymbol{S}^{-1}$, is exposed as the pure static routine `josephPseudoUpdate` and reused for scalar pseudo-observation constraints (e.g. the GIM ionosphere prior via `applyPseudoObservation`) and for the IEKF pass of Section 11.3.

Post-fit residuals. After the update the post-fit residuals and their (diagonal) covariance are

$$\boldsymbol{r} = \boldsymbol{\nu} - \boldsymbol{H}\,\delta\boldsymbol{x}, \qquad S_{r,ii} = R_{ii} - \left(\boldsymbol{H}\,\boldsymbol{P}_{k|k}\,\boldsymbol{H}^{\mathrm{T}}\right)_{ii}, \tag{146}$$

of which, by default, only the diagonal of S_r is materialised. When the off-diagonal-R kernel is enabled (`enable_full_r_kernel`, default off) *and* a dense $\boldsymbol{R}$ was supplied (BSSD / baseline-DD epochs whose $\boldsymbol{R}$ carries SD/DD cross-covariance), the full symmetrised $S_r = \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}_{k|k}\,\boldsymbol{H}^{\mathrm{T}}$ is instead formed and stored, so the redundancy and w-test QC of Section 11.4 honour correlated observations via S_r^{-1}; the diagonal — and hence Equation (147) — is identical either way. The standardised residual that drives gating (Section 11.4) is

$$s_i = \frac{r_i}{\sqrt{\max\{\,S_{r,ii},\,R_{ii},\,10^{-12}\,\}}}. \tag{147}$$

The denominator floor at the *working* R_{ii} (the inflated noise actually used in $\boldsymbol{R}$, not the formal noise) is a deliberate robustness compromise: in ambiguity-absorption cases — double-differenced phase with an ambiguity gain near unity — the rigorous denominator $S_{r,ii}$ collapses toward $R\,\boldsymbol{S}^{-1}R$, which would register near-zero residuals as many-σ outliers. The strictly correct standardisation uses $S_{r,ii}$ alone; the floor trades that rigour for stability and is documented as such in source.

11.3 Iterated relinearisation (IEKF)

For long initial position offsets a single linearisation of the Equation (6) per epoch can be insufficient. When `processing.relinearize_geometry` is enabled (and the run is not tightly-coupled IMU), the engine performs a prior-anchored iterated update — a Gauss–Newton / iterated EKF step — after the primary update.

Anchor. The iteration is anchored to the *stashed pre-update prior* $(\boldsymbol{x}^{-}, \boldsymbol{P}^{-})$ that the converged primary Equations (138) and (145) read, captured only when relinearisation is armed (`getLastUpdatePriorX` / `getLastUpdatePriorP`). This is the prior *after* any in-epoch ambiguity augmentation and outlier reset, so it is dimension-matched to the posterior; it is not a top-of-loop snapshot.

Gauss–Newton step. Let $\boldsymbol{x}_i$ be the current linearisation iterate (the committed state at which `measurement` was linearised, with innovations $\boldsymbol{z} - h(\boldsymbol{x}_i)$ and $\boldsymbol{H} = \partial h/\partial \boldsymbol{x}\big|_{\boldsymbol{x}_i}$). The prior-anchored iterate (`KalmanFilter::updateFromPrior`, via `josephPseudoUpdate`) is

$$\boxed{\begin{aligned} \boldsymbol{\nu} &= \big(\boldsymbol{z} - h(\boldsymbol{x}_i)\big) - \boldsymbol{H}(\boldsymbol{x}^- - \boldsymbol{x}_i), && (148)\\ \boldsymbol{K} &= \boldsymbol{P}^- \boldsymbol{H}^{\mathrm{T}} \big(\boldsymbol{H}\boldsymbol{P}^- \boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R}\big)^{-1}, \qquad \boldsymbol{x}^+ = \boldsymbol{x}^- + \boldsymbol{K}\,\boldsymbol{\nu}, && (149)\\ \boldsymbol{P}^+ &= (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\,\boldsymbol{P}^-\,(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})^{\mathrm{T}} + \boldsymbol{K}\,\boldsymbol{R}\,\boldsymbol{K}^{\mathrm{T}}. && (150) \end{aligned}}$$

The offset term $-\boldsymbol{H}(\boldsymbol{x}^- - \boldsymbol{x}_i)$ in Equation (148) re-expresses the innovation about the fixed prior and is what makes this a proper MAP / Gauss–Newton iteration: because $\boldsymbol{K}$ and $(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\boldsymbol{P}^-$ are always derived from the same fixed prior $(\boldsymbol{x}^-, \boldsymbol{P}^-)$, the covariance does *not* contract a second time and no information is double-counted.

Firing condition. The driver `Application::runGeometryRelinearization` (`src/core/app/Application.cpp`) fires the iteration when all of the following hold:

1. `processing.relinearize_geometry` is on and a prior was stashed at the primary update;
2. the position change since the primary update exceeds a data-dependent threshold — `relinearize_dpos_threshold_phase_m` (default 1 m) when phase is present, or `relinearize_dpos_threshold_code_only_m` (default 100 m) for code-only epochs;
3. the per-run cap `relinearize_max_count` (-1 = unlimited) is not exhausted.

The inner loop recomputes geometry and re-applies Equations (148) to (150) against the *fixed* converged exclusion set, for at most five iterations, declaring convergence when the position step falls below 10^{-4} m. Because GNSS geometry is nearly linear at the metre level, convergence is typically reached in a single pass and the threshold of item 2 means the iteration rarely fires at all. The tightly-coupled IMU configuration instead takes a single legacy *naive* second pass (re-applying the measurement against the posterior), since error-state attitude/bias feedback conflicts with a clean prior re-derivation.

11.4 Innovation and residual gating

The engine performs *no* χ^2 innovation-covariance test. Outlier rejection (`KalmanFilter::validatePostFitResiduals`) operates on the standardised *post-fit* residuals s_i of Equation (147) against a fixed scalar threshold, supplemented by robust median-based spatial detectors for pivot slips. The two top-level thresholds are hardcoded `GnssConfig` defaults (`residual_threshold_` = 6.0 and `bad_meas_percent_` = 0.5 in `src/core/config/GnssConfig.cpp`) that are deliberately not exposed to JSON: the per-row rejection threshold is $s_{\max} = 6.0$ (read at runtime via `getResidualThreshold`) and the complete-reset trigger fraction is 0.5 (`getBadMeasPercent`); there are no `residual_threshold_` or `bad_meas_percent_` config keys to search for. The dispatch row and criterion can nonetheless be *replaced* wholesale by the Baarda data-snooping gate below — itself JSON-exposed.

Per-row rejection. If $\max_i |s_i| > 6.0$, the worst row is dispatched by type: a phase outlier triggers an ambiguity reset (with BSSD pivot handling and a sub-cycle λ-fraction floor), while a code outlier is excluded from the epoch. Correlated same-satellite phase rows on other bands are reset alongside it.

Baarda data-snooping gate (optional override). When `estimation.enable_data_snooping` is set, the fixed-σ gate triple (gate stat, gate row, gate crit) = ($\max_i |s_i|$, arg max, 6.0) is *replaced* in `validatePostFitResiduals` by the Baarda w-test: over *checkable* rows — those whose redundancy $r_i = S_{r,ii}/R_{ii}$ meets a floor ($r_i \geq 0.1$; low-redundancy rows are skipped as uncheckable, since a blunder there is absorbed into the estimate rather than the residual) — the worst $|w_i|$ is selected and the gate fires on $|w_i| >$ `estimation.data_snooping_critical` (default 3.29). Both the criterion *and* the dispatched row can thus differ from the σ gate; the type-based dispatch of the selected row is otherwise unchanged. The w-test statistic and redundancy are produced by the a-posteriori QC diagnostics layer (`computeResidualQc_`, gated by `estimation.enable_variance_factor`): the variance factor $\hat{\sigma}_0^2 = \left(\sum_i v_i^2/R_{ii}\right)/\mathrm{dof}$, the per-row redundancy r_i, the w-test, and the minimal detectable bias (MDB). This layer honours a correlated R through S_r^{-1} when the full-R kernel is active (see Equation (146)).

Complete-reset backstop. If the count of rejected rows exceeds 0.5 × (original pre-peel observation count), the filter covariance is re-inflated (`resetFilterCovariance`) — a hard reset that, in the smoother, marks a segment boundary (Section 11.5).

Robust pivot-slip detectors (BSSD). Before the per-row gate, single-difference phase rows are grouped by (pivot, signal) and screened for a coherent pivot slip using median / median-absolute-deviation (MAD) statistics: a forced pivot handover fires when a group is large enough, a majority of its rows exceed threshold, the group median exceeds a metre-scale floor, and the MAD-to-median ratio is small (a tight cluster indicating a common-mode shift rather than scattered noise). Cross-band pooled and code-side variants apply analogous tests with their own floors, and non-pivot multi-band slips are confirmed by integer-cycle consistency ($|r|/\lambda_{\mathrm{band}}$ rounding to the same nonzero integer across bands).

Down-weighting (not gating). Two paths inflate $\boldsymbol{R}$ rather than reject: a persistent EWMA-bias term adds the squared mean bias to R_{ii} once an arc has warmed up, and a Huber post-fit scheme inflates R for sub-cycle rows whose $|s_i|$ exceeds the Huber knee (innovation one-shot or post-fit IRLS variants, toggled by `enable_huber_r_inflation` and `enable_huber_innovation`). These modify the working $\boldsymbol{R}$ that feeds Equations (142) and (147), and are the reason the standardised residual floors at the working noise rather than the formal noise.

11.5 Backward RTS smoother

In post-processing modes, enabling `processing.run_smoother` runs a fixed-interval Rauch–Tung–Striebel recursion [20] over the stored forward trajectory (`include/filter/Smoother.hpp`, `src/core/filter/Smoother.cpp`). The flag is force-disabled under `AppMode::RealTime`, since smoothing is non-causal and inherently a post-processing operation.

Forward-pass capture. Rather than reconstruct the a-priori from $\boldsymbol{F}$ and $\boldsymbol{Q}$ on the backward pass, the forward pass caches, per epoch, both the one-step prediction $\tilde{\boldsymbol{x}}_{k|k-1}, \tilde{\boldsymbol{P}}_{k|k-1}$ and the posterior $\hat{\boldsymbol{x}}_{k|k}, \boldsymbol{P}_{k|k}$, together with $\boldsymbol{F}_k$ and the full set of core and dynamic key→index maps (`capturePriorSnapshot` / `capturePosteriorSnapshot` in `src/core/app/Application.cpp`). A capture-ordering invariant makes the stored prior exactly $\boldsymbol{F}_k\hat{\boldsymbol{x}}_{k-1|k-1}$: the a-priori is recorded immediately after `predict()` and before any cycle-slip reseeding or measurement update, and the a-posteriori is recorded *before* integer ambiguity resolution, so the smoother consumes *float* ambiguities only.

Recursion. Initialised at the terminal posterior $\hat{\boldsymbol{x}}_{N|N} = \hat{\boldsymbol{x}}_{N|N}$, $\boldsymbol{P}_{N|N} = \boldsymbol{P}_{N|N}$, the backward recursion is

$$\boxed{\begin{aligned} \boldsymbol{G}_k &= \boldsymbol{P}_{k|k}\,\boldsymbol{F}_{k+1}^{\mathrm{T}}\,\tilde{\boldsymbol{P}}_{k+1|k}^{-1}, && (151)\\ \hat{\boldsymbol{x}}_{k|N} &= \hat{\boldsymbol{x}}_{k|k} + \boldsymbol{G}_k\big(\hat{\boldsymbol{x}}_{k+1|N} - \tilde{\boldsymbol{x}}_{k+1|k}\big), && (152)\\ \boldsymbol{P}_{k|N} &= \boldsymbol{P}_{k|k} + \boldsymbol{G}_k\big(\boldsymbol{P}_{k+1|N} - \tilde{\boldsymbol{P}}_{k+1|k}\big)\boldsymbol{G}_k^{\mathrm{T}}. && (153) \end{aligned}}$$

The smoother gain $\boldsymbol{G}_k$ is realised as an LDL^{T} solve — the predicted covariance is symmetrised, then $\boldsymbol{G}_k = \big(\tilde{\boldsymbol{P}}_{k+1|k}^{-1}(\boldsymbol{P}_{k|k}\boldsymbol{F}^{\mathrm{T}})^{\mathrm{T}}\big)^{\mathrm{T}}$ via `ldlt.solve(...).transpose()` — never an explicit inverse, and the smoothed covariance Equation (153) is symmetrised after the update. If the factorisation fails, the step is abandoned and epoch k keeps its forward posterior.

Variable-dimension reconciliation. Because $\dim \boldsymbol{x}_k \neq \dim \boldsymbol{x}_{k+1}$ in general, Equations (151) to (153) are restricted at each step to the *common-key subspace* — the states present at both k and $k{+}1$, found by intersecting the dynamic key→index maps (`collectCommonIndices`, less any keys in the handover exclusion set of the segment-boundary paragraph below). Let $\boldsymbol{\Pi}_k$ and $\boldsymbol{\Pi}_{k+1}$ select the common rows in each epoch's state space; the implemented state update is then

$$\hat{\boldsymbol{x}}_{k|N} = \hat{\boldsymbol{x}}_{k|k} + \boldsymbol{\Pi}_k^{\mathrm{T}}\,\boldsymbol{G}_{k,C}\big(\boldsymbol{\Pi}_{k+1}\hat{\boldsymbol{x}}_{k+1|N} - \boldsymbol{\Pi}_k\tilde{\boldsymbol{x}}_{k+1|k}\big), \tag{154}$$

with the analogous projection on Equation (153). States outside the subspace retain their forward a-posteriori value. A subtle indexing invariant: $\tilde{\boldsymbol{x}}_{k+1|k}$, $\tilde{\boldsymbol{P}}_{k+1|k}$ and $\boldsymbol{F}_{k+1}$ are sized for epoch k's layout (the predict step does not grow the state), so they are addressed with epoch k's common indices, whereas $\hat{\boldsymbol{x}}_{k+1|N}$, $\boldsymbol{P}_{k+1|N}$ live in epoch $k{+}1$'s layout.

Segment boundaries at resets and frame changes. Four mechanisms of differing severity terminate or trim backward information flow at ambiguity resets, teardowns, and pivot handovers:

- A *complete* filter reset at $k{+}1$ (the covariance re-inflation of Section 11.4, recorded as `is_discontinuity`) is a hard segment boundary: it severs the dynamic link $\tilde{\boldsymbol{x}}_{k+1|k} = \boldsymbol{F}\hat{\boldsymbol{x}}_{k|k}$, so no $\boldsymbol{G}_k$ is meaningful and backward flow is terminated — epoch k keeps its forward posterior.
- A *single-satellite* ambiguity reset is handled softly: that satellite's ambiguities are excluded from the common subspace for the $k \to k{+}1$ step, because the re-seeded large-variance prior at $k{+}1$ carries no information backward and smoothing it would only inject noise. The exclusion matches by satellite prefix and is intentionally slightly over-broad, always erring toward the safe forward fallback.
- A *dimension change* $\dim \tilde{\boldsymbol{x}}_{k+1|k} \neq \dim \hat{\boldsymbol{x}}_{k|k}$ is a hard boundary: predict never grows the state, so any size mismatch signals a pre-predict long-gap teardown (`Application::handleGap` prunes ambiguity / iono / sat-bias states before `predict()`), across which epoch k's indices would address the wrong states in the $k{+}1$ a-priori — backward flow stops and epoch k keeps its forward posterior.
- A *BSSD pivot handover* between k and $k{+}1$ re-references one or more constellations in place; it is reconciled or segmented per constellation as described next.

BSSD pivot-handover reconciliation. When the pivot map changes between epochs ($\texttt{pivot_satellites}_k \neq \texttt{pivot_satellites}_{k+1}$) the smoothed posterior at $k{+}1$ sits in the *new* pivot frame while the cached a-priori $\tilde{\boldsymbol{x}}_{k+1|k}$ (captured pre-handover, in epoch k's layout) is

still in the *old* frame, so the two are not directly comparable in Equation (152). `Smoother::prepareHandoverFrame` restores comparability per constellation on *copies* of the $k{+}1$ smoothed estimate (leaving the reported $k{+}1$ result in its own frame). A handover recorded as *clean* by `PivotManager` (old/new pivots matching `clean_handovers`) is inverted back to the old pivot p_0 signal-group by signal-group through the public static `Smoother::inverseReReferenceGroup` — the exact inverse of the forward re-reference. For a group with old-pivot datum row d,

$$x'(i) = x(i) - x(d), \quad x'(d) = 0, \qquad P'(i,\cdot) = P(i,\cdot) - P(d,\cdot), \quad P'(\cdot,j) = P(\cdot,j) - P(\cdot,d), \tag{155}$$

applied over *full* rows and columns (the forward re-reference coupled the group to every other state, so the inverse must too) with the datum row/column zeroed last. A signal group is invertible only if the old pivot still carries a live state on it *and* the forward leg re-referenced the signal to the new pivot; orphaned groups (the new pivot lacked the signal), disturbed or forced handovers, and every handover constellation's ionosphere and satellite-bias states are instead *segmented* — added to the `exclude_keys` set that `collectCommonIndices` drops from the common subspace, so those states fall back to the forward posterior.

Static-position degeneracy. In static PPP the receiver position is a (near-)constant state, so the position-block smoother gain propagates essentially the same correction to every epoch and the smoothed position is degenerate — constant across epochs — *by construction*. This is expected, not a defect: the smoother's value accrues on the non-position states (troposphere, ionosphere, float ambiguities) and on kinematic runs where position carries genuine epoch-to-epoch dynamics.

12 Stochastic Model and Quality Control

This document covers the *measurement* side of the estimator's error budget: the strategies that populate the observation-noise covariance $\boldsymbol{R}$, and the a-posteriori quality-control (QC) layer that validates an epoch's residuals against that $\boldsymbol{R}$. The $\boldsymbol{R}$ produced here is the block-diagonal skeleton of Equation (8); this document supplies the per-strategy variance interiors of that skeleton. The QC layer is built on the same post-fit residual statistics the filter already computes for outlier rejection, but reframes them as the classical geodetic detection theory of Baarda and Teunissen [27].

Two scopes are deliberately *out* of this document and only cross-referenced. First, the standardised-residual outlier path ($|s| > 6$), the Huber re-weighting, and the EWMA $\boldsymbol{R}$-inflation that react to a detected blunder live in Section 11.4 — this document explains how the QC statistics feed an *alternative* gate but does not re-derive the dispatch. Second, the per-state process-noise covariance $\boldsymbol{Q}$ (clock random walk, ZWD, ambiguity, IMU bias) is purely a *dynamics*-side quantity and lives with the smoother (Equation (140)); everything below is measurement noise only.

12.1 The stochastic-model dispatch

Every undifferenced observation (r, i, s) is assigned a scalar variance by a strategy object selected from the `StochasticModelType` enum (`include/config/GnssConfigTypes.hpp`, the four-way {`Constant`, `Elevation`, `SNR`, `SatelliteBlock`} set). The dispatch lives in `src/core/model/StochasticModel.cpp`; each variant returns a variance in m^2 that becomes one diagonal entry of $\boldsymbol{R}$. The three geometry-aware strategies share a common low-elevation guard: when $\sin\theta \le 10^{-6}$ the elevation amplification is suppressed and the un-amplified variance is returned, so a satellite at the horizon does not inflate to a numerically infinite weight (`computeElevation`). The `Constant` strategy has no elevation term at all — it returns its a-priori variance directly from the dispatch (`computeConstant`) and never reaches the guard.

(a) Constant. The trivial strategy returns the a-priori variance of unit weight unchanged,

$$\sigma^2_{r,i,s} = \sigma^2_{0,\mathrm{type}(s)}, \tag{156}$$

with σ_0 drawn from `filter.measurement_noise` (`computeConstant`). This is the reference against which the other strategies are multiplicative.

(b) Elevation. The classical elevation-dependent model scales the zenith-referenced variance by an inverse power of $\sin\theta$,

$$\boxed{\sigma^2_{r,i,s} = \sigma^2_{\mathrm{zen}} \left(\sin\theta^i_r\right)^{-2p_{\mathrm{type}}}, \qquad p_P = p_\Phi = 1.0.} \tag{157}$$

The shipped defaults of `filter.code_elevation_power` and `filter.phase_elevation_power` are both unity, so the delivered model is *symmetric*: code and phase alike follow the legacy $\csc^2\theta$ variance law (`computeElevation`). The exponent in code is $-2p_{\mathrm{type}}$ because the strategy works in variance, not σ: $\left(\sin\theta\right)^{-2p}$ is exactly $\left(\sin\theta^{-p}\right)^2$. Rows whose observable code does not begin with `L` — everything but carrier phase, including Doppler and the empty-`obs_type` fallback paths — take the code power.

Both exponents are configurable per observable class, and the asymmetric setting $p_\Phi = 0.5$ is the one alternative with a physical argument behind it, so it is worth recording why it is an *option* rather than the default. It gives a $\csc\theta$ variance ($\csc^{1/2}\theta$ in σ), a markedly gentler amplification at low elevation. The argument for it is that phase noise near the horizon is dominated by carrier-loop SNR and by antenna phase-centre miscalibration, neither of which follows the $\csc^2\theta$ mapping that governs code multipath and pseudorange path-length error; and that the milder amplification keeps low-elevation phase inside the useful range of the standardised-residual cycle-slip test instead of deweighting it into irrelevance. The argument against it is comparability — the code branch, and every archived benchmark, run at unity — and that is the position the loader now ships (`GnssConfig::getPhaseElevationPower`, whose documentation carries the amplification ratios at a 10° mask).

(c) SNR (SIGMA-ε). When carrier-to-noise density is available the strategy uses the Hadás–Bosy variance-additive form [23]. Writing the measured (or synthesised) density as $\mathrm{C/N_0}$ in dB Hz,

$$\boxed{\sigma^2_{\mathrm{SNR}} = a^2 + b^2\, 10^{-\mathrm{C/N_0}/10},} \tag{158}$$

where a is the high-SNR floor ($\sigma \to a$ as $\mathrm{C/N_0} \to \infty$) and b scales the SNR-dependent contribution; both have separate code/phase values (`snr_params_.code_a/b`, `phase_a/b`). The *variance*-additive composition in Equation (158) is mathematically the correct combination of two independent noise sources. The legacy σ-additive alternative $\sigma = a + b\,10^{-\mathrm{C/N_0}/10}$ is explicitly rejected in `computeSnr` because squaring it produces an unphysical $2ab\,10^{-\mathrm{C/N_0}/10}$ cross-term that has no noise-process interpretation — it would couple the two contributions as if they were correlated when they are not.

When no `S...` observable is present the model does not switch functional form; instead it *synthesises* a density from elevation so the noise curve stays continuous across the boundary (`synthesizeCn0FromElevation`):

$$\mathrm{C/N_0}(\theta) = 50 - 15\left(1 - \max(0, \sin\theta)\right) \ \ [\mathrm{dB\,Hz}], \tag{159}$$

i.e. a 50 dB Hz zenith maximum dropping 15 dB to the horizon. The density is then clamped to the configured working range [`snr_floor_dbhz`, `snr_ceil_dbhz`]. Two refinements follow: on code rows

a per-band scale s^2_{band} from the SatelliteBlock code table multiplies σ^2_{SNR} (wider-bandwidth signals such as L5/E5a get $\sim\sqrt{10}$ less correlator-noise σ, hence a factor $\sim 1/10$ in variance), while phase stays band-flat because PLL noise is governed by carrier-loop bandwidth, not chip rate. Finally the Wieser hybrid step pipes the SNR variance through Equation (157) so that elevation-correlated multipath — which pure SNR weighting under-weights, since the carrier loop tracks cleanly even while the range is corrupted — is still captured.

(d) SatelliteBlock. The most detailed strategy keys the zenith σ on the transmitting satellite's ANTEX block type, then applies observable-specific frequency-band and tracking-attribute scales (`computeSatBlock`):

$$\boxed{\sigma^2_{r,i,s} = \left(\sigma_{\text{block}(i),\,\text{type}}\, s_{\text{band}}\, s_{\text{attr}}\right)^2 \left(\sin\theta\right)^{-2p_{\text{type}}}.} \tag{160}$$

Here $\sigma_{\text{block}(i),\text{type}}$ is looked up by substring-matching the satellite's ATX type string against `block_code_std_m` / `block_phase_std_m` (falling back to `default_code_std_m` / `default_phase_std_m` when no block matches); s_{band} is the per-band factor keyed on the RINEX band character (`obs_type[1]`); and s_{attr} is the tracking-attribute factor keyed on `obs_type[2]` via a *per-system* table, because RINEX-3 attributes carry constellation-specific meaning (a `C` attribute is the noisy GPS C/A but the high-precision Galileo E1C pilot). The attribute lookup order is `[system][attr]` → `[default][attr]` → 1.0. Code and phase carry separate band/attribute tables (`SatBlockNoiseParams::ObservableScale`) since chip-rate calibration governs code while carrier-tracking calibration — a much smaller band gradient — governs phase. As with the other geometry-aware strategies the block variance is finally amplified by Equation (157); phase only reaches this path when `apply_to_phase` is set, otherwise it takes the flat `default_phase_std_m` route in the top-level dispatch.

All four strategies populate diagonal entries only; the undifferenced $\boldsymbol{R}$ is therefore block-diagonal, exactly the skeleton of Equation (8). Structured off-diagonal correlation appears only after differencing (Section 12.5).

12.2 A-posteriori variance factor

The entire Kalman-side QC layer described in this and the following three sections is a *default-off* diagnostic. `computeResidualQc_` is called from `KalmanFilter::update` only when `estimation.enable_variance_factor` or `estimation.enable_data_snooping` is set, and the correlation-aware kernel of Section 12.5 additionally requires `estimation.enable_full_r_kernel`; all three default to false. With them off the filter's only outlier gate is the fixed $|s| > 6$ path of Section 11.4, the variance factor keeps its initialised value of unity, and none of the redundancy numbers, w-tests or minimal detectable biases below are computed at all.

Once an epoch has been updated, the post-fit residuals $\hat{\boldsymbol{v}}$ provide an internal-consistency check on whether the assumed $\boldsymbol{R}$ was correctly scaled. The a-posteriori *variance factor* (estimated variance of unit weight) is

$$\boxed{\hat\sigma_0^2 = \frac{\Omega}{\text{dof}}, \qquad \Omega = \hat{\boldsymbol{v}}^{\mathrm{T}}\boldsymbol{R}^{-1}\hat{\boldsymbol{v}} = \sum_i \frac{\hat v_i^{\,2}}{R_{ii}}\,\Bigg|_{\boldsymbol{R}\,\text{diag}}, \qquad \text{dof} = \operatorname{tr}\boldsymbol{\Theta},} \tag{161}$$

where Ω is the weighted sum of squared residuals and the degrees of freedom dof is the trace of the redundancy matrix $\boldsymbol{\Theta}$ (Equation (166) below), i.e. the *effective* redundancy of a recursively-filtered epoch rather than the naive $m - n$ of a single batch (`computeResidualQc_`, `src/core/filter/KalmanFilter.cpp`). Because $\boldsymbol{R}$ is carried in absolute units, the a-priori variance of unit weight is

exactly $\sigma_0^2 = 1$; a well-scaled model therefore yields $\hat{\sigma}_0^2 \approx 1$, with $\hat{\sigma}_0^2 \gg 1$ signalling under-weighted (optimistic) noise and $\hat{\sigma}_0^2 \ll 1$ pessimistic noise.

The variance factor is a *reporting-only* rescale: it never feeds back into the propagated covariance $\boldsymbol{P}$. When `enableVarianceFactor()` is set, the filter *logs* a copy of the position σ scaled by $\sqrt{\hat{\sigma}_0^2}$, i.e. $\sigma_{\text{pos}} \to \sigma_{\text{pos}} \sqrt{\hat{\sigma}_0^2}$, leaving the filter state untouched (`validatePostFitResiduals`, `src/core/filter/KalmanFilter.cpp`).

The same quantity has a batch incarnation in the normal-equation accumulator of the PPP NEQ-export path (constructed only under `enableNeqExport()` in `Application::run`). Given normal matrix $\boldsymbol{N}$, right-hand side $\boldsymbol{u}$, and the pre-reduced $\boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l}$ term, the accumulator forms (`NeqAccumulator::solve`)

$$\Omega = \boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l} - \boldsymbol{u}^{\mathrm{T}}\hat{\boldsymbol{x}}, \qquad \mathrm{dof} = n_{\text{obs}} - (n_{\text{params}} + n_{\text{local}}), \tag{162}$$

$$\hat{\sigma}_0^2 = \frac{\Omega}{\mathrm{dof}}, \qquad \sigma_i = \sqrt{\hat{\sigma}_0^2 \left(\boldsymbol{N}^{-1}\right)_{ii}}. \tag{163}$$

Here $\hat{\boldsymbol{x}} = \boldsymbol{N}^{-1}\boldsymbol{u}$ is the least-squares solution, the Schur-reduced local parameters n_{local} are subtracted from the redundancy alongside the retained parameters n_{params}, and each retained parameter's formal σ is the $\hat{\sigma}_0^2$-scaled square root of the inverse normal matrix diagonal — the batch analogue of Equation (161)'s recursive dof.

Neither of these is the network producer's *operational* variance factor. The undifferenced network estimator does not form Ω/dof at all; it estimates a robust unit-weight σ as a scaled median-absolute-deviation over the weighted phase-leveling residual rows $\tilde{v}_i = |\hat{v}_i| \sqrt{w_i}$,

$$\hat{\sigma}_{0,\text{net}} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \mathrm{median}_i\, \tilde{v}_i\big), \tag{164}$$

with the 1.4826 consistency factor recovering a Gaussian σ from the MAD and `raim_sigma0_floor` (default 1.0) flooring it. Unlike the Kalman variance factor of Equation (161), $\hat{\sigma}_{0,\text{net}}$ is emphatically *not* reporting-only: it drives a within-epoch RAIM adaptation loop whenever it exceeds `network.raim_sigma0_trigger`, a whole-epoch and per-constellation product withhold above `network.raim_withhold_sigma0`, and the scale on the emitted URA. Those three responsibilities live in three separate seams: `networkSolveGlue` (`filter/NetworkSolveGlue`) owns the within-epoch loop and both withhold gates, the per-constellation one through `NetworkEstimator::decidePerConstWithhold`; `networkPostFitRaim` (`filter/NetworkRaim`) owns the post-fit pass and the exponentially-weighted fold of $\hat{\sigma}_{0,\text{net}}$ into the emitted URA scale; and `networkSolPrep` (`filter/NetworkSolPrep`) owns the emission-side prune that actually removes a withheld constellation's satellites. Sections 12.6 and 12.7 develop the adaptation loop and the pooled statistics; the emitted-product side is not treated here.

A third, non-operational variance-factor instrument exists on the network path. `NetworkCarryVce` estimates Floor and Rate variance components of the carry-row stochastic model by residual-form least-squares variance-component estimation at fixed weight, purely to check the hand-set carry floors against a measured $\hat{\sigma}$. It is default-off (`network.carry_vce`), never feeds back into the solve, and the feedback path `network.carry_vce_apply` is a hard configuration-load error — the class has no apply method at all.

12.3 Redundancy numbers, the Baarda w-test, and the MDB

The detection theory rests on the post-fit residual covariance $\boldsymbol{Q}$ (the covariance of $\hat{\boldsymbol{v}}$),

$$\boldsymbol{Q} \;=\; \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}^{+}\,\boldsymbol{H}^{\mathrm{T}}, \tag{165}$$

where $\boldsymbol{P}^{+}$ is the *updated* (post-fit) state covariance, so that $\boldsymbol{Q}$ is the covariance of $\hat{\boldsymbol{v}}$ propagated through the gain. Its diagonal $\boldsymbol{Q}_{ii}$ is built per row in `KalmanFilter::update` as $R_{ii} - (\boldsymbol{H}\boldsymbol{P}^{+}\boldsymbol{H}^{\mathrm{T}})_{ii}$; the full matrix is formed only under the off-diagonal kernel of Section 12.5.

Redundancy number. The (local) redundancy of observation i is

$$\boxed{r_i \;=\; \frac{\boldsymbol{Q}_{ii}}{R_{ii}} \;=\; (\boldsymbol{\Theta})_{ii}, \qquad \boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i \in [0,1],} \tag{166}$$

clamped to $[0,1]$ for numerical safety in both branches of `computeResidualQc_`. $r_i \to 1$ means the observation is fully checked by the geometry (a blunder lands almost entirely in the residual); $r_i \to 0$ means it is *uncheckable* (a blunder is absorbed into the estimate and leaves no residual signature). The sum $\mathrm{dof} = \sum_i r_i = \operatorname{tr}\boldsymbol{\Theta}$ is the effective number of degrees of freedom used in Equation (161).

The network producer reimplements Equation (166) in the batch/arrowhead algebra of the Schur-conditioned shared covariance, $r_i = 1 - w_i\,\boldsymbol{a}_i^{\mathrm{T}}\operatorname{Cov}(\boldsymbol{x})\,\boldsymbol{a}_i$ with $\boldsymbol{a}_i = [\,\boldsymbol{a}_{s,i} \mid \boldsymbol{a}_{r,i}\,]$ and the same $[0,1]$ clamp — algebraically the batch twin of $r_i = \boldsymbol{Q}_{ii}/R_{ii}$ (`NetworkNormalEquations::rowRedundancy`, gated by `network.raim_redundancy_normalize`, default off). Rows falling below the `kRedFloor` redundancy floor are dropped from the $\hat{\sigma}_{0,\mathrm{net}}$ pool of Equation (164) as uncheckable — the network analogue of the KF snooping gate's `kUncheckableRedundancy` cutoff below, at a much smaller constant because it guards only the $\sqrt{r_i}$ standardisation, not a reset decision.

A structurally *exact* instance of $r_i = 0$ governs the network producer's entire phase-side QC design, and it is worth stating as a theorem rather than as a numerical caveat. Whenever each (station, satellite, signal) arc carries its own free ambiguity — the configuration in force whenever `network.phase_leveling` is off — every phase row is fitted *exactly* by its own parameter. The post-fit phase residual then vanishes identically, r_i is exactly zero, and no phase fault is detectable from the residuals at any magnitude: an undifferenced network with per-arc float ambiguities carries *no* phase-side redundancy whatsoever. Only when the ambiguities are stiffened by a carried prior do the phase rows acquire redundancy, and therefore fault-detection power — which is why the producer's phase RAIM (Section 12.6) is gated on phase-leveling in the first place, rather than merely working less well without it. The measure-only instrument that reports the collapse is `networkPostfitSigma0` (`filter/NetworkFrameDiag`), whose pooled phase variance factor is identically zero in that configuration; the zero is the finding, not a defect. Its pools deliberately exclude two row classes on the same reasoning: rows touching a shared spherical-harmonic column, which are pseudo-observations rather than data (Section 12.8), and the carry-coupling rows, which are a statement about the carry rather than about the epoch's measurements.

***w*-test.** Baarda's data-snooping statistic standardises each residual by its own post-fit standard deviation. The diagonal (uncorrelated) form is

$$w_i \;=\; \frac{\hat{v}_i}{\sqrt{\boldsymbol{Q}_{ii}}} \tag{167}$$

(the diagonal branch of `computeResidualQc_`), which — crucially — divides by $\sqrt{\boldsymbol{Q}_{ii}}$, the *true* residual standard deviation, not by $\sqrt{R_{ii}}$. This is the Studentised residual proper; under the null hypothesis $w_i \sim \mathcal{N}(0,1)$, whereas the s used by the fixed gate floors the denominator at R and is therefore conservative. When the residual covariance is non-diagonal the correlated Teunissen

generalisation is used instead (the `use_full` branch of the same routine):

$$\boxed{w_i \;=\; \frac{\left(\boldsymbol{Q}^{-1}\hat{\boldsymbol{v}}\right)_i}{\sqrt{\left(\boldsymbol{Q}^{-1}\right)_{ii}}}.} \tag{168}$$

Equation (168) reduces to Equation (167) when $\boldsymbol{Q}$ is diagonal, but accounts for the off-diagonal correlation among residuals introduced by differencing [25, 27].

Minimal detectable bias. The smallest blunder on observation i that the w-test will detect, at the configured significance and a conventional 80% test power, is

$$\boxed{\mathrm{MDB}_i \;=\; \frac{\sigma_i\,\delta_0}{\sqrt{r_i}}, \qquad \delta_0 = \texttt{dataSnoopingCritical()} + 0.8416,} \tag{169}$$

with $\sigma_i = \sqrt{R_{ii}}$ the a-priori observation σ (both branches of `computeResidualQc_`). The non-centrality δ_0 is the critical value plus $\Phi^{-1}(0.80) = 0.8416$ (the 20% false-negative quantile, i.e. $\beta = 0.20$ power); for the default critical $k = 3.29$ this gives $\delta_0 \approx 4.13$. The $1/\sqrt{r_i}$ factor is the geometric leverage: a poorly-checked observation ($r_i \to 0$) has an arbitrarily large MDB — the code returns $+\infty$ when r_i falls below `kRedFloor` — so a blunder there is by construction undetectable from the residual alone. The MDB is a Kalman-filter QC quantity only: the network producer implements no minimal detectable bias, relying instead on the robust $\hat{\sigma}_{0,\mathrm{net}}$ RAIM loop of Equation (164).

12.4 Data-snooping gate override

The QC statistics are not merely reported; when `enableDataSnooping()` is set they drive an *alternative, parallel* outlier gate that can reselect the row the fixed $|s| > 6$ path missed. Both gates share the same downstream dispatch inside `KalmanFilter::validatePostFitResiduals`; only the *statistic, index, and critical value* differ. By default the gate is seeded from the fixed-σ triple $(\max_i |s_i|, \arg\max, 6)$ so the dispatch is byte-identical when snooping is off.

When snooping is enabled the gate is instead reseeded from the w-test over *checkable* rows only: rows whose redundancy falls below the `kUncheckableRedundancy` cutoff are skipped, because their residual is unreliable and resetting the corresponding ambiguity would discard good information. (The same cutoff screens the summary log, so what is reported and what is gated agree.) Over the surviving rows the gate fires on

$$\max_{\{i\,:\,r_i \geq \texttt{kUncheckableRedundancy}\}} |w_i| \;>\; \texttt{dataSnoopingCritical()}, \tag{170}$$

and the worst checkable $|w_i|$ row becomes the handled index. Because w_i uses the true residual σ (Equation (167)) rather than the R-floored s, this gate can catch a blunder on a high-leverage row whose floored s stayed below 6, and it overrides the s pick: the downstream Melbourne–Wübbena tiebreaker still operates in the σ domain, so when the w-test selected a row the σ cut missed, no σ-candidate is collected and the w-selected index is handled directly. This is the complement of the fixed standardised-residual path in Section 11.4, which this document does not re-derive.

12.5 Correlation-aware full-$\boldsymbol{R}$ kernel

The diagonal forms above assume $\boldsymbol{R}$ and $\boldsymbol{Q}$ are diagonal, which holds for undifferenced PPP. For BSSD and baseline-DD epochs the differencing operator $\boldsymbol{D}$ of Section 5 maps the diagonal

undifferenced covariance into a *dense* one, $\boldsymbol{R}_{\mathrm{SD/DD}} = \boldsymbol{D}\,\boldsymbol{R}_{\mathrm{ZD}}\,\boldsymbol{D}^{\mathrm{T}}$, carrying the structured SD/DD cross-covariance between observations that share a pivot or reference. Under the off-diagonal kernel (`estimation.enable_full_r_kernel`, default off, which additionally requires that a dense $\boldsymbol{R}$ was actually supplied for the epoch — the `dense_r_supplied` test in `KalmanFilter::update`) the full residual covariance Equation (165) is built densely and symmetrised, and the QC quantities follow without approximation:

$$\boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i = \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \mathrm{dof} = \operatorname{tr}\boldsymbol{\Theta}, \tag{171}$$

$$\Omega = \hat{\boldsymbol{v}}^{\mathrm{T}}\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}, \qquad w_i = \frac{\big(\boldsymbol{Q}^{-1}\hat{\boldsymbol{v}}\big)_i}{\sqrt{(\boldsymbol{Q}^{-1})_{ii}}}, \qquad \mathrm{MDB}_i = \frac{\sqrt{R_{ii}}\,\delta_0}{\sqrt{r_i}}. \tag{172}$$

$\boldsymbol{R}^{-1}$ and $\boldsymbol{Q}^{-1}$ are obtained by LDL^{T} solves against the identity, exploiting the symmetric positive-definiteness of both. The kernel is consistent by construction: the diagonal of the full $\boldsymbol{\Theta}$ and of the full Teunissen w equal the per-row quantities of Section 12.3 whenever the off-diagonals vanish, so the *outlier-handling path is unchanged behind the flag* — it only gains correctness on differenced epochs where the residual correlations are real. Note the narrow scope of the flag: the gain and Joseph-form update already honour a dense $\boldsymbol{R}$ whenever one is supplied, independently of it; the flag decides only whether the *post-fit residual* covariance is formed densely. This is the measurement-side counterpart to the structured correlation Section 5 introduces on the model side.

12.6 Producer-side RAIM: within-epoch adaptation

The network producer's response to a tripped $\hat{\sigma}_{0,\mathrm{net}}$ is not a single exclusion but a choice of *granularity*, made from a robust two-way decomposition of the epoch's standardised residuals. Taking the largest standardised residual m_{ri} of each (station r, satellite i) carried-ambiguity cell as a sparse two-way table and applying a Tukey median polish,

$$m_{ri} \;\approx\; \mu \;+\; \alpha_r \;+\; \beta_i \;+\; \gamma_{ri}, \tag{173}$$

splits the contamination into a grand effect μ, a per-station row effect α_r, a per-satellite column effect β_i, and an isolated-arc interaction γ_{ri} (`medianPolish2Way`, `filter/NetworkRobustStats`). Sweeping medians rather than means is what makes the split usable: a minority of very large cells — the actual outliers — does not distort the additive effects, so a bad satellite lighting up a whole column is separable from a bad station lighting up a whole row, and genuinely isolated arc slips are left in the interaction term.

Each effect is a candidate culprit, and each must clear effect$/\hat{\sigma}_{0,\mathrm{net}} > k_{\mathrm{RAIM}}$ against the configured gate (`network.raim_k_sigma`) to be eligible at all. The three are then ranked surgical-first: on equal score an arc beats a satellite beats a station, and the station branch is additionally de-rated by a bias factor (`network.raim_station_drop_bias`, active under `network.raim_surgical_station`) so the coarsest response — which discards good arcs along with bad — is taken only when it dominates by that margin. A consensus rule short-circuits arc-by-arc peeling: when at least `network.raim_min_stations` of one satellite's arcs are over the gate in a single iteration, the arc candidate is redirected to the whole satellite, because a corrupted *shared* clock or phase bias is a network-wide fault rather than a stack of independent local ones.

The response itself distinguishes a bad *measurement* from a stale *parameter*, and that distinction is the load-bearing design decision. A whole-station culprit is excluded — its data are bad, and deweighting or exclusion is the right tool for bad data. A satellite or arc culprit, by contrast, is overwhelmingly a cycle slip, and a slipped ambiguity does not need less weight, it needs a new

value. The epoch is therefore replayed over a freshly rebuilt normal-equation system with the culprit's ambiguities *refloated*: their measurements retained at full weight, so the local block stays positive-definite and no drop-induced singularity is introduced, but their carried prior withheld, so the re-solve is free to find the new value and absorb the slip locally. The loop iterates until $\hat{\sigma}_{0,\mathrm{net}}$ falls below the trigger or a small fixed pass cap is reached; only the rare, already-contaminated epochs pay for a re-solve, and any failure reverts to the last good solution (`networkSolveGlue`).

Arming honesty: `network.raim` and `network.raim_within_epoch` both default to false, and *both* tiers are additionally gated on the phase-leveling carry being warm — the within-epoch loop inside `networkSolveGlue`, the post-fit pass at the `networkPostFitRaim` call site. With `network.phase_leveling` off there are no carried ambiguities to refloat and, by the redundancy-collapse argument of Section 12.3, no phase-side redundancy to test at all, so the producer's RAIM is inert by construction rather than by policy.

12.7 Pooled variance factors and the per-constellation carve-out

Equation (164) pools only the carried-ambiguity *phase* rows. Two extensions widen or partition that support without altering the estimator; both are default-off.

Code and iono-constraint pools. In the historical arrangement the code rows and the per-line-of-sight $\leftrightarrow$ spherical-harmonic iono-constraint rows are screened by no tier whatsoever. Under `network.raim_we_code_iono` the same scaled-MAD estimator is applied to each of them over its own disjoint row support,

$$\hat{\sigma}_{0,\mathrm{code}} = 1.4826\ \mathrm{median}_{i\in\mathcal{C}}\,\tilde{v}_i, \qquad \hat{\sigma}_{0,\mathrm{iono}} = 1.4826\ \mathrm{median}_{i\in\mathcal{I}}\,\tilde{v}_i, \tag{174}$$

each with its own trigger, and each row assigned to exactly one of the three supports. Neither pool carries the floor of Equation (164), and neither displaces the phase pool as the driver of the whole-epoch ceiling. The adaptation gains two response classes matched to what these rows can actually diagnose: an epoch-scoped station exclusion, and a network-wide *deweighting* of one satellite's code rows — a fallback, not a drop, in keeping with the project's standing preference for degrading a measurement over discarding it.

Per-constellation carve-out. When the global ceiling trips, the default response withholds the whole epoch's product. Under `network.per_const_withhold` the same pool is instead partitioned by the ambiguity's satellite constellation and each part judged on its own,

$$\hat{\sigma}_{0,c} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \mathrm{median}\{\tilde{v}_i : i \in \mathcal{P}_c\}\big), \tag{175}$$

over exactly the rows $\mathcal{P}_c$ that the global statistic pools — same statistic, same ceiling, restricted support. A constellation is published if and only if $\hat{\sigma}_{0,c}$ clears that ceiling *and* $|\mathcal{P}_c|$ meets a minimum row count, since too few rows cannot certify cleanliness inside a contaminated epoch (`NetworkEstimator::decidePerConstWithhold`).

The approximation is stated rather than hidden. The residuals entering Equation (175) are evaluated at the *joint* solution, so a contamination in one constellation perturbs the shared per-station nuisances (receiver clock, tropospheric delay, slant ionosphere) and leaks into another constellation's residuals; and each row's redundancy is its share of the joint system's redundancy, not of a hypothetical per-constellation solve. $\hat{\sigma}_{0,c}$ therefore under-separates cross-constellation coupling, and the outcome "every constellation individually clean while the global statistic tripped" is the honest edge of that approximation — it is published with a warning and counted, not silently

absorbed. The rank and observability gate is deliberately left with whole-epoch semantics: a rank deficiency invalidates the declared datum itself, so the product would be a meaningless least-norm split and no healthy subset exists to carve out.

12.8 Station-relative deweighting of the ionosphere constraint

A single-layer spherical-harmonic VTEC surface structurally cannot represent per-line-of-sight slant structure: the large majority of LOS residuals against the fitted field exceed 3σ at any degree and at every network scale. That exceedance is a chronic, roughly common-mode background, so an *absolute* 3σ gate would engage every station and diagnose nothing. The genuinely hostile sites — equatorial-anomaly and scintillation-belt stations whose true slant ionosphere lies beyond the field's representability — stand out only as a *relative* excess. That is the reason the gate below is network-relative and robust rather than absolute (`NetworkIonoDeweightGate`).

Each station's per-epoch misfit statistic m_r is first smoothed by an exponentially-weighted moving average with forgetting factor $\phi = 2^{-1/W}$, i.e. a half-life of W epochs, then standardised against the network's own robust centre and scale. Writing $\mathcal{M}$ for the set of mature, non-statically-listed stations,

$$z_r \;=\; \frac{\bar{m}_r - \mathrm{med}}{1.4826\,\mathrm{MAD}}, \qquad \mathrm{med} = \mathrm{median}_{r'\in\mathcal{M}}\,\bar{m}_{r'}, \qquad \mathrm{MAD} = \mathrm{median}_{r'\in\mathcal{M}}\big|\bar{m}_{r'} - \mathrm{med}\big|. \tag{176}$$

The median centring removes the common-mode background; the MAD scale tolerates a poison minority, its 50% breakdown point being precisely the property an absolute threshold lacks. A minimum mature population is required before the robust centre is formed at all, and statically-listed stations are excluded from it so that a known-bad site cannot mask the detection of others.

Detection is hysteretic — a station engages at $z_r > \sigma_{\mathrm{hi}}$ and releases at $z_r < \sigma_{\mathrm{lo}}$, with σ_{lo} forced below σ_{hi} — and while engaged the target weight follows a smooth IGG-style ramp, unity at the release threshold and equal to a configured floor $w_{\min}$ at and above the engage threshold, quadratic between:

$$\boxed{w_r^{*} = w_{\min} + (1 - w_{\min})\,(1 - t)^2, \qquad t = \frac{\mathrm{clamp}\big(z_r,\,\sigma_{\mathrm{lo}},\,\sigma_{\mathrm{hi}}\big) - \sigma_{\mathrm{lo}}}{\sigma_{\mathrm{hi}} - \sigma_{\mathrm{lo}}}.} \tag{177}$$

The realised weight slews toward w_r^{*} by at most a fixed increment per clean epoch and is clamped to $[w_{\min}, 1]$, which keeps the gate from oscillating against its own effect on the field fit.

The action is purely on $\boldsymbol{R}$. The factor w_r multiplies the *information* of that station's spherical-harmonic $\leftrightarrow$ slant-iono pseudo-observation rows, equivalently $\sigma^2 \mapsto \sigma^2/w_r$. Because those rows are one joint pseudo-observation coupling the station's per-LOS slant ionosphere to the shared field in both directions, a single w_r relaxes both directions uniformly; $w_r \to 0$ decouples that station's ionosphere entirely — it then floats on its own geometry-free combination, still observable, so no rank hole opens — while the station continues to serve the shared clocks, biases, datum graph and position. It is the surgical alternative to dropping a hostile station, and it is a weighting change only: no state is removed and no datum is altered.

Two honesty notes. Datum-change epochs, pivot handover included, are excluded outright — the gate is neither fed nor advanced on them, because their residual spikes are datum artefacts rather than station misfit — so every weight is simply held across them. And the adaptive path is default-off (`network.iono_constraint_deweight_adaptive`); with it off and the static station list empty, every w_r is unity and the producer is byte-identical to the ungated build. The gate is advanced once per epoch from the serial post-solve harvest in `networkShHealth` (`filter/NetworkShHealth`) and read (const, race-free) during the parallel per-station assembly.

12.9 State-domain emission gates

Everything above tests *residuals* against $\boldsymbol{R}$. The producer also carries two gates that test the *solved state* and act on the emitted product rather than on the estimation. A reader who meets them as "QC" should know both that they are a different kind of test and where they live — in neither case the seam whose name most suggests it.

Spherical-harmonic field health. Under `network.sh_health_gate` (default off) the solved VTEC coefficients are read back out of the shared solution, the dual-layer degree-zero terms are summed to a total mean VTEC, and that value is tested against a physically admissible band together with a ceiling on the RMS of all coefficients. An out-of-band epoch must repeat for a configured number of consecutive epochs before the gate engages, which debounces the single-epoch excursion. The verdict is *emission-only* by construction: the solve, the carry stash and every QC tier continue to run while emission is withheld, so an era lasting hours cannot expire the carry the way a skip-epoch response would. A failed coefficient lookup leaves the gate state unchanged rather than fabricating a release. The gate has no release-or-degrade policy — a sick era withholds for its full duration — and that is an open policy item, not a design conclusion. The verdict is formed in `networkSolveGlue`, the only pre-emission seam holding the solved shared vector; `networkShHealth` (`filter/NetworkShHealth`) owns the health *probe*, the deweight fold of Section 12.8 and the carry variance-component update, not this verdict.

Clock-runaway quarantine. Under `network.clock_runaway_check` (default off) the epoch's satellite clock solutions are grouped by constellation, and a constellation is dropped from the emission when the median of $|\text{clock} - \text{median}_{\text{all}}|$ exceeds a magnitude ceiling, or the median |drift| exceeds a rate ceiling. Like the carve-out of Section 12.7, this degrades scope rather than the whole product. The code documents an inversion hazard instead of defending against it: the magnitude test is referenced to the global median over *all* clock satellites, so if the runaway constellation supplies half or more of them the comparison inverts — the healthy minority trips the ceiling and is quarantined while the runaway passes. The single-constellation degenerate case is warned at configuration load. The quarantine lives in `networkSolPrep` (`filter/NetworkSolPrep`); `networkClockQc` (`filter/NetworkClockQc`) is a different tier entirely — the two-state oscillator model with its normalised-innovation fault test, the decoupled emit-clock-rate estimator, and the per-satellite carried-clock sanity monitor.

13 Estimator Infrastructure: WLS Seeding, State Initialisation, and Pivot Management

The Kalman filter of Section 11 is a *local* linear estimator: every update assumes the state lies inside a small neighbourhood of the linearisation point, and every dynamic state—ambiguity, per-satellite ionosphere, receiver code bias, satellite bias, inter-system offset (Section 3)—must be born with a value and a covariance before it can be updated. This document covers the *support* machinery that supplies those preconditions: the weighted-least-squares (WLS) epoch solve that drops the receiver inside the linearisation radius (Section 13.1); the first-epoch residual-differencing seeds that give each newly created state a physically informed prior (Section 13.2); and the between-satellite pivot selection and disturbance logic that keep the single-difference (SD) reference frame stable across an arc (Section 13.3). None of this is filtering proper—it is the scaffolding that makes the filter's local-linear assumptions valid at the first epoch and after every discontinuity.

The lossless covariance re-referencing algebra (the $\boldsymbol{T}\,\boldsymbol{P}\,\boldsymbol{T}^{\mathrm{T}}$ S-transform that re-anchors a group onto a new pivot) is deliberately *out of scope* here: it is derived in Section 5 and implemented once in `include/filter/CovarianceReReference.hpp`, which both the single-receiver and network handover paths call. This section names that seam where the pivot logic invokes it and otherwise refers the reader to the differencing derivation.

13.1 The WLS epoch estimator: receiver-clock seed and post-gap re-seed

Before the filter can run, the receiver position must be close enough to truth that the geometry Jacobian Equation (6) is valid to sub-decimetre. Broadcast pseudoranges alone fix this: a GPS-only, code-only iterated Gauss–Newton solve over the compact core state

$$\boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{r}^{\mathrm{T}} & c\,\mathrm{d}t_r & \mathrm{ZWD} \end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^5, \tag{178}$$

with $\boldsymbol{r} \in \mathbb{R}^3$ the ECEF receiver position, $c\,\mathrm{d}t_r$ the receiver clock in metres, and ZWD the zenith wet delay (`src/core/math/WlsEstimator.cpp`). At least `kCoreSize` $= 5$ code observations are required before the solve is attempted; the iteration cap is `kMaxIter` $= 5$.

For each code measurement i the prefit residual removes every modelled term and the current state's contribution,

$$\widetilde{z}_i = P_i - \left[\, P_i^{\mathrm{mod}} + (\rho_i - \rho_i^0) + c\,\mathrm{d}t_r + m_w(i)\,\mathrm{ZWD} \,\right], \tag{179}$$

where P_i^{mod} is the modelled pseudorange, $\rho_i - \rho_i^0$ the change in geometric range relative to the range at the iteration's linearisation point, and $m_w(i)$ the wet mapping function (`WlsEstimator::predictObs`, evaluated once per row from the design-row loop of `WlsEstimator::estimate`). The geometric range is *re-evaluated every iteration* about the running position estimate inside that loop, so this is a genuine Gauss–Newton relinearisation rather than a single-shot least-squares fit.

The design row for satellite i is the negated line-of-sight stacked with the clock and wet-mapping partials, assembled in the same loop:

$$\boldsymbol{A}_i = \begin{bmatrix} -\boldsymbol{e}_i^{\mathrm{T}} & 1 & m_w(i) \end{bmatrix} = \begin{bmatrix} -e_x & -e_y & -e_z & 1 & m_w(i) \end{bmatrix}, \tag{180}$$

with $\boldsymbol{e}_i$ the receiver-to-satellite unit vector (the same line-of-sight that drives Equation (7)). Stacking all rows into $\boldsymbol{A}$, the prefit residuals into $\widetilde{z}$, and the inverse measurement variances onto the diagonal weight $\boldsymbol{W}$ (clamped from below by `kMinVariance` $= 10^{-6}$ in that same loop), the normal-equations step is the boxed Gauss–Newton update

$$\boxed{\Delta\boldsymbol{\xi} = \left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right)^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\widetilde{z},} \tag{181}$$

applied once per Gauss–Newton iteration of `WlsEstimator::estimate`. Iteration terminates on a position increment $\|\Delta\boldsymbol{r}\| <$ `kTol` $= 0.1\,\mathrm{m}$ or at `kMaxIter`. The normal matrix is guarded against rank deficiency: if

$$\det\!\left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right) \le 10^{-6} \tag{182}$$

the solve aborts and the epoch is left un-seeded (the determinant guard immediately preceding the step), which protects against the degenerate single-constellation, near-coplanar geometry that would otherwise produce a wild position step.

On convergence the estimator returns both the state $\boldsymbol{\xi} = [\boldsymbol{r}^{\mathrm{T}},\ c\,\mathrm{d}t_r,\ \mathrm{ZWD}]^{\mathrm{T}}$ and its formal covariance, the inverse normal matrix

$$\boldsymbol{P}_{\boldsymbol{\xi}} = \left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right)^{-1} \tag{183}$$

(the result assembly at the tail of `WlsEstimator::estimate`). Of this pair the filter consumes only the *clock*: `Application::handleWlsSeeding` writes $c\,\mathrm{d}t_r$ into the receiver-clock state, and only when one of three triggers fires—the first epoch, a data gap, or a clock *jump* of at least `kClockJumpThreshSec` $= 0.5$ ms between the WLS and filter clock estimates (in practice the jump, not the gap, is the dominant trigger). The position, the zenith wet delay, and the covariance (183) are computed but not consumed as filter seeds: the position prior is the RINEX-header `approx_position` plus the antenna offset, and the WLS position re-seeds the filter only on a kinematic (non-IMU) long-gap cold start (`Application::handleGap`); the ZWD estimate reaches only a trace-CSV column; and the formal covariance has no filter consumer at all. The whole seeding path is skipped under BSSD, where the filter carries no receiver-clock state for the WLS clock to seed (`handleWlsSeeding` early-returns on `!hasClock()`). The clock seed's recurring role is *post-gap re-seeding*: the receiver clock is the most volatile state across a data gap (an unsteered oscillator can drift arbitrarily while no measurements constrain it), so after a gap the WLS clock estimate re-anchors $c\,\mathrm{d}t_r$ before the phase filter resumes, rather than letting the filter chase a stale clock prior through a sequence of large innovations.

13.2 First-epoch state seeding by residual differencing

The WLS solve of Section 13.1 seeds the receiver clock (and, on a kinematic cold start, position); it does not touch the *differential* states. Every such state—inter-system bias, phase clock, uncombined receiver code bias, per-satellite ionosphere, carrier ambiguity—is created empty and must be seeded from the measurement residuals. The unifying idea (`src/core/measurement/StateInitializer.cpp`) is that each such state is precisely the quantity left in a particular *difference* of residuals once all already-modelled terms are removed; estimating it is a matter of forming that difference and taking a robust (median) central value. The shared prefit residual is

$$\tilde{z}^i_{r,s} = \Phi/P^i_{r,s} - \big(\text{modelled} + c\,\mathrm{d}t_r + m_w\,\mathrm{ZWD} + \mathrm{ISB} + g_{\mathrm{trop}} + \alpha_s \widehat{I}^i + \beta_s b_r + b^i\big), \tag{184}$$

the observation minus every term the model can already account for, evaluated at the current state (`StateInitializer::computeRawResidual`, which also forms the tropospheric gradient projection $g_{\mathrm{trop}} = m_g(g_N \cos\mathrm{az} + g_E \sin\mathrm{az})$). The five *differential* seeds (a)–(e) below are differences of (184); a sixth, network-only pre-seed (f) anchors the receiver clock itself.

(a) Inter-system bias. Each non-GPS constellation g carries a receiver clock offset relative to the GPS time scale (Section 3). Because the GPS clock is the datum that the WLS step has already absorbed into $c\,\mathrm{d}t_r$, the ISB of g is the constellation-median code residual minus the GPS-median residual:

$$\boxed{\mathrm{ISB}_g = \mathrm{med}_{i\in g}\ \tilde{z}^i - \mathrm{med}_{i\in \mathrm{GPS}}\ \tilde{z}^i} \tag{185}$$

(`StateInitializer::initializeInterSystemBiases`). Medians, not means, are used throughout so a single gross code outlier cannot capsize the seed. A hard guard requires at least one GPS code observation at the seeding epoch (the GPS-median guard in that same routine): if GPS is absent it returns `false` and *retries the next epoch* rather than latching the ISB states as initialised against an arbitrary datum. This avoids the failure mode where a constellation that happens to acquire first becomes the implicit zero, and a GPS satellite rising later is then forced to wear the whole offset.

(b) Phase clock. In the uncombined model the phase observations see a receiver clock that differs from the code clock by the receiver phase–code hardware delay (Section 3). Its seed is the median

over satellites carrying *both* a code and a phase observation of the code-minus-phase residual:

$$\delta_\Phi = \mathrm{med}_i \left(\widetilde{z}_P^i - \widetilde{z}_\Phi^i \right) \tag{186}$$

(`StateInitializer::initializePhaseClockOffset`). The seed's covariance is then set from the code noise, tightened relative to the (looser) default initial standard deviation but *independent* of the n differences that formed the median (the covariance-seeding tail of that routine):

$$\sigma_{\delta_\Phi} = \max\!\Big(\tfrac{\sigma_P}{\sqrt{n}},\ \sigma_P \Big), \qquad \boldsymbol{P}(\delta_\Phi, \delta_\Phi) = \sigma_{\delta_\Phi}^2, \tag{187}$$

with the corresponding row and column of $\boldsymbol{P}$ first zeroed so the seeded state starts uncorrelated. The max against σ_P makes the $\sigma_P/\sqrt{n}$ branch unreachable—$\sigma_P/\sqrt{n} \le \sigma_P$ for every $n \ge 1$—so the seeded variance is σ_P^2 regardless of n: a conservative floor at the single-measurement code noise, not a $1/\sqrt{n}$ promise.

(c) Uncombined receiver code bias / IFB. In the uncombined PPP-RTK model each non-reference code band carries a receiver differential code bias (DCB) or inter-frequency bias (IFB) state relative to the constellation's BIA clock-reference band (Section 3). Forming the residual difference between band j and the reference band, removing the differential ionospheric contribution (the two bands scale the same slant ionosphere by different α), and dividing by the difference of receiver-bias Jacobian coefficients gives

$$\boxed{\widehat{b}_{r,j} = \frac{\Delta\widetilde{z}_j - \Delta(\alpha_j\, \widehat{I})}{\Delta\beta_j} = \frac{(\widetilde{z}_j - \widetilde{z}_{\mathrm{ref}}) - (\alpha_j - \alpha_{\mathrm{ref}})\, \widehat{I}^i}{\beta_j - \beta_{\mathrm{ref}}}} \tag{188}$$

(`StateInitializer::initializeCodeBiases`), median-aggregated over satellites and subject to the same $|\widehat{b}| > 200\,\mathrm{m}$ reject as the ionosphere seed below. The Jacobian coefficient β_s depends on the bias parameterisation (`StateInitializer::resolveReceiverBias`): for a pure IFB state it is the unit coefficient $\beta = 1$, whereas for a geometry-free / ionosphere-free DCB state against reference bands of frequencies f_1, f_2 it is

$$\mu_1 = 1, \quad \mu_2 = \frac{f_1^2}{f_2^2}, \quad \Delta\mu = \mu_1 - \mu_2, \qquad \beta_j = \begin{cases} \mu_1/\Delta\mu & j = \text{band 1} \\ \mu_2/\Delta\mu & j = \text{band 2}, \end{cases} \tag{189}$$

guarded by $|\Delta\mu| \ge 10^{-12}$ to avoid dividing by a near-degenerate frequency separation.

A third, higher-priority branch precedes both cases for GLONASS: under `enable_glonass_ifb` each FDMA band (G1, G2) carries one bias state *per FCN slot* k with $\beta = 1$ (`StateKeys::makeGlonassPerSlotIfbKey`), because the per-slot carrier $f = f_{\mathrm{base}} + k\,\Delta f$ gives a slot-dependent receiver group delay that a single per-band state cannot represent; the central channel $k = 0$ is the held datum (no state) and the CDMA bands (G3/G4/G6) fall through to the legacy per-band treatment above. This branch runs before the clock-reference lookup because GLONASS usually ships no clock-reference pair in the `.BIA` product and would otherwise early-return before reaching any IFB logic; with the flag off a non-reference GLONASS band carries no IFB state at all (the per-slot and non-reference-band branches of `resolveReceiverBias`).

(d) Per-satellite slant ionosphere. Each tracked satellite's slant ionospheric delay (Section 3) is recovered from the geometry-free combination of two code bands. With ionospheric scale factors $\alpha_s = f_{\mathrm{ref}}^2/f_s^2$ referenced to GPS L1 (Equation (1)), the seed is

$$\boxed{\widehat{I}^i = \frac{(P_1 - P_1^{\mathrm{mod}}) - (P_2 - P_2^{\mathrm{mod}})}{\alpha_1 - \alpha_2}} \tag{190}$$

(`StateInitializer::computeIonosphereSeed`), where P_k^{mod} is the modelled pseudorange on band k. Two structural guards apply: the denominator must satisfy $|\alpha_1 - \alpha_2| \geq 10^{-6}$ (the denominator guard in that routine), and the seed is rejected unless it is finite and $|\widehat{I}^i| < 200\,\mathrm{m}$ (its magnitude reject). The 200 m reject is a physical sanity bound—a slant ionospheric delay larger than this is never real and signals a band mis-pairing or a gross code outlier, which must not be latched into a per-satellite state. The seed additionally applies two BIA-coverage gates so the differenced code is bias-consistent: *presence* (a `.BIA` entry exists on the band, with the zero-valued BeiDou C2I/C6I entries treated as missing), and—outside the network producer, which keeps presence-only semantics—coverage *honesty* (`bias_coverage_requires_apply`), which skips the seed when an entry is loaded but not *applied* to ($P -$ mod) so the raw hardware bias still survives. The honesty gate matters precisely because the failure it blocks—a zero-valued BeiDou entry injecting 30–60 m—sits well under the 200 m reject above and would not be caught by it.

(e) Carrier ambiguity. The float ambiguity that subsequently feeds integer resolution (Section 15) is what remains of the *phase* residual once every other modelled and estimated term has been stripped (`StateManager::computeAmbiguitySeed`, the shared routine every phase-seeding call site invokes),

$$\widehat{N}^i_{r,s} = \Phi^i_{r,s} - \text{modelled} - \left(c\,\mathrm{d}t_r + \delta_\Phi\right) - m_w\,\mathrm{ZWD} - \mathrm{ISB} - g_{\mathrm{trop}} + \alpha_s\,\widehat{I}^i, \tag{191}$$

i.e. the phase observation minus its model, receiver clock *and* phase clock δ_Φ (seed (b)), wet delay, inter-system bias, and tropospheric gradient, plus the slant ionospheric phase advance $+\alpha_s\,\widehat{I}^i$ (seed (d))—the phase leads by the same α_s the code lags, so the iono enters with the sign opposite to (184). Both the phase clock and the slant iono are removed here because they are already estimated states in the uncombined model; the one term this subtraction *cannot* remove is a receiver clock still sitting at zero on a network cold start, which is what pre-seed (f) exists to anchor. By construction this is the phase residual after removing every term the filter already estimates, so the seeded ambiguity absorbs exactly the integer cycle count plus the residual receiver/satellite phase-bias remainder—the quantity whose integer part AR recovers once the satellite phase bias is supplied and the receiver phase bias cancels under between-satellite differencing.

(f) Cold receiver-clock pre-seed (network path). Seeds (a)–(e) assume the receiver clock has already been solved—true on the PPP path, where the WLS clock seed of Section 13.1 runs first. The network producer does not run that WLS seed, so on the network path nothing pre-solves the receiver clock before the ambiguity of (e) is seeded: `computeAmbiguitySeed` then subtracts a clock term of zero and the entire free-running receiver-clock offset leaks uniformly into every cold ambiguity seed. The opt-in `seedColdReceiverClock` (`network.cold_clock_seed`, default off) closes this by writing the median GPS code residual—the very `gps_median` that ISB seed (a) computes and discards—into x(clock). It is self-limiting: it overwrites only a *cold* clock, treating any $|x(\text{clock})| >$ `kColdClockEpsilonM` $= 10^{-6}$ m as already solved and latching it untouched, and it returns `false` (retry next epoch, do *not* latch) when the epoch carries no GPS code observation—mirroring the GPS-reference contract of seed (a) (`StateInitializer::seedColdReceiverClock`).

13.3 BSSD pivot management

Between-satellite single differencing (BSSD, Section 5.1) re-expresses each constellation's ambiguities relative to a designated *pivot* satellite p_g, cancelling the receiver phase bias and rendering the SD ambiguities integer. The pivot is not arbitrary: a poor choice degrades every SD row it anchors, and an unstable choice forces costly re-referencing handovers. `src/core/filter/PivotManager.`

cpp selects and maintains it; the baseline double-difference path uses a structurally analogous src/core/filter/DdPivotManager.cpp.

13.3.1 Lexicographic candidate ranking

Candidates are ordered *lexicographically*: a hard signal-coverage tier first, then a continuous score within the tier. Coverage dominates because the pivot must track every signal any non-pivot tracks—a non-pivot signal the pivot does not carry is orphaned and dropped—so *any* full-coverage candidate outranks *any* partial-coverage candidate regardless of score (the lexicographic comparator in PivotManager::buildEpochMaps). Concretely, candidate a beats b iff its phase-signal count is higher, or the counts tie and its score is higher.

Within a coverage tier the tie-break score combines float-convergence and geometry (PivotManager::scorePivotCandidate):

$$\boxed{\mathrm{score}(i) = -\min\!\big(1,\ \sqrt{v_{\max}(i)}\,\big) + \frac{\mathrm{el}(i)}{90} - \tfrac{1}{2}\,\mathbb{1}\!\left[\mathrm{el}(i) < 20^\circ\right],} \tag{192}$$

where $v_{\max}(i)$ is the worst (largest) float-ambiguity variance among the satellite's tracked phase signals and $\mathrm{el}(i)$ is its elevation in degrees. The first term penalises an un-converged pivot: on handover the new pivot's variance is added to every same-group SD state (the diagonal of the re-reference, $\boldsymbol{P}'(i,i) = \boldsymbol{P}(i,i) + \boldsymbol{P}(p_g,p_g) - 2\boldsymbol{P}(i,p_g)$ — the diagonal of the Section 5, implemented in include/filter/CovarianceReReference.hpp), so a pivot whose own state is still settling injects that uncertainty into the entire constellation; the penalty saturates at -1 (one metre of equivalent 1σ ambiguity uncertainty), with a flat -1 sentinel for a candidate that has *no* state yet (the negative-max_amb_variance sentinel branch of that score). On the first BSSD epoch every candidate in a tier shares the sentinel, so the score collapses to elevation alone and no special first-election branch is needed. The el/90 term prefers high-elevation pivots, and the explicit $-\frac{1}{2}$ sub-20° penalty hard-discourages a low-elevation pivot whose tropospheric and multipath residuals exceed the formal stochastic model regardless of how converged its float state looks. Both score terms only ever *rank* candidates that have already cleared a hard elevation gate: a satellite below pivot_min_el (getBssdPivotMinElevation) is barred from election outright—in the freshest-peer scan, the online pick, and the sticky-retention test, all three in PivotManager::selectPivotCandidates—so the $-\frac{1}{2}$ penalty only breaks ties among survivors rather than being the mechanism that keeps low satellites out. The gate falls through deliberately when *no* satellite clears the mask (a genuine code-only-float constellation): the pick reverts to the precomputed best/highest-elevation satellite so the candidate set is never emptied (the fallback from pickExclusionAwareBest to the precomputed best_sat / highest_sat). Sticky retention—the code's own *elevation hysteresis*—suppresses epoch-to-epoch flip-flop, but on that same *absolute* gate, not a challenger comparison: the incumbent is kept whenever it still clears pivot_min_el, is not excluded, and stays within the toe spread of Section 13.3.2 (the sticky branch of selectPivotCandidates), with no score evaluated on that branch, so a strictly better challenger cannot displace a qualifying incumbent; the score re-ranking runs only once the incumbent has *failed* the gate.

13.3.2 Ephemeris-freshness guardrail

A pivot anchored to a stale broadcast ephemeris steps *every* SD row that references it, because its extrapolated orbit and clock drift away from the fresher peers it is differenced against (observed in production as a Galileo pivot pinned to a one-hour-old I/NAV record that detonated the entire SD frame while its peers were minutes fresh). The guardrail is *spread-gated*: a candidate is barred

from pivot election when its absolute ephemeris age $|t_{\text{obs}} - t_{\text{oe}}|$ exceeds the constellation's freshest qualified peer by more than

$$\texttt{kPivotToeSpreadMaxS} = 1200 \text{ s} = 20 \text{ min} \tag{193}$$

(the `freshestToeAgeAbs` scan in `PivotManager::selectPivotCandidates`, applied both at the online pick and at the sticky-retention test). Gating on the *spread* rather than an absolute age means that when every candidate is comparably fresh the ranking is untouched—only an outlier-stale candidate is displaced. The barred satellite remains usable as a non-pivot; the freshest peer always passes (its difference from itself is zero), so the candidate set is never emptied.

13.3.3 Pivot disturbance detection and handover

By the BSSD construction a held pivot's own SD ambiguity is identically zero with zero variance—it *is* the reference, so $x(p_g) = 0$ and $\boldsymbol{P}(p_g, p_g) = 0$ exactly. Any departure from that frozen state signals that the filter has, through process noise or an update, perturbed the reference and the SD frame is no longer self-consistent. `isPivotDisturbed` flags this when, for any of the pivot's phase signals,

$$\boxed{\left|x(p_g)\right| > 10^{-6} \quad \text{or} \quad \boldsymbol{P}(p_g, p_g) > 10^{-12}} \tag{194}$$

(`PivotManager::isPivotDisturbed`). The two thresholds are numerical-zero tolerances, not tuned gates: 10^{-6} m on the value and 10^{-12} m^2 on the variance are tight enough to catch any genuine leakage of state onto the pivot while tolerating floating-point round-off in the re-reference arithmetic. To prevent the predict step from *manufacturing* a spurious disturbance, the pivot's process noise is suppressed during prediction so $\boldsymbol{P}(p_g, p_g)$ does not grow off zero (Section 11); without that suppression every epoch's $\boldsymbol{Q}$ would trip (194) and force a needless handover.

When a handover does fire—either because the incumbent was disturbed or because ranking elected a different satellite—the outgoing pivot is classified. A *clean* handover (the old pivot's state is invertible and frame-consistent) can be re-referenced losslessly and remains smoother-inverse-referenceable, so the RTS smoother can later undo it; a *forced* or disturbed handover cannot be inverted cleanly and marks a frame discontinuity. In either case the actual covariance re-anchoring—subtract the new pivot's row/column from each non-pivot group member, then zero the new pivot's row/column—is performed in place by `CovReReference::applyBssd` in `include/filter/CovarianceReReference.hpp`, the single shared implementation of the $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ transform Section 5; it is *not* re-derived here. The baseline DD path differs structurally: rather than a subtract-and-freeze on a single pivot it negates the new-pivot row/column and subtracts from the others (a different S-transform), then re-keys the affected ambiguity states via `StateManager::renameAmbiguity` (`DdPivotManager::handleHandover`). Both paths share only the post-transform positive-definiteness guard (`enforcePsd`); the single-receiver user path runs the transform unguarded, since at user scale the groups are small and handovers rare enough that floating-point error in $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ does not accumulate.

Part IV
Cycle-slip detection and repair

14 Cycle-slip detection and repair

The uncombined filter (Section 4) never forms a linear combination on the measurement-update path: per-signal $P^i_{r,s}$ and $\Phi^i_{r,s}$ enter Equations (4) and (5) directly. Cycle slips are instead screened by a bank of *diagnostic* combinations, each constructed so that the geometric range ρ^i_r, the clocks $c(\mathrm{d}t_r - \mathrm{d}t^i)$ and the troposphere T^i_r cancel, leaving an observable whose only between-epoch variation is measurement noise plus any integer carrier discontinuity. Three detectors are compiled and registered: Melbourne–Wübbena (`src/core/slip/MwSlipDetector.cpp`), the primary and the only one armed by default; GRAPHIC (`src/core/slip/GraphicSlipDetector.cpp`); and the extra-wide-lane detector (`src/core/slip/EwlSlipDetector.cpp`), which additionally needs a third frequency. The latter two are opt-in (Section 14.1) and no bundled config enables them. The ionosphere-free phase detector (`src/core/slip/IfPhaseSlipDetector.cpp`) is *compiled and armed on the network path* — instantiated per non-pivot station whenever `network.cycle_slip.if_phase_td` is set (Section 14.2.5) — but stays staged on the single-receiver user/kinematic path, where the position-uncertainty gate it needs is unsafe. The geometry-free phase detector (`src/core/slip/GfSlipDetector.cpp`) is by contrast still *staged* — present in the tree but excluded from the build (it is commented out of the explicit CMake source list, unlike the IF detector beside it) — and its algebra is recorded here for completeness. The user/kinematic bank is composed at run time from the config block `estimation.cycle_slip`; the network producer composes its own per-station set from a parallel `network.cycle_slip` block (Section 14.1).

Throughout this section we suppress the receiver index r (single-receiver PPP / PPP-RTK) and write $f_1 > f_2$ for the two carriers of a band pair, sorted by frequency so that $f_1 - f_2 > 0$ and $\lambda_1 < \lambda_2$ (`src/core/slip/MwSlipDetector.cpp`, the band-frequency sort). Carrier phase is read from RINEX in cycles and scaled by $\lambda_s = c/f_s$ before combining, so Φ^i_s below is always in metres, consistent with Equation (5). We reserve the operator $\delta\,\cdot$ for a *temporal* integer slip step (e.g. δN_s is the change in the ambiguity on signal s across the slip epoch); this is deliberately distinct from the between-satellite single difference $\Delta\cdot$ of Section 5, which also uses Δ.

14.1 Detector registry and composition

The detectors are not hard-wired. Each implements a narrow polymorphic contract `ISlipDetector` (`include/slip/ISlipDetector.hpp`) — a single per-epoch entry point

$$\texttt{detect}(\text{SlipEpochContext}) \;\longrightarrow\; \{\,\text{SlipEvent}\,\},$$

plus lifecycle hooks (`resetSatellite`, `resetSatelliteSignal`, `enableTracing`, `setCorrections`). A `SlipEvent` is the union of the legacy per-detector result structs; its `granularity` field distinguishes a *per-signal* flag (GRAPHIC, GF, IF: one carrier) from a *band-difference* flag (MW, EWL: a wide-lane integer $\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2$ that, on its own, cannot attribute the jump to one signal — the two-unknown problem Equation (209) resolves downstream). Inputs are bundled in `SlipEpochContext` so the signature is uniform even though only the IF detector consumes the extra 3D position sigma and receiver ECEF it carries — and on the network path, where it is armed, that gate is satisfied by construction because the estimator pins station positions.

`SlipDetectorRegistry::createEnabled` (`src/core/slip/SlipDetectorRegistry.cpp`) builds the config-selected set from `estimation.cycle_slip.{mw,graphic,ewl,gf,if}` in a

deterministic order — MW first (it is the primary and the sole source of the wide-lane integers consumed by ambiguity resolution (Section 15) and the BSSD pivot machinery (`src/core/filter/PivotManager.cpp`)), then the auxiliaries. MW defaults on; the auxiliary detectors default off, preserving the historical MW-only behaviour. A kind that is enabled in config but for which the registry holds no creator is skipped with a warning rather than aborting, so a forward-looking config degrades gracefully. Two kinds skip for distinct reasons: `gf`, whose `.cpp` is excluded from the build (Section 14.2.4); and `if`, whose `.cpp` *is* compiled but which is instantiated directly on the network path (Section 14.2.5) rather than through this registry, so `SlipDetectorRegistry` registers no `IonoFreePhase` creator either. `CycleSlipManager` drives the whole set through one loop and recovers the MW detector's richer API (WL-lock queries, pivot-handover hints, per-satellite quality scoring) through a typed down-cast (`asMelbourneWubbena`) rather than widening the base interface.

The network producer does not use this registry at all, for any detector. Each station's set is constructed directly in the `buildStation` lambda of `Application::networkSetup_` (`src/core/app/ApplicationNetworkSetup.cpp`) from a *parallel* config block, `network.cycle_slip.{mw_detect, turbo_edit, if_phase_td}` — distinct keys from their `estimation.cycle_slip` namesakes, and all three default off. The two paths share the detector *classes* but neither the composition mechanism, nor the config block, nor the response doctrine: the network refloats where the user path repairs (Section 14.3). One consequence of composing by hand is load-bearing. With `network.cycle_slip.mw_detect` on and `network.cycle_slip.turbo_edit` off, the station is given an `MwSlipDetector` but a null `TurboEditResolver`, and that null pointer is precisely what steers the detection branch away from integer repair and onto the refloat path.

`turbo_edit` (Section 14.4) sits a level above the registry: it is *not* a detector kind but a *coordination mode* that, when the MW detector is enabled, pairs the MW wide-lane integer with the geometry-free phase jump to recover per-signal integers. It is therefore documented separately from the detector bank below.

14.2 Test-statistic combinations

14.2.1 Melbourne–Wübbena (wide-lane)

The Melbourne–Wübbena (MW) observable is the wide-lane carrier phase minus the narrow-lane code. Introducing the wide- and narrow-lane wavelengths

$$\lambda_{\mathrm{WL}} \;:=\; \frac{c}{f_1 - f_2}, \qquad \lambda_{\mathrm{NL}} \;:=\; \frac{c}{f_1 + f_2}, \tag{195}$$

the engine forms (`MwSlipDetector::computeMwMeters`)

$$\boxed{\mathrm{MW}^i \;=\; \underbrace{\frac{f_1\,\Phi_1^i - f_2\,\Phi_2^i}{f_1 - f_2}}_{\text{wide-lane phase}} \;-\; \underbrace{\frac{f_1\,P_1^i + f_2\,P_2^i}{f_1 + f_2}}_{\text{narrow-lane code}}.} \tag{196}$$

The range-additive terms common to Equations (4) and (5) — geometry, clocks, troposphere — cancel between the two bracketed combinations. The slant ionosphere also cancels: the wide-lane phase carries the ionospheric coefficient $-(f_1\mu_1 - f_2\mu_2)/(f_1 - f_2)$ and the narrow-lane code carries $+(f_1\mu_1 + f_2\mu_2)/(f_1 + f_2)$, and with $\mu_s = f_1^2/f_s^2$ (Equation (1)) these two coefficients are equal, so the difference (196) removes the ionosphere exactly to first order. What remains is a near-constant equal to the wide-lane ambiguity plus hardware bias,

$$\mathrm{MW}^i \;\approx\; \lambda_{\mathrm{WL}}\, N_{\mathrm{WL}}^i \;+\; b_{\mathrm{MW}}^i \;+\; \varepsilon_{\mathrm{MW}}, \qquad N_{\mathrm{WL}}^i = N_1^i - N_2^i, \tag{197}$$

so a wide-lane slip $\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2$ appears as a step of $\lambda_{\mathrm{WL}}\,\delta N_{\mathrm{WL}}$. For GPS L1/L2, $\lambda_{\mathrm{WL}} \approx 0.861\,\mathrm{m}$ while the dominant error is the metre-level narrow-lane *code* noise; with code noise $\sigma_P \sim 0.3\,\mathrm{m}$ the MW noise is $\sigma_{\mathrm{MW}} \approx \sigma_P/(\sqrt{2}\,\lambda_{\mathrm{WL}}) \sim 0.25$ wide-lane cycles, i.e. code-noise-limited. The integer-ready (BIA-corrected) variant subtracts the satellite MW bias b^i_{MW}, which has the same algebra as Equation (196) applied to the OSB phase and code biases of Equations (4) and (5).

The level series $\mathrm{MW}^i(k)$ is screened against a robust adaptive gate whose deviation statistic is configurable via `estimation.cycle_slip.mw_detect_reference`. The recommended choice, and the one every bundled real-time and replay config sets explicitly, is the first time difference $\mathrm{TD1}(k) = \mathrm{MW}^i(k) - \mathrm{MW}^i(k-1)$, gated against the median-absolute-deviation of its own history; the second time difference $\mathrm{TD2}(k) = \mathrm{MW}^i(k) - 2\,\mathrm{MW}^i(k-1) + \mathrm{MW}^i(k-2)$ is available (`mw_td_order`) for curvature immunity at a $\sqrt{6}$ noise penalty versus the $\sqrt{2}$ of TD1 (the white-noise scaling of each statistic is the single source of truth in `MwSlipDetector::activeStatScale`). TD1 is *not* the loader default: a config that leaves `estimation.cycle_slip.mw_detect_reference` unset runs the legacy Hatch mean, which is kept as the default for backward compatibility and as an A/B arm alongside the windowed local median (`hatch_mean`, `local_median`). The threshold itself is fully data-driven — no stochastic-model σ enters — combining a per-arc MAD, an EWMA-smoothed cross-satellite MAD floor and an elevation $1/\sin\theta$ shape, with a breadth cascade (band-pair → constellation → global-no-GLONASS) for sparse or cold-start arcs; the empirical construction lives in `MwSlipDetector::computeAdaptiveThreshold` and is intentionally not reproduced here.

14.2.2 Extra-wide-lane (EWL)

For three-or-more frequency satellites the same MW algebra (Equation (196)) is applied to a pair of *non-L1* carriers (`EwlSlipDetector::computeEwlMwMeters`; band digit ≥ 2). Because the two carriers are spectrally close — $f_1 - f_2$ small — the extra-wide-lane wavelength

$$\lambda_{\mathrm{EWL}} \;=\; \frac{c}{f_1 - f_2} \tag{198}$$

is very long: GPS L2/L5 gives $\lambda_{\mathrm{EWL}} \approx 5.86\,\mathrm{m}$ and Galileo E5a/E5b $\approx 9.77\,\mathrm{m}$. Metre-level code noise then maps to $\lesssim 0.05$ EWL cycles, so the EWL integer $N_{\mathrm{EWL}} = N_2 - N_5$ resolves in a single epoch and a slip δN_{EWL} is sized with near-certainty. Together with the L1/L2 wide-lane (Equation (197)) this yields two independent band-difference constraints,

$$\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2, \qquad \delta N_{\mathrm{EWL}} = \delta N_2 - \delta N_5, \tag{199}$$

narrowing the per-signal triple $(\delta N_1, \delta N_2, \delta N_5)$ to a single remaining degree of freedom [5].

14.2.3 GRAPHIC (single-frequency half-sum)

The GRAPHIC combination averages the code and phase on a *single* band (`GraphicSlipDetector::computeGraphic`):

$$\mathrm{GR}^i_s \;=\; \tfrac{1}{2}\big(P^i_s + \Phi^i_s\big), \tag{200}$$

with Φ^i_s in metres. Because the slant ionosphere enters $+\mu_s I^i_{r,1}$ on the code and $-\mu_s I^i_{r,1}$ on the phase of the *same* signal (Equations (4) and (5)), the half-sum cancels it to first order — GRAPHIC is ionosphere-free without a second frequency, at the cost of retaining half the code noise. The combination still contains the geometric range, clocks and troposphere plus the constant half-ambiguity $\frac{1}{2}\lambda_s N^i_s$, so the between-epoch first difference

$$\mathrm{dGR}^i_s(k) \;=\; \mathrm{GR}^i_s(k) - \mathrm{GR}^i_s(k-1) \tag{201}$$

removes the constant ambiguity (and cancels range and clocks down to receiver-motion / code-noise level). A δN-cycle slip on signal s therefore produces a jump of $\frac{1}{2}\lambda_s\,\delta N$, and the integer estimate is $\delta N = \mathrm{round}\big(2\,\mathrm{dGR}/\lambda_s\big)$. With $\sigma_P \sim 0.3\,\mathrm{m}$ the post-difference noise is $\sim 0.3\,\mathrm{m}$, so sensitivity is noise-limited to a few L1 cycles; in practice a hard floor `kMinThresholdM` $= 0.5\,\mathrm{m}$ governs on quiet arcs (≈ 5 L1 cycles), keeping GRAPHIC a coarse backstop rather than a sub-cycle detector [1, 29].

14.2.4 Geometry-free phase (staged)

The geometry-free (GF) observable is the between-band phase difference

$$\mathrm{GF}^i \;=\; \Phi_1^i - \Phi_2^i \;\approx\; (\mu_2 - \mu_1)\, I_{r,1}^i \;+\; (\lambda_1 N_1^i - \lambda_2 N_2^i), \tag{202}$$

which removes geometry, clocks and troposphere exactly but retains the *difference* of the two ionospheric delays. Since $I_{r,1}^i$ varies smoothly, a second-order time difference

$$\mathrm{TD2}_{\mathrm{GF}}(k) \;=\; \mathrm{GF}^i(k) - 2\,\mathrm{GF}^i(k-1) + \mathrm{GF}^i(k-2) \tag{203}$$

cancels the ionospheric trend up to quadratic order over the sampling interval, leaving noise plus slip (`GfSlipDetector::computeGfMeters` for Equation (202); the TD2 update is the level-jump test statistic). The detector is intrinsically high-rate ($\geq 1\,\mathrm{Hz}$): at coarse rate the residual ionospheric curvature defeats (203). It is present in the tree but excluded from the build.

The TD2 statistic (203) is not orphaned by that staging. `slip/IonoVariabilityMonitor` forms the same second difference per satellite on the two lowest RINEX bands (`selectPrimaryPair`, `computeGfMeters`) and *is* compiled — but it is a process-noise *monitor*, not a detector. Its per-satellite output is the clamped squared ratio of a recent-window MAD-derived σ of $\mathrm{TD2}_{\mathrm{GF}}$ to a baseline-window one,

$$m^i \;=\; \mathrm{clamp}\left(\left(\hat{\sigma}_{\mathrm{recent}}/\hat{\sigma}_{\mathrm{baseline}}\right)^2,\;\; 1,\;\; m_{\max}\right), \tag{204}$$

and the ratio form is what makes it self-normalising: the slowly varying obliquity-curvature content of Equation (202) sits in *both* windows and cancels, so a quiet sky returns $m^i \approx 1$ at any elevation and the $1/\sin^2\theta$ obliquity scaling already carried by the ionospheric process noise is not double-counted. The consequence for this chapter is that the monitor treats cycle slips as isolated single-epoch outliers that the MAD is designed to *reject* — the exact opposite of a detection statistic — so it must not be read as a substitute for the staged detector. Both of its arming keys default off: `estimation.adaptive_iono_process_noise.enable` on the user path, where m^i scales the static ionospheric spectral density into a per-satellite zenith override consumed by `StateManager::setIonoProcessNoiseOverrides`, and `network.adaptive_iono_q` per station on the network path, where it instead scales the deviation-channel innovation of the only time-linked slant-iono element the producer carries. The window lengths and the clamp $m_{\max}$ are empirical and stay in config.

14.2.5 Ionosphere-free phase (staged on the user path; armed on the network path)

The dual-frequency ionosphere-free (IF) phase combination uses the coefficients

$$c_1 = \frac{f_1^2}{f_1^2 - f_2^2}, \qquad c_2 = \frac{f_2^2}{f_1^2 - f_2^2}, \qquad c_1 - c_2 = 1, \tag{205}$$

giving $\Phi_{\mathrm{IF}}^i = c_1\Phi_1^i - c_2\Phi_2^i$ and the effective wavelength $\lambda_{\mathrm{IF}} = \left|c_1\lambda_1 - c_2\lambda_2\right|$ (`IfPhaseSlipDetector::computeIfWavelengthMeters`); for GPS L1/L2 $\lambda_{\mathrm{IF}} \approx 0.107\,\mathrm{m}$. Rather than screen the raw

combination, the detector forms the IF observed-minus-computed (OMC), differences it between satellites against a high-elevation pivot p_g to remove the receiver clock, the inter-system offset $c\,\delta t_r^{(g)}$ and the satellite phase clock, then between epochs to remove the slowly-varying wet troposphere and the constant ambiguity:

$$\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g}(k) = \big[\Phi_{\mathrm{IF}}^{i}(k) - \Phi_{\mathrm{IF}}^{p_g}(k)\big] - \big[\Phi_{\mathrm{IF}}^{i}(k-1) - \Phi_{\mathrm{IF}}^{p_g}(k-1)\big], \tag{206}$$

which is near zero for a clean arc and jumps by $\sim \delta N\,\lambda_{\mathrm{IF}}$ on a slip. The between-satellite step does not cancel the differential geometric error $(\boldsymbol{e}^i - \boldsymbol{e}^{p_g})^{\mathrm{T}}\delta\boldsymbol{r}$, so flagging must be gated by the filter's 3D position sigma. That precondition — a static site with a precise a priori position — has no guarantee on the single-receiver kinematic path, where the detector has no instantiation route and the `estimation.enable_if_slip_detector` user toggle stays inert; the network estimator, by contrast, *pins* every station position and so satisfies the gate by construction. It is there that the detector is armed.

On the network path (the `buildStation` lambda of `Application::networkSetup_`, gated on `network.cycle_slip.if_phase_td`) an `IfPhaseSlipDetector` is instantiated per *non-pivot* station — the pivot's ambiguities are gauge-pinned, so on the legacy datum path there is nothing there to refloat — and, under the prioritised phase datum `network.phase_datum_prioritized`, per station, because the former pivot then carries non-anchored arcs that still need detection. It is built with `setDeferFlagging(true)`: it caches each per-epoch $\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g}$ of Equation (206) but does *not* threshold locally. The driver (`networkSlipIfTdDriver` in `src/core/slip/NetworkSlipQc.cpp`) instead *pools* the deferred jumps across stations by signal pair and removes their median med — the network common mode, a frame/datum wobble that is elevation-independent and is therefore removed in raw metres. Each residual is then normalised to a zenith equivalent,

$$z \;=\; \big(\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g} - \mathrm{med}\big)\,\sin\theta, \tag{207}$$

before the pooled robust scale $\hat{s} = 1.4826\,\mathrm{median}\,|z|$ is formed, and the scale is re-inflated by $1/\sin\theta$ again at each satellite's own threshold, so *one* scale is valid across the whole sky. The map is the free function `ifElevMap` ($1/\sin\theta$, clamped at 5° so it cannot blow up at the already elevation-masked horizon). The effective scale $\hat{s}_{\mathrm{eff}}$ is an empirical-Bayes blend of that pooled prior with the station-pair's own EWMA scale, shrinkage weight $w = n/(n+\kappa)$ in the station-pair's clean sample count n, so a quiet reference station and a noisy modernised-signal station each get their own envelope rather than one pooled threshold set by the majority. A satellite at elevation θ is flagged when

$$\big|\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g} - \mathrm{med}\big| \;>\; \max\!\left(\frac{k\,\hat{s}_{\mathrm{eff}}}{\sin\theta},\;\; c_{\mathrm{floor}}\,\lambda_{\mathrm{IF}}\right), \tag{208}$$

with k a deliberately loose MAD multiplier and the floor *proportional* to the $(1,1)$ IF jump $\lambda_{\mathrm{IF}} = c/(f_1+f_2) \approx 10.7\,\mathrm{cm}$, kept in raw metres so a one-cycle simultaneous slip is caught at any elevation. Clean survivors — and only clean survivors — then fold into the station-pair EWMA, so the scale never absorbs the event it must catch. The five knobs (κ, k, c_{floor}, the minimum pool size and the EWMA weight) live under `network.cycle_slip.if_phase_td_*` and are snapshot once inside the driver.

A flag never repairs and never resets filter state in place. It restarts the detector's own arc unconditionally (arc hygiene), and its two estimator-side consequences are gated separately inside `refloat_pair`: the flagged pair enters the Card-4 invariant-carry slip funnel only when that carry is armed, and both band ambiguities are marked pending-reseed — the actual refloat — only once the phase-leveling carry is warm. On a cold or absent carry the flag is emitted as diagnostic telemetry

and changes nothing, so a clean run log is not evidence that the arm acted; the run-end counters are. The sharp integer decision is left to a post-estimation phase validation that is not yet built, and the loose gate is intentional — this is a coarse first pass, not the discriminating solver of Section 14.4.

The pool cannot see a slip on the between-satellite reference itself: such a jump is common-mode across every satellite of that station's pool and is removed with med. The driver therefore folds in a second, independent flag source — the detector's own reference-satellite reset keys, parsed by `parseIfPairResetKey` and refloated through the same `refloat_pair`. Both bands of the pair must refloat because the IF combination cannot localise which one slipped. The reference-satellite election that produces those keys is itself gated by `IfPhaseSlipDetector::RefGateParams`, pushed only on the network path and armed by `network.cycle_slip.if_phase_td_ref_gate` (default off); the staged user/kinematic path keeps the legacy fixed gate.

14.3 Network slip QC: raw evidence and refloat over repair

The IF-phase driver of Section 14.2.5 is one of three arms of the network producer's slip layer. All three are free functions in `slip/NetworkSlipQc`, called in a fixed serial, station-ordered sequence from the per-epoch PASS-A slip seam (`Application::networkPassASlipScan_`). The other two are `networkSlipDrainT0Repairs`, which drains the phase-leveling carry's integer repairs, and `networkSlipDrainMwRefloat`, which drains confident wide-lane integer jumps and the receiver's own LLI loss-of-lock bit. Their shared evidence is a fourth component, `slip/NetworkRawObservables`.

Shared evidence. `networkComputeRawObservables` forms, once per epoch per (station, satellite) on the two lowest distinct bands whose phase *and* matching code are both present, the same MW combination of Equation (196) and the same GF combination of Equation (202), plus the first band's phase-minus-code level and the per-band RINEX LLI byte. These are absolute *levels*, not differences. `networkQualifyRawObservables` then adjudicates each observable's step against the immediately adjacent processed epoch — $\mathrm{MW}^i(k) - \mathrm{MW}^i(k-1)$ and $\mathrm{GF}^i(k) - \mathrm{GF}^i(k-1)$ — under the same threshold shape as Section 14.2.5: a multiple of an empirical scale, floored from below. The empirical term is a per-station robust EWMA of that station's *own* clean steps, built from the same $1.4826\,\mathrm{median}\,|\cdot|$ estimator (exposed for test as `networkRobustStepScale`); the floor is a *physical* integer bound kept in raw metres — a fraction of a wide-lane cycle λ_{WL} for MW, a few centimetres for GF. The gate therefore degrades to physics when the empirical term is cold or degenerate, never to a stochastic-model σ, which is the same discipline as the MW adaptive threshold of Section 14.2.1. The ordering invariant also matches the IF-TD pool arm: each satellite is thresholded with the *pre-update* scale and only the clean survivors are folded into the EWMA, so the scale can never absorb the event it exists to catch. A step is measurable only against an *adjacent* predecessor; across a gap it is structurally unmeasurable, and the code is explicit that absence of evidence is neither evidence of a slip nor evidence of its absence. The multipliers and floor values live in `SlipQcParams`, not here.

Response doctrine. On the network path a detection *refloats* — it marks the ambiguity pending-reseed so the next pass re-estimates it from that epoch's data — and never applies an integer. This is a deliberate asymmetry against the user-path TurboEdit repair of Section 14.4, and the reason is the network's missing satellite phase biases: without them the network wide-lane integer is not trustworthy, so a wrong integer written into a carried value would bias the gauge, whereas a refloat costs only re-convergence. The one exception is `networkSlipDrainT0Repairs`, which shifts the phase-leveling carry by $\delta N_s\,\lambda_s$ *continuously* — variance unchanged, covariance-preserving — and only for an integer that is both confident (small rounding residual, Equation (214)) and of sane

magnitude. A low-confidence detection is left untouched rather than repaired, refloated or dropped, because the float wide-lane's marginal repairs are measured to be predominantly false positives and acting on them discards a good carry.

Status. Every arm is default *off*: `network.cycle_slip.mw_detect` arms the MW/LLI drain, `network.cycle_slip.mw_lli_corroborate` the LLI qualification and `network.cycle_slip.gap_instrument` the arc-resumption instrument. The raw observables are computed at all only when some consumer asked for them (`SlipQcParams::anyConsumer`), and the adjudication above runs only when a *decision* card is armed — a run with the raw trace alone computes the levels and adjudicates nothing. The drains therefore guard every use of the verdicts on whether the evidence exists this epoch; without it the LLI arm stays decisive, because a gate that fired on missing data would be worse than the chatter it exists to fix. The qualification never *excludes* an LLI assertion: an assertion acts when a measured MW or GF step corroborates it, or when the station's own LLI behaviour is still judged credible by a two-state hysteresis on the EWMA of its LLI-set fraction (`networkLliCredibilityStep`). Only an uncorroborated assertion at a demonstrably non-credible station is downgraded, and it is then emitted as suppressed and counted rather than silently dropped.

The raw step evidence has a second consumer outside this chapter. With `network.carry_invariant_innov_corroborate` armed (default off), the Card-4 innovation gate on the carried double differences will not re-tap a chord whose arc the raw observables positively measured as continuous. That gate is fail-safe in one direction only — measured-and-flat suppresses, absence of evidence still re-taps — so the slip response is narrowed, never removed. The gate itself is estimation-side and is described in Section 19.

14.4 TurboEdit dual-frequency resolution

The TurboEdit resolver is a *pure* MW+GF joint integer solver (`src/core/slip/TurboEditResolver.cpp` and its header `include/slip/TurboEditResolver.hpp`). It owns the per-arc geometry-free jump history and the integer arithmetic, and returns per-signal integer repairs plus a reset-fallback set without mutating filter state; it is wired into both the post-processing `CycleSlipManager` and the real-time engine, and enabled by `estimation.cycle_slip.turbo_edit`.

That key is default *off*. The bundled real-time and SBF-replay configs turn it on; the internal PPP-RTK AR config leaves it off. The network producer has a separate key, `network.cycle_slip.turbo_edit`, and with that off but `network.cycle_slip.mw_detect` on the same MW detector runs as a pure detector and the network refloats rather than repairing (Section 14.3). Of the sub-toggles below, the soft-LLI bridge (`turbo_edit_bridge_lli`) is likewise default off, while the MW-only code-domain veto (`turbo_edit_mw_only_veto`) is default on.

14.4.1 The 2×2 system

For a dual-frequency phase pair (s_1, s_2) on satellite i, ordered higher-frequency first ($f_{s_1} > f_{s_2}$, hence $\lambda_{s_1} < \lambda_{s_2}$), the resolver pairs two scalar slip observables. The first is the MW wide-lane integer of Equation (197), taken *as input* from the MW detector and never recomputed (the empirically tuned MW path of Section 14.2.1 is the single source of δN_{WL}):

$$\delta N_{\mathrm{WL}} \;=\; \delta N_{s_1} - \delta N_{s_2} \qquad \text{(one equation, two unknowns).} \tag{209}$$

The second is the GF phase jump in metres. Writing the GF level as $\mathrm{GF}(t) = \lambda_{s_1} L_{s_1}(t) - \lambda_{s_2} L_{s_2}(t)$ (carrier in cycles), a simultaneous slip steps its ambiguity content by

$$\delta\mathrm{GF} \;=\; \lambda_{s_1}\,\delta N_{s_1} - \lambda_{s_2}\,\delta N_{s_2}, \tag{210}$$

estimated in-arc by the second-order time difference $\mathrm{TD2}(t) = \mathrm{GF}(t) - 2\,\mathrm{GF}(t-1) + \mathrm{GF}(t-2)$ of Equation (203), which cancels the ionospheric delay up to quadratic order so that at the slip epoch it equals the integer jump plus phase noise only. The TD2 chain requires three *consecutive* epochs; non-consecutive history forces a chain re-seed (Section 14.4.4).

Stacking Equations (209) and (210) gives the linear map from the integer slip pair to the two observed jumps:

$$\boxed{\begin{pmatrix} \delta N_{\mathrm{WL}} \\ \delta\mathrm{GF} \end{pmatrix} \;=\; \underbrace{\begin{pmatrix} 1 & -1 \\ \lambda_{s_1} & -\lambda_{s_2} \end{pmatrix}}_{\boldsymbol{A}} \begin{pmatrix} \delta N_{s_1} \\ \delta N_{s_2} \end{pmatrix}.} \tag{211}$$

The determinant $\det \boldsymbol{A} = \lambda_{s_1} - \lambda_{s_2}$ is non-zero for any genuine cross-band pair — it is the cm-scale quantity `denom` of Equation (213), $\approx -0.0539\,\mathrm{m}$ for GPS L1/L2, *not* the $\sim 0.86\,\mathrm{m}$ wide-lane wavelength — so the system is always invertible:

$$\boldsymbol{A}^{-1} = \frac{1}{\lambda_{s_1} - \lambda_{s_2}} \begin{pmatrix} -\lambda_{s_2} & 1 \\ -\lambda_{s_1} & 1 \end{pmatrix}. \tag{212}$$

The implementation does not invert the full matrix; it solves the second row for δN_{s_2} and back-substitutes the first (`TurboEditResolver.cpp`):

$$\delta N_{s_2} \;=\; \frac{\delta\mathrm{GF} - \lambda_{s_1}\,\delta N_{\mathrm{WL}}}{\lambda_{s_1} - \lambda_{s_2}}, \qquad \delta N_{s_1} \;=\; \delta N_{s_2} + \delta N_{\mathrm{WL}}, \tag{213}$$

with $\delta\mathrm{GF}$ instantiated either by the in-arc TD2 or, across a data gap, by the bridged jump of Section 14.4.4. The denominator $\lambda_{s_1} - \lambda_{s_2}$ (the variable `denom` in the source) equals $\approx -0.0539\,\mathrm{m}$ for GPS L1/L2, so the amplification factor on the wide-lane integer is $\lambda_{s_1}/(\lambda_{s_1} - \lambda_{s_2}) \approx -3.53$: the GF equation must localise δN_{s_2} to well within ± 0.5 cycle despite that lever, which is what the split-capability gate below enforces.

14.4.2 Integer rounding and validation

The float δN_{s_2} from Equation (213) is rounded and its residual recorded:

$$d_2 = \mathrm{round}\big(\delta N_{s_2}\big), \quad f = \big|\delta N_{s_2} - d_2\big|, \quad d_1 = d_2 + \delta N_{\mathrm{WL}}. \tag{214}$$

A candidate is *accepted* as a repair only if the rounding is clean ($f \le 0.30$, the constant `kMaxFractional`; a $1 - 0.30 = 0.70$ cycle margin to the nearest wrong integer), the magnitudes are sane ($|d_1|, |d_2| \le$ `kMaxSlipCycles`), and the pair is non-trivial ($d_1 \neq 0 \vee d_2 \neq 0$). A non-trivial accepted candidate yields `action="repair"`; $(d_1, d_2) = (0, 0)$ is resolved as no-slip and the arc keeps rolling; otherwise the satellite is reset.

Before the joint solve is *trusted*, the GF evidence must be sharp enough to discriminate the integer lattice. The float δN_{s_2} carries $\sigma_{\mathrm{TD2}}/|\lambda_{s_1} - \lambda_{s_2}|$ cycles of noise, so the solve is permitted only when

$$\sigma_{\mathrm{TD2}} \;\le\; \texttt{turbo_edit_split_max_sigma} \cdot \big|\lambda_{s_1} - \lambda_{s_2}\big| \qquad (\texttt{splitCapable}), \tag{215}$$

with σ_{TD2} the arc's own MAD-derived σ when mature, else the cross-satellite tier floor, else NaN (NaN fails by design — "no estimate" is not evidence of sharpness). The default `estimation.cycle_slip.turbo_edit_split_max_sigma` is 0.2 cycles. This gate is the single mechanism making the resolver rate-adaptive: millimetric 1 s noise passes trivially, while coarse-rate (30 s) ionospheric residual passes only while the arc is quiet. On an MW-flagged pair failing the gate the resolver falls back to a satellite reset; a GF-only exceedance on a split-incapable arc is *ignored*, not reset. When $\delta N_{\mathrm{WL}} = 0$ (an equal slip $\delta N_{s_1} = \delta N_{s_2}$, to which MW is blind) a GF-only flag fires on |TD2| exceeding an adaptive threshold $\tau = \max(k_\sigma \hat{\sigma}_{\mathrm{TD2}}, 0.04\,\mathrm{m})$, with k_σ the `turbo_edit_sigma_multiplier`; the floor sits just below $|\lambda_{s_1} - \lambda_{s_2}| \approx 0.054\,\mathrm{m}$ so a ±1-cycle equal slip remains detectable.

14.4.3 MW-only / code-event null line

The asymmetric case is MW flagged but the GF jump *affirmatively zero*. For a fixed $\delta N_{\mathrm{WL}} = k$, every integer pair $(\delta N_{s_1}, \delta N_{s_1} - k)$ predicts, via Equation (210), a GF jump

$$\delta\mathrm{GF} \;=\; \lambda_{s_2} k - (\lambda_{s_2} - \lambda_{s_1})\,\delta N_{s_1}, \tag{216}$$

whose zero crossing sits at $\delta N_{s_1} = \lambda_{s_2} k / (\lambda_{s_2} - \lambda_{s_1}) \approx 4.53\,k$ for GPS L1/L2 ($\approx 3.95\,k$ for L1/L5). Every small pair $|\delta N_{s_1}| \le 3$ then predicts $|\delta\mathrm{GF}| \ge 8\,\mathrm{cm}$ — orders above phase noise — so a flat GF leaves only *null-line* pairs of $(4, 3)$ / $(9, 7)$-class magnitude: implausible multi-cycle simultaneous slips on both signals with no loss of lock. The Occam-simpler explanation is a narrow-lane *code* excursion (multipath, NLOS, DLL re-lock), since $\mathrm{MW} = \Phi_{\mathrm{WL}} - P_{\mathrm{NL}}$ and one wide-lane cycle $\approx 0.86\,\mathrm{m}$ of narrow-lane code.

Accordingly the pair is classified a code-domain artifact — and the caller *skips* the phase reset — when the GF jump is affirmatively zero within a discrimination-bounded gate,

$$|\mathrm{TD2}| \le \underbrace{\texttt{kCodeEventZeroSigma} \cdot \hat{\sigma}_{\mathrm{MAD}}}_{\text{zero gate}} \quad\wedge\quad \text{zero gate} \le \underbrace{\texttt{kVetoZeroGateMaxLambdaFrac} \cdot |\lambda_{s_1} - \lambda_{s_2}|}_{\text{discrimination cap}}, \tag{217}$$

with `kCodeEventZeroSigma` = 3.5 (≈ 99.95% two-sided) and `kVetoZeroGateMaxLambdaFrac` = 0.5. The cap is the rate-adaptive safeguard: a zero gate wider than half the smallest GF-visible quantum cannot distinguish stillness from a small slip, so the veto stands down and the MW mandate governs. The classification is further trust-gated — veto enabled, MW flagged, GF not exceeding, no post-slip cooldown, ionospherically stable constellation, no LLI on either signal, and MAD maturity ≥ 10 samples (stricter than detection's 5, because affirming stillness is a state-*preserving* decision). The implied null-line pair is computed for the trace only, never applied; `did_slip` stays false so the GF chain keeps rolling. A genuine null-line slip is not lost: it jumps that signal's Kalman phase innovation by $\ge |4\lambda_{s_1}| \approx 0.76\,\mathrm{m}$ this very epoch, far beyond any residual gate, so the residual-driven reset corrects it one epoch late. The behaviour is toggled by `estimation.cycle_slip.turbo_edit_mw_only_veto` (default on).

14.4.4 Gap-spanning GF bridge

The TD2 chain needs three consecutive epochs; any per-signal dropout or receiver gap re-seeds the chain, making the post-gap TD2 *blind* to a re-lock slip whose GF level jump lives *between* the two chains. When a chain break occurs within `estimation.cycle_slip.turbo_edit_max_gap_s`, the resolver stashes the pre-gap GF level $\mathrm{GF}_{\mathrm{ref}}$, its timestamp, the arc's EWMA GF rate $\dot{\mathrm{GF}}$ and the pre-gap spacing dt_0, and on return forms the drift-corrected across-gap jump

$$\boxed{\delta\mathrm{GF}_{\mathrm{gap}} \;=\; \mathrm{GF}_{\mathrm{now}} - \mathrm{GF}_{\mathrm{ref}} - \dot{\mathrm{GF}}\,\Delta t, \qquad \Delta t = t_{\mathrm{now}} - t_{\mathrm{ref}},} \tag{218}$$

which then plays the role of δGF in Equation (213). Here $\dot{\text{GF}}$ is an EWMA ($\alpha = 0.1$) of the per-epoch GF rate, chosen so its noise does not amplify when projected over the gap. Because the un-cancelled ionospheric residual is random-walk over the gap, the detection threshold and the code-event zero gate are widened by the factor $w = \sqrt{\Delta t / dt_0}$ (for $\Delta t > dt_0$), and the split-capability σ becomes $\sigma_{\text{bridge}} = \hat{\sigma}_{\text{MAD}}\, w$; the zero-gate *discrimination cap* stays un-widened, since the integer lattice of Equation (211) does not stretch with the gap. The bridge cap default is 15 s in code, raised to 60 s in the bundled real-time and replay configs.

An MW-flagged pair across the gap is attributed three ways. (i) A *code event* (`action="code_event"`) when $\delta\text{GF}_{\text{gap}}$ is affirmatively zero within the widened gate — the same null-line logic as Equation (217), applied across the gap — preserving phase through a re-acquisition DLL transient. (ii) A *repair* or *moot* when the joint solve runs with $\delta\text{GF} = \delta\text{GF}_{\text{gap}}$; per signal it is dispatched by observation-domain continuity, a signal observed at every epoch through the gap kept its ambiguity (its nonzero δN is a genuine *repair*), whereas a signal that missed an epoch was pruned-and-reseeded on return (its nonzero δN is *moot* — already absorbed, mandate suppressed). (iii) A *reset* when the across-gap σ is too coarse for the split, or the candidate set fits zero or two integers with no deferral.

Because the GF equation alone is degenerate along the null-line lattice of Equation (216) — $+0.735$ m fits both $(0, -3)$ and $(9, 4)$ at millimetre level; only the integer k separates them — the bridged solve demands a *unique* surviving candidate after three guards: the fractional gate of Equation (214); a GF-separation guard requiring the candidate's own predicted jump to clear the zero gate, $|\lambda_{s_1} d_1 - \lambda_{s_2} d_2| >$ `zero_gate`, lest a marginal noise exceedance mint a multi-cycle repair from noise; and a sibling-pair zero affirmation, whereby a satellite's *other* GF arc that stayed chain-continuous and unflagged through the bridged interval affirms $\delta N = 0$ for both its signals and prunes any candidate demanding a nonzero integer on an affirmed signal. The wide-lane integer feeding the bridged solve is taken in trust order: the published MW integer k; failing that, at the flag epoch, round$(f) \pm 3$ around the (NL-code-biased) instantaneous re-acquisition float; and on deferral epochs the settled across-gap MW Hatch mean. When the float wide-lane evidence is unsettled or the lattice is ambiguous (two or more acceptable integers), the solve is *deferred* rather than guessed (`estimation.cycle_slip.turbo_edit_wl_settle_max_s`), terminating in a later repair, reset or timeout-reset.

14.4.5 Atomic reconciliation

Per-satellite candidates are reconciled atomically: a cross-pair integer disagreement on a shared signal demotes the whole satellite to reset, and a satellite ending in the reset set has its code-event, moot, clean and pending classifications *withdrawn* — contradictory evidence always resolves to the conservative outcome. Arc bookkeeping keeps the TD2 validity precondition honest: a non-consecutive update partially resets the chain but preserves the slowly-varying MAD calibration window, an absolute gap beyond the configured maximum wipes the arc, and an accepted slip re-seeds with a short cooldown to suppress the TD2 echo; only clean (non-exceeding, non-bridged) TD2 samples feed the MAD window.

Part V
Integer ambiguity resolution

15 Integer ambiguity resolution

Integer ambiguity resolution (AR) converts the real-valued, bias-contaminated phase ambiguities of the float solution into the integer carrier-cycle counts they represent. The engine implements MLAMBDA [26] over a between-satellite-differenced ambiguity sub-problem extracted from the filter state and covariance. The operative quantity is the ambiguity differenced against a per-constellation pivot p_g (Section 5); the same integer-estimation machinery serves both the single-receiver and the baseline modes, but *what makes that differenced ambiguity an integer differs by mode.* The undifferenced ambiguity $N^i_{r,s}$ of the phase observation equation (Equation (5)) carries a receiver phase bias $b_{r,s}$ and a satellite phase bias b^i_s. In single-receiver PPP / PPP-RTK the pivot difference is a *between-satellite single difference* $\Delta\,\cdot$ (Section 5.1), which cancels the receiver bias $b_{r,s}$ (common to every tracked satellite) but *not* the satellite bias; the residual $\Delta N^{i,p_g}_{r,s}$ is integer only once b^i_s has been removed by a consistent external phase-bias product (BIA / SSR, Section 17). In baseline mode the state already holds true double differences $\nabla\Delta\,\cdot$ (Section 5.3), where the additional between-receiver difference cancels the satellite bias as well, so $\nabla\Delta N$ is integer intrinsically. We retain the resolver's internal label $\nabla\Delta$ for the differenced ambiguity in the equations below — the algebra is identical either way — with the understanding that it denotes the between-satellite single difference Δ in the single-receiver case and the full double difference only in baseline mode. The implementation is split along a driver/kernel seam: `src/core/filter/AmbiguityResolver.cpp` is the *driver* — state-vector assembly, metre↔cycle rescaling, per-pair λ lookup, DD formation, the WL/NL decomposition, and application of the returned integer fix — while the decorrelation-agnostic *numerical core* (the LDL^{T} factorisation, the $\boldsymbol{Z}$-reduction, the Schnorr–Euchner search, the bootstrap success rate, and the full acceptance battery, all as pure free functions over ($\hat{\boldsymbol{N}}$ cycles, $\boldsymbol{Q}_{\hat{N}}$ cycles^2) with no `StateManager` / `KalmanFilter` / config coupling) lives in the `GnssProcessor::ils` namespace of `src/core/filter/IlsCore.{hpp,cpp}`. That same `ils` core is the single implementation the PPP-RTK network producer also calls (Section 19), though there it runs under its own gate set and purely as a read-only fixability probe — the producer is float-only at the time of writing (Section 15.7). Configuration for the single-receiver and baseline paths described below is under the JSON path `estimation.ambiguity_resolution.*`.

15.1 The integer least-squares problem

The float solution delivers a real-valued ambiguity estimate $\hat{\boldsymbol{N}}$ (in cycles) with covariance $\boldsymbol{Q}_{\hat{N}}$, both obtained by sub-vector extraction from the filter state $\boldsymbol{x}$ and covariance $\boldsymbol{P}$. Integer ambiguity resolution is the weighted closest-lattice-point problem

$$\boxed{\check{\boldsymbol{N}} \;=\; \arg\min_{\boldsymbol{a}\,\in\,\mathbb{Z}^n} \left(\hat{\boldsymbol{N}}-\boldsymbol{a}\right)^{\mathrm{T}} \boldsymbol{Q}_{\hat{N}}^{-1} \left(\hat{\boldsymbol{N}}-\boldsymbol{a}\right),} \tag{219}$$

i.e. minimisation of the squared Mahalanobis distance from $\hat{\boldsymbol{N}}$ to a lattice point $\boldsymbol{a}$ in the metric of $\boldsymbol{Q}_{\hat{N}}^{-1}$. This is an NP-hard integer least-squares (ILS) program; LAMBDA and its MLAMBDA variant make it tractable in three invariant stages — (i) an LDL^{T} factorisation of $\boldsymbol{Q}_{\hat{N}}$, (ii) a decorrelating integer $\boldsymbol{Z}$-transform (lattice reduction), and (iii) a depth-first shrinking-ellipsoid search

over the reduced lattice — none of which carries a tuning parameter. We follow the hat/check convention of the notation block: $\hat{\boldsymbol{N}}$ is the float estimate, $\check{\boldsymbol{N}}$ the integer-fixed solution.

Float extraction and rescaling to cycles. The filter state stores per-band ambiguity states in *metres* ($\lambda_s N$, one per tracked (satellite, signal)), not cycles, so the resolver first forms the between-satellite-differenced float and covariance by explicit differencing against the pivot p_g and then rescales by wavelength. (In baseline mode the state already holds the double difference and the differencing step below collapses to a direct selection; see Section 15.6.) For a differenced pair k with satellite index $i(k)$ and pivot index $p(k)$ (`src/core/filter/AmbiguityResolver.cpp`):

$$y_k^{\nabla\Delta} = \boldsymbol{x}_{i(k)} - \boldsymbol{x}_{p(k)}, \qquad \text{(DD float, m)} \tag{220}$$

$$\boldsymbol{Q}_{kl}^{\nabla\Delta} = \boldsymbol{P}_{i_k i_l} - \boldsymbol{P}_{i_k p_l} - \boldsymbol{P}_{p_k i_l} + \boldsymbol{P}_{p_k p_l}, \qquad \text{(DD covariance, m}^2\text{)} \tag{221}$$

$$\hat{\boldsymbol{N}}_k = y_k^{\nabla\Delta}/\lambda_k, \qquad (\boldsymbol{Q}_{\hat{N}})_{kl} = \boldsymbol{Q}_{kl}^{\nabla\Delta}/(\lambda_k\lambda_l). \qquad \text{(cycle domain)} \tag{222}$$

The rescaling (222) is load-bearing: running MLAMBDA on metre-domain floats would round toward integer-*metre* values with no carrier-cycle semantics, and the ratio test (Section 15.5) would degenerate to ≈ 1 irrespective of float quality. The per-pair wavelength λ_k is resolved with GLONASS FDMA per-satellite frequencies where applicable, and $\boldsymbol{Q}^{\nabla\Delta}$ is symmetrised, $\boldsymbol{Q}^{\nabla\Delta} \leftarrow \frac{1}{2}(\boldsymbol{Q}^{\nabla\Delta} + \boldsymbol{Q}^{\nabla\Delta\mathrm{T}})$, before factorisation. The resolver elects its own DD reference: the highest-elevation member of each (constellation, signal) group among the AR-eligible ambiguities, with all remaining members of the group differenced against it. This is deliberately *not* the filter's BSSD pivot — under BSSD the pivot's ambiguity is frozen at exactly zero variance, which excludes it from the candidate set (Section 15.4), so the resolver differences against a second reference and the resulting quantity is a difference of single differences. The two elections also run on different rules: the BSSD pivot is chosen on a hard phase-coverage tier plus a convergence/elevation score and an ephemeris-freshness bar (Section 5), the AR reference on elevation alone (`AmbiguityResolver::buildDdPairs`).

15.2 Decorrelation: LDL^{T} factorisation and the integer Z-transform

Factorisation. MLAMBDA factorises the symmetric positive-definite ambiguity covariance as

$$\boldsymbol{Q}_{\hat{N}} \;=\; \boldsymbol{L}^{\mathrm{T}}\,\boldsymbol{D}\,\boldsymbol{L}, \qquad \boldsymbol{L} \text{ unit lower-triangular}, \;\; \boldsymbol{D} = \mathrm{diag}(d_1, \ldots, d_n), \;\; d_i > 0, \tag{223}$$

factored in reverse index order (the Chang–Yang MLAMBDA convention). A non-positive pivot $d_i \le 0$ aborts the fix, since it signals a non-PD covariance. After reduction, d_i is the *conditional variance* of the i-th decorrelated ambiguity given the later ones; it feeds the success-rate gate of Section 15.5 directly.

The Z-transform. The central device of LAMBDA [26] is a unimodular integer transform $\boldsymbol{Z} \in \mathbb{Z}^{n\times n}$, $|\det \boldsymbol{Z}| = 1$, that re-parameterises

$$\boldsymbol{z} = \boldsymbol{Z}^{\mathrm{T}}\hat{\boldsymbol{N}}, \qquad \boldsymbol{Q}_{\hat{z}} = \boldsymbol{Z}^{\mathrm{T}}\,\boldsymbol{Q}_{\hat{N}}\,\boldsymbol{Z}, \tag{224}$$

so that the transformed ambiguities are nearly decorrelated (small off-diagonal $\boldsymbol{L}$ entries) and their conditional variances are ordered. We write the decorrelated float vector as $\bar{\boldsymbol{z}} := \boldsymbol{Z}^{\mathrm{T}}\hat{\boldsymbol{N}}$ (the quantity the search of Section 15.2 actually rounds; implemented as `z_float = Z.transpose()*a` in `ils::mlambda` (`src/core/filter/IlsCore.cpp`)), with components $\bar{z}_k$. Because $\boldsymbol{Z}$ is unimodular the integer minimiser maps back exactly, $\check{\boldsymbol{N}} = (\boldsymbol{Z}^{\mathrm{T}})^{-1}\check{\boldsymbol{z}}$ with $(\boldsymbol{Z}^{\mathrm{T}})^{-1}$ also integer; decorrelation

therefore does not alter the solution of (219) — it only makes the search ellipsoid closer to spherical, collapsing the enumeration count by orders of magnitude. The reduction interleaves two operations sweeping $j = n - 2$ downward.

Integer Gauss transformations zero the magnitude of sub-diagonal $\boldsymbol{L}$ entries by integer column operations,

$$\mu = \lceil L_{ij} \rfloor, \qquad \boldsymbol{L}_{(:,j)} \mathrel{-}= \mu\, \boldsymbol{L}_{(:,i)}, \qquad \boldsymbol{Z}_{(:,j)} \mathrel{-}= \mu\, \boldsymbol{Z}_{(:,i)}, \tag{225}$$

where $\lceil \cdot \rfloor$ is round-to-nearest, keeping every $|L_{ij}| \le \frac{1}{2}$.

Permutations (conditional adjacent swaps) reorder ambiguities when doing so reduces the later conditional variance. With $\bar{d}_j = d_j + L_{j+1,j}^2\, d_{j+1}$, the swap fires when

$$\bar{d}_j + \varepsilon \;<\; d_{j+1}, \qquad \varepsilon = 10^{-6}, \tag{226}$$

after which d_j, d_{j+1}, the affected rows/columns of $\boldsymbol{L}$ and the columns of $\boldsymbol{Z}$ are updated by the standard rank-one formulas ($\eta = d_j/\bar{d}_j$, $\lambda = d_{j+1} L_{j+1,j}/\bar{d}_j$, $d_j \leftarrow \eta\, d_{j+1}$, $d_{j+1} \leftarrow \bar{d}_j$), and the sweep restarts from $j = n - 2$. The guard $\varepsilon = 10^{-6}$ prevents thrashing on near-equal variances and is a numerical safeguard, not an estimator knob; it has no config exposure.

The shrinking-ellipsoid search. The reduced problem is solved by depth-first enumeration of lattice points inside the ellipsoid $\sum_i (z_i - \hat{z}_i)^2/d_i \le \chi^2$, in Schnorr–Euchner order (nearest integer first, then zig-zagging outward). Recursing from the last level $k = n - 1$ down to 0,

$$\hat{z}_k = \bar{z}_k + \sum_{i>k} (z_i - \hat{z}_i)\, L_{ik}, \qquad \text{(conditional centre)} \tag{227}$$

$$z_k = \lceil \hat{z}_k \rfloor, \qquad \mathrm{dist}_k = \mathrm{dist}_{k+1} + \frac{(\hat{z}_k - z_k)^2}{d_k}. \qquad \text{(partial distance)} \tag{228}$$

When a complete integer vector reaches level 0 with cumulative distance below the current bound χ^2, it is recorded and the bound shrinks to the worst retained candidate, contracting the ellipsoid as better candidates are found. The search (`ils::search` in `src/core/filter/IlsCore.cpp`) retains the $m = 2$ best candidates (for the ratio test) sorted by ascending squared distance, under a hard iteration cap (`kMaxIter` = 10 000). Its outputs are the two smallest objective values $s_1 \le s_2$ of (219) and the best integer vector $\check{\boldsymbol{z}}$, back-transformed via $\check{\boldsymbol{N}} = (\boldsymbol{Z}^{\mathrm{T}})^{-1} \check{\boldsymbol{z}}$.

15.3 Wide-lane / narrow-lane decomposition (PPP path)

Standard PPP and PPP-RTK AR use a *wide-lane-first* decomposition when Melbourne–Wübbena (MW)-resolved wide-lane (WL) integers are available, halving the search dimension and exploiting the long WL wavelength [13]. The driver attempts the WL-constrained narrow-lane (NL) fix before any unconstrained LAMBDA and — critically — does *not* fall through to an unconstrained search when WL information exists but the constrained fix fails: a wrong-WL fix is worse than no fix, so "no NL AR for satellites whose WL is float" is the enforced hierarchy (`src/core/filter/AmbiguityResolver.cpp`).

WL from Melbourne–Wübbena (integer by contract). The DD wide-lane integer is differenced directly from the MW detector's single-difference WL integers (`src/core/slip/MwSlipDetector`), not searched:

$$\nabla\Delta N_{\mathrm{WL}} = N_{\mathrm{WL}}^{i} - N_{\mathrm{WL}}^{p_g} \in \mathbb{Z}, \qquad \lambda_{\mathrm{WL}} = \frac{c}{f_1 - f_2}. \tag{229}$$

NL search variable. For each dual-frequency DD pair the float narrow-lane ambiguity (ionosphere-free phase combination, in NL cycles) is formed by the transform $\boldsymbol{T}$,

$$\hat{n}_{\mathrm{NL}} = \frac{f_1\,\nabla\Delta N_1 + f_2\,\nabla\Delta N_2}{f_1+f_2} = \frac{1}{f_1+f_2}\left(\frac{f_1}{\lambda_1}\,\nabla\Delta_1 + \frac{f_2}{\lambda_2}\,\nabla\Delta_2\right), \qquad \lambda_{\mathrm{NL}} = \frac{c}{f_1+f_2}, \tag{230}$$

with reduced covariance $\boldsymbol{Q}_{\mathrm{NL}} = \boldsymbol{T}\,\boldsymbol{Q}^{\nabla\Delta}\,\boldsymbol{T}^{\mathrm{T}}$. Only NL-paired DDs enter LAMBDA; unpaired single-signal DDs are excluded so they cannot contaminate the ratio test.

Integer-recentring shift. The float $\hat{n}_{\mathrm{NL}}$ is *not* integer-valued in general: substituting $N_2 = N_1 - \mathrm{WL}$ gives $\hat{n}_{\mathrm{NL}} = N_1 - \frac{f_2}{f_1+f_2}\,\mathrm{WL}$, which for GPS L1/L2 with $\mathrm{WL} = 1$ sits at $N_1 - 0.4379$ — neither integer is meaningfully closer and the ratio test degenerates near 1.6 regardless of float quality. The resolver therefore adds the known-WL offset so the search variable is integer in expectation,

$$\tilde{n}_k = \hat{n}_{\mathrm{NL},k} + \frac{f_2}{f_1+f_2}\,\nabla\Delta N_{\mathrm{WL},k} \;\approx\; N_{1,k} \;\in\; \mathbb{Z} + \text{noise}, \tag{231}$$

runs MLAMBDA on $\tilde{\boldsymbol{n}}$ (a constant per-row shift leaves $\boldsymbol{Q}_{\mathrm{NL}}$ unchanged), then back-shifts $\check{n}_{\mathrm{NL}} = \check{N}_1 - \frac{f_2}{f_1+f_2}\,\mathrm{WL}$.

Back-transform to per-signal DDs. The fixed integers map back to the two original signal DDs in metres,

$$\nabla\Delta_1^{\mathrm{fix}} = \left(\check{n}_{\mathrm{NL}} + \tfrac{f_2}{f_1+f_2}\,\nabla\Delta N_{\mathrm{WL}}\right)\lambda_1, \qquad \nabla\Delta_2^{\mathrm{fix}} = \left(\check{n}_{\mathrm{NL}} - \tfrac{f_1}{f_1+f_2}\,\nabla\Delta N_{\mathrm{WL}}\right)\lambda_2, \tag{232}$$

which re-enter the filter through the constrained update of Section 15.6.

State-basis variants. When `estimation.state_ambiguity_basis` = `nl_wl` the state holds NL/WL ambiguities directly and there are no per-band DD pairs to enumerate; today only the MW-resolved WL integers are applied as constraint rows (a full NL LAMBDA search with WL pre-constrained is future work in this basis). The complementary WL-only mode (`estimation.ambiguity_resolution.wl_only`, in the `per_band` basis) applies the cycle-domain constraint $\nabla\Delta_1/\lambda_1 - \nabla\Delta_2/\lambda_2 = \nabla\Delta N_{\mathrm{WL}}$ as a single pseudo-observation per pair without any NL search, with covariance inflation on the affected single-difference states when MW re-locks at a new integer so the state snaps within one epoch. Both WL-constraint paths (this one and the `nl_wl`-basis constraint below) build their pseudo-observation noise from a *separate* knob, $\boldsymbol{R} = \sigma_{\mathrm{wl}}^2\,\boldsymbol{I}$ with $\sigma_{\mathrm{wl}}^2 =$ `wl_only_pseudo_var` (10^{-4} cycles^2, note the cycle domain), *not* the metre-domain σ_{hold}^2 of the NL/DD feedback in (237). This `per_band` `wl_only` path carries no cross-frequency consistency check: it gates each DD WL only on the per-arc fractional residual (`wl_max_frac`, default 0.40, both arcs). The cross-frequency parity check lives instead in the `nl_wl` basis (`applyMwWlConstraintsNlWl_` in `src/core/filter/AmbiguityResolver.cpp`), where it rejects all three WL DDs of a triple-frequency satellite when the telescoping canonical cycle-count consistency $\nabla\Delta_{ab} - \nabla\Delta_{ac} + \nabla\Delta_{bc} \neq 0$ fails; it does *not* run in the `per_band` `wl_only` path.

15.4 Partial / subset ambiguity resolution

Partial AR operates at *two* levels: a static upstream candidate gate that screens which ambiguities ever enter the integer problem, and an optional dynamic backward-elimination search that recovers a fixable subset when the full set fails validation.

Upstream candidate gating (always on). Subset selection begins when the DD pairs are built (`src/core/filter/AmbiguityResolver.cpp`). A satellite/signal ambiguity is admitted to the integer set only if it passes every filter:

- **Initialised** — the state entry is non-zero (seeded by data), and **non-frozen** — its variance is non-zero (excludes a BSSD-frozen pivot).
- **Elevation** — $\theta_r^i \geq \theta_{\mathrm{AR}}$ (`elevation_mask`).
- **Resolvable** — the satellite is not flagged unresolvable for want of bias/ATX (BIA, ATX) coverage (single-receiver path; in baseline DD the analogous filter is the `exclude_keys` set of non-integer-capable DDs from `MeasurementModel`).
- **Constellation** — $g(i) \in$ `active_systems`, when that list is non-empty.
- **Pairable** — the (constellation, signal) group holds ≥ 2 satellites, so a pivot difference exists.
- **Mature** (`min_arc_info`, default $0 =$ off) — both constituent arcs of a candidate have accumulated at least $\sum_k \sin^2 \theta_{r,k}^i$ of elevation-weighted code information since arc birth. Maturity is measured in *information*, not epochs, because an arc climbing off the mask and one at zenith accumulate at very different rates under the elevation stochastic model, so two arcs of equal age are not equally determined. The counter restarts on *both* ways an arc can end — the key leaving the state vector, and a refloat in place, which keeps the key and would otherwise let a fresh arc inherit its pre-slip age (`AmbiguityResolver::refreshArcAges_`, keyed on `StateManager::isAmbiguityPendingReseed`). Scope is narrow: the gate is tested only on the WL-constrained NL candidate set of Section 15.3, not on the baseline DD, WL-only, or unconstrained full-set paths. Its purpose is defensive — one immature candidate spoils the joint ratio test of (233) for the whole set — but it is a wrong-fix guard, not a convergence accelerator: it admits a candidate once it is already well determined, which is precisely when the float needs least help.

The constellation filter keeps a system in the *float* solution while excluding it from the candidate set — useful when one constellation's residual model bias would drag down the ratio test.

Dynamic backward-elimination subset (`enable_partial_ar`, default off). With the candidate set fixed, the default behaviour is all-or-nothing: the full set is searched and either passes all of Section 15.5 or the epoch stays float. When `enable_partial_ar` is set, a failure of the full-set battery triggers a greedy backward elimination (`ils::selectFixableSubset` in `src/core/filter/IlsCore.cpp`, reached through the driver-side `selectFixableSubset_` config adapter) instead of falling straight back to float. Starting from the full candidate set, each iteration re-runs the complete MLAMBDA pipeline of Sections 15.1 to 15.2 on the current subset and re-tests the *same* acceptance battery — ratio lower/upper bound (233), bootstrap success rate (234), and per-entry fractional cap (235). On any failure it drops one entry and retries, down to a single ambiguity:

- if the *fractional cap* is the failing gate, the entry with the largest $|\hat{\boldsymbol{N}}_k - \check{\boldsymbol{N}}_k|$ (nearest a half-cycle boundary) is removed;
- otherwise the entry with the largest formal float variance $(\boldsymbol{Q}_{\hat{N}})_{kk}$ — the least precise ambiguity, whose removal most improves both the bootstrap rate and the search conditioning — is removed.

Because the accepted subset is always re-validated by the full battery on its own covariance sub-block, this can only fix *fewer* ambiguities than the full set, never a worse-conditioned set: a wrong elimination order costs fixes, not integrity, and the path is bit-identical to the legacy behaviour when off. The same routine serves the per-band SD, baseline DD, and WL-constrained NL paths uniformly, operating purely in the decorrelation-agnostic cycle domain; each caller maps the surviving indices back to its own state rows and feeds them through the constrained update of Section 15.6.

15.5 Validation: ratio test and fixed-failure-rate gate

A fix is committed only if it passes all three of the following gates (a logical AND). The ratio and fixed-failure-rate (FFR) gates are theoretically distinct: one bounds the discrimination between the two best candidates, the other bounds the absolute success probability.

Ratio test (discrimination).

$$\boxed{\text{ratio} = \frac{s_2}{s_1} \;\geq\; \tau_{\text{ratio}}, \qquad s_1 \leq s_2 \text{ the two smallest ILS objectives,}} \tag{233}$$

with s_1, s_2 from the search of Section 15.2 and threshold τ_{ratio} (`ratio_threshold`). A second upper-bound rejection ratio $> \tau_{\text{ratio}}^{\max}$ (`ratio_upper_bound`) catches the pathological case of a degenerate covariance direction — a near-zero s_1 left by a previous wrong-integer hold — that would otherwise produce ultra-confident but meaningless ratios.

Fixed-failure-rate gate (bootstrap success rate). A geometry-driven, self-tuning lower bound on the ILS success probability,

$$\boxed{P_{s,\text{boot}} = \prod_{i=1}^{n} \operatorname{erf}\!\left(\frac{1}{2\sqrt{2\,d_i}}\right) \;\geq\; P_s^{\min},} \tag{234}$$

where the d_i are the post-reduction LDL^{T} conditional variances of (223) and $P_s^{\min}$ is `min_success_rate`. Per dimension the factor is $\Pr\big(|\mathcal{N}(0, d_i)| < \frac{1}{2}\big)$ — the probability that sequential bootstrapped rounding lands on the correct integer — and the product is the standard Teunissen lower bound on the true ILS success rate. This ties the acceptance threshold to the actual problem geometry rather than to a fixed ratio alone.

Per-pair residual backstop. The joint ratio test can pass on a borderline set in which one pair sits near a half-cycle boundary; the fix is rejected if any single DD's fractional residual exceeds a cap,

$$\max_k \big|\hat{\boldsymbol{N}}_k - \check{\boldsymbol{N}}_k\big| \;\leq\; \kappa_{\text{frac}} \quad \text{(cycles)}, \tag{235}$$

with $\kappa_{\text{frac}} =$ `nl_max_per_pair_frac`.

Convergence guards. Before any search the resolver short-circuits AR while the float is immature — if the epoch count is below `min_epochs` or the position standard deviation $\sqrt{\boldsymbol{P}_{00} + \boldsymbol{P}_{11} + \boldsymbol{P}_{22}}$ exceeds `max_position_sigma_m` — because a marginal fix accepted before the covariance settles can lock onto wrong integers and then poison every subsequent epoch.

15.6 Feedback: constrained pseudo-observation update

Accepted integers are *not* fed back by the classical perfect back-substitution (the $R = 0$ conditioning form)

$$\boldsymbol{x}_{\text{fix}} = \boldsymbol{x} - \boldsymbol{Q}_{\boldsymbol{aa}}\, \boldsymbol{Q}^{\nabla\Delta-1}\big(y^{\nabla\Delta} - \check{\boldsymbol{N}}\big), \qquad \boldsymbol{P}_{\text{fix}} = \boldsymbol{P} - \boldsymbol{Q}_{\boldsymbol{aa}}\, \boldsymbol{Q}^{\nabla\Delta-1}\boldsymbol{Q}_{\boldsymbol{aa}}^{\mathrm{T}}, \tag{236}$$

but via a *finite-variance Joseph-form pseudo-observation* (`src/core/filter/AmbiguityResolver.cpp`; kernel `KalmanFilter::josephPseudoUpdate` in `include/filter/KalmanFilter.hpp`). The reason is structural: (236) drives the fixed ambiguity rows of $\boldsymbol{P}$ to ≈ 0 variance, so the next

epoch's LAMBDA sees a near-singular $\boldsymbol{Q}^{\nabla\Delta}$ and emits meaningless inflated ratios that trip the upper-bound guard of (233) indefinitely. A finite-variance pseudo-observation is *intended* to keep $\boldsymbol{P}$ well-conditioned so that subsequent epochs can re-confirm the integer (asymptoting toward it) or detect a slip through its residual. Whether it in fact does so is an open question at the time of writing: the resolver carries a read-only conditioning probe (`qbCondition` / `rebuildQbCycles` in `src/core/filter/AmbiguityResolver.cpp`) that eigen-decomposes the cycle-domain DD covariance either side of every fix and reject, added precisely because tightening σ^2_{hold} was observed to *reduce* the fix count and destabilise the kinematic solution — the signature of a covariance being driven singular by the constraint rather than merely tightened by it. The mechanism argued for below is therefore the design intent, not a validated result.

For each accepted DD pair k the constraint row, innovation and noise are

$$\boxed{\boldsymbol{H}_{(k,\,i)} = +1, \quad \boldsymbol{H}_{(k,\,p)} = -1, \qquad v_k = \lambda_k\,\check{\boldsymbol{N}}_k - \big(\boldsymbol{x}_i - \boldsymbol{x}_p\big), \qquad \boldsymbol{R} = \sigma^2_{\text{hold}}\,\boldsymbol{I},} \tag{237}$$

followed by $(\boldsymbol{x}_{\text{fix}}, \boldsymbol{P}_{\text{fix}}) = \textsc{JosephUpdate}(\boldsymbol{x}, \boldsymbol{P}, \boldsymbol{H}, \boldsymbol{v}, \boldsymbol{R})$, with $\sigma^2_{\text{hold}} =$ `hold_variance`. The DD/baseline path uses a single-column selector $\boldsymbol{H}_{(k,\,\text{idx})} = +1$ since the corresponding state value already *is* the double difference. Fix-and-hold mode (`estimation.ambiguity_resolution.mode` = 3) re-applies the same constraint each epoch to suppress float drift. This pseudo-observation feedback is the resolution of the mode-3 destabilisation that the earlier $R = 0$ form (236) induced; (236) survives in the code only as the documented, rejected legacy approach.

15.7 Network-producer AR (both mechanisms default off)

The PPP-RTK network producer of Section 19 is *float-only*. Two integer mechanisms exist in the producer's per-epoch seam sequence, both default-off, and neither changes the emitted satellite-bias product in its shipped state. They are documented here because their algebra is settled, not because they are in use.

Producer AR: a log-only fixability probe. Under `network.ambiguity_resolution.enable` (default `false`) the seam `filter/NetworkProducerAr` reconstructs each station's float ambiguities from the reduced solution (`NetworkEstimator::reconstructStation`), forms an *independent* between-station double-difference basis — a star topology per (constellation, signal), restricted to GPS and Galileo because GLONASS FDMA leaves a per-slot receiver term that survives the difference — builds the exact DD covariance from `NetworkEstimator::jointConditionalCovariance`, rescales it to cycles by (222), and calls `ils::ilsFix`. The result is *logged and discarded*: nothing is re-pinned, nothing is re-solved, and the run is byte-identical whether the block executes or not. The one piece of invariant algebra that makes the probe meaningful is that the between-station DD ambiguity

$$\nabla\Delta N^{i,p_g}_{r_1 r_2,\,s} \;=\; \Delta N^{i,p_g}_{r_1,\,s} - \Delta N^{i,p_g}_{r_2,\,s} \tag{238}$$

is invariant under the producer's choice of phase-bias S-basis: the satellite bias b^i_s cancels in the between-receiver difference exactly as it does in the baseline DD of Section 5.3, so the probe reports the network's intrinsic integer recoverability rather than a property of the datum it happens to hold. The convergence gauge the probe emits is the median DD formal standard deviation in cycles; it has to fall well below the half-cycle scale before (219) is reachable at all, and it is the quantity to watch rather than the fix count. Two configuration caveats: `ils_max_dim` (default 0 = uncapped) exists because the search does not return on the full-network basis at operational station counts, and caps it to the lowest-formal-σ subset in the manner of Section 15.4; and `wl_cascade` is parsed and echoed in the load-time log but has no consumer anywhere in the tree — it is inert.

SD extra-wide-lane integers: Route B and the A2 datum. The second mechanism (`filter/NetworkSdEwlAr`, gated on `network.sd_ewl.fix_enable`, default `false`) decides an integer for the between-satellite single-differenced extra-wide-lane ambiguity. Its motivation is a rank argument: the float Melbourne–Wübbena row that the enclosing SD-EWL layer folds is a linear combination of observables the epoch's normal equations already contain, so it adds *no new rank*; the only genuinely new content available from that row is integrality. Rounding the SD statistic directly is circular — it would require the satellite bias difference $b^{i}_{\mathrm{EWL}} - b^{p_g}_{\mathrm{EWL}}$ the network is solving for. Route B breaks the circle by differencing the SD MW statistic *between stations*, which is the same double difference as (238):

$$\boxed{\left(\mathrm{MW}^{i}_{r_1} - \mathrm{MW}^{p_g}_{r_1}\right) - \left(\mathrm{MW}^{i}_{r_2} - \mathrm{MW}^{p_g}_{r_2}\right) \;=\; \lambda_{\mathrm{EWL}}\,\nabla\Delta N^{i,p_g}_{r_1 r_2,\mathrm{EWL}} \;+\; \nabla\Delta\varepsilon^{\mathrm{MW}},} \tag{239}$$

in which the satellite bias pair cancels exactly (same satellite pair on both sides; each station's receiver term already cancelled inside its own single difference), leaving a pure integer against averaged MW noise at the extra-wide-lane wavelength. An integer decided this way uses no a-priori satellite bias and therefore cannot be poisoned by a wrong one — the reason Route B is preferred over rounding against a bias product. The cancellation in (239) carries a precondition that is easy to overlook and is not an implementation detail: b_{EWL} is a per-*signal* combination, so both stations' arcs must sit on the same two tracking channels, or a per-signal bias difference is folded in as though it were an integer. The seam therefore elects a network-common channel pair per satellite by station consensus and admits only stations holding that exact pair.

What (239) yields is a double difference, while the row to be constrained needs a single-differenced integer per (station, satellite). The A2 datum supplies one: within each connected component elect a reference station r_0, define $\Delta n^{i,p_g}_{r_0} := 0$, and propagate the rounded DD fixes outward along a spanning tree, the per-station free integer being absorbed by the link satellite. The asserted quantity is then

$$\Delta n^{i,p_g}_{r} \;=\; \Delta N^{i,p_g}_{r,\mathrm{EWL}} - \Delta N^{i,p_g}_{r_0,\mathrm{EWL}}, \tag{240}$$

the true SD integer minus a constant per (satellite pair, component) — which is exactly the classical wide-lane fractional-cycle-bias convention modulo λ_{EWL} [5, 13], and is absorbed harmlessly into the estimated bias the network reports. Two consequences follow from (240) and are load-bearing: the per-station free integer, and hence the spanning tree itself, cancels at the point of use, so tree churn is not a defect to be defended against; and r_0's own integers are absorbed into the constant, making the choice of r_0 a datum decision rather than a measurement.

Acceptance is the scalar case of the battery in Section 15.5: a rounded DD is admitted on its fractional residual, as in (235), together with the $n = 1$ instance of the bootstrap bound (234),

$$P_{s,\mathrm{round}} = \mathrm{erf}\!\left(\frac{1}{2\sqrt{2}\,\sigma}\right), \qquad \sigma \text{ in cycles}, \tag{241}$$

so the same failure-rate semantics apply to a single rounding as to the n-dimensional search. Finally, the mechanism is subject to the standing rule that a stiff, time-linked quantity needs a wired event response: the fixed row writes a λ_{EWL}-quantised value into satellite bias states that run at very small process noise, so an integer that *changes* between epochs would inject a step those states relax out of only over hours. The wired response is withhold-on-disagreement — when this epoch's integer differs from the previous one on a *continuous* arc the fix is withheld for that arc rather than repaired, since on a disagreement epoch neither value is known to be right; an arc that restarted (slip, gap, re-acquisition) replaces the memory without being charged a disagreement, because there the integer legitimately changed. The memory is used only to veto, never to supply a value, which is what keeps this short of a carried zero-process-noise hold — that increment is deliberately not built.

Status. Both mechanisms are default-off and neither has a graded, datum-invariant field result. `network.sd_ewl.fix_enable` is additionally force-disarmed at load unless the float row layer (`network.sd_ewl.enable`) is armed, and that layer is itself inert without `network.phase_leveling` and an MW-class slip detector, since it reads the arc accumulator those build. A reader should assume the emitted network product is float.

15.8 Configuration summary

All tunable thresholds live under the JSON path `estimation.ambiguity_resolution.*`, parsed in `src/core/config/GnssConfigLoad.cpp` with defaults in `include/filter/AmbiguityResolver.hpp`. The invariant estimator theory of Sections 15.1 to 15.6 carries no knobs — changing it changes the mathematics, not a policy. The two network-producer mechanisms of Section 15.7 are configured elsewhere and separately, under `network.ambiguity_resolution.*` and `network.sd_ewl.*` (parsed in `src/core/config/GnssConfigLoadNetwork.cpp`); both master switches default to `false` and are omitted from the table below.

Symbol	Config key (`...ambiguity_resolution.`)	Default	Role
—	`mode`	`0`	0 float / 1 continuous / 2 instantaneous / 3 fix-and-hold
τ_{ratio}	`ratio_threshold`	`3.0`	ratio-test acceptance (233)
$\tau_{\text{ratio}}^{\max}$	`ratio_upper_bound`	`1e4`	degenerate-covariance reject (233)
$P_s^{\min}$	`min_success_rate`	`0.999`	FFR gate (234)
κ_{frac}	`nl_max_per_pair_frac`	`0.15`	per-pair residual cap (235)
θ_{AR}	`elevation_mask`	`20°`	candidate elevation mask
σ^2_{hold}	`hold_variance`	`0.001` m^2	pseudo-obs $\boldsymbol{R}$ (237)
$n^{\text{ep}}_{\min}$	`min_epochs`	`10`	convergence guard
$\sigma^{\max}_{\text{pos}}$	`max_position_sigma_m`	`0.05` m	convergence guard
$\sum \sin^2\theta_{\min}$	`min_arc_info`	`0.0` (off)	per-candidate arc-maturity gate, WL/NL path only (Section 15.4)
—	`wl_only`	`false`	MW-WL-only, no NL search
σ^2_{wl}	`wl_only_pseudo_var`	`1e-4` cyc^2	WL-constraint pseudo-obs $\boldsymbol{R}$ (Section 15.3)
—	`wl_max_frac`	`0.40`	per-arc WL fractional-residual gate (Section 15.3)
—	`enable_partial_ar`	`false`	backward-elimination subset fix (Section 15.4)
—	`active_systems`	`[]` (all)	constellation subset (Section 15.4)
—	`state_ambiguity_basis` †	`per_band`	`per_band` vs `nl_wl` dispatch

† Path is `estimation.state_ambiguity_basis` — one level above the `ambiguity_resolution` block.

Part VI
Inertial aiding

16 Inertial aiding

When an IMU is configured (`imu.enabled`; the sibling boolean `estimation.aiding.imu` is the `AidingSourceManager` selector and back-compat-defaults to `imu.enabled`, so enabling the legacy flag also enables the aiding-IMU source — an explicit `estimation.aiding.imu` wins, see `src/core/config/GnssConfigLoad.cpp`), the inter-epoch time update of Section 11.1 is replaced by a first-order strapdown mechanization in the Earth-centred Earth-fixed (ECEF) frame. The

mechanization itself lives in `src/core/filter/ImuIntegrator.cpp`; the live per-epoch predict routes through `Application::run` → `AidingSourceManager::collectPredict` → `src/core/aiding/ImuAidingSource.cpp`, whose `PredictContribution` is spliced into the filter covariance by `KalmanFilter::predictAided` (`src/core/filter/KalmanFilter.cpp`). The real-time engine (`RealTimeEngine::run`) drives the identical `predictAided` / `collectPredict` path; the only difference from the batch run is ingest — a live sensor feeds `ImuIntegrator::addSample`, the streaming counterpart to the CSV `loadData`, which appends to the same sorted sample buffer so streaming and batch yield identical `integrate` results.

This `IAidingSource` / `AidingSourceManager` abstraction (`include/aiding/`) is the composable seam through which every non-GNSS modality — IMU, ZUPT, NHC, baro, wheel odometry, vision — contributes up to three things (a `StateManager`-owned state block, a predict-time Φ/Q sub-block, and either independent measurement rows or extra Jacobian *columns* on existing GNSS rows). The IMU is its first and privileged client; the legacy monolithic `KalmanFilter::predictImu` is retained but no longer on the live path (superseded by `predictAided`). The `ImuAidingSource` replicates `predictImu`'s arithmetic verbatim — the strapdown equations below are unchanged by the refactor.

The formulation is an *error-state* extended Kalman filter. The mechanization propagates the nominal navigation state $(\boldsymbol{r}, \boldsymbol{v}, \boldsymbol{C}_b^e)$ deterministically from the inertial samples, while a 15-element INS error vector

$$\delta\boldsymbol{x}_{\text{ins}} = \begin{bmatrix}\delta\boldsymbol{r} & \delta\boldsymbol{v} & \delta\boldsymbol{\psi} & \delta\boldsymbol{b}_a & \delta\boldsymbol{b}_g\end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^{15} \tag{242}$$

carries the linearised transition and process noise that drive the covariance. Three attitude-error states $\delta\boldsymbol{\psi}$, three accelerometer-bias states $\boldsymbol{b}_a$, and three gyro-bias states $\boldsymbol{b}_g$ augment the existing position state (and, when `estimation.estimate_velocity` is set, the three velocity states); the augmented block is allocated contiguously by `StateManager` (accessors `getAttitudeIndex`, `getAccelBiasIndex`, `getGyroBiasIndex`).

Remark 1 (Implemented scope). The block order of Equation (242) ($[\delta\boldsymbol{r}, \delta\boldsymbol{v}, \delta\boldsymbol{\psi}, \delta\boldsymbol{b}_a, \delta\boldsymbol{b}_g]$) is fixed by `ImuIntegrator`. *No* lever-arm state and *no* sensor scale-factor states are carried, and there is no IMU-to-antenna lever-arm term in the GNSS measurement model (Section 16.4). These are roadmap items, not part of the current implementation; the corresponding terms are omitted from every equation below rather than written and then ignored.

16.1 Strapdown mechanization

For each IMU sample the raw specific force $\tilde{\boldsymbol{a}}$ and angular rate $\tilde{\boldsymbol{\omega}}$ (both in the body frame) are bias-corrected with the current filter estimates,

$$\boldsymbol{f}_b = \tilde{\boldsymbol{a}} - \boldsymbol{b}_a, \qquad \boldsymbol{\omega}_b = \tilde{\boldsymbol{\omega}} - \boldsymbol{b}_g, \tag{243}$$

and integrated over the sample interval Δt. The attitude direction-cosine matrix $\boldsymbol{C}_b^e$ (body → ECEF) is updated to first order with Earth-rotation compensation,

$$\boxed{\boldsymbol{C}_b^e \;\leftarrow\; \boldsymbol{C}_b^e\big(\boldsymbol{I} + [\boldsymbol{\omega}_b]_\times\,\Delta t\big) \;-\; [\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{C}_b^e\,\Delta t,} \tag{244}$$

where $\boldsymbol{\omega}_{ie}^e = [\,0,\,0,\,\omega_e\,]^{\mathrm{T}}$ is the ECEF Earth-rotation vector ($\omega_e = 7.292\,115\,146\,7 \times 10^{-5}\,\mathrm{rad\,s^{-1}}$, `common/Constants.hpp`) and $[\cdot]_\times$ is the skew-symmetric (cross-product) matrix. To curb numerical drift, $\boldsymbol{C}_b^e$ is re-orthonormalised by singular-value decomposition every 100 samples. The velocity

and position propagate as

$$\boxed{\begin{aligned} \boldsymbol{v}_{k+1} &= \boldsymbol{v}_k + \big(\boldsymbol{C}_b^e\,\boldsymbol{f}_b + \boldsymbol{g}^e(\boldsymbol{r}_k) - 2\,[\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{v}_k\big)\Delta t, \\ \boldsymbol{r}_{k+1} &= \boldsymbol{r}_k + \tfrac{1}{2}\big(\boldsymbol{v}_k + \boldsymbol{v}_{k+1}\big)\Delta t. \end{aligned}} \tag{245}\tag{246}$$

Equation (245) applies the Coriolis acceleration $-2\,[\boldsymbol{\omega}_{ie}^e]_\times \boldsymbol{v}$, while the centrifugal acceleration is folded into the gravity vector $\boldsymbol{g}^e$ (i.e. $\boldsymbol{g}^e$ is a *plumb-bob* gravity, gravitational plus centrifugal); Equation (246) uses a trapezoidal position update.

The gravity vector is a point-mass radial model plus the centrifugal term (`ImuIntegrator::computeGravityEcef`):

$$\boldsymbol{g}^e(\boldsymbol{r}) = -\,\frac{\mu}{\|\boldsymbol{r}\|^3}\,\boldsymbol{r} \;+\; \omega_e^2\,[\,p_x,\,p_y,\,0\,]^{\mathrm{T}}, \qquad \mu = 3.986\,005\times10^{14}\,\mathrm{m^3\,s^{-2}}. \tag{247}$$

The normal-gravity (Somigliana) flattening and the J_2 zonal term are *not* modelled; over a single GNSS epoch interval the resulting deterministic acceleration error is small and is absorbed by the estimated accelerometer bias $\boldsymbol{b}_a$.

16.2 INS error-state dynamics

Linearising Equations (244) to (246) about the nominal trajectory yields the continuous error dynamics $\dot{\delta\boldsymbol{x}}_{\mathrm{ins}} = \boldsymbol{F}_{\mathrm{ins}}\,\delta\boldsymbol{x}_{\mathrm{ins}} + \boldsymbol{G}_{\mathrm{ins}}\,\boldsymbol{w}$, with, in the block order of Equation (242),

$$\boldsymbol{F}_{\mathrm{ins}} = \begin{bmatrix} \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & -2[\boldsymbol{\omega}_{ie}^e]_\times & -[\boldsymbol{f}^e]_\times & \boldsymbol{C}_b^e & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & -[\boldsymbol{\omega}_{ie}^e]_\times & \boldsymbol{0} & -\boldsymbol{C}_b^e \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \end{bmatrix}, \qquad \boldsymbol{f}^e := \boldsymbol{C}_b^e\,\boldsymbol{f}_b. \tag{248}$$

Both sensor biases are modelled as pure random walks (zero deterministic dynamics, the null bottom rows of Equation (248)). Two terms of the textbook ECEF error model are deliberately dropped: the position-error feedback row ($[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{r},\delta\boldsymbol{r}} = -[\boldsymbol{\omega}_{ie}^e]_\times$) and the gravity-gradient coupling $[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{v},\delta\boldsymbol{r}} = +\,\partial\boldsymbol{g}^e/\partial\boldsymbol{r}$ (the gradient of the *plumb* gravity of Equation (247), which already subsumes the centrifugal $-[\boldsymbol{\omega}_{ie}^e]_\times^2$; the leading sign follows from differentiating the velocity update Equation (245)). This is a first-order approximation that is accurate over the short epoch interval of a GNSS-aided run. The discrete transition is taken to first order,

$$[\boldsymbol{\Phi}_{\mathrm{ins}}]_k = \boldsymbol{I} + \boldsymbol{F}_{\mathrm{ins}}\,\Delta t. \tag{249}$$

The process noise maps the continuous accelerometer / gyro white noise and the bias random walks through

$$[\boldsymbol{Q}_{\mathrm{ins}}]_k = \boldsymbol{G}_{\mathrm{ins}}\,\boldsymbol{W}\,\boldsymbol{G}_{\mathrm{ins}}^{\mathrm{T}}\,\Delta t, \quad \boldsymbol{G}_{\mathrm{ins}} = \begin{bmatrix} \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{C}_b^e & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & -\boldsymbol{C}_b^e & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} \end{bmatrix}, \quad \boldsymbol{W} = \mathrm{diag}\big(\sigma_a^2,\sigma_g^2,\sigma_{ba}^2,\sigma_{bg}^2\big)\otimes\boldsymbol{I}_3, \tag{250}$$

with the spectral densities $\sigma_a, \sigma_g, \sigma_{ba}, \sigma_{bg}$ read from `imu.{accel_noise_density, gyro_noise_density, accel_bias_instability, gyro_bias_instability}` (defaults in `config/GnssConfigTypes.hpp`). Over one GNSS epoch the per-sample $([\boldsymbol{\Phi}_{\mathrm{ins}}]_k, [\boldsymbol{Q}_{\mathrm{ins}}]_k)$ accumulate by the standard recursion

$$\boldsymbol{\Phi}_{\mathrm{ins}} \leftarrow [\boldsymbol{\Phi}_{\mathrm{ins}}]_k\,\boldsymbol{\Phi}_{\mathrm{ins}}, \qquad \boldsymbol{Q}_{\mathrm{ins}} \leftarrow [\boldsymbol{\Phi}_{\mathrm{ins}}]_k\,\boldsymbol{Q}_{\mathrm{ins}}\,[\boldsymbol{\Phi}_{\mathrm{ins}}]_k^{\mathrm{T}} + [\boldsymbol{Q}_{\mathrm{ins}}]_k. \tag{251}$$

16.3 Covariance splice and attitude feedback

The accumulated 15×15 pair $(\boldsymbol{\Phi}_{\text{ins}}, \boldsymbol{Q}_{\text{ins}})$ is spliced (*assigned*, not accumulated) onto an identity-seeded full-state transition $\boldsymbol{F}$ and onto the random-walk process-noise baseline $\boldsymbol{Q}$ over the indices the INS owns; the non-INS states (clock, troposphere, ionosphere, ambiguities, ...) retain their standard random-walk model from Section 11.1. The covariance is then propagated once and symmetrised,

$$\boxed{\boldsymbol{P} \leftarrow \boldsymbol{F}\,\boldsymbol{P}\,\boldsymbol{F}^{\mathrm{T}} + \boldsymbol{Q}, \qquad \boldsymbol{P} \leftarrow \tfrac{1}{2}\big(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}}\big).} \tag{252}$$

The nominal navigation states $(\boldsymbol{r}, \boldsymbol{v})$ are overwritten by the mechanised values of Equations (245) to (246), and the attitude error is reset, $\delta\boldsymbol{\psi} := \mathbf{0}$, because the rotation increment has already been applied directly to $\boldsymbol{C}_b^e$ in Equation (244). After the subsequent GNSS measurement update the corrected attitude error feeds back multiplicatively into the DCM and is again reset,

$$\boldsymbol{C}_b^e \leftarrow \big(\boldsymbol{I} + [\delta\boldsymbol{\psi}]_\times\big)\boldsymbol{C}_b^e, \qquad \delta\boldsymbol{\psi} := \mathbf{0}, \tag{253}$$

followed by SVD re-orthonormalisation (`KalmanFilter::update`). This closed-loop reset keeps $\delta\boldsymbol{\psi}$ a small-angle quantity throughout, so the linearisation in Equation (248) stays valid.

16.4 Coupling modes and the GNSS update

How the GNSS observations correct the inertial errors depends on the coupling mode (`imu.coupling_mode`).

Loosely coupled (default). The GNSS code and carrier-phase Jacobians of Section 4.3 touch only the position (and velocity) columns. The INS error states $\delta\boldsymbol{\psi}, \boldsymbol{b}_a, \boldsymbol{b}_g$ therefore receive *no direct* observation partials; they are corrected only indirectly, through the off-diagonal cross-covariances that Equation (252) builds between the position/velocity block and the attitude/bias block. The attitude correction is then realised by the feedback of Equation (253).

Tightly coupled. Tightly-coupled mode requires the velocity block, so the loader force-enables it independently of the user's `estimation.estimate_velocity` — a `TightCoupled coupling_mode` with the flag left false flips it true and logs `"forcing estimate_velocity=true"` (`GnssConfigLoad.cpp`), guaranteeing the three velocity states and hence the Doppler rows the TC Jacobian below attaches to. In this mode the *Doppler* observation alone gains direct attitude and accelerometer-bias columns. This is the one aiding contribution that re-parameterizes existing GNSS rows rather than appending its own: each relinearisation pass calls `AidingSourceManager::applyTcJacobianInputs` → `ImuAidingSource::applyTcJacobianInputs`, which feeds the current attitude and specific force to `MeasurementModel::setImuState` (`src/core/measurement/MeasurementModel.cpp`). With $\boldsymbol{f}^e = \boldsymbol{C}_b^e \boldsymbol{f}_b$ the specific force in ECEF and $\boldsymbol{e}_r^i$ the unit line-of-sight of Equation (7), the Doppler measurement function h_{D} contributes

$$\boxed{\frac{\partial h_{\text{D}}}{\partial \delta\boldsymbol{\psi}} = -\,[\boldsymbol{f}^e]_\times^{\mathrm{T}}\,\boldsymbol{e}_r^i, \qquad \frac{\partial h_{\text{D}}}{\partial \boldsymbol{b}_a} = -\,(\boldsymbol{C}_b^e)^{\mathrm{T}}\,\boldsymbol{e}_r^i.} \tag{254}$$

The specific force fed into Equation (254) is a static-rest approximation $\boldsymbol{f}_b \approx (\boldsymbol{C}_b^e)^{\mathrm{T}}(-\boldsymbol{g}^e) - \boldsymbol{b}_a$ (`ImuAidingSource.cpp`); it is exact only when the platform's true acceleration is negligible against gravity. The *code and carrier-phase rows carry no attitude or bias partials in either mode* — only Doppler couples directly to the inertial errors.

Remark 2 (Relinearisation under tight coupling). The tightly-coupled path retains the legacy single *naive* second-pass relinearisation rather than the prior-anchored iterated EKF of Section 11.3: re-deriving the gain from a fixed prior conflicts with the multiplicative attitude/bias feedback of Equation (253), which has already mutated $\boldsymbol{C}_b^e$ between passes. The aiding layer exposes this as a capability query (`AidingSourceManager::anySourceRequiresNaiveRelinearization`, which the IMU source answers true for in tight coupling), so any future nominal-feedback source forces the same path; the migration off the direct `tightlyCoupledImu()` checks in `Application.cpp` / `KalmanFilter.cpp` is still in progress.

Part VII
Real-time SSR corrections

17 Real-time SSR corrections

The real-time engine consumes network corrections in *State Space Representation* (SSR): per-satellite, per-signal refinements to the broadcast orbit, clock, and hardware biases, streamed as RTCM 3.x messages (legacy 1057–1068 / 1240–1270) or as their IGS-SSR `4076` repackaging [9, 21]. Each SSR quantity refines *exactly one* term already present in Equation (4) and Equation (5) of the Foundations document; none introduces a new state. This is what separates the precise real-time mode from broadcast-only operation: with SSR applied, $c\,\mathrm{d}t^i$ and $\boldsymbol{r}^i$ are centimetre-grade and the satellite biases B_s^i, b_s^i are *known* rather than absorbed into the ambiguity. The corrections in this document are the rover-side *consumer*: the same on-disk store is the sink for both a third-party IGS-RTS stream and the engine's own network estimator (Section 19), so the application math below is identical regardless of who produced the corrections.

Decoded records are cached per satellite in `products/SsrCorrectionStore` (a process-wide singleton for the live stream, or a caller-owned instance for the network producer's closed-loop test); the consumers (`model/ModelGeometry`, `corrections/GnssCorrections`, `measurement/MeasurementModel`) query the store and fall through to broadcast-only when it is empty, so every equation below collapses to the broadcast-only observation model in post-processing. The streaming and decoding plumbing — RTCM frame parse, the per-satellite mutex-guarded cache, the real-time push loop — belongs to Section 18; this document covers only the store-driven correction math, which is invariant to how records arrived. There is no `enable_ssr` flag: SSR application is implicit and store-driven.

17.1 Orbit correction — refines ρ_r^i

The orbit message carries a delta in the satellite's radial / along-track / cross-track (RAC) frame, anchored to the broadcast ephemeris of issue IODE. The frame is built from the broadcast position $\boldsymbol{r}_{\mathrm{brdc}}^i$ and velocity $\dot{\boldsymbol{r}}_{\mathrm{brdc}}^i$ (`products/SsrCorrectionStore.cpp`):

$$\hat{\boldsymbol{e}}_a = \frac{\dot{\boldsymbol{r}}_{\mathrm{brdc}}^i}{\|\dot{\boldsymbol{r}}_{\mathrm{brdc}}^i\|}, \quad \hat{\boldsymbol{e}}_c = \frac{\boldsymbol{r}_{\mathrm{brdc}}^i \times \dot{\boldsymbol{r}}_{\mathrm{brdc}}^i}{\|\boldsymbol{r}_{\mathrm{brdc}}^i \times \dot{\boldsymbol{r}}_{\mathrm{brdc}}^i\|}, \quad \hat{\boldsymbol{e}}_r = \hat{\boldsymbol{e}}_a \times \hat{\boldsymbol{e}}_c. \tag{255}$$

The RAC delta is propagated to the query epoch with its rate term and rotated into ECEF. With $\delta t = t - t_{\mathrm{SSR}}$ the age of the correction, $\boldsymbol{\delta}_{\mathrm{rac}} = (\delta_r, \delta_a, \delta_c)^{\mathrm{T}}$ its components and $\dot{\boldsymbol{\delta}}_{\mathrm{rac}}$ their rate,

$$\boxed{\boldsymbol{r}^i \;=\; \boldsymbol{r}_{\mathrm{brdc}}^i \;-\; \big[\,\hat{\boldsymbol{e}}_r \;\; \hat{\boldsymbol{e}}_a \;\; \hat{\boldsymbol{e}}_c\,\big]\big(\boldsymbol{\delta}_{\mathrm{rac}} + \dot{\boldsymbol{\delta}}_{\mathrm{rac}}\,\delta t\big).} \tag{256}$$

The *subtraction* follows the RTCM sign convention $\boldsymbol{\delta}$ = broadcast − precise (`products/SsrCorrectionStore.cpp`). The corrected $\boldsymbol{r}^i$ then enters $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line-of-sight $\boldsymbol{e}_{\hat{r}}^i$ of the Foundations document unchanged: the orbit correction never touches an observation equation directly, only the geometry (Section 4) that feeds it. The delta is applied only when its IODE matches that of the broadcast ephemeris the baseline orbit was evaluated from (Equation (260)); on a live cutover the baseline position is deliberately re-referenced to the ephemeris *version* the SSR delta was issued against rather than dropped to broadcast grade (`model/ModelGeometry.cpp`, via `LiveEphemerisStore::computeSatellitePV` with the SSR IODE). Because the clock is queried against the same broadcast ephemeris, the broadcast clock's *radial* component cancels consistently against Equation (258). The decoded RAC scale factors are fixed by RTCM 10403.3 (radial δ_r a 22-bit two's-complement field at 0.1 mm, along/cross at 0.4 mm, rates at 1 and 4 μm/s respectively; `rtcm/RtcmSsrDecoder.cpp`). The orbit record also carries a *reference point* for the corrected $\boldsymbol{r}^i$: legacy RTCM SSR corrects to the satellite antenna phase centre (APC), while IGS-SSR `4076` and the network product correct to the centre of mass (CoM) (`rtcm/RtcmSsrDecoder.cpp`). Under `realtime.honor_ssr_reference_point` (RTB-5; default OFF, byte-identical) the consumer records whether the applied overlay was APC-referenced (`ssr_orbit_apc_applied_` in `model/ModelGeometry.cpp`) so that `PppModel` suppresses its own satellite-PCO term on that geometry rather than double-counting the phase-centre offset the orbit correction has already resolved.

17.2 Clock correction — refines $-c\,\mathrm{d}t^i$

The clock message carries a quadratic polynomial in time-from-epoch, optionally augmented by a high-rate additive delta δC_{hr}. The total satellite-clock correction, in metres, is

$$\Delta C^i(t) \;=\; C_0 + C_1\,\delta t + C_2\,\delta t^2 + \delta C_{\mathrm{hr}}, \qquad \delta t = t - t_{\mathrm{SSR}}, \tag{257}$$

converted to seconds and *added* to the baseline clock (`products/SsrCorrectionStore.cpp`):

$$\boxed{c\,\mathrm{d}t^i \;=\; c\,\mathrm{d}t^i_{\mathrm{base}} \;+\; \Delta C^i(t),} \tag{258}$$

where $\mathrm{d}t^i_{\mathrm{base}}$ is the precise CLK sample (post-processing) or the broadcast clock polynomial (live rooftop mode, `products/LiveEphemerisStore`). This term enters both undifferenced observation equations through $-c\,\mathrm{d}t^i$, identically on code and phase. The high-rate clock is additive *on top of* the polynomial, not a replacement (`corrections/GnssCorrections.cpp`); the C_2 field is a 27-bit two's-complement quantity at 2×10^{-8} m/s^2.

Split-cadence fast clocks. A third, structurally different way for the stored clock to move is a *fast tick* (`SsrCorrectionStore::pushFastClock`), which the network producer can run between full solves on decimated-out epochs. A fast tick re-bases exactly three fields of an *existing* clock record — C_0 and the two epoch stamps — so (257) is thereafter evaluated at the fast δt while retaining the slow era's C_1 and C_2:

$$\Delta C^i(t) = C_0^{\mathrm{fast}} + C_1\,(t - t_{\mathrm{fast}}) + C_2\,(t - t_{\mathrm{fast}})^2. \tag{259}$$

Two omissions are what make this safe for a consumer, and both are contract, not accident. The discontinuity counter is *not* advanced: a fast row is a continuation of the same clock, not a new arc, so it must not trip the ambiguity reset of the discontinuity paragraph below at the fast cadence. And the shared per-satellite SSR header is *not* restamped, so the code-bias, phase-bias, URA and slant-iono channels do not inherit the fast clock's age off one refresh. The method is also refresh-only

— it returns false rather than creating a record, so a consumer can never be handed a satellite that exists on the fast channel alone with no bias set to go with it. It is wired both in-process (`filter/NetworkFastClock`) and on replay of an on-disk product (`products/NetworkProductIO`), and is default OFF (`network.fast_clock.enable`, which the loader also disarms when the run's posture leaves no epoch for a tick to ride).

17.3 IOD gate at correction-set boundaries

Orbit and clock corrections are deltas against a *specific* broadcast ephemeris, so they are applied only when the SSR record's IODE matches that of the broadcast ephemeris from which the baseline orbit/clock was evaluated:

$$\text{apply} \iff \big(\text{IODE}_{\text{brdc}} < 0\big) \ \vee \ \big(\text{IODE}_{\text{SSR}} < 0\big) \ \vee \ \big(\text{IODE}_{\text{brdc}} = \text{IODE}_{\text{SSR}}\big), \tag{260}$$

the gate being permissive when either side is unknown (`SsrCorrectionStore::iodeMatches`, `products/SsrCorrectionStore.hpp`). A combined orbit+clock record (RTCM 1060/1066, IGS-SSR sub-type 3) is preferred over separate messages because it guarantees a single shared IODE (`products/SsrCorrectionStore.cpp`). Since RTF-2 the two overlays gate on *separate* anchors: the orbit delta on the record's orbit `iode` (`model/ModelGeometry.cpp`), the clock polynomial on a distinct `clock_iode` field (`corrections/GnssCorrections.cpp`). For a standard stream `clock_iode` inherits the orbit anchor in `getOrbitClock`, so the two coincide and (260) reads as the single gate written; only a `brdc_rts` producer, which stamps its own clock IODE, makes them differ. At a correction-set boundary — the provider issuing a new ephemeris reference, hence a new IODE — the gate (260) simply declines the mismatched record until the matching broadcast ephemeris is in hand, avoiding a discontinuous jump in $\boldsymbol{r}^i$ or $c\,\mathrm{d}t^i$. Records older than `max_age_s` (`processing.ssr_max_age_s`, default 30 s) are treated as absent by default. The RTA-9(b) stale-hold ladder relaxes this binary clock gate into three rungs (`SsrCorrectionStore::classifyClockAge`): with `processing.ssr_clock_stale_hold_factor` $f > 1$ a clock aged in (`max_age_s`, $f \cdot$ `max_age_s`] is *StaleHeld* — still applied as the extrapolated $C_0 + C_1\,\delta t$ of (257), with a matching URA-staleness R inflation on the consuming rows (`corrections/GnssCorrections.cpp`, `measurement/MeasurementModel.cpp`) — while a record beyond the hold window is *Rejected*. The default $f = 0$ collapses the ladder back to the plain fresh-or-absent gate (byte-identical).

17.4 Satellite biases — refine $-B_s^i$ and $-b_s^i$

SSR delivers *observable-specific biases* (OSB): one value per (satellite, RINEX-3 signal) pair, in metres. They are applied as a *known* additive correction to the modelled observable, with no state and no Jacobian row (`measurement/MeasurementModel.cpp`). The code OSB refines B_s^i in the undifferenced code equation:

$$\boxed{P_{r,s}^i = \rho_r^i + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t_r^{(g)} + T_r^i + \mu_s I_{r,1}^i + B_{r,s} \underbrace{- B_s^i}_{\text{SSR OSB}} + \varepsilon^P,} \tag{261}$$

and the phase OSB refines b_s^i in the undifferenced phase equation:

$$\boxed{\Phi_{r,s}^i = \rho_r^i + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t_r^{(g)} + T_r^i - \mu_s I_{r,1}^i + \lambda_s N_{r,s}^i + b_{r,s} \underbrace{- b_s^i}_{\text{SSR OSB}} + \varepsilon^\Phi.} \tag{262}$$

The bias decoded from the stream is the satellite bias *present in* the observation, so it is removed from the clean model prediction with a +1 sign convention (`kSsrCodeBiasSign` = `kSsrPhaseBiasSign` =

+1, `measurement/MeasurementModel.cpp`); the predicted observable accumulates it into the predicted observable as `ssr_bias_term` (`MeasurementModel.cpp`), summed alongside the receiver clock, troposphere, ionosphere, estimated code/phase biases and ambiguity. Biases are queried per *raw* signal only — an ionosphere-free composite key such as `C1W_C5X` returns zero (code) or `nullopt` (phase) and is skipped, consistent with the uncombined parameterisation. The code OSB is a 14-bit field at 0.01 m, the phase OSB a 20-bit field at 0.1 mm (`rtcm/RtcmSsrDecoder.cpp`). The application path is *ungated* by default: `ssr_bias_term` loads whenever the store carries the exact RINEX signal key. An optional A5 staleness gate (`processing.enable_ssr_bias_max_age`, default OFF) rejects a bias whose obs-vs-record age exceeds `processing.ssr_bias_max_age_s` (default 30 s), holding the last value over sub-limit gaps out to `ssr_bias_stale_hold_factor` × that age rather than manufacturing a model step (`measurement/MeasurementModel.cpp`). The corollary of the application path — avoiding a redundant estimated bias on the same row — is the subject of the next subsection.

17.5 Satellite-bias STATE gate vs. SSR code biases

The uncombined model can also *estimate* a per-(satellite, signal) code-bias state (`estimation.estimate_sat_biases`; see Section 3). That estimated state and a known SSR code OSB both enter the code row with unit coefficient and opposite physical roles, so allowing both for the same (i, s) would be doubly redundant: the stream already fixes the satellite code bias, and a free state collinear with it is near-unobservable and merely soaks up noise. State allocation therefore consults *both* correction sources before creating a sat-bias state (`MeasurementModel::ensureRequiredStates`):

$$\text{create sat-bias}(i,s) \iff \neg\,\text{hasBIA}(i,s) \;\wedge\; \neg\,\text{hasSsrCB}(i,s) \;\wedge\; \text{pivotCovers}(i,s), \tag{263}$$

where hasBIA tests the static `.BIA` product (`PppModel::hasBiasEntry`) and hasSsrCB tests the live stream (`SsrCorrectionStore::hasCodeBias`, surfaced as `ssrCoversCodeSignal`). Two design points make the gate correct:

- **Presence, not magnitude.** `hasCodeBias` is an exact-key *presence* test on the per-signal map, distinct from `getCodeBias` $\neq 0$: a provider's present-but-zero entry still defines that signal's datum frame, so the estimator must not re-estimate it. The two queries are required to agree exactly so a covered signal is never both estimated *and* SSR-corrected on one row.
- **Per-signal, not global.** IGS streams are partial (e.g. Galileo E1 covered, E5a/E5b not). The gate keys on the exact (i, s) pair, so a signal the stream does *not* carry keeps its estimated state — its only available code correction — while a covered sibling on the same satellite is deferred to the OSB. A strict per-epoch prune (`pruneSatBiasStatesExcept`) drops the state when SSR coverage arrives mid-arc and re-creates it (warm-restarted from the stashed cache) if coverage later disappears.

In post-processing the live store is empty, so hasSsrCB is uniformly false and the gate reduces to the static-`.BIA` test alone.

Network-producer exemption. When the engine *is* the network estimator (`ProcessingMode::PPP_RTK_Network`), the roles invert: it *estimates* the satellite code/phase biases as states to emit as a product (Section 19). Importing SSR biases there would double-count and render those very states unobservable, so both the `ssr_bias_term` application path (261) and the hasSsrCB branch of the state gate (263) are forced off on the producer side (`ssrCoversCodeSignal` returns false). The producer's own emitter (`products/NetworkProductEmitter.cpp`) then pushes the estimated

clock and code/phase biases into a *separate* store instance, closing the loop against a held-out user consumer.

Integer recoverability via consistent phase biases. Removing b_s^i is precisely what restores the integer property of $N_{r,s}^i$ promised in the phase observation equation of the Foundations document. Once the satellite phase bias is known, the residual non-integer part of the ambiguity is the receiver phase bias $b_{r,s}$ *alone*, which cancels under between-satellite single differencing $\Delta\cdot$ (Section 5), leaving an integer single-difference ambiguity admissible to AR. This is the operational content of a *consistent* phase-bias product: the provider must generate b_s^i on the same network datum whose integer property the rover then exploits. The phase-bias message therefore carries three AR-critical metadata fields per signal (`rtcm/RtcmTypes.hpp`): an integer-preservation indicator, a two-bit wide-lane integer indicator, and a four-bit *discontinuity counter*.

Discontinuity handling. An incremented discontinuity counter signals that the provider's phase bias jumped between correction sets, breaking the integer connection of that arc. The engine treats this as an ambiguity-arc boundary: the affected $N_{r,s}^i$ is reset before the Kalman update (`realtime/RealTimeEngine.cpp`, via `StateManager::noteSsrDiscontinuity`), so the post-jump phase is re-initialised rather than fixed against a stale integer offset. This is the phase-bias analogue of the orbit/clock IOD gate (260): both protect state continuity across correction-set boundaries, one by declining a mismatched delta, the other by resetting the dependent ambiguity.

17.6 GLONASS

GLONASS SSR is decoded and stored on the same footing as the CDMA constellations: the legacy block (orbit 1063, clock 1064, code bias 1065, combined 1066, URA 1067, high-rate clock 1068, phase bias 1266) and the IGS-SSR `4076` subtypes in the GLONASS message-indicator band (IM = 41–47) both map onto the identical orbit/clock/bias structs (`rtcm/RtcmSsrDecoder.cpp`). The orbit, clock and bias *equations* above apply unchanged; nothing in Equations (256), (258), (261) and (262) is constellation-specific.

The one GLONASS-specific subtlety is on the *time tag*, not the correction math. GLONASS SSR carries its content epoch as a time-of-day field (DF389, seconds since the start of the GLONASS UTC(SU)+3 h day) rather than a GPS time-of-week. The decoder promotes it to the GPS scale through `TimeUtils::glonassTodToGpsTime` (`rtcm/RtcmSsrDecoder.cpp`); without that promotion the SSR epoch would land at $\{0, 0\}$, the freshness check (260)'s sibling `max_age_s` gate would declare every GLONASS record stale, and the satellite would silently fall back to broadcast grade. Once promoted, the per-record content epoch drives $\delta t = t - t_{\mathrm{SSR}}$ in Equations (256) and (258) exactly as for GPS. Whether the resulting GLONASS observable then *contributes* to a between-satellite-differenced solution is a separate, estimator-side opt-in (its FDMA per-channel receiver delay does not cancel in SD); see Section 5.

17.7 Ionosphere — $\mu_s I_{r,1}^i$ off-wire, via the slant channel

For completeness, and to forestall a natural misreading of the term: although the SSR standard defines a global VTEC spherical-harmonic message (IGS-SSR IM = 201) and regional STEC products, *neither is decoded by this engine*. The IGS-SSR classifier rejects IM = 201 as unsupported (`rtcm/RtcmSsrDecoder.cpp`), so nothing on the RTCM wire writes the slant-iono state $I_{r,1}^i$. On that wire the ionospheric term $\mu_s I_{r,1}^i$ is therefore informed only by (i) the per-satellite iono *state*

estimated by the filter itself and (ii) the optional GIM/IONEX deterministic prior (`atmosphere/GlobalIonosphereMap`, an independent VTEC→STEC mapping unrelated to the SSR stream).

The atmosphere loop is, however, closed *off* the RTCM wire, through the same `SsrCorrectionStore` the clock and bias overlays use. The store carries a dedicated undifferenced slant-iono channel (`SsrSlantIono`, via `SsrCorrectionStore::pushSlantIono` / `getSlantIono`): the network producer's emit seam (`filter/NetworkEmitStore`) evaluates its co-estimated SH-VTEC field (Section 8.2.4) at the consumer's pierce points and pushes a per-LOS slant delay and 1σ, and the consumer (`RealTimeEngine::processEpoch`) folds each fresh, same-epoch record into an ionosphere-weighted (BLUP) pseudo-observation on the matching slant-iono state (`KalmanFilter::applyPseudoObservation`) — exactly the iono-weighted constraint that makes single-epoch AR feasible. The fold is gated by `closed_loop.consume_network_iono` (default OFF, so no rows are built and the update is byte-identical) and admits a value only when the producer refreshed it this epoch (`max_age` $= \frac{1}{2}$ the epoch spacing), so a satellite the producer did not re-supply floats rather than inheriting a stale delay.

Three properties of the channel are worth stating because they are inherited from the field, not chosen here. The supply is GPS and Galileo only — GLONASS is excluded from the SH field itself, so a consumer's GLONASS slant states simply float. When a second shell is armed the pushed value is the *sum* over shells, each evaluated at its own pierce geometry, so the consumer is layer-agnostic by construction. And the per-LOS σ is the field's own representativeness floor (Equation (87)), optionally root-sum-squared with the SH coefficients' *formal* σ propagated through the same weighted basis and additionally gated on fresh OSB coverage for that satellite, under `network.iono_supply_formal_sigma` (default OFF, byte-identical). On the consumer side the rows are not independent: they are folded as a per-constellation common-mode-free block with exactly the noise structure of Equation (83), and frozen BSSD pivots (and any zero-variance slant state) are excluded from it for exactly the reason given there — the common-mode block exists to absorb the datum offset of the *remaining* rows, so the datum satellite must not appear as a measurement.

What remains open is a *standardised* on-wire STEC product: the in-process channel is a closed-loop store hand-off, not an IGS-SSR/RTCM message either side can yet decode from a third-party stream.

17.8 Provenance and code pointers

Both transports map onto the same internal correction structs (`rtcm/RtcmTypes.hpp`) through a single decoder (`rtcm/RtcmSsrDecoder.cpp`). Legacy RTCM uses per-constellation six-message blocks (orbit, clock, code bias, combined orbit+clock, URA, high-rate clock) plus a separate phase-bias block (1265–1270); IGS-SSR `4076` repackages the same payloads under a message-indicator (IM) whose *tens* place ($\lfloor \mathrm{IM}/10 \rfloor$) selects the constellation (GPS $2x$, GLONASS $4x$, Galileo $6x$, QZSS $8x$, BeiDou $10x$, SBAS $12x$) and whose *ones* place (IM mod 10) selects the subtype (1=orbit, 2=clock, 3=combined, 4=high-rate, 5=code bias, 6=phase bias, 7=URA) — note this per-block subtype ordering differs from the legacy SSR block ordering, so the legacy mapping is not reused (`RtcmSsrDecoder.cpp`). VTEC (IM 201) and NavIC (IM 141–147) are rejected by the message-indicator classifier *before* the header is parsed, so they return `ok = false` with an empty header — this is the decode-side reason the ionospheric term carries no *on-wire* SSR contribution (the off-wire `SsrSlantIono` store channel of the ionosphere subsection is unaffected).

The invariant correction definitions of Equations (256), (258) and (260) live in the static, deterministic `applyOrbitToPosition` / `applyClockToBias` / `iodeMatches` members of `products/SsrCorrectionStore`, which is the seam this document documents. The stateful streaming layer — the mutex-guarded per-satellite cache with wall-clock freshness gating, the `RealTimeEngine::`

`feedRtcmFrame` push loop, and the obs↔SSR synchronisation that bounds how far the obs stream may run ahead of lagging corrections — belongs to Section 18 and is kept separate so the correction math can be reasoned about (and tested) in isolation. The one user-facing SSR-related estimator config is the warm-up gate `estimation.defer_until_corrections_ready` (`config/GnssConfigLoad.cpp`), which defers estimation until a solvable floor of satellites is broadcast-plus-SSR centimetre-grade, so the filter seeds code and phase together rather than cold-starting a contaminated partition on metre-grade broadcast clocks.

Alongside the correction records the store also carries a per-epoch producer *heartbeat* (`NetworkProducerStatus`), which is what lets a consumer distinguish "the producer withheld this epoch" from "the producer died" — a withheld epoch still writes its status line even though it publishes no corrections. The wire token set is deliberately forward-compatible: an unrecognised token from a newer producer maps to `State::Unknown` and is surfaced, never fatal (`SsrCorrectionStore::parseState / stateName`). One of the tokens now reports a withhold driven by the health of the producer's own SH-VTEC field; the *policy* behind that withhold — what engages it, what it suppresses, and what it does not — belongs to Section 19 and is not restated here.

Part VIII
Real-time engine

18 Real-time engine

The real-time path reuses the same estimator, measurement model and state manager as post-processing, but reorganises the control flow around a *streaming* contract: complete RTCM 3 (or SBF) frames arrive one at a time over a network socket, are decoded into observations / corrections / broadcast ephemeris, and a buffered epoch is pushed through the Section 11 predict/update cycle the instant its boundary is detected. Where post-processing iterates a closed list of epochs in time order with the entire orbit / clock product set already on disk, the engine sees each epoch exactly once, with corrections that may lead or lag the observations, and can never look ahead. The entry point is `Application::runRealTime` (`src/core/app/ApplicationRealtime.cpp`), which constructs a `RealTimeEngine` (`src/core/realtime/RealTimeEngine.cpp`) and drives it from a single-threaded multi-source poll loop. This section documents the parts that are *specific* to that path; the estimation mathematics, observation equations and SSR correction model are documented elsewhere and only cross-referenced here.

18.1 Streaming control flow

The poll loop holds one `RealtimeStream` per logical source and services them round-robin without ever blocking on a single one:

- a *primary* stream carrying rover observations (RTCM 3 MSM or Septentrio SBF, from a file replay, an NTRIP mountpoint, or a raw TCP socket);
- an optional *corrections* stream (typically an IGS-RTS NTRIP mountpoint carrying SSR);
- an optional *ephemeris* stream (broadcast nav for an obs-only station that does not co-broadcast 1019/1020/...).

Each iteration drains a bounded burst from every active source through its framer and dispatches complete frames to the engine; the primary obs drain is byte-capped per iteration so a high-rate observation stream cannot starve the sparse-but-critical SSR / nav streams (`pumpFrames`). The loop exits when every source goes inactive (file EOF or socket teardown), when a cooperative stop is requested (Ctrl+C / console-close handler), or when the `processing.max_duration_s` data-time budget latches; on exit the final buffered epoch is flushed before teardown so no observations or trace rows are lost.

A socket source is treated as *active* while its client intends to run, not merely while momentarily connected, so a transient drop or the brief pre-connect window does not end the session prematurely; the client reconnects underneath.

18.2 NTRIP transport

Caster connection. `NtripClient` (`src/core/realtime/NtripClient.cpp`) is an NTRIP v2 client built on libcurl. A background thread holds the HTTP GET open and pushes received bytes into a mutex-guarded ring buffer with a hard cap (4 MiB); the buffer applies back-pressure by dropping the oldest bytes when full. Consumers pull through a blocking `read` or a timeout-bounded `tryRead` (the latter is what the multi-stream loop uses so one stream cannot block the others). The request carries the NTRIP v2 headers (`Ntrip-Version:Ntrip/2.0`, a `User-Agent`); the mountpoint is the URL path; HTTP Basic authentication (`CURLAUTH_BASIC`) is supplied when a username is configured, and an optional NMEA GGA sentence is POSTed for VRS mountpoints.

Transport security. The transport defaults to plain HTTP. Plain-HTTP casters (IGS-IP, `igs-ip.net`, SBF replay) work out of the box, byte-identically. A `use_tls` flag selects the `https` URL scheme and forces `CURLUSESSL_ALL`, which is required for port-443 casters such as CDDIS that offer no plain-HTTP endpoint. Certificate verification (`SSL_VERIFYPEER`/`VERIFYHOST`) stays on unless a testing-only `tls_insecure` override disables it for self-signed or mis-chained casters. Both flags default off; a GNSS-only / IGS-IP run is unaffected.

Reconnection. The receive loop reconnects automatically. After any transfer ends it classifies the outcome — transport error, a genuine non-200 HTTP status (with actionable hints for 401 / 403 / 404), a zero-byte idle mountpoint, or a clean stream end — logs it, and always backs off for `reconnect_delay_s` before retrying, so a misconfigured mountpoint surfaces as an error rather than spinning invisibly. A non-200 *ICY* response (NTRIP v1 casters answer "ICY 200 OK", reported by libcurl as code 0) is treated as success.

18.3 RTCM 3 framing and CRC

`RtcmStreamReader` (`src/core/realtime/RtcmStreamReader.cpp`) turns an arbitrary byte stream into discrete RTCM 3 frames, decoupling transport from parsing. An RTCM 3 frame is

$$\underbrace{\texttt{0xD3}}_{\text{preamble}}\ \underbrace{[\,6\text{ reserved}\,|\,10\text{-bit length }L\,]}_{2\text{ bytes}}\ \underbrace{[\,\text{payload, }L\text{ bytes}\,]}\ \underbrace{[\,\text{CRC-24Q}\,]}_{3\text{ bytes}}, \tag{264}$$

so the total frame length is $3 + L + 3$. `nextFrame` scans for the `0xD3` preamble (discarding non-preamble bytes), reads the 10-bit payload length, waits until the full $3 + L + 3$ bytes are buffered, and verifies the trailing 24-bit Qualcomm CRC (CRC-24Q, table-driven) computed over header+payload. A CRC mismatch advances past the candidate preamble byte and keeps scanning — a corrupt or misframed byte never wedges the reader. Only CRC-valid frames are emitted, so the

downstream parser (`RtcmParser`) may assume frame integrity. The reader tracks frames-extracted and bytes-discarded counters for stream-health diagnostics.

Septentrio SBF. A parallel binary path decodes Septentrio SBF frames into the same observation / ephemeris outputs (`src/core/sbf/`): `SbfStreamReader` does the `$@` (`0x24 0x40`) sync and CRC-16/XMODEM check, `SbfParser` dispatches blocks by ID, `Meas3Decoder` unpacks the `MeasEpoch` / `Meas3Ranges` observations (GLONASS FDMA slots resolved through `SbfSignalMap`), and `SubframeAssembler` rebuilds broadcast ephemeris from raw-nav subframes. Only the framing and feed seam (Sections 18.1 and 18.9) are covered here. The `Meas3Ranges` differential range reconstruction — the direct analogue of (265) and (266) below, and the one part of the SBF path that is report-worthy algebra rather than ICD bookkeeping — is *not* yet derived in this reference; it is a known gap, not material documented elsewhere. The remaining SBF detail (`SubframeAssembler` ICD scaling, the `SbfSignalMap` tables, the framing constants) belongs in the Doxygen headers and is deliberately out of scope here.

The top-level `RtcmParser` (`src/core/rtcm/RtcmParser.cpp`) reads the 12-bit message type DF002 from the *first 12 bits of the payload* (not the header) and dispatches: 1005/1006 station coordinates, 1033 receiver/antenna descriptor and 1230 GLONASS code-phase biases (both surfaced-not-consumed), 4076 IGS-SSR, legacy SSR (1057–1068, 1240–1270), broadcast ephemeris (1019/1020/1042/1044/1046), and MSM4 / MSM5 / MSM7 observations. The week needed to promote the various truncated epoch fields to a full `GnssTime` comes from the parser's per-stream time tracker, not from wall-clock at each frame (Section 18.6).

18.4 MSM observation decode

MSM observation messages are decoded by a single body (`decodeMsmImpl`, `src/core/rtcm/RtcmMsmDecoder.cpp`) parameterised over an `MsmVariant` descriptor that supplies the per-field bit widths, scale factors, sentinels and capability flags. Three variants are emitted, MSM4, MSM5 and MSM7 (MSM6 is described by the same variant mechanism, but no variant entry is instantiated for it, so it is not currently produced). The header carries a 64-bit satellite mask, a 32-bit signal mask, and an $N_{\text{sat}} \times N_{\text{sig}}$ cell mask; the number of cells is $N_{\text{cell}} = \text{popcount(cell mask)}$. Each field block is packed contiguously across all satellites or all cells, in mask order, and decoded into parallel vectors before recombination.

Range reconstruction. For each satellite the rough range in milliseconds is assembled from the 8-bit integer (DF397) and the 10-bit modulo (2^{-10} ms, DF398) parts, and each cell adds a signed fine term scaled per variant. Writing c for the speed of light, the pseudorange and carrier phase for a cell are

$$P = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\, 2^{-10} + n^{P}_{\text{fine}}\, \sigma_P\Big), \tag{265}$$

$$\Phi_{[\text{m}]} = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\, 2^{-10} + n^{\Phi}_{\text{fine}}\, \sigma_\Phi\Big), \qquad \Phi_{[\text{cyc}]} = \Phi_{[\text{m}]}/\lambda, \tag{266}$$

with fine scales $\sigma_P = 2^{-24}$, $\sigma_\Phi = 2^{-29}$ for MSM4 and $\sigma_P = 2^{-29}$, $\sigma_\Phi = 2^{-31}$ for MSM7 (all in ms). RTCM-defined sentinel values mark invalid cells and are skipped. Carrier phase is emitted in cycles, matching the RINEX-3 convention consumed by the observation model (Section 4); when the wavelength λ cannot be resolved the phase is carried in metres as a fallback. CNR (DF408/DF403) and, for MSM5/7, Doppler from the combined rough+fine phase-range-rate are also produced; the half-cycle-ambiguity bit sets the LLI half-cycle flag.

DF419 extended-satellite-info gating. A subtle framing hazard governs the per-satellite block. The 4-bit DF419 "extended satellite information" field is present in the bitstream for *every* constellation in MSM5/MSM7, not only GLONASS; only GLONASS *assigns it meaning* (the FDMA frequency-channel number). The decode therefore separates two concerns:

- *bitstream consumption* is gated on the MSM variant (`honour_ext_sat_info`, true for MSM5/MSM7, false for MSM4/MSM6) — when set, $4\,N_{\text{sat}}$ bits are read for all systems;
- *interpretation* is gated separately on the system descriptor; only GLONASS maps the value to a channel.

This is the fix for the GPS L1/L2 geometry-free sign anomaly: reading DF419 only for GLONASS (or omitting it for MSM7) under-consumes the per-satellite block by $4\,N_{\text{sat}}$ bits, so the per-cell fine pseudoranges are read from the wrong bit offset. The corruption preserves the rough range but biases the fine terms, producing an L1/L2 split of order hundreds of metres with the wrong sign on the geometry-free combination — plausible-looking but wrong observations that survive a naive over-read check. The gate matches RTCM 10403.3 and the cssrlib reference. A complementary framing guard in the parser (`warnIfUnderConsumed`) warns whenever a decode leaves ≥ 8 unread payload bits, catching exactly this class of silent misalignment that the bit reader's over-read flag cannot.

18.5 Broadcast nav, SSR and station coordinates

Broadcast ephemeris. Messages 1019 (GPS LNAV), 1044 (QZSS), 1046 (Galileo I/NAV), 1042 (BeiDou D1) and 1020 (GLONASS) are decoded by `RtcmNavDecoder` into a common `BroadcastEphemeris` and pushed into the process-wide `LiveEphemerisStore`, which is what lets the geometry layer resolve satellite orbits with no SP3/CLK file loaded. The per-message field order differs by constellation (e.g. QZSS leads with its clock block); GLONASS carries a sign-magnitude PZ-90 state integrated separately.

SSR corrections. Both the legacy SSR ranges and the IGS-SSR 4076 container are decoded by `RtcmSsrDecoder` and pushed into the shared `SsrCorrectionStore` carrying the real SSR header (provider epoch and IOD-SSR) so staleness and IOD diagnostics reflect the provider solution rather than wall-clock receipt. Orbit, clock, combined, code-bias, phase-bias, URA and high-rate-clock records are stored per satellite; the orbit / clock query paths consult the store transparently, so no per-epoch wiring is needed. The correction *mathematics* — the radial/along/cross orbit projection, the clock and bias application, the iono model — is documented in Section 17 and is not repeated here. Two real-time-specific reactions to the stream are worth noting:

- an incremented IGS-SSR phase-bias *discontinuity counter* for a signal breaks the integer property of that signal's ambiguity, so the engine resets the affected ambiguity before the next update (`StateManager::noteSsrDiscontinuity`);
- every SSR push re-attempts a held observation epoch (Section 18.7), since the freshest correction horizon has just advanced.

Station coordinates (1005/1006). `decodeStationCoords` reads the antenna reference point (ARP) as three signed 38-bit ECEF components in 0.1 mm units; 1006 additionally carries a 16-bit antenna height. For a reference station that streams its own coordinates this provides an a-priori seed (Section 18.9).

18.6 Time-system handling

Streaming messages carry truncated, system-specific epoch fields that must be promoted to a common GPS-time `GnssTime`. Each parser instance keeps a continuous-GPS-seconds tracker of the last decoded stream epoch and promotes every mod-week / time-of-day field by *nearest continuation* to that tracker — the week (or day) minimising $|\Delta t|$, RTKLIB's `adjweek` semantics (`RtcmParser::anchorGpsSeconds`). The host clock (`TimeUtils::nowContinuousGpsSeconds`, POSIX UTC plus the leap offset) is consulted *only* at cold start, on the first time-bearing frame after construction, so a skewed host clock or a promotion race at a week rollover cannot re-week an established stream. Each successful MSM/SSR epoch advances the tracker (`updateTimeAnchor`); the truncated broadcast-nav weeks only read it. The anchor below is this tracker's current value.

- **Flat ms-of-week systems** (GPS/GAL/QZS/SBAS/BDS MSM): the 30-bit epoch is milliseconds-of-week; BeiDou adds the constant 14 s BDT→GPS offset (with a week-rollover guard).
- **GLONASS** carries its 30-bit MSM epoch as a 3-bit day-of-week plus a 27-bit time-of-day in ms — not 27 bits total; reading it short misaligns the satellite mask and corrupts the whole GLONASS decode. The ms-of-day is promoted by `TimeUtils::glonassTodToGpsTime`, which carries the reported time-of-day through the GLONASS = UTC(SU)+3 h ("Moscow") day frame and back to GPS time, anchoring on the stream-time tracker and re-adding leap seconds:

$$t_{\mathrm{GPS}} = \left(t^{0}_{\mathrm{day}} + \tilde{t}_{\mathrm{TOD}} \right) - \Delta_{\mathrm{Moscow}} + \Delta_{\mathrm{leap}}, \tag{267}$$

  where t^{0}_{day} is the Moscow-day midnight nearest the anchor and $\tilde{t}_{\mathrm{TOD}}$ is the reported time-of-day placed into that day by resolving the midnight wrap toward the nearest match (± 12 h). The redundant 3-bit day-of-week is left unread. The same conversion promotes the 1020 ephemeris `toe` and any GLONASS SSR epoch, so observations, orbits and corrections share one clock.
- **GLONASS FDMA wavelength.** GLONASS L1/L2 are frequency-division: each slot transmits on $f = f_{\mathrm{base}} + k\,\Delta f$ with channel $k \in [-7, +6]$, so the carrier wavelength is per-satellite. The channel is decoded from DF419 as $k = v - 7$ for the valid range and used in Equation (266); it is also carried on the observation so the slot map and MW combinations can resolve frequency. When DF419 is absent (MSM4/MSM6) or reserved the phase is carried in metres rather than fabricating a bogus $k = -7$.

Each decoded record stamps its own content epoch (`epoch_time`), which is what the synchronisation logic of Section 18.7 compares against, rather than wall-clock arrival.

18.7 Observation/SSR synchronisation

PPP-RTK needs observations time-*aligned* with the corrections, not pinned to wall-clock. In a live session the obs stream commonly runs ahead of a lagging SSR stream; processing an obs epoch that no satellite's SSR can cover would drive the whole epoch broadcast-grade (and, with `drop_broadcast_grade`, wipe it). The opt-in `processing.enable_obs_ssr_sync` hold buffer guards against this. Let t_{obs} be the obs epoch and define the SSR coverage horizon from the freshest stored SSR content epoch (`SsrCorrectionStore::newestSsrEpoch`) plus the maximum SSR age:

$$t_{\mathrm{horizon}} = t^{\mathrm{newest}}_{\mathrm{SSR}} + \tau_{\max}, \qquad \tau_{\max} = \texttt{ssr_max_age_s}. \tag{268}$$

The decision is:

- $t_{\mathrm{obs}} \le t_{\mathrm{horizon}}$: SSR covers the epoch — process normally and clear any hold;

- $t_{\text{obs}} > t_{\text{horizon}}$: *defer* the epoch. The held buffer keeps the latest such epoch only (skip-to-latest semantics), and `drainHeldObs` re-attempts it the moment an SSR push advances the horizon.

A liveness backstop prevents a dead SSR feed from starving the filter forever: if a hold persists past `obs_ssr_sync_max_hold_s` the engine latches a degraded "outage" and passes every epoch through (broadcast-grade / dropped via the existing path) until SSR re-covers the obs. The held epoch is drained with skip-to-latest *bypassed* so the intended SSR-aligned target is not discarded, and the final `flush` bypasses both the hold and skip-to-latest so the last epoch is never lost. The whole feature is inert when disabled or in any non-live-SSR run.

A separate, coarser `enable_obs_skip_to_latest` drops obs epochs whose lag behind *wall-clock* GPS time exceeds a bound, so a backlogged stream fast-forwards to the freshest epoch; the skip returns before any continuity state is touched, so the resume correctly trips the gap path (Section 18.10).

18.8 Ephemeris management

`LiveEphemerisStore` retains, per satellite, the current record and one previous record, and resolves the orbit/clock against whichever is appropriate. Two policies matter in real time.

- **Freshest-toe selection.** An incoming record is rejected when its `toe` (expressed in GPS-continuous seconds, `toeGpsContinuous`, so BDT/GLONASS scales are reconciled) is older than the held record beyond a sub-second epsilon, so a stale or re-ordered broadcast never overwrites a newer one. An authoritative IGS-stream record is additionally protected from being clobbered by a receiver-decoded record at equal `toe`: receiver nav is accepted only as a genuine update or as a fallback once the IGS stream has stalled past a threshold (the GLONASS nominal `toe` step).
- **toe-aware / IOD-aware pivoting.** For orbit and clock evaluation the store picks the record whose `toe` is nearest the query epoch (`pick`); for an SSR-corrected satellite it first re-references to the record the SSR delta was computed for (`pickForSsr`: match the SSR IODE against the current or retained-previous record). Across an ephemeris cutover the receiver can be one IOD ahead of the SSR stream; without this re-reference the IOD mismatch would needlessly drop the satellite to broadcast-grade.

18.9 A-priori seeding from RTCM 1005/1006

A *live* SBF/RTCM observation stream carries no RINEX `APPROX POSITION` record (a file-fed source does; see the third bullet below). Without a finite a-priori the first `computeModels` rejects every satellite on a position-sanity check (almost all sit below the elevation mask when the seed is the default Earth-centred fallback at the geographic pole), so WLS never runs and the filter cannot bootstrap. The engine therefore seeds the filter a-priori from the stream itself until the Kalman filter takes ownership of the position:

- from a *PVTGeodetic* solution in an SBF stream (`feedSbfFrame`); and
- from an RTCM *1005/1006* station ARP (`feedRtcmFrame`), the RTCM analogue of the PVT seed, which makes `receiver.reference_position` optional for any station streaming 1005/1006; and
- from the RINEX header of a *file-fed* source, reduced from the marker to the antenna reference point through `ANTENNA:DELTA H/E/N` (`RinexParser::getArpPositionXyz`, over `Coordinates::markerToArp`). The ARP, not the monument, is the frame the position state and the ATX PCO/PCV applied to it live in, so seeding on `APPROX POSITION XYZ` verbatim would start the filter one eccentricity out of frame. The reduction is an exact no-op on a zero or absent

eccentricity, so a live SBF/RTCM stream is unaffected; it is observable only for the file-fed closed-loop consumer, whose header carries a real eccentricity. There is no double application — `PppModel::initializeState` reduces from the *raw* marker, and only while the state position is still zero or non-finite.

A configured `reference_position` always wins. The seed is applied only while `first_epoch_` holds; the startup-defer gate keeps that flag set through warmup, so a 1005/1006 message arriving a few seconds into the stream (these come less often than the 1 Hz obs) still seeds before the first estimation epoch. Once the first epoch is fully processed the flag clears and the filter owns the position — the ARP / PVT is thereafter only logged for comparison, not re-applied.

18.10 The per-epoch real-time update

`processEpoch` runs the same predict/update pipeline as post-processing, with streaming-specific guards layered on the front and the post-processing-only stages removed. In order:

1. **Lag / sync / duration guards.** The wall-clock backlog warning, the obs↔SSR hold of Section 18.7, the skip-to-latest drop, and the `max_duration_s` latch all run before any continuity state is touched.
2. **Step / gap detection.** $\Delta t = t_k - t_{k-1}$ (with a week-rollover correction). A non-advancing $\Delta t \leq 0$ is a duplicate (a co-timestamped second SBF block or a retransmit) and is skipped cleanly — folding it in would double-count the measurement and feed $\Delta t = 0$ into the process-noise build, which hard-throws. A Δt beyond the gap tolerance flags a gap.
3. **Model + startup gate.** Observations are filtered to the active systems and run through `computeModels`. When `defer_until_corrections_ready` is armed, estimation is deferred (no predict/update) until a per-constellation cm-grade quorum of broadcast+SSR satellites is present (or a timeout forces a degraded start), so the filter seeds code and phase together on a clean state instead of cold-starting on metre-grade broadcast clocks.
4. **Clock / gauge seeding.** In modes with a receiver-clock state a WLS clock estimate re-seeds the filter clock on a jump, gap, or first epoch. Under BSSD (no receiver-clock state) a single per-epoch median-of-code-innovations receiver clock, common to all satellites, is subtracted from every code and phase observable — an exactly SD-invariant re-gauge that keeps the per-satellite bias states at DCB scale rather than absorbing the full unsteered receiver clock.
5. **Predict.** `predictAided` over $(t_{k-1}, t_k]$ — identical to the shared aiding path; for a GNSS-only stream it delegates byte-identically to `predict`(Δt) of Section 11.
6. **Cycle-slip detection / repair.** The MW detector plus the optional TurboEdit MW+GF joint repair and soft-LLI gap bridge run pre-update, sharing the same detectors and repair machinery as Section 14. These are enabled under `estimation.cycle_slip`; TurboEdit is high-rate-only and seeds its GF-arc gap tolerance from the configured stream rate (a live stream has no RINEX interval header).
7. **Measurement update.** The same outlier / forced-pivot-handover rebuild loop as the post-processing driver (shared via `EpochUpdateSupport`): rebuild on `NeedsMeasurementRebuild`, forward each constellation's forced pivot handover, re-inflate reset-satellite states, clear MW/TurboEdit arc history for KF-reset signals, and peel one outlier per pass. An optional soft a-priori ZWD pseudo-observation anchors the BSSD [up, ZWD] near-null subspace.
8. **Ambiguity resolution.** After a successful float update, MLAMBDA AR runs on the committed state with the same configuration surface as post-processing, fed the MW-resolved wide-lane integers (Section 15).

9. **State extraction + trace.** The posterior is synced back to the PPP state and trace rows are written.

Post-processing stages skipped in real time. The non-causal and batch stages have no place in a streaming, look-ahead-free pipeline and are deliberately absent:

- the backward RTS Section 11 is force-disabled under `AppMode::RealTime` — smoothing requires the full forward trajectory and cannot run causally;
- geometry relinearisation (the IEKF second pass) is a post-processing-only refinement and is not replicated here;
- precise SP3/CLK products are not loaded when a live broadcast+SSR source supplants them (a precise-product *file* replay still loads them);
- higher-order ionosphere is unsupported in the streaming pipeline (no GIM source) and is warned-and-skipped if enabled.

Part IX
Network mode

19 Network mode

This section documents the multi-station network estimator (`ProcessingMode::PPP_RTK_Network`) that *produces* the satellite-side corrections the single-receiver user modes consume. Where every other mode in this reference *differences away* the satellite clock and hardware biases (Section 5) or imports them as known SSR quantities (Section 17), the network producer turns that relationship around: it estimates the satellite clock and the satellite *phase* and *code* biases *explicitly*, jointly across a network of stations whose coordinates are known, and emits them in IGS-SSR convention for a downstream consumer. The closed loop is deliberate — the same single-receiver PPP-RTK engine documented elsewhere becomes the product test, run on a station *held out* of the network solve.

Scope and status. The numerical core — the arrowhead normal-equation kernel (`filter/NetworkNormalEquations`) and the semantic estimator that drives it (`filter/NetworkEstimator`) — is implemented and validated on a synthetic noise-free network (truth recovery up to the declared S-transform). The full pipeline is live: the `PPP_RTK_Network` enum, its forced-uncombined `DifferencingMode::None` (`measurement/MeasurementBuilder`), the explicit satellite phase-bias state in `filter/StateManager`, and the mode dispatch in `Application::run` all exist. `Application::runNetwork` is no longer the estimator: it is a thin epoch-loop orchestrator that calls a fixed ordered sequence of single-concern seams (Section 19.1), and every step this chapter derives — per-station design-row assembly from each station's `MeasurementModel` (`networkPassAHead_` / `networkPassB_`), the S-basis datum with its null-space rank gate (Section 19.4, in `networkSolveGlue`), the between-epoch time update, and the product emitter (`NetworkProductEmitter`, driven from `networkEmitStore`) — runs inside one of them. The mathematics below describes the implemented estimator core and the locked design.

This chapter treats the single-epoch solve. The between-epoch carry is layered, and the layers differ in status. The base *diagonal* D2 shared carry — each shared parameter's posterior value

and variance re-injected as a diagonal prior on the next epoch's shared border, inflated by $q\,\Delta t$ and dropped on a datum change (`networkInjectD2CarryPrior`, `app/NetworkRunContext`) — is *unconditional*: it carries no enable flag, so the shipped producer is a filtered, not a snapshot, solve. The upgrades that ride on top of it — the C1 full-covariance shared carry (`network.shared_carry_full_cov`, folded through `addSharedPriorDense`), the Card-4 invariant DD-carry (`network.carry_invariant`, `setDdPriors`), the station-local phase-leveling / ZWD / receiver-code-bias carries folded through `addStationPrior`, and the MW-WL / EWL / lane-coordinate widelane layers — are each large default-off subsystems in their own right and are only referenced here, not derived.

19.1 Producer architecture: the decomposed epoch loop

The rest of this chapter is mathematics. This subsection is the map that says where each piece of it executes, because the answer is no longer "in one function".

Why the shape changed. The producer began as a single driver function that reached roughly 14,700 lines. At that size it could not be reasoned about as a whole, could not be exercised below the granularity of an entire run, and could not be reviewed: a reader looking for the datum build had no way to bound where it was *not*. It was therefore decomposed into single-concern *seams*, and the decomposition chose the strongest available safety property. Relocation is *verbatim* — bodies move, they are not rewritten — and each increment was gated on a matrix of golden runs required to be *byte-identical* to the pre-cut binary's output. That gate is what licenses the claim that the equations below still describe the code: the driver shrank by roughly a factor of eight, to under 1,800 lines of epoch-loop orchestration, without the numbers moving. The gate's coverage should be stated plainly, though: `docs/network_decomposition_plan.md` records per-increment byte gates through increment 7b only, and the later cuts (solve glue, carry install, DD-prior build, producer AR, MW/WL glue, re-level, teardown) landed with no recorded gate in that status section.

Four stages. A network run decomposes into four stages — three translation units and the seam sequence between them.

- **Setup** (`app/ApplicationNetworkSetup`). `networkValidateInputs_` rejects an unrunnable configuration before anything is allocated; `networkSetup_` then builds the station roster and its synchroniser, the product sink and its declared S-basis contract, and — when the closed loop is armed — the consumer engine (Section 19.7). Its output is one `NetworkSetupResult`, filled in place and deliberately neither copyable nor movable: three run-long references pin its member addresses, so the member declaration order in that header is normative rather than cosmetic.
- **Assembly** (`app/ApplicationNetworkAssembly`), two passes per epoch. PASS A is *layout*: it runs each present station's geometry models and pre-allocates the shared satellite states, taking the datum (pivot) station *first* so that the phase-bias columns admitted to the shared block are exactly those backed by a pivot-receiver ambiguity pin — the local half of the datum (Section 19.4) constrains the column layout, not merely the solve. PASS B is *assembly*: each station's undifferenced design block is computed with no receiver-side differencing and then *split*, the satellite clock and phase-bias columns scattering into the shared border $\boldsymbol{A}_{s,r}$ and the remainder becoming the private block $\boldsymbol{A}_{r,r}$ of Equation (269); the three receiver-position columns are dropped, because network station coordinates are held fixed. Equation (272) is formed from exactly these two pieces.
- **The epoch seams** — the ordered sequence tabulated below, which *is* the body of the loop.

- **Teardown** (`filter/NetworkTeardown`). The run-end readout, the service counters and the summary writers live in `networkTeardown`, which also returns the process exit code; the driver's tail is literally a `return` of that call.

The seam contract. A seam is a free function over the run context, the configuration, the logger and the trace bundle, opening with an alias preamble that reconstructs the driver's former local names so the relocated body reads as it did before the cut. The contract each seam header states is that it *owns no state*: every carried object is a reference into the context. Two structural adaptations to pure relocation are worth recognising when reading the code. A relocated block that used to `continue` the epoch loop cannot, so it returns an `EpochAction` — `Proceed`, `SkipEpoch` or `StopRun` (`filter/NetworkRunTypes`) — which keeps the per-skip bookkeeping verbatim inside the seam while the caller's `continue` preserves the original timing-bracket bypass. And a block whose locals were read further down the loop returns a result struct (`NetworkSolveGlueResult` and its siblings) that the call site re-aliases.

Why there is a context object. Carried state, configuration snapshots and counters live in one place: `NetworkRunContext` (`app/NetworkRunContext`), holding a `const NetworkKnobs` snapshot of the configuration — taken in the original getter read order — the cross-epoch `NetworkCarryState`, and `NetworkServiceCounters`. The rule is that a seam introduces no function-local carry: a relocated block that leaves a local behind silently *forks* the carry, and one decomposition pass was spent deleting the dead aliases that pattern produced. Persistence carries its own discipline, the *three-place rule*: a carry field that is a snapshot member must appear in the warm-resume fingerprint, in the restore, and in the snapshot write, all three in `NetworkRunContext`. A field present in only two of the three yields a warm resume that silently disagrees with the run that wrote it.

The ordered epoch seams. The table below lists the per-epoch call sequence in driver order. Each call is bracketed by the driver's section timers, and rows marked *default off* are inert unless explicitly armed — a point worth keeping in view, since most of the newer machinery in this producer is in that state.

seam call (driver order)	concern (translation unit)
networkFastClockTick	sub-epoch fast-clock tick between solution epochs; *default off* (filter/NetworkFastClock)
networkPassAHead_	PASS A head: per-station geometry and shared-state pre-allocation, datum station first (app/ApplicationNetworkAssembly)
networkPassASlipScan_	PASS A tail and the slip scans that fill the slip funnel (app/ApplicationNetworkAssembly, slip/NetworkSlipQc)
networkAssemblyPrefilter_	raw slip observables; cross-station observability prune (app/ApplicationNetworkAssembly)
networkBuildPhaseDatumCandidates then networkElectPhaseAnchors / networkElectEnsembleDatum / networkElectChainDatum	election of the *local* half of the datum: the candidate census and coverage gate, then whichever realisation is armed (filter/NetworkPhaseDatum, filter/NetworkChainDatum)
networkAssemblyRegisterColumns_	clear the layout, register this epoch's shared columns, open the arrowhead system (app/ApplicationNetworkAssembly)
networkSdEwlAr	single-difference extra-widelane integer decision; *default off* (filter/NetworkSdEwlAr)
networkPassB_	PASS B produce/consume: the split design blocks handed to addStation (app/ApplicationNetworkAssembly, filter/NetworkCoupling)
networkBuildEnsembleGauges, networkBuildCarryDdPriors	sum-to-zero phase gauges; Card-4 DD carry priors (filter/NetworkDdPriorBuild)
networkSolveGlue	shared datum build, rank gate and drop-and-heal, D2 carry injection, the constrained solve, within-epoch RAIM re-solve, withhold gates (filter/NetworkSolveGlue, calling NetworkSbasis, NetworkPhaseDatum, NetworkRelevel)
networkInstallCarry	DD posterior read-back into the invariant carry; *default off* (filter/NetworkInstallCarry)
networkProducerAr	producer-side AR diagnostic and normal-equation export; read-only and log-only, *default off* (filter/NetworkProducerAr)
networkSolPrep	map to the solution epoch; per-satellite discontinuity signalling (filter/NetworkSolPrep)
networkEmitStore	emit re-base, phase-bias continuity filter, store push, product write, closed-loop consume (filter/NetworkEmitStore)
networkFastClockHarvest	fast-clock harvest off the accepted solution; *default off* (filter/NetworkFastClock)
networkIonoDiag, networkDdFrac, networkShDevVtec, networkShHealth, networkLeveling	diagnostic sweeps plus the SH-deviation and phase-leveling carry harvests (filter/NetworkIonoDiag, NetworkDdFrac, NetworkShDevVtec, NetworkShHealth, NetworkLeveling)
networkPostFitRaim	post-fit network RAIM and its attribution pass (filter/NetworkRaim)
networkClockQc	two-state clock update and clock fault QC (filter/NetworkClockQc)
networkPostfitSigma0, networkShSpectrum	post-fit variance factor and reduced-normal spectrum; *default off* (filter/NetworkFrameDiag)
networkStashStatus	carry re-stash and the epoch-status record (filter/NetworkStashStatus)
networkWriteStateSnapshot	warm-resume state snapshot (app/NetworkRunContext)

Reading key. The three derivations that follow land in three places in that sequence. The rank deficiency and its minimum-constraint datum (Section 19.4) are built and gated inside networkSolveGlue. The arrowhead Schur reduction (Section 19.5) happens in the kernel that the same seam invokes, on normals accumulated station-by-station during PASS B. The closed loop (Section 19.7) — emission, product write and, when armed, the held-out consumer step — is networkEmitStore. Everything else in the table is quality control, diagnostics or carry bookkeeping wrapped around those three.

19.2 Joint multi-station observation model

The producer runs on the same *undifferenced, uncombined* observations as single-receiver PPP (Section 4), but stacks the equations of every station $r \in \mathcal{R}$ into one system. The defining structural fact is the partition of the unknowns into a block *shared* by all stations and a block *private* to each:

- **Shared (satellite-side).** The estimable satellite clock $c\,\mathrm{d}t^i$, the per-signal satellite code bias B^i_s, and the per-signal satellite *phase* bias $\bar{b}^i_s$. These columns are *identical* across every station — a satellite's clock and hardware delays do not depend on who is observing it. (Orbits are held fixed to a precise product here; orbit determination is out of scope for the producer.) When `network.estimate_iono_sh` is set, the shared block carries a fourth family: the coefficients a_{nm}, b_{nm} of a global single-layer spherical-harmonic VTEC expansion (`SharedParamKind::VtecSh`, registered through `NetworkEstimator::registerVtecCoeff`) — optionally a second, plasmasphere-side shell as well (`estimation.global_vtec_sh.layer2_max_degree`, default 0, i.e. single-layer; both shells' coefficients coexist on the shared border and the pin row sums them) — network-wide rather than per-satellite. This co-estimated ionosphere model replaces the external GIM and forces satellite code-bias estimation on — the two are jointly load-bearing for the per-satellite slant-iono columns below — and its columns Schur-reduce onto the shared border exactly like the satellite-indexed families.
- **Private (station-side).** For each station r: its receiver clock $c\,\mathrm{d}t_r$, the inter-system time offsets $c\,\delta t^{(g)}_r$, the zenith wet troposphere ZWD_r (and gradients), the per-satellite slant ionosphere $I^i_{r,1}$, the receiver biases, and one float ambiguity $N^i_{r,s}$ per tracked carrier — plus, only under the chain datum (`network.phase_datum_eq34`, default off), a per-(receiver, system, signal) receiver *phase*-bias split $\delta_{r,s}$ re-derived fresh each epoch (`StateKeys::makeReceiverPhaseBiasKey`), which is what stops the receiver's fractional phase bias lumping into the *emitted* satellite phase bias under that realisation. Station coordinates are *known* (the network constrains them), so position is not a free parameter.

Write the shared satellite-parameter vector as $\boldsymbol{x}_s$ and station r's private nuisance vector as $\boldsymbol{x}_r$. The undifferenced observation rows of station r linearise (Section 4) to

$$\boldsymbol{y}_r \;=\; \boldsymbol{A}_{s,r}\,\boldsymbol{x}_s \;+\; \boldsymbol{A}_{r,r}\,\boldsymbol{x}_r \;+\; \boldsymbol{\varepsilon}_r, \qquad r \in \mathcal{R}, \tag{269}$$

where $\boldsymbol{y}_r$ is the prefit residual (observed minus computed), $\boldsymbol{A}_{s,r}$ is the Jacobian with respect to the *shared* satellite columns, and $\boldsymbol{A}_{r,r}$ is the Jacobian with respect to station r's *own* nuisances. Crucially there is no $\boldsymbol{A}_{r,r'}$ term for $r' \neq r$: a station's observations touch only the shared block and its own private block. That sparsity is the structural engine of Section 19.5.

No receiver-side differencing. Unlike the PPP-RTK user mode, the producer applies *no* between-satellite or between-receiver differencing. The differencing strategy is forced to `DifferencingMode::None` (`MeasurementBuilder.cpp`), overriding any `between_satellite_sd` toggle that may be present in a network config. Differencing would cancel exactly the satellite-side terms the producer is trying to *estimate*; the network instead carries them as explicit columns and resolves the resulting rank deficiency with a minimum-constraint datum (Section 19.4).

19.3 The explicit satellite phase-bias state

The single defining contrast with the user side is the treatment of the satellite phase bias b^i_s (here written $\bar{b}^i_s$ once promoted to a state). On the user side this term is never an explicit unknown:

between-satellite single differencing (Section 5) cancels the *receiver* phase bias, the satellite phase bias arrives as a *known* SSR product, and what remains is an integer single-difference ambiguity. The user handles biases *implicitly*.

The producer is what *generates* that known product, so it must carry the satellite phase bias as a genuine estimated state. The state vector therefore gains a per-(satellite, signal) phase bias column,

$$\bar{b}_s^i \in \boldsymbol{x}_s, \qquad \text{keyed by } \texttt{makeSatPhaseBiasKey}(i, s) = \texttt{"}i\texttt{|}s\texttt{|PB"}, \tag{270}$$

held in `StateManager::sat_phase_bias_indices_`, a map kept *deliberately disjoint* from the satellite *code* bias map (`sat_bias_indices_`); the trailing `|PB` discriminator keeps the two namespaces from colliding (`include/filter/StateKeySchema.hpp`). Its process noise is a tiny random walk (`network.process_noise.sat_phase_bias`, legacy `filter.process_noise.sat_phase_bias_q`; default two orders of magnitude below the code-bias density), reflecting that satellite phase hardware delays are nearly constant but not strictly so: a $q = 0$ random walk would over-converge the diagonal of $\boldsymbol{P}$ and leave the column deaf to genuine slow drift (`src/core/filter/StateManager.cpp`).

Remark 3. The estimated $\bar{b}_s^i$ is not the "raw" satellite phase hardware delay — that quantity is not separately estimable. It is the delay *lumped with the network pivot-receiver ambiguity/clock datum* (Odijk's δ^s, [17]): the estimable combination that, when removed by a user receiver, restores the integer property of that user's single-difference ambiguities. This lumping is exactly why a *consistent* phase-bias product matters (Section 17): the producer's datum and the consumer's integer recovery must be the same S-system.

19.4 Rank deficiency and the minimum-constraint S-basis

The undifferenced network system Equation (269) is rank deficient. The same linear dependencies that the user side disposes of by differencing are present here as exact *gauge freedoms* of the joint normal matrix: there is no observation that distinguishes a common shift of all satellite clocks from an opposite shift of all receiver clocks, nor the split of an absolute phase bias between the satellite column and the per-station ambiguity datum, nor an inter-system bias (ISB) common to a constellation. These are precisely the S-system ambiguities of [25]: the data fix only *estimable combinations*, and a *datum* must be chosen to pin the remaining null space.

Exact gauge, soft elimination. The chosen resolution is a *minimum-constraint* datum applied as exact hard pins to the reduced shared system, never a hard structural elimination of columns. Concretely, each gauge is realised as a single *Dirichlet pin* of one shared column c to a datum value (almost always 0): the column's known contribution is moved to the right-hand side, its row and column in the reduced normal matrix are zeroed, and a diagonal entry is set so that $x_c =$ value holds exactly (`NetworkEstimator::solve`, `src/core/filter/NetworkEstimator.cpp`). The pin is applied to a *copy* of the reduced normals (the kernel's `solveSharedConstrained` seam), leaving the estimator's own reduced system pristine so per-station back-substitution (Section 19.5) stays valid. The pinned diagonal is set commensurate with the reduced normals' own scale rather than to unity, so that a genuinely *under*-specified datum still surfaces as a near-zero *relative* pivot in the positive-definiteness guard, independent of the network's overall weighting magnitude.

In S-transform language, if $\boldsymbol{x}$ collects the rank-deficient undifferenced parameters and $\boldsymbol{S}$ is the S-basis selection, the emitted satellite corrections are the estimable combination $\boldsymbol{S}\,\boldsymbol{x}$; choosing the datum *is* choosing $\boldsymbol{S}$, and the hard pins above are the minimal set of rows of $\boldsymbol{S}$ that remove the deficiency. The two halves of the datum live in two places:

- **Shared-block gauges** — the overall clock datum (one per constellation), the per-constellation phase-bias↔ambiguity datum, and the ISB datum — are the elements that touch the shared satellite columns. These go through the explicit `NetworkEstimator::Datum` (a list of column pins) and are applied in `solve`.
- **Local datum elements** — a network *pivot receiver*'s own ambiguity/clock pin, which the emitted satellite phase bias absorbs (Odijk Eq. 4/9) — live on that one station's private block. They are realised by the *caller* as tight prior rows folded into that station's $\boldsymbol{A}_{r,r}$, $\boldsymbol{W}$, and $\boldsymbol{y}$, not through the shared `Datum` (`NetworkEstimator.hpp`). Station priors are also what make each local block $\boldsymbol{N}_{rr}$ full rank for the Schur step (Section 19.5).

Selectable datum realisations. The pins just described are the *legacy default*; each half of the datum now admits alternatives, selected by config and recorded in the `SbasisDeclaration`, all producing the same estimable $\boldsymbol{S}\,\boldsymbol{x}$ but distributing the gauge differently. The clock/ISB datum offers three modes (`SbasisDeclaration::IsbDatum`): the hard `PerConstellationSatClockPin` above (default), a `SumToZeroPerConstellation` constraint (`network.clock_datum_sum_to_zero`) that spreads the pin across all satellites of a constellation, and a `BroadcastAnchoredPerSatellite` soft anchor (`network.clock_datum_broadcast_anchored`) tying each satellite clock to its broadcast value at a declared 1σ. The ambiguity↔phase-bias datum likewise offers three (`AmbDatum`): the local `PivotReceiverZeroAmbiguity` pin above (default) — whose anchor may optionally be elected *per* (satellite, signal) at the highest-coverage tracking receiver rather than at the single clock pivot (`network.phase_datum_prioritized`, default off; `networkElectPhaseAnchors`, `filter/NetworkPhaseDatum`), which is the same one-pin-per-freedom constraint realised at a different station, so that a satellite low or unseen at the clock pivot still keeps a clean gauge — a `SumToZeroPerReceiver` ensemble (`network.phase_datum_ensemble`), and a `SingleReceiverChain` realisation (`network.phase_datum_eq34`). When satellite code biases are estimated (the W6 slice), their own gauge — a per-constellation sum-to-zero (`CodeBiasDatum::SumToZeroPerConstellation`) — is folded as explicit constraint rows through `NetworkEstimator::addSharedConstraintRows`, not as column pins. Whichever realisations are active, the rank gate below is declared against the matching S-basis dimension.

Remark 4. A rigorous null-space rank gate — eigen-decompose the pre-constraint reduced normals and assert that the count of small eigenvalues equals the *declared* S-basis dimension — is implemented as `NetworkSbasis::assertNullSpaceRank` and is invoked on `reducedSharedNormals()` *before* the constrained solve, against the `SbasisDeclaration`'s `declared_dimension` (the solve seam `networkSolveGlue`, `filter/NetworkSolveGlue`, which also builds the datum via `NetworkSbasis::buildSharedDatum` — see Section 19.1). It complements the positive-definiteness guard: the guard catches the exact-deficiency (under-specified datum) case on the pinned pivot, while the rank gate certifies the reduced deficiency against the declared gauge dimension and withholds any epoch whose null-space count disagrees. Under the prioritized phase datum the gate first attempts a *drop-and-heal* pass — soft-anchoring exactly the near-null phase-bias columns (`healNearNullPhaseBias`, `filter/NetworkPhaseDatum`) — and re-gates once; on the legacy pivot datum there is nothing to heal, the heal returns zero, and the epoch is withheld immediately.

19.5 Per-station Schur reduction (arrowhead normals)

Stacking Equation (269) over all stations and forming weighted normal equations $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{A}\,\boldsymbol{x} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{y}$ yields a *bordered block-arrow* ("arrowhead") system, because the station–station off-diagonal

blocks are exactly zero (stations couple only through $\boldsymbol{x}_s$):

$$\begin{bmatrix} \boldsymbol{N}_{ss} & \boldsymbol{N}_{s1} & \boldsymbol{N}_{s2} & \cdots & \boldsymbol{N}_{sR} \\ \boldsymbol{N}_{1s} & \boldsymbol{N}_{11} & \boldsymbol{0} & \cdots & \boldsymbol{0} \\ \boldsymbol{N}_{2s} & \boldsymbol{0} & \boldsymbol{N}_{22} & \cdots & \boldsymbol{0} \\ \vdots & & & \ddots & \\ \boldsymbol{N}_{Rs} & \boldsymbol{0} & \boldsymbol{0} & \cdots & \boldsymbol{N}_{RR} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_s \\ \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_R \end{bmatrix} = \begin{bmatrix} \boldsymbol{u}_s \\ \boldsymbol{u}_1 \\ \boldsymbol{u}_2 \\ \vdots \\ \boldsymbol{u}_R \end{bmatrix}, \tag{271}$$

with the per-station blocks built from that station's pre-formed design rows,

$$\boldsymbol{N}_{ss}^r = \boldsymbol{A}_{s,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{A}_{s,r}, \quad \boldsymbol{N}_{sr}^r = \boldsymbol{A}_{s,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{A}_{r,r}, \quad \boldsymbol{N}_{rr}^r = \boldsymbol{A}_{r,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{A}_{r,r}, \tag{272}$$

and $\boldsymbol{u}_s^r = \boldsymbol{A}_{s,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{y}_r$, $\boldsymbol{u}_r^r = \boldsymbol{A}_{r,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{y}_r$. A full (non-diagonal) $\boldsymbol{W}_r$ is supported, so correlated undifferenced measurement covariance (e.g. shared pre-corrections) propagates exactly (`addStationObservations`, `NetworkNormalEquations.hpp`).

Lossless marginalisation. The arrowhead structure is solved by Schur-eliminating each station's private block onto the shared border. Each $\boldsymbol{N}_{rr}^r$ is factorised once ($\mathrm{LDL}^{\mathrm{T}}$) when the station is added; the elimination accumulates

$$\tilde{\boldsymbol{N}}_{ss} = \sum_{r \in \mathcal{R}} \left(\boldsymbol{N}_{ss}^r - \boldsymbol{N}_{sr}^r \left(\boldsymbol{N}_{rr}^r\right)^{-1} \boldsymbol{N}_{sr}^{r\mathrm{T}} \right) + \boldsymbol{N}_{ss}^{\mathrm{prior}}, \tag{273}$$

$$\tilde{\boldsymbol{u}}_s = \sum_{r \in \mathcal{R}} \left(\boldsymbol{u}_s^r - \boldsymbol{N}_{sr}^r \left(\boldsymbol{N}_{rr}^r\right)^{-1} \boldsymbol{u}_r^r \right) + \boldsymbol{u}_s^{\mathrm{prior}}, \tag{274}$$

the reduced shared system $\tilde{\boldsymbol{N}}_{ss}\, \boldsymbol{x}_s = \tilde{\boldsymbol{u}}_s$ (`NetworkNormalEquations::reduce`). The reduction is a *lossless* marginalisation, not an approximation: solving Equations (273) and (274) for $\boldsymbol{x}_s$ and back-substituting per station,

$$\boldsymbol{x}_r = \left(\boldsymbol{N}_{rr}^r\right)^{-1} \left(\boldsymbol{u}_r^r - \boldsymbol{N}_{sr}^{r\mathrm{T}}\, \boldsymbol{x}_s \right), \tag{275}$$

reproduces the dense stacked solve to machine precision. Fill-in is confined to the satellite border and each station's elimination is independent, so the cost scales linearly in the station count and the per-station eliminations parallelise (`reconstructStation`, `NetworkNormalEquations.cpp`). The shared-block priors $\boldsymbol{N}_{ss}^{\mathrm{prior}}$ in Equation (273) — soft physical priors and any additive exact-gauge contributions — are added pre-reduction via `addSharedInfo` and carry through the Schur complement unchanged.

Remark 5. This arrowhead-with-exact-reduction topology mirrors the NGS M-PAGES `NgsLsq::NormalEquations::reduce` / `reconstructLocals` batch network solver, specialised so that the M-PAGES "global" block is the shared satellite border and the per-epoch swept block becomes the R independent station blocks. The design panel rejected a two-stage covariance-form alternative precisely because the normal-equation Schur form is exact and cheaper here.

19.6 Constellation scope

The producer estimates GPS and Galileo, with BeiDou supported on the same footing; **GLONASS phase biases are excluded**. The reason is structural, not a tuning choice: GLONASS is FDMA, so each satellite transmits on a slightly different frequency and the receiver inter-channel (hardware) bias does *not* cancel cleanly across satellites the way it does for the CDMA constellations. The satellite/receiver phase-bias bookkeeping that underpins the explicit-bias S-system

(Sections 19.3 and 19.4) assumes a per-signal bias common across a constellation's satellites; the per-frequency GLONASS inter-channel bias breaks that assumption and would corrupt the phase datum. GLONASS is therefore excluded from the network *phase*-bias product by design (`docs/ssystem_conformance_2026-06-15.md`); the user side excludes it from ambiguity resolution for the same reason. A *code-only* GLONASS mode is nonetheless supported behind `network.estimate_glonass` (`enableGlonassNetwork`, default off): it admits GLONASS to the solve carrying a satellite clock and a per-slot receiver inter-frequency bias but *no* FDMA phase bias, so the phase-datum bookkeeping above is untouched. The mode is mutually exclusive with the W6 satellite code-bias slice — GLONASS admission suppresses the GPS/Galileo receiver DCB that code-bias separation needs, so `network.estimate_sat_code_bias` together with `network.estimate_glonass` is a hard load error — and therefore also with the co-estimated SH-VTEC ionosphere, which forces the code-bias slice on. (This is distinct from the float-only GLONASS support the user engine carries for positioning; with the code-only mode off, the network *producer* generates no GLONASS bias products at all.)

19.7 The closed loop: producer → held-out user

The emitted product is one `NetworkSolutionEpoch` per content epoch: per-satellite clock C_0 plus per-signal code and phase biases, keyed by RINEX satellite id and serialised in IGS-SSR convention by `NetworkProductEmitter` (`NetworkEstimator::toSolutionEpoch`). That product is exactly the SSR stream the single-receiver engine already knows how to consume: the satellite clock refines $-c\,\mathrm{d}t^i$ and the observable-specific biases refine $-B^i_s$ and $-b^i_s$ as *known* corrections, restoring the integer ambiguity property exactly as described in Section 17.

The validation strategy follows directly from this symmetry. The companion `ProcessingMode::PPP_RTK_User` mode runs the ordinary uncombined PPP-RTK pipeline but consumes the network's own bias/atmosphere product. Holding one station *out* of the network solve and then positioning it as a user against the produced corrections turns the consumer into a closed-loop product test: the held-out station's recovered position and fixed ambiguities certify that the producer's S-system datum and the consumer's integer recovery are mutually consistent. The `PPP_RTK_User` consumer flows through the standard PPP path (it is not a separate estimator); only the producer takes the joint multi-station pipeline of this section.

That test exists in two mechanised forms. The *in-process* harness (`network.closed_loop.enable`) builds a consumer engine from a derived configuration during setup and hands it an *owned* SSR correction store rather than the process-wide one, so the producer's own products can never leak into a store some other component reads; the consumer is then stepped from the emit seam `networkEmitStore`, after that epoch's products have been pushed, which is what makes the loop closed *within* the epoch. Its consumer runs a between-satellite single-difference user model by default, or an undifferenced one under `closed_loop.consumer_undifferenced`. The *offline replay* driver (`network.closed_loop.replay_products_path` → `Application::runConsumerReplay`, `app/ApplicationConsumerReplay`) short-circuits `runNetwork` before any producer setup and fills the same owned store from an already-written product file through the same reader path, so the recorded stream's discontinuity bookkeeping is reconstructed exactly and a position-domain arm costs minutes rather than a producer run. One boundary is worth stating because it limits what a replay arm proves: the in-process SH-VTEC slant-ionosphere supply has no product-wire record, so `closed_loop.consume_network_iono` is inert on the replay path and the consumer floats its own ionosphere there.

20 Worked example: estimability in a two-receiver network

This section makes the rank deficiency and minimum-constraint datum of Section 19.4 concrete on the smallest network that still shows the structure: two receivers $\mathcal{R} = \{A, B\}$ and four satellites $i \in \{1, 2, 3, 4\}$. We strip everything that is not essential to the estimability argument — positions are known so the geometric range is removed, a single frequency is used, the troposphere and ionosphere are absent, and the data are noise-free — so that every number below is *exact* and the gauge freedoms are visible by inspection. The point is not the size of the system but its null space.

20.1 Code only: clocks and one gauge freedom

With geometry removed, the single-frequency pseudorange of Section 4 collapses to a difference of two clock lumps,

$$P_r^i \;=\; a_r \;-\; g^i, \qquad a_r := c\,\mathrm{d}t_r + B_r, \quad g^i := c\,\mathrm{d}t^i + B^i, \tag{276}$$

where a_r is the receiver clock and g^i the satellite clock. On a single frequency the receiver and satellite *code biases* B_r, B^i are inseparable from the respective clocks, so each "clock" here is already a clock + code-bias lump — the IF-vs-DCB split of Section 19.3 degenerates to nothing with one band. There are $|\mathcal{R}| + |\{i\}| = 2 + 4 = 6$ unknowns and $2 \times 4 = 8$ observations.

Take as ground truth (metres)

$$a_A = 5, \quad a_B = 3, \qquad g^1 = 2, \quad g^2 = -1, \quad g^3 = 4, \quad g^4 = 0.5,$$

which through Equation (276) produces the eight pseudoranges

P_r^i	$i=1$	$i=2$	$i=3$	$i=4$
$r=A$	3	6	1	4.5
$r=B$	1	4	−1	2.5

The deficiency, seen two ways. *(i) Invariance.* Add any constant κ to *all six* unknowns, $a_r \mapsto a_r + \kappa$ and $g^i \mapsto g^i + \kappa$; the difference $a_r - g^i$, and therefore every observation, is unchanged. The truth above and the shifted set $\{15, 13, 12, 9, 14, 10.5\}$ fit the data identically. Absolute clocks are unknowable; only differences are.

(ii) Singular normals. Ordering the unknowns $\boldsymbol{x} = [\,a_A, a_B, g^1, g^2, g^3, g^4\,]^{\mathrm{T}}$ and using unit weights, the normal matrix $\boldsymbol{N} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$ is

$$\boldsymbol{N} = \begin{bmatrix} 4 & 0 & -1 & -1 & -1 & -1 \\ 0 & 4 & -1 & -1 & -1 & -1 \\ -1 & -1 & 2 & 0 & 0 & 0 \\ -1 & -1 & 0 & 2 & 0 & 0 \\ -1 & -1 & 0 & 0 & 2 & 0 \\ -1 & -1 & 0 & 0 & 0 & 2 \end{bmatrix}, \qquad \boldsymbol{N}\,\mathbf{1} = \mathbf{0}, \tag{277}$$

which is 6×6 but has rank 5: the all-ones vector $\mathbf{1} = [1, 1, 1, 1, 1, 1]^{\mathrm{T}}$ is an exact null vector (every row sums to zero), and it is precisely the common-shift direction of view (i). The system $\boldsymbol{N}\boldsymbol{x} = \boldsymbol{u}$ cannot be solved without one added constraint — a *datum*.

Two datum choices, same fit. Pinning one clock removes the gauge. Two natural choices give *different* parameter values that fit the eight observations *equally and exactly*:

datum (S-basis)	a_A	a_B	g^1	g^2	g^3	g^4
$a_A \equiv 0$ (reference receiver)	0	−2	−3	−6	−1	−4.5
$g^1 \equiv 0$ (reference satellite)	3	1	0	−3	2	−1.5

Under $a_A \equiv 0$, every satellite-clock estimate comes out as $\hat{g}^i = g^i - a_A^{\text{true}}$: the reference receiver's clock is *absorbed* into all of them. The estimated "satellite clock" is not the physical clock; it is the physical clock plus the datum we chose — the *lumping* that the user must add back. This is exactly why a producer must *declare* its datum (Section 19.7).

What is invariant. The estimable quantities — combinations annihilated by the null vector $\mathbf{1}$, i.e. differences — are identical in both bases:

estimable quantity	$a_A \equiv 0$	$g^1 \equiv 0$	(truth)
$a_A - a_B$	2	2	2
$g^2 - g^1$	−3	−3	−3
$g^3 - g^1$	2	2	2
$a_B - g^4$	2.5	2.5	$(= P_B^4)$

Only the absolute labels are convention; the differences are physics. This is the S-system of [25] in six parameters.

Remark 6 (Why the producer cannot difference). Form the double difference of this code-only model, $\nabla\Delta P_{AB}^{i,1} = (P_A^1 - P_A^i) - (P_B^1 - P_B^i)$. The between-satellite step gives $g^i - g^1$ for *both* receivers, so the between-receiver step yields exactly $\mathbf{0}$. Double differencing annihilates the entire model, because DD is built to delete clocks and here clocks are all there is (Section 5). A DD user would estimate nothing; the network producer instead *keeps* the clocks and pins one datum. That degeneracy is the producer's reason for existence in miniature.

20.2 Adding carrier phase: the satellite phase bias needs the network

Now add the single-frequency carrier phase. With the same clock lumps it reads

$$\Phi_r^i \;=\; a_r - g^i \;+\; b^i \;+\; N_r^i, \tag{278}$$

where b^i is the satellite phase bias — the producer's target — and N_r^i is the float ambiguity (in metres). The receiver phase bias and the receiver code-minus-phase offset are constant per receiver and have been absorbed into N_r^i; we keep b^i explicit because it is what the producer must emit. Extend the truth with

$$b^1 = 0.10, \qquad b^2 = -0.20, \qquad b^3 = 0.05, \qquad b^4 = 0.15,$$
$$N_A^i : 10,\ -5,\ 20,\ 8, \qquad\qquad N_B^i : 12,\ -3,\ 18,\ 9,$$

giving the eight carrier-phase observations

Φ_r^i	$i=1$	$i=2$	$i=3$	$i=4$
$r = A$	13.10	0.80	21.05	12.65
$r = B$	13.10	0.80	17.05	11.65

Counting: one receiver vs. two. The clocks are fixed by the code, so per satellite the phase contributes $b^i + N_r^i$. With *one* receiver this is a single equation $b^i + N_A^i$ in two unknowns: the satellite phase bias has no separate existence at all — it is inseparable from that one receiver's ambiguity, and not even redundant. With *two* receivers there are two equations,

$$b^i + N_A^i, \qquad b^i + N_B^i, \tag{279}$$

in three unknowns (b^i, N_A^i, N_B^i): rank 2, one deficiency per satellite. The estimable two-dimensional space now contains the *between-receiver single-difference ambiguity* $\nabla N_{AB}^i = N_A^i - N_B^i$ — a quantity that simply does not exist with one receiver, and is exactly the integer a user resolves (Section 5). The second receiver is what gives b^i an identity shared across stations.

Datum and solution. The joint code + phase system has a 1-dimensional clock gauge (as before) plus one phase-bias↔ambiguity gauge per satellite: $b^i \mapsto b^i + c_i$, $N_r^i \mapsto N_r^i - c_i$ for all r. The minimum-constraint datum used here is the clock pin $a_A \equiv 0$ — S-transform-equivalent to the reference-*satellite* clock pin $g^1 \equiv 0$ that the estimator's S-basis actually declares (Section 19.4: the shared-block gauge pins a satellite clock per constellation, not a receiver clock) — plus the reference-receiver ambiguity pins $N_A^i \equiv 0$ for every i (the local datum on the pivot receiver, folded into station A's rows). Solving gives

	$i=1$	$i=2$	$i=3$	$i=4$
emitted phase bias $\bar{b}^i = b^i + N_A^i$	10.10	−5.20	20.05	8.15
$\hat{N}_A^i$ (datum)	0	0	0	0
$\hat{N}_B^i = N_B^i - N_A^i$	2	2	−2	1

(the clocks are unchanged from Section 20.1). One checks the fit directly, e.g. $\hat{\Phi}_B^3 = a_B - g^3 + \bar{b}^3 + \hat{N}_B^3 = -2 - (-1) + 20.05 + (-2) = 17.05$, matching the observation.

Reading the result. Two things are worth dwelling on.

- **The emitted bias is dominated by the lump.** The physical satellite phase bias of satellite 1 is $b^1 = 0.10$ m, but the *emitted* estimable bias is $\bar{b}^1 = 10.10$ m: it has absorbed receiver A's ambiguity (10 m) for that satellite. This is Odijk's $\bar{b}^i = b^i +$ (pivot-receiver ambiguity) (Section 19.3, Eq. 4/9) made arithmetic. A consumer recovers integers only by removing the *same* lumped quantity — producer datum and consumer integer recovery must be one S-system.
- **The network unlocks the integer.** The estimable $\hat{N}_B^i = N_B^i - N_A^i = \nabla N_{AB}^i$ is the between-receiver single-difference ambiguity, the very quantity an RTK user fixes to an integer. It is identically absent in a single-receiver problem; the second station is what brings it into existence. The satellite phase bias becomes a definite, transferable number for the price of pinning one reference ambiguity — and that pin is what it carries.

This four-satellite, two-receiver instance is, up to the geometry and the second frequency it suppresses, exactly the synthetic network the estimator core is validated on: the `RecoversSyntheticTruthUpToSTransform` test (`tests/test_network_estimator.cpp`) builds rows of exactly this shape — at four receivers and five satellites, from a random truth — pins the reference-satellite clock datum $g^1 \equiv 0$ (S-transform-equivalent to the $a_A \equiv 0$ used above) through `NetworkEstimator::solve`, and asserts recovery of the satellite-side parameters *up to the declared*

S-transform — i.e. recovery of the lumped $\bar{b}^i$ and the single-difference ambiguities above, not of the unobservable physical b^i.

Part X
Reference

21 Pipeline and architecture diagrams

This section sketches the control- and data-flow structures that the prose chapters describe algebraically: the per-epoch estimation loop (Figure 1), the shared geometry kernel behind the three differencing regimes (Figure 2), the real-time SSR correction pipeline (Figure 3), the real-time streaming front-end that feeds it (Figure 4), and the PPP-RTK network producer's joint estimator (Figure 5). Boxes name the function or class that realises each step, so the diagrams double as a map into `src/`.

A Generated reference tables

The tables in this appendix are generated from the source tree by `scripts/gen_report_tables.py`. They are not maintained by hand: a renamed flag, a new cycle-slip detector, or a changed state-key delimiter appears here on the next run of the generator. *Known gap:* the feature-flag table de-duplicates on the C++ member name rather than on the JSON path, so a section master switch named `enable` in more than one config sub-struct appears only once — several `network.*` masters, including those governing the default-off subsystems this appendix exists to disclose, are consequently absent. Treat the flag table as indicative and `GnssConfigLoad.cpp` / `GnssConfigLoadNetwork.cpp` as authoritative until `gen_report_tables.py` keys on the path. The fragments under `generated/` are `.gitignore`d and rebuilt on demand (`build.ps1 -Regen` / `build.sh --regen`); to see whether anything moved, re-run the generator and check `git status` (`--check` is not read-only — it rewrites the fragments before comparing them).

A.1 Processing modes

Table 1: `ProcessingMode` – the estimation algorithm selector.

Value	Meaning
`PPP`	Precise Point Positioning (single receiver, external products).
`PPP_RTK`	PPP with RTK enhancements (single receiver, uncombined).
`BaselineSingle`	Double-differenced single baseline: one rover + one reference station.
`BaselineMulti`	Double-differenced multi-receiver network (star topology: N rovers against one reference).
`PPP_RTK_Network`	Minimum-constraint S-system network PRODUCER: joint multi-station estimator emitting satellite clock + code/phase bias (+ atmosphere) under a declared S-basis.

Value	Meaning
PPP_RTK_User	PPP-RTK CONSUMER of network PPP_RTK_Network products (post-processing closed-loop counterpart): uncombined PPP-RTK + network SSR bias/atmosphere consume + S-basis cross-check.

A.2 Execution mode and data source

Table 2: AppMode – top-level post-processing vs. real-time dispatch.

Value	Meaning
PostProcessing	Batch processing from RINEX / product files.
RealTime	Streaming / live processing from RTCM data.

Table 3: DataSource – bundled fixtures vs. auto-download.

Value	Meaning
Internal	Use files under internal_data_root only.
Production	Auto-download from CDDIS into data_root.

A.3 Differencing regime

Table 4: DifferencingMode – the measurement differencing regime.

Value	Meaning
None	Raw observables (PPP / PPP-RTK without SD).
BetweenSatellite	Per-constellation single difference vs. pivot (PPP-RTK SD).
DoubleDifference	Between-receiver + between-satellite (baseline processing).

A.4 Config-to-differencing mapping

The differencing regime is resolved once per run by the factory below (verbatim from MeasurementBuilder.cpp):

```
DifferencingMode differencingModeFor(const GnssConfig& config) {
    // The network producer runs on undifferenced + uncombined rows and estimates
    // the satellite-side bias lumps explicitly -- it must NEVER apply receiver-side
    // BSSD even if a between_satellite_sd toggle is present in its config.
    if (config.getProcessingMode() == ProcessingMode::PPP_RTK_Network)
                                        return DifferencingMode::None;
    if (config.isBaselineMode())        return DifferencingMode::DoubleDifference;
    if (config.enableBetweenSatelliteSd()) return DifferencingMode::BetweenSatellite;
    return DifferencingMode::None;
}
```

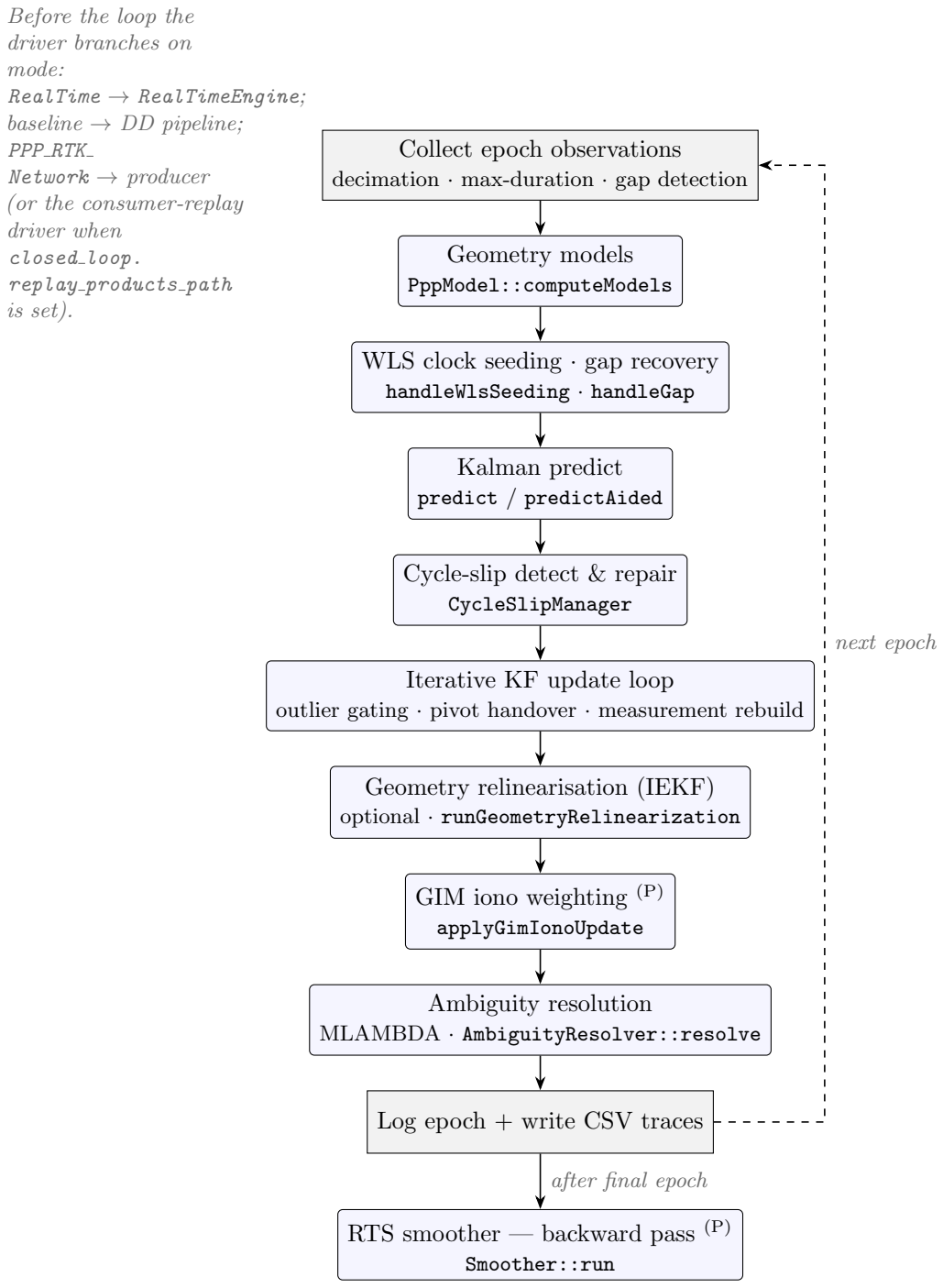


Figure 1: Per-epoch post-processing loop in `Application::run`. Solid arrows are the intra-epoch sequence; the dashed arrow is the epoch-to-epoch feedback. Steps tagged (P) run in post-processing only and are skipped on the real-time path (`RealTimeEngine`). The smoother runs once after the forward pass completes.

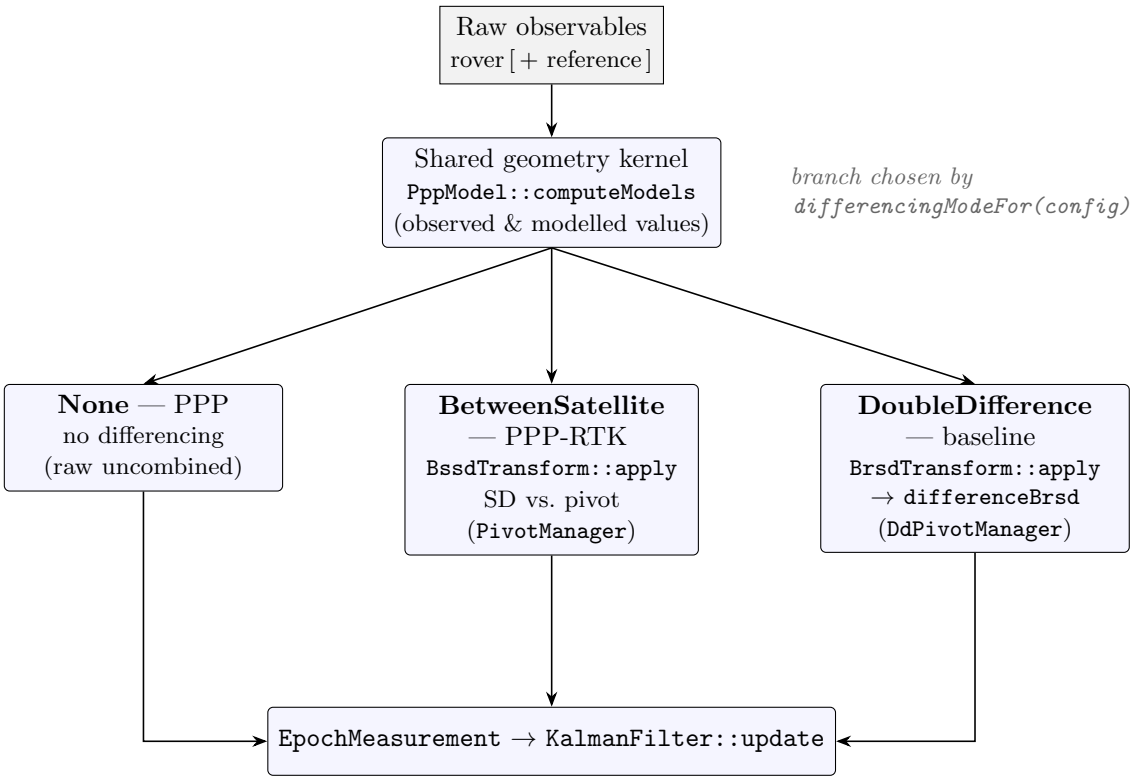


Figure 2: The three differencing regimes share one geometry kernel. Undifferenced residuals are computed once; the strategy selected by `differencingModeFor` then applies no transform (PPP), a between-satellite single difference against a pivot (PPP-RTK), or a between-receiver → between-satellite double difference (baseline) before the common Kalman update.

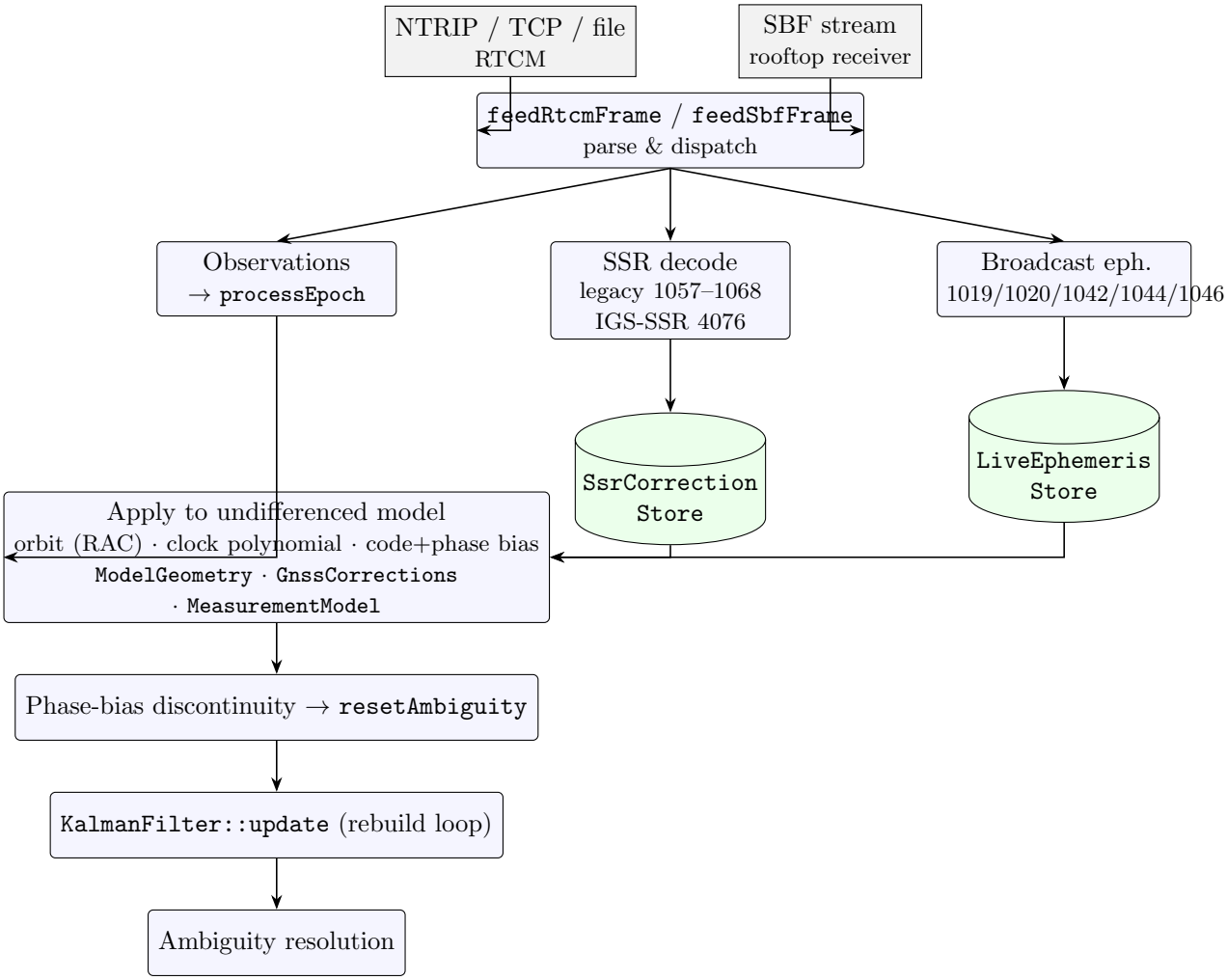


Figure 3: Real-time SSR correction pipeline. Observations, SSR corrections, and broadcast ephemerides are decoded from the byte stream into two stores; the undifferenced observation model then overlays the precise orbit (RAC frame), clock polynomial, and code/phase biases before the Kalman update. A phase-bias discontinuity counter forces an ambiguity reset ahead of the update. See `RealTimeEngine::processEpoch`.

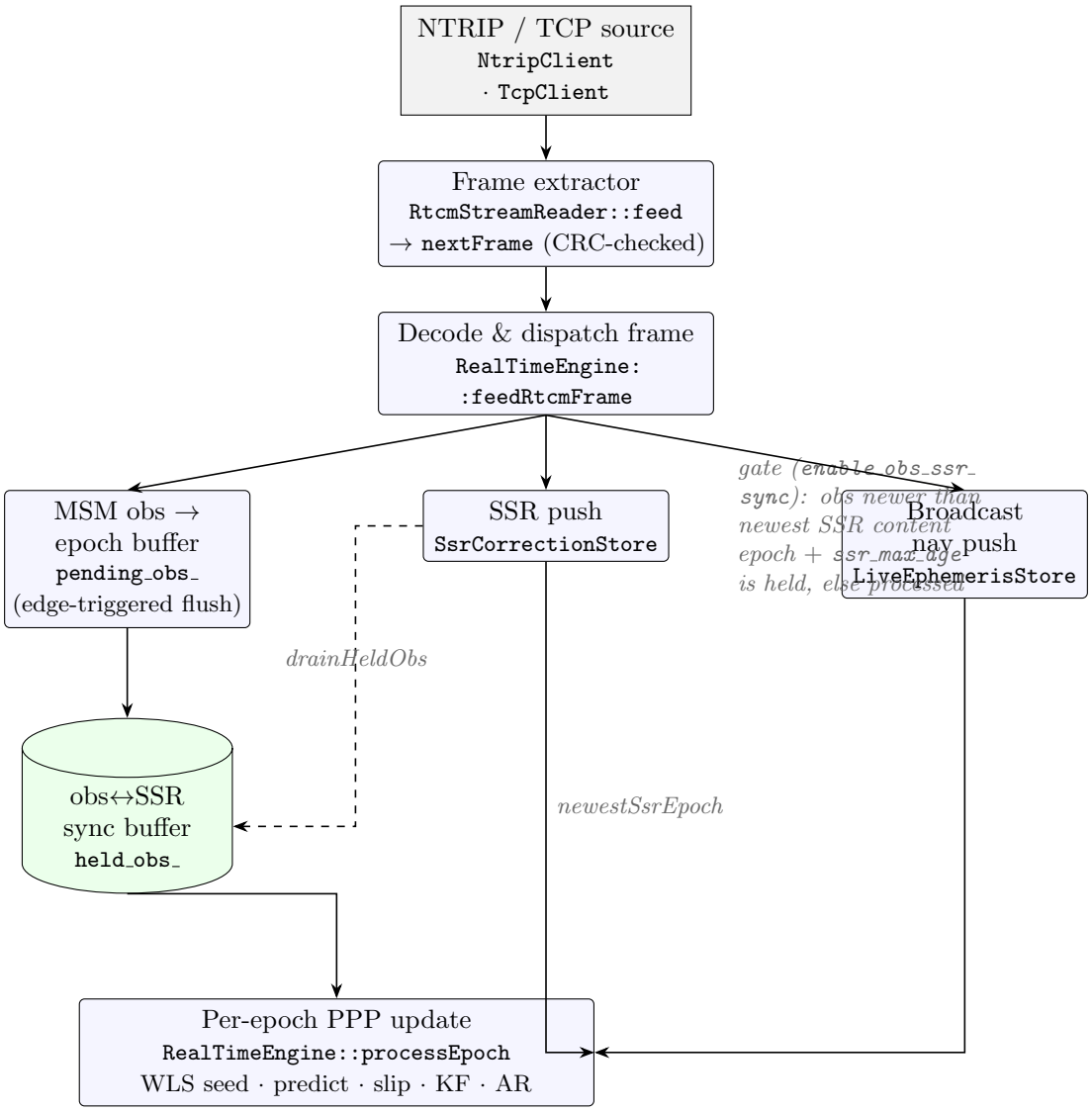


Figure 4: Real-time streaming front-end ahead of the SSR correction stage of Figure 3. Raw bytes from `NtripClient/TcpClient` are reassembled into CRC-validated RTCM3 frames by `RtcmStreamReader`, then decoded and dispatched by `feedRtcmFrame`: MSM observations accumulate in an edge-triggered epoch buffer, while SSR corrections and broadcast ephemerides flow into their stores. When `enable_obs_ssr_sync` is set, an obs epoch that runs ahead of the SSR coverage horizon (newest SSR content epoch + `ssr_max_age`) is held in `held_obs_` and re-attempted by `drainHeldObs` once an SSR push advances the horizon; otherwise it is passed straight to `processEpoch`. See Section 18.

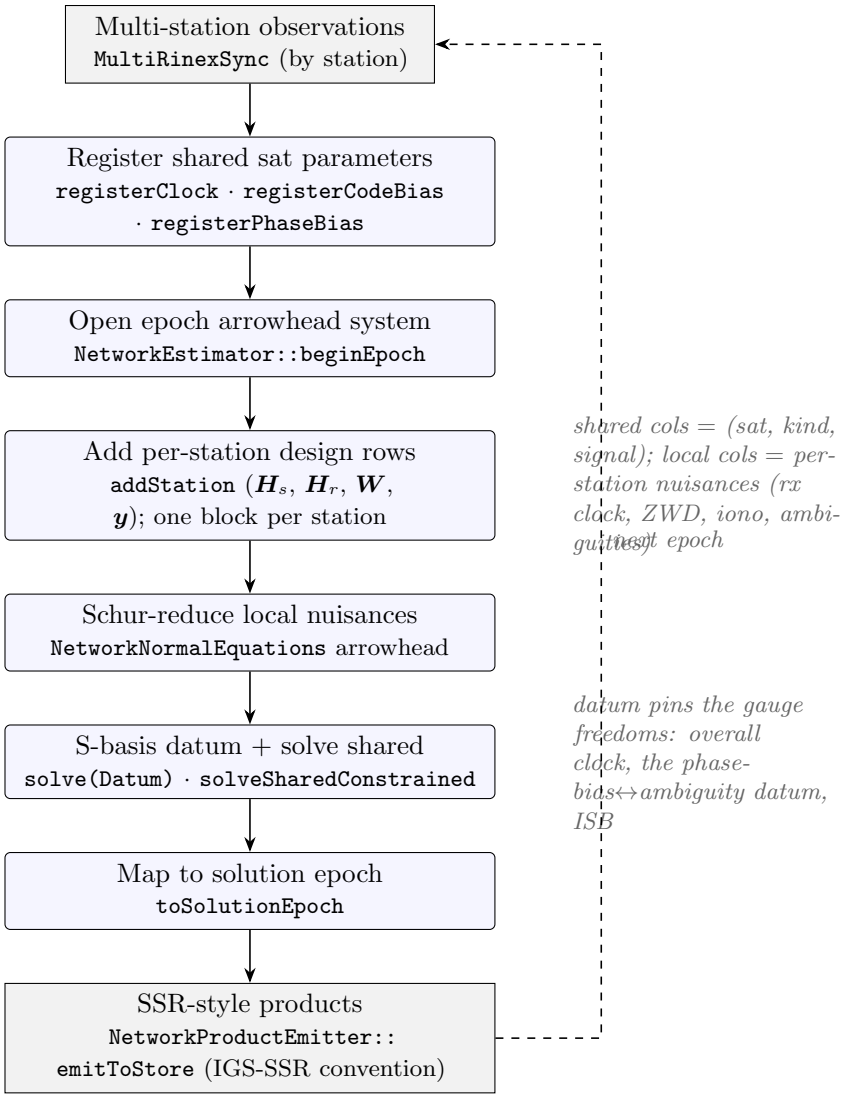


Figure 5: PPP-RTK network producer joint estimator (`NetworkEstimator`). Each epoch the shared satellite-side columns—one per (satellite, parameter-kind, signal) for the estimable clock, code bias, and phase bias—are re-registered at the top of every epoch (`reset`, then `registerClock` / `registerCodeBias` / `registerPhaseBias`, plus `registerVtecCoeff` for the network-wide SH-VTEC coefficients when `network.estimate_iono_sh` is on), after which `beginEpoch` opens an arrowhead normal-equation system of that width. `addStation` contributes one pre-formed block of local-vs-shared design rows per station (the network applies no receiver-side differencing); the local nuisances are Schur-reduced out, leaving a reduced shared system that is rank-deficient by the gauge freedoms. `solve` pins those with an explicit minimum-constraint S-basis datum on a copy of the reduced system, and `toSolutionEpoch` maps the shared estimate to an emittable record that `NetworkProductEmitter` serializes in IGS-SSR convention. The `Application::runNetwork` driver runs this core in a live per-epoch loop over the synced `MultiRinexSync` stations, but is itself only orchestration: the two-pass station assembly (`networkPassAHead_`/`networkPassB_`, `app/ApplicationNetworkAssembly`) feeds `addStation`, the datum and solve run in `networkSolveGlue`, and the emit step in `networkEmitStore` — which, when `network.closed_loop.enable` is armed, additionally drives a held-out PPP-RTK consumer against the freshly emitted products, a between-satellite single-difference (BSSD) consumer by default or an undifferenced one under `closed_loop.consumer_undifferenced`. See Section 19 and Section 19.1.

A.5 Feature toggles

These are the loop- and state-vector-level switches.

Table 5: Top-level boolean feature toggles, parsed from the `jval<bool>` calls in `GnssConfigLoad.cpp` and `GnssConfigLoadNetwork.cpp`. The legacy alias is the pre-refactor flat key still accepted as a fallback.

JSON path	Default	Legacy alias
estimation.accumulate_phase_windup	off	accumulate_phase_windup
estimation.adaptive_iono_process_noise.trace	off	adaptive_iono_q_trace
estimation.aiding.baro	off	aiding_baro
estimation.aiding.nhc	off	aiding_nhc
estimation.aiding.vision	off	aiding_vision
estimation.aiding.wheel	off	aiding_wheel
estimation.aiding.zupt	off	aiding_zupt
estimation.ambiguity_resolution.enable_partial_ar	off	ar_enable_partial_ar
estimation.ambiguity_resolution.wl_only	off	ar_wl_only
estimation.between_satellite_sd.enable_glonass	off	enable_glonass_bssd
estimation.between_satellite_sd.enabled	off	enable_between_satellite_sd
estimation.cycle_slip.ewl	off	cycle_slip_ewl
estimation.cycle_slip.gf	off	cycle_slip_gf
estimation.cycle_slip.graphic	off	cycle_slip_graphic
estimation.cycle_slip.mw	on	cycle_slip_mw
estimation.cycle_slip.turbo_edit	off	cycle_slip_turbo_edit
estimation.cycle_slip.turbo_edit_bridge_lli	off	cycle_slip_turbo_edit_bridge_lli
estimation.cycle_slip.turbo_edit_mw_only_veto	on	cycle_slip_turbo_edit_mw_only_veto
estimation.defer_until_corrections_ready.drop_broadcast_grade	off	defer_until_corrections_ready_drop_broadcast_grade
estimation.defer_until_corrections_ready.require_all_above_mask	off	defer_until_corrections_ready_require_all_above_mask
estimation.enable_cio_frame	off	enable_cio_frame
estimation.enable_data_snooping	off	enable_data_snooping
estimation.enable_full_r_kernel	off	enable_full_r_kernel
estimation.enable_huber_innovation	off	enable_huber_innovation
estimation.enable_huber_r_inflation	on	enable_huber_r_inflation
estimation.enable_if_slip_detector	off	enable_if_slip_detector
estimation.enable_iono_rate	off	enable_iono_rate

JSON path	Default	Legacy alias
estimation.enable_subcycle_slip_suppression	on	enable_subcycle_slip_suppression
estimation.enable_variance_factor	off	enable_variance_factor
estimation.estimate_ifb	off	estimate_ifb
estimation.estimate_phase_clock	off	estimate_phase_clock
estimation.estimate_sat_biases	off	estimate_sat_biases
estimation.estimate_sat_clocks	off	estimate_sat_clocks
estimation.estimate_sat_phase_biases	off	estimate_sat_phase_biases
estimation.estimate_tropo_gradients	off	estimate_tropo_gradients
estimation.estimate_velocity	off	estimate_velocity
estimation.global_vtec_sh.rep_error_d5_keep	off	global_vtec_sh_rep_error_d5_keep
estimation.global_vtec_sh.rep_error_demean	off	global_vtec_sh_rep_error_demean
estimation.global_vtec_sh.sun_fixed	on	global_vtec_sh_sun_fixed
estimation.glonass_ifb.enable	off	enable_glonass_ifb
estimation.lock_signals_to_clock_ref	off	lock_signals_to_clock_ref
estimation.promote_sat_code_bias	off	promote_sat_code_bias
estimation.require_bia_for_signals	off	require_bia_for_signals
estimation.residual_history.enabled	off	residual_history_enabled
network.adaptive_iono_q	off	network_adaptive_iono_q
network.ambiguity_resolution.enable_partial_ar	on	network_ar_enable_partial_ar
network.ambiguity_resolution.wl_cascade	off	network_ar_wl_cascade
network.carry_conditional_coupling	off	carry_conditional_coupling
network.carry_coupling_joint_whiten	off	carry_coupling_joint_whiten
network.carry_coupling_sh_marginalize	off	carry_coupling_sh_marginalize
network.carry_coupling_time_update	off	carry_coupling_time_update
network.carry_invariant	off	network_carry_invariant
network.carry_invariant_emit_rebase	on	network_carry_invariant_emit_rebase
network.carry_invariant_honesty	off	network_carry_invariant_honesty
network.carry_invariant_innov_calibrate	off	network_carry_invariant_innov_calibrate
network.carry_invariant_innov_corroborate	off	network_carry_invariant_innov_corroborate
network.carry_invariant_innov_gate	off	network_carry_invariant_innov_gate
network.carry_invariant_innov_refold	off	network_carry_invariant_innov_refold
network.carry_rx_code_bias	off	carry_rx_code_bias
network.carry_vce	off	carry_vce

JSON path	Default	Legacy alias
network.carry_vce_apply	off	carry_vce_apply
network.carry_zwd	off	carry_zwd
network.cb_admission	off	cb_admission
network.cb_excursion_reinit	off	cb_excursion_reinit
network.cb_excursion_reinit_all_sats	off	cb_excursion_reinit_all_sats
network.cb_kaula_prior	off	network_cb_kaula_prior
network.clock_datum_broadcast_anchored	off	network_clock_datum_broadcast_anchored
network.clock_datum_sum_to_zero	off	network_clock_datum_sum_to_zero
network.clock_disc_on_set_change	off	network_clock_disc_on_set_change
network.clock_model	off	network_clock_model
network.clock_monitor	off	network_clock_monitor
network.clock_monitor_couple_raim	off	network_clock_monitor_couple_raim
network.clock_runaway_check	off	network_clock_runaway_check
network.clock_signal_basis_hold	off	network_clock_signal_basis_hold
network.code_bias_disc_jump_test	off	network_code_bias_disc_jump_test
network.code_datum_time_linked	off	network_code_datum_time_linked
network.cold_clock_seed	off	cold_clock_seed
network.consume_phase_bias_sigma	off	network_consume_phase_bias_sigma
network.consume_ura	off	network_consume_ura
network.cycle_slip.gap_instrument	off	network_cycle_slip_gap_instrument
network.cycle_slip.if_phase_td	off	network_cycle_slip_if_phase_td
network.cycle_slip.if_phase_td_ref_gate	off	network_cycle_slip_if_phase_td_ref_gate
network.cycle_slip.mw_detect	off	network_cycle_slip_mw_detect
network.cycle_slip.mw_lli_corroborate	off	network_cycle_slip_mw_lli_corroborate
network.cycle_slip.turbo_edit	off	network_cycle_slip_turbo_edit
network.emit_clock_disc	off	network_emit_clock_disc
network.emit_clock_rate	off	network_emit_clock_rate
network.emit_combined_sigma	off	network_emit_combined_sigma

JSON path	Default	Legacy alias
network.emit_epoch_status_record	off	network_emit_epoch_status_record
network.emit_generation_id	off	network_emit_generation_id
network.emit_generation_time	off	network_emit_generation_time
network.emit_pb_filter	off	network_emit_pb_filter
network.emit_pb_filter_emit_sigma	off	network_emit_pb_filter_emit_sigma
network.emit_pb_filter_gate_v2	off	network_emit_pb_filter_gate_v2
network.emit_phase_bias_sigma	off	network_emit_phase_bias_sigma
network.emit_ura	off	network_emit_ura
network.estimate_glonass	off	estimate_glonass
network.estimate_glonass	off	estimate_glonass
network.estimate_iono_sh	off	estimate_iono_sh
network.estimate_sat_code_bias	off	estimate_sat_code_bias
network.exclude_unmodeled_antenna_stations	off	network_exclude_unmodeled_antenna_stations
network.fast_clock.allow_single_band	off	network_fast_clock_allow_single_band
network.fast_clock.emit	off	network_fast_clock_emit
network.fast_clock.shadow	off	network_fast_clock_shadow
network.iono_constraint_deweight_adaptive	off	iono_constraint_deweight_adaptive
network.iono_supply_formal_sigma	off	iono_supply_formal_sigma
network.lane_carry.bridge_meas	on	network_lane_carry_bridge_meas
network.lane_carry.fold_blocked	off	network_lane_carry_fold_blocked
network.lane_carry.lane_meas	on	network_lane_carry_lane_meas
network.lane_carry.parallel_components	off	network_lane_carry_parallel_components
network.lane_carry.skip_pinned_increments	off	network_lane_carry_skip_pinned_increments
network.mw_wl.bridge_sample_debias	off	network_mw_wl_bridge_sample_debias
network.mw_wl.d5_dump	off	network_mw_wl_d5_dump
network.mw_wl.ewl_enable	off	network_mw_wl_ewl_enable
network.mw_wl.ewl_fold_rows	off	network_mw_wl_ewl_fold_rows
network.mw_wl.f6_all_blocks	off	network_mw_wl_f6_all_blocks
network.mw_wl.f6_dump	off	network_mw_wl_f6_dump
network.mw_wl.fix_enable	off	network_mw_wl_fix_enable
network.mw_wl.lane_trace	off	network_mw_wl_lane_trace
network.mw_wl.uwl_cold_fix	off	network_mw_wl_uwl_cold_fix

JSON path	Default	Legacy alias
network.mw_wl.uwl_dual_top	off	network_mw_wl_uwl_dual_top
network.mw_wl.uwl_enable	off	network_mw_wl_uwl_enable
network.mw_wl.uwl_family_pairs	off	network_mw_wl_uwl_family_pairs
network.mw_wl.uwl_mid_enable	off	network_mw_wl_uwl_mid_enable
network.mw_wl.uwl_sib_gate	off	network_mw_wl_uwl_sib_gate
network.mw_wl.wl_inject_enable	off	network_mw_wl_wl_inject_enable
network.parallel_carry	off	parallel_carry
network.parallel_reduce	off	parallel_reduce
network.parallel_seam	off	parallel_seam
network.parallel_stations	on	parallel_stations
network.per_const_withhold	off	network_per_const_withhold
network.phase_bias_exclude_unmatched_antenna	off	network_phase_bias_exclude_unmatched_antenna
network.phase_datum_ensemble	off	network_phase_datum_ensemble
network.phase_datum_eq34	off	network_phase_datum_eq34
network.phase_datum_eq34_carry_shift	off	network_phase_datum_eq34_carry_shift
network.phase_datum_eq34_extend_only	off	network_phase_datum_eq34_extend_only
network.phase_datum_eq34_longevity_election	off	network_phase_datum_eq34_longevity_election
network.phase_datum_eq34_pin_round	on	network_phase_datum_eq34_pin_round
network.phase_datum_prioritized	off	network_phase_datum_prioritized
network.phase_datum_sticky	on	network_phase_datum_sticky
network.phase_leveling	off	phase_leveling
network.phase_leveling_refloat_at_convergence	off	phase_leveling_refloat_at_convergence
network.phase_leveling_warmup_on_convergence	off	phase_leveling_warmup_on_convergence
network.pivot_vanish_return_signal	off	network_pivot_vanish_return_signal
network.product_flush_per_epoch	off	product_flush_per_epoch
network.raim	off	network_raim
network.raim_attribution	off	network_raim_attribution
network.raim_redundancy_normalize	off	network_raim_redundancy_normalize
network.raim_refloat_maturity_gate	off	network_raim_refloat_maturity_gate
network.raim_surgical_station	on	network_raim_surgical_station

JSON path	Default	Legacy alias
network.raim_we_code_iono	off	network_raim_we_code_iono
network.raim_within_epoch	off	network_raim_within_epoch
network.require_signal_integer_for_ar	off	network_require_signal_integer_for_ar
network.scoped_handover_response	off	network_scoped_handover_response
network.serialize_disc_counters	off	network_serialize_disc_counters
network.serialize_sbasis_contract	off	serialize_sbasis_contract
network.service_mode	off	network_service_mode
network.sh_health_gate	off	sh_health_gate
network.shared_carry_full_cov	off	shared_carry_full_cov
network.state_snapshot_carry	off	state_snapshot_carry
network.state_snapshot_carry_diagonal	off	state_snapshot_carry_diagonal
network.state_snapshot_keep_history	off	state_snapshot_keep_history
network.state_snapshot_require	off	state_snapshot_require
network.station_pd_pin_aware	off	network_station_pd_pin_aware
network.sticky_pivot	off	network_sticky_pivot
network.trace_anchor_info	off	trace_anchor_info
network.trace_clock_full	off	trace_clock_full
network.trace_dd_fractional	off	trace_dd_fractional
network.trace_epoch_status	off	trace_epoch_status
network.trace_estimability	off	trace_estimability
network.trace_events	off	trace_events
network.trace_flag_rates	off	trace_flag_rates
network.trace_ionosphere	off	trace_ionosphere
network.trace_phase_anchor	off	trace_phase_anchor
network.trace_postfit_keyed	off	trace_postfit_keyed
network.trace_postfit_sigma0	off	trace_postfit_sigma0
network.trace_section_timers	off	trace_section_timers
network.trace_sh_spectrum	off	trace_sh_spectrum
network.trace_slip_events	off	trace_slip_events
network.trace_slip_raw	off	trace_slip_raw
network.trace_ssr_consumer	off	trace_ssr_consumer
network.trace_station_residuals	off	trace_station_residuals
network.trace_stations	on	trace_stations
network.trace_zwd	off	trace_zwd
network.ura_retro_bump	off	network_ura_retro_bump
network.use_sif_coordinates	off	use_sif_coordinates
network.write_run_manifest	off	write_run_manifest
processing.ar_ignore_mw_wl	off	ar_ignore_mw_wl
processing.auto_report	off	auto_report
processing.enable_neq_export	off	enable_neq_export

JSON path	Default	Legacy alias
processing.enable_noncausal_slip_bridge	off	enable_noncausal_slip_bridge
processing.enable_obs_skip_to_latest	off	enable_obs_skip_to_latest
processing.enable_obs_ssr_sync	off	enable_obs_ssr_sync
processing.enable_orbit_normal_attitude	off	enable_orbit_normal_attitude
processing.enable_ssr_bias_max_age	off	enable_ssr_bias_max_age
processing.enable_station_info	off	enable_station_info
processing.enable_trig_sp3_interp	off	enable_trig_sp3_interp
processing.exclude_eclipsing_satellites	off	exclude_eclipsing_satellites
processing.inhouse_bias_estimation	off	inhouse_bias_estimation
processing.network_consumer.bias	on	network_consumer_bias
processing.network_consumer.clock	on	network_consumer_clock
processing.network_consumer.enable	off	network_consumer_enable
processing.network_consumer.undifferenced	on	network_consumer_undifferenced
processing.relinearize_geometry	off	relinearize_geometry
processing.report_async	off	report_async
processing.run_smoother	off	run_smoother
processing.ssr_bias_band_fallback	off	ssr_bias_band_fallback
processing.uncombined_enabled	off	uncombined_enabled
receiver.enable_antenna_orientation	off	enable_antenna_orientation
receiver.enable_antenna_substitution	off	enable_antenna_substitution
receiver.enable_itrf_velocity	off	enable_itrf_velocity

A.6 Model-correction switches

Table 6: Physical model on/off switches (ModelCorrections). These gate the correction terms applied to the undifferenced observation model.

Field	Default	Effect
enable_troposphere	on	Saastamoinen ZHD + GMF dry mapping (slant hydrostatic delay).

Field	Default	Effect
enable_klobuchar_ionosphere	off	Klobuchar ionospheric delay model (single-frequency fallback only). IF combinations and DD short-baselines cancel ionosphere geometrically; PPP-RTK uncombined estimates per-satellite slant iono states. In all three of those configs the Klobuchar model is a no-op — but constructing the Ionosphere instance still pays for an extra Model::Geometry load and the Klobuchar coefficient parse. Default off. Enable only for single-frequency PPP runs that need a-priori iono.
enable_higher_order_ionosphere	off	Apply second-order ($B \cdot \cos(\theta) \cdot STEC/f^3$) and third-order ($\eta \cdot STEC^2/f^4$) ionospheric delays as a per-signal observation-level correction, evaluated at the GIM-derived IPP using a centered tilted dipole geomagnetic field. Requires a loaded IONEX GIM as the absolute-STEC source; the engine auto-loads the GIM when this flag is on even if estimation.ionosphere_weighting.enabled is false. No-op when no GIM resolves (warning logged at startup). Magnitudes: I_2 ˜1–2 cm at L1 during high TEC, I_3 ˜mm; the IF combination only partially cancels these, so the largest visible benefit is in standard PPP (IF). For PPP-RTK uncombined and short-baseline DD the higher-order term mostly cancels in the SD / DD differencing, so this flag is closer to a consistency knob than a bias remover. Default off.
enable_relativistic_correction	on	Periodic relativistic clock correction (-2 rv / c).
enable_shapiro_delay	on	Shapiro (gravitational path) delay.
enable_solid_earth_tide	on	IERS 2010 solid Earth tide displacement.
enable_ocean_tide_loading	on	Ocean tidal loading displacement (BLQ).
enable_pole_tide	on	Pole tide displacement (from ERP): IERS 2010 Sec. 7.1.4 rotational deformation.
enable_ocean_pole_tide	off	Ocean pole tide load displacement (IERS 2010 Sec. 7.1.5 / Desai 2002), added on top of enable_pole_tide. Requires the opole load grid (files.opt_grid_path or ¡data¿/static/opole/). ˜0.1-2 mm; default off keeps runs byte-identical.
enable_atmospheric_loading	off	Atmospheric pressure loading (S1/S2 thermal-tide site displacement). Requires per-station S1/S2 load coefficients (files.atm_tide_grid_path or ¡data¿/static/apl/). Sun-synchronous → does NOT average out in static daily PPP. Default off keeps runs byte-identical.
enable_satellite_pco	on	Satellite antenna phase centre offset.
enable_satellite_pcv	on	Satellite antenna phase centre variation.

Field	Default	Effect
enable_receiver_pco	on	Receiver antenna phase centre offset.
enable_receiver_pcv	on	Receiver antenna phase centre variation.
enable_receiver_cross_system_atx_fallback	off	RECEIVER-ONLY: when the receiver ATX entry lacks a non-GPS frequency, substitute the nearest GPS band (E1-¿G01, E5a-¿G05, B1C-¿G01, B2a-¿G05, ...). Valid: receiver phase pattern is frequency-dependent on shared hardware. NEVER applied to the satellite antenna (E01 and G01 are different spacecraft). Opt-in.
enable_atx_frequency_fallback	on	When the ATX entry doesn't have PCO/PCV for the requested frequency (common for L5 / E5* / B2a on older satellites or receiver antennas), look up the nearest available frequency on the same antenna and use its values instead of silently skipping the correction. Cross-frequency PCO drift on a single physical antenna is typically ¡1 cm — better than the few-cm vertical residual that comes from skipping a frequency entirely. Set false to revert to the legacy "skip if missing" behavior for A/B comparison.
require_atx_coverage	off	When true, drop observations whose (sat, signal) lacks complete ATX coverage on **both** the satellite and receiver entries — i.e., satellite PCO+PCV and receiver PCO+PCV must all resolve to non-empty entries for the row's exact frequency, *without* enable_atx_frequency_fallback being used to substitute a neighbouring frequency. Default false (legacy behaviour: observations are kept and any missing PCO/PCV is silently zeroed). Set true when you've explicitly chosen 'signal_mode: "all"' and want the ATX file to be the authoritative whitelist, dropping rows that would otherwise process without proper antenna calibration.
require_atx_coverage_for_ar	off	When true, AR (WL constraints, LAMBDA NL search) skips (sat, signal) pairs lacking strict ATX coverage even though the underlying observations are still used in the float Kalman update. Softer counterpart to require_atx_coverage: lets you keep "all"-signal float estimation alive while restricting integer fixing to the ATX-curated subset. Default false (no AR-side gating).

Field	Default	Effect
restrict_ar_for_substituted_atx	off	When true, AR additionally skips (sat, signal) pairs whose ATX correction was computed via the frequency-fallback substitution chain (rather than an exact entry). Targets specifically the cases where the antenna model is approximate; signals lacking ATX entirely (no substitution applied) are still allowed. Use alongside or instead of require_atx_coverage_for_ar depending on which kind of imperfection you want to gate on. Default false.
loosen_amb_q_for_substituted_atx	off	When true, ambiguity states whose underlying (sat, signal) had their ATX correction frequency-fallback-substituted use a looser process noise (ambiguity_q_substituted_atx) instead of the standard process_noise.ambiguity. Compensates for the small but non-zero modeling error introduced by the substituted PCV/PCO. Default false.
apply_phase_shifts	off	When true, RINEX phase observations have the receiver-applied "SYS / PHASE SHIFT" values subtracted at parse time, recovering the unshifted phase convention. Default false because most modern BIA products (CODE, CAS, GFZ) are calibrated against the receiver-shifted phases as recorded in RINEX, so leaving the shifts in place is the standard PPP convention. Set true only when your bias product is explicitly calibrated for the unshifted convention, or when running without external BIA and using in-house bias estimation that benefits from a uniform shift-free input.
enable_phase_windup	on	Carrier-phase wind-up correction.
enable_code_phase_bias	on	Code/phase bias corrections from BIA product files.
bias_coverage_requires_apply	off	When true, the "already corrected?" gates in
refine_emission_time	on	When true, Geometry::computeSatelliteGeometry runs a

A.7 Logging and trace channels

Table 7: Logging and CSV-trace channels (LoggingConfig). Each toggles one spdlog file logger or one per-epoch CSV trace.

Field	Default	Channel
enable_main_log	on	logs/main_log.log
enable_gnss_config	on	logs/gnss_config.log

Field	Default	Channel
enable_kalman_filter	on	logs/kalman_filter.log
enable_smoother	on	logs/smoother.log
enable_state_manager	on	logs/state_manager.log
enable_ppp_model	on	logs/ppp_model.log
enable_measurement_model	on	logs/measurement_model.log
enable_mw_slip_detector	on	logs/mw_slip_detector.log
enable_if_slip_detector	on	logs/if_slip_detector.log
enable_gf_slip_detector	on	logs/gf_slip_detector.log
enable_turbo_edit_resolver	on	logs/turbo_edit_resolver.log
enable_cycle_slip_manager	on	logs/cycle_slip_manager.log
enable_ambiguity_resolver	on	logs/ambiguity_resolver.log
enable_corrections	on	logs/corrections.log
enable_geometry	on	logs/geometry.log
enable_troposphere	on	logs/troposphere.log
enable_ionosphere	on	logs/ionosphere.log
enable_antenna	on	logs/antenna.log
enable_antenna_model	on	logs/antenna_model.log
enable_ocean_tide	on	logs/ocean_tide.log
enable_planetary_model	on	logs/planetary_model.log
enable_stochastic_model	on	logs/stochastic_model.log
enable_rinex_parser	on	logs/rinex_parser.log
enable_product_parser	on	logs/product_parser.log
enable_product_cache	on	logs/product_cache.log
enable_resolver	on	logs/resolver.log
enable_state_initializer	on	logs/state_initializer.log
enable_bssd_pivot_manager	on	logs/bssd_pivot_manager.log
enable_bssd_transform	on	logs/bssd_transform.log
enable_ppp_utils	on	logs/ppp_utils.log
enable_downloader	on	logs/downloader.log
enable_ntrip	on	logs/ntrip.log
enable_rt_engine	on	logs/rt_engine.log
enable_imu	on	logs/imu.log
enable_state_trace	on	logs/state_trace.csv
enable_ambiguity_trace	on	logs/ambiguity_trace.csv
enable_ionosphere_trace	on	logs/ionosphere_trace.csv
enable_sat_bias_trace	on	logs/sat_bias_trace.csv
enable_code_bias_trace	on	logs/code_bias_trace.csv (receiver DCB/IFB)
enable_sat_status_trace	on	logs/satellite_status_trace.csv

Field	Default	Channel
enable_dop_trace	on	logs/dop_trace.csv
enable_residual_trace	on	logs/measurement_residuals.csv
enable_wls_trace	on	logs/wls_trace.csv
enable_smoother_trace	on	logs/smoother_trace.csv
enable_mw_slip_trace	on	logs/mwwl_trace.csv
enable_if_slip_trace	on	logs/if_slip_trace.csv
enable_gf_slip_trace	on	logs/gf_slip_trace.csv
enable_turbo_edit_trace	on	logs/turbo_edit_trace.csv
enable_windup_trace	off	logs/windup_trace.csv (phase wind-up diagnostic; off by default)
kalman_filter_concise_log	off	

A.8 Cycle-slip detector registry

Table 8: Cycle-slip detectors known to SlipDetectorRegistry. *Active* detectors have a creator in the factory table and run when their flag is set; *staged* detectors are declared in the SlipDetectorKind enum but excluded from the build (the registry warns and skips them if requested). The $k\sigma$ column is the MAD gate multiplier from CycleSlipConfig.

Detector	Config flag	Default	$k\sigma$	Status	Combination
MelbourneWubbena	estimation.cycle_slip.mw	on	8.0	**active**	Wide-lane MW combination (primary, always available).
Graphic	estimation.cycle_slip.graphic	off	6.0	**active**	GRAPHIC (code+phase average) per-signal detector.
ExtraWideLane	estimation.cycle_slip.ewl	off	5.0	**active**	Extra-wide-lane MW for 3+ frequency satellites.
GeometryFree	estimation.cycle_slip.gf	off	–	*staged*	2nd-order time-differenced geometry-free (STAGED).
IonoFreePhase	estimation.cycle_slip.ifp	off	–	*staged*	Double-differenced ionosphere-free phase OMC (STAGED).

A.9 State-vector key schema

Table 9: Dynamic state-vector key formats, reconstructed from the `make*Key` bodies in `StateKeySchema.hpp`. Angle-bracketed fields are runtime values; literal characters are the delimiter / infix / prefix / suffix constants.

State	Key format	Builder
Ambiguity	`<satellite>_<signal>`	`makeAmbiguityKey`
NL Ambiguity	`<satellite>_NL_<band>`	`makeNlAmbiguityKey`
WL Ambiguity	`<satellite>_WL_<band_a>_<band_b>`	`makeWlAmbiguityKey`
DD Ambiguity	`DD|<cons>|<sat_i>|<code>|<pivot>`	`makeDdAmbiguityKey`
Ionosphere	`<satellite>`	`makeIonosphereKey`
IFB	`<system>_IFB<band>`	`makeIfbKey`
DCB	`<system>_DCB`	`makeDcbKey`
Glonass Per Slot IFB	`<system>_IFB<band>_k`	`makeGlonassPerSlotIfbKey`
Receiver Phase Bias	`<system>_RPB_<signal>`	`makeReceiverPhaseBiasKey`
Satellite Bias	`<satellite>|<signal>`	`makeSatBiasKey`
Satellite Phase Bias	`<satellite>|<signal>|PB`	`makeSatPhaseBiasKey`
Satellite Clock	`<satellite>|SC`	`makeSatClockKey`
System Time Offset	`<system>`	`makeSystemTimeOffsetKey`
Pivot Seed	`<cons>|<band>`	`makePivotSeedKey`
Vtec Sh	`|||<trig>`	`makeVtecShKey`

References

[1] Simon Banville and Richard B. Langley. "Mitigating the impact of ionospheric cycle slips in GNSS observations". In: *Journal of Geodesy* 87.2 (2013), pp. 179–193. DOI: 10.1007/s00190-012-0604-1.

[2] Jean-Paul Berrut and Lloyd N. Trefethen. "Barycentric Lagrange Interpolation". In: *SIAM Review* 46.3 (2004), pp. 501–517. DOI: 10.1137/S0036144502417715.

[3] Guang Chen and Thomas A. Herring. "Effects of atmospheric azimuthal asymmetry on the analysis of space geodetic data". In: *Journal of Geophysical Research: Solid Earth* 102.B9 (1997), pp. 20489–20502. DOI: 10.1029/97JB01739.

[4] Shailen D. Desai. "Observing the pole tide with satellite altimetry". In: *Journal of Geophysical Research: Oceans* 107.C11 (2002), p. 3186. DOI: 10.1029/2001JC001224.

[5] Jianghui Geng and Yehuda Bock. "Triple-frequency GPS precise point positioning with rapid ambiguity resolution". In: *Journal of Geodesy* 87.5 (2013), pp. 449–460. DOI: 10.1007/s00190-013-0619-2.

[6] *GLONASS Interface Control Document: Navigational Radiosignal in Bands L1, L2 (Edition 5.1)*. Russian Institute of Space Device Engineering. Moscow, 2008.

[7] M. Mainul Hoque and Norbert Jakowski. "Estimate of higher order ionospheric errors in GNSS positioning". In: *GPS Solutions* 12.2 (2008), pp. 87–97. DOI: 10.1007/s10291-007-0069-5.

[8] IAU SOFA Board. *SOFA Tools for Earth Attitude*. Software version 18; http://www.iausofa.org/. International Astronomical Union, Standards of Fundamental Astronomy (SOFA). 2021.

[9] International GNSS Service Real-Time Working Group. *IGS State Space Representation (SSR) Format, Version 1.00*. Format specification. RTCM message 4076 IGS-SSR sub-types. International GNSS Service (IGS), 2020.

[10] P. J. de Jonge. "A processing strategy for the application of the GPS in networks". PhD thesis. Delft, The Netherlands: Delft University of Technology, 1998.

[11] J. A. Klobuchar. "Ionospheric time-delay algorithm for single-frequency GPS users". In: *IEEE Transactions on Aerospace and Electronic Systems* AES-23.3 (1987), pp. 325–331. DOI: 10.1109/TAES.1987.310829.

[12] Daniel Landskron and Johannes Boehm. "VMF3/GPT3: refined discrete and empirical troposphere mapping functions". In: *Journal of Geodesy* 92.4 (2018), pp. 349–360. DOI: 10.1007/s00190-017-1066-2.

[13] D. Laurichesse et al. "Integer ambiguity resolution on undifferenced GPS phase measurements and its application to PPP and satellite precise orbit determination". In: *Navigation* 56.2 (2009), pp. 135–149. DOI: 10.1002/j.2161-4296.2009.tb01750.x.

[14] Jean Meeus. *Astronomical Algorithms*. Richmond, VA, USA: Willmann-Bell, 1991. ISBN: 0-943396-35-2.

[15] Oliver Montenbruck and Eberhard Gill. *Satellite Orbits: Models, Methods and Applications*. Berlin, Heidelberg: Springer, 2000. DOI: 10.1007/978-3-642-58351-3.

[16] A. E. Niell. "Global mapping functions for the atmosphere delay at radio wavelengths". In: *Journal of Geophysical Research: Solid Earth* 101.B2 (1996), pp. 3227–3246. DOI: 10.1029/95JB03048.

[17] D. Odijk. "Fast precise GPS positioning in the presence of ionospheric delays". In: *Publications on Geodesy* 52 (2003).

[18] Gérard Petit and Brian Luzum. *IERS Conventions (2010)*. IERS Technical Note 36. International Earth Rotation and Reference Systems Service (IERS). Frankfurt am Main: Verlag des Bundesamts für Kartographie und Geodäsie, 2010. ISBN: 3-89888-989-6.

[19] William H. Press et al. *Numerical Recipes: The Art of Scientific Computing*. 3rd. Cambridge: Cambridge University Press, 2007.

[20] H. E. Rauch, F. Tung, and C. T. Striebel. "Maximum likelihood estimates of linear dynamic systems". In: *AIAA Journal* 3.8 (1965), pp. 1445–1450. DOI: 10.2514/3.3166.

[21] *RTCM Standard 10403.3: Differential GNSS Services – Version 3*. Special Committee No. 104; State Space Representation messages 1057–1068, 1240–1270, 4076. Radio Technical Commission for Maritime Services. Arlington, VA, USA, 2016.

[22] J. Saastamoinen. "Atmospheric correction for the troposphere and stratosphere in radio ranging of satellites". In: *The Use of Artificial Satellites for Geodesy*. Ed. by S. W. Henriksen, A. Mancini, and B. H. Chovitz. Vol. 15. Geophysical Monograph Series. Washington, D.C.: American Geophysical Union, 1972, pp. 247–251. DOI: 10.1029/GM015p0247.

[23] J. Sanz Subirana, J. M. Juan Zornoza, and M. Hernández-Pajares. *GNSS Data Processing, Volume I: Fundamentals and Algorithms*. ESA TM-23/I. Noordwijk, The Netherlands: ESA Communications, 2013.

[24] Tomoji Takasu and Akio Yasuda. "Development of the Low-Cost RTK-GPS Receiver with an Open Source Program Package RTKLIB". In: *International Symposium on GPS/GNSS*. Jeju, Korea, 2009, pp. 4–6.

[25] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.

[26] P. J. G. Teunissen. "The least-squares ambiguity decorrelation adjustment: a method for fast GPS integer ambiguity estimation". In: *Journal of Geodesy* 70.1–2 (1995), pp. 65–82. DOI: 10.1007/BF00863419.

[27] P. J. G. Teunissen and O. Montenbruck, eds. *Springer Handbook of Global Navigation Satellite Systems*. Cham, Switzerland: Springer, 2017. ISBN: 978-3-319-42926-7. DOI: 10.1007/978-3-319-42928-1.

[28] G. Wübbena. "Software developments for geodetic positioning with GPS using TI 4100 code and carrier measurements". In: *Proceedings of the First International Symposium on Precise Positioning with the Global Positioning System*. Rockville, MD, USA, 1985, pp. 403–412.

[29] Thomas P. Yunck. "Coping with the atmosphere and ionosphere in precise satellite and ground positioning". In: *Environmental Effects on Spacecraft Positioning and Trajectories*. Geophysical Monograph Series 73 (1993), pp. 1–16. DOI: 10.1029/GM073p0001.